

TECHNISCHE UNIVERSITÄT DRESDEN

FAKULTÄT INFORMATIK
INSTITUT FÜR SYSTEMARCHITEKTUR
PROFESSUR FÜR DATENBANKEN
PROF. DR.-ING. WOLFGANG LEHNER
PROF. APL. DR.-ING. DIRK HABICH

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Aktive cache-bewusste Hardware-Adaption: Eine Cache-Engine für Trie-basierte Indexstrukturen

Benjamin-Elias Probst
(Geboren am 11. April 1996 in Potsdam)

Hochschullehrer: apl. Prof. Dr.-Ing. Dirk Habich
Betreuer: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, 2. Juli 2026

Aufgabenstellung

Aktive cache-bewusste Hardware-Adaption:

Eine Cache-Engine für Trie-basierte Indexstrukturen

Kontext und Motivation

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanksysteme. Die Professur für Datenbanken an der TU Dresden adressiert die hardware-nahe Optimierung solcher Systeme. Seit der Einführung des Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Techniken und verwandter Algorithmenfamilien vorgeschlagen — von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation —, doch nutzt fast jede Veröffentlichung eigene Datenstrukturen, verschiedene Lastprofile und inkonsistente Messmethodik. Ergebnisse sind dadurch über Veröffentlichungen hinweg schwer vergleichbar. Zuletzt sind die Gewichtungen über Verbesserungen einzelner Algorithmus-Bestandteile derzeit nicht trennbar.

Problemstellung

Suchverfahren — von ausgewogenen Suchbäumen (B-/B⁺-Bäume) über Tries und Radix-Bäume bis zu spezialisierten, eigenentwickelten Konstrukten — spannen einen großen, aber konzeptionell einheitlichen, *Entwurfsraum (design space)* auf: Bereits eine zwei- bzw. dreikomponentige Datenstruktur erlaubt astronomisch viele Entwürfe, und ein konkreter Algorithmus ist ein einzelner Punkt in diesem Raum, beschrieben durch viele einzelne Entwurfsentscheidungen — wie Knoten, Verweise und Daten repräsentiert, durchlaufen, im Speicher angeordnet, und alloziert werden [IZA⁺18, IZH⁺18]. Cache-line-bewusstes Verhalten entsteht dabei nicht an einer einzelnen dieser Stellen, sondern an *vielen verschiedenen strukturellen Stellen* eines Algorithmus bzw. dessen Speicherverwaltung und aus dem Zusammenspiel multipler konzeptioneller Schichten eines Algorithmus. Der Stand der Technik umfasst zahlreiche ausgereifte Verfahren, doch optimiert jedes nur einzelne dieser Stellen und belegt seinen Nutzen in einer eigenen Messumgebung, was den Fortschritt über die Verfahren hinweg schwer vergleichbar macht. Es fehlt ein gemeinsames, generisches Konzept und **Prinzip**, das cache-line-bewusstes Verhalten, neben anderen Messschwerpunkten, an den verschiedenen Stellen eines Suchalgorithmus — austauschbar, systematisch und fair — messen und auswerten kann.

Zielsetzung

Ziel der Arbeit ist es, ein solches Prinzip (ein Rahmenwerk) zu finden, zu entwerfen und prototypisch als Experimentier-Framework umzusetzen, der mit Konzentrationspunkt auf eine dynamische Zusammensetzung der verteilten Algorithmus Konzepte: (i) cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegt, (ii) Algorithmen des Standes der Technik als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert, (iii) cache-line-bewusstes Verhalten sowohl an den einzelnen direkt und indirekt betroffenen Stellen als auch für den gesamten Algorithmus auf gemeinsamer CPU-only-Basis misst und auswertet und (iv) auf Basis der feingliedrigen Messergebnisse den besten Gesamtalgorithmus als eigenständige Binary hinter einem einheitlichen Suchalgorithmus-Interface ausliefert. Als Prüfling für die testweise Eingliederung einer neuen Technologie in den Stand der Technik trägt die Arbeit PRT-ART bei (einen Probst-Redirect-Tree-/ART-Hybriden), der als konzeptionell neuartiger Suchalgorithmus untersucht wird, allerdings nur einige neue Aspekte innerhalb der Klassifizierung von Suchalgorithmen liefert und keinen vollständigen Algorithmus.

Die hier geforderte Trennbarkeit der einzelnen Algorithmus-Bestandteile wird in der vorliegenden Arbeit durch das Achsen-Framework (Kapitel 2) und die drei-granulare Mess-Methodik (Kapitel 3) realisiert.

Teilaufgaben

1. Systematische Erfassung der allgemeinen Entwurfsbestandteile von Suchalgorithmen — insbesondere der cache-bewussten — und Identifikation der Stellen, an denen cache-line-bewusste und suchalgorithmus-spezifische Entscheidungen wirksam werden. Dazu zählt die systematische Untergliederung der Entwurfsbestandteile in Entwurfsebenen einer verallgemeinerten Algorithmus-Kategorisierung

-
2. Rekonstruktion von Entwürfen des Stands der Technik — möglichst aus dem Originalcode der jeweiligen Veröffentlichung und Original Compiler Binary linking — (ART, HOT, Masstree, CoCotrie, START, B²-Tree, Wormhole, SuRF sowie weitere Verfahren) als explizite, vergleichbare Konfigurationen.
 3. Entwurf und Implementierung von PRT-ART als Prüfling mit mehreren eigenen Entwurfsbestandteilen, der gegen den erfassten Stand der Technik antritt.
 4. Aufbau einer reproduzierbaren Mess-Pipeline (Binary → CSV → LaTeX → Diagramm) mit profilbewusstem Workload-Routing aller im Stand der Technik gefundenen Workload-Frameworks über *alle* verfügbaren Workloads und Hardware-Counter-Erfassung.
 5. Systematische, möglichst vollständige Messung über die Konfigurationen in drei verpflichtenden Messreihen — (A) Prüfling gegen den Stand der Technik, (B) systematische Variation der Entwurfsentscheidungen, (C) Merge/Regression alter gegen neue Varianten —, wobei jede Reihe in drei Granularitäten erhoben wird: *Micro-Benchmarking* (jeder einzelne Entwurfsbestandteil innerhalb der Suchalgorithmus-Hülle wird über das gemeinsame Interface seiner Kategorie isoliert gegen Lastprofile gemessen), *Makro-Benchmarking* (die einzelnen Interface-Operationen des Suchalgorithmus gegen Beispiel-Lasten bzw. Test-Teilbereiche der Algorithmus-Bestandteile, z. B. Einfügen, Punktsuche, Löschen) und *Gesamt-Benchmarking* (etablierte YCSB-ähnliche Lastprofile und vergleichbare Frameworks über jede Rekombination der Entwurfsbestandteile).
 6. Auswertung, an welchen Stellen und in welchen Kombinationen cache-line-bewusste Entscheidungen für welche datentyp-spezifischen Lastmuster vorteilhaft oder nachteilig sind — ermittelt aus der Rangbildung über die drei Benchmarking-Granularitäten und aufbereitet als auswertbare Diagramme und Tabellen.
 7. Automatisierung der Systemerkennung und Binary-Auswahl des optimalsten cache-line aware Algorithmus, anhand von empirisch belegten Messwerten, zusammen mit einer anhand der Messwerte dynamisch gesteuerten cache-line aware Anpassung des Algorithmus-Verhaltens als statische heuristische System-Richtlinie für das Algorithmusverhalten unter verschiedenen Lasten und Operationen. Diese sogenannte Extraktion soll als XML-Lastprofil-Ergebnis für die testende Umgebung für einen Gesamtalgorithmus unter Referenz verschiedener angewendeter Lastprofile abgelegt werden, sie ist für jeden Architekturfokus wiederverwendbar, um automatisch Heuristiken zu erstellen.

Methodik und Evaluation

Die Evaluation umfasst neben dem CPU-only-Kern auch SIMD- und nebenläufige Multi-Threading-Pfade und vergleicht die Konfigurationen gegen etablierte Baselines, darunter eine nicht-baumartige Hash-Tabelle (`flat_hash_map`) als Gegenprobe innerhalb derselben Gattung. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik — etwa `url`, `dna`, `protein`, `xml`, `tpcds-id` und `trec-terms` — unter variierten Datenvolumina, Keylängen, Präfixüberlappungen, lokaler Dichte, Skew, Treffer-/Fehlerquoten und Value-Größen. Gemessen werden Exact-/Prefix-Lookup, Prefix-Enumeration, Einfügen/Aktualisieren/Löschen unter allen Varianten der C++ standard-Interfaces bekannter `std::map` und `std::vector` Container-Hüllen, p50/p95/p99-Latenz, Durchsatz, Bytes pro Key sowie Hardwarezähler (Cycles, Instructions, L1/L2/LLC- und dTLB-Misses, Branch-Mispredictions). Zunächst wird das Single-Thread-Verhalten untersucht, danach folgen leseparallele Multi-Thread-Szenarien.

Faire Vergleichbarkeit wird durch feste Regeln gesichert: ein gemeinsamer Minimalmodus (Common-Denominator) wird vom PRT-ART-Native-Modus getrennt, alle Verfahren erhalten identische Schlüsselräume und Seeds, fehlende Fähigkeiten werden als N/A ausgewiesen statt verdeckt emuliert, und Compiler, Flags, ISA-Pfad, Allokator sowie Commit-Hash jeder Baseline werden protokolliert. Für die Reproduzierbarkeit wird jede Gesamt-Algorithmus-Konfiguration in mehreren getrennten Läufen vermessen, deren Streuung Teil der Aussage bleibt (Perzentile über HDR-Histogramme statt Mittelung);

jeder Datensatz trägt eine Akte aus Quelle, Prüfsumme und Seed-Regel, um den Originalcode zweifelsfrei nachzuweisen.

Umfangsabgrenzung

Der Kernbeitrag ist CPU-fokussiert mit SIMD-Erweiterungen und Multi-Threading, beschränkt sich dabei aber auf leseparallele Verfahren mit serialisiertem Schreiben (multi-read, single-write). Die GPU liegt vorerst außerhalb des Umfangs (zukünftig erweiterbar); volle Schreibparallelität und eine Engine-/Optimierer-Integration sind optionale Erweiterungen. Alle Erweiterungen der Architekturen sind ihrerseits dynamische Erweiterungen gegen den CPU-Standard-Lauf und führen zu tieferen Mess-Permutationen mit multipler Laufzeit. Die systemoptimierte Binary lässt sich dabei wahlweise mit oder ohne Mess-Observer übersetzen — als Debug-, Mess- oder Release-Variante.

Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

*Aktive cache-bewusste Hardware-Adaption:
Eine Cache-Engine für Trie-basierte Indexstrukturen*

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 2. Juli 2026

Benjamin-Elias Probst
Mat.-Nr. 4510512

Kurzfassung

Suchalgorithmen und Indexstrukturen sind ein Kernbaustein moderner In-Memory-Datenbanken. Seit dem Adaptive Radix Tree wurde eine Vielzahl cache-bewusster Optimierungen vorgeschlagen—von Knoten-Layout über Prefetching und Succinct-Repräsentationen bis zur Allokation—, doch wurden sie nie in einem einzigen, fairen Messrahmen kombiniert und verglichen: Jede Veröffentlichung nutzt eigene Datenstrukturen, Lastprofile und Messmethodik, wodurch Ergebnisse über Veröffentlichungen hinweg schwer vergleichbar sind. Diese Arbeit stellt ein Cache-Engine-Framework vor, dessen Module auf Binärebene durch eine einzige, ABI-stabile C-Schnittstelle getrennt sind (je Modul genau ein exportiertes Lebewesen); dahinter liegt eine dreischichtige Abstraktion (CacheEngine → ExecutionEngine → SearchEngine), die die Such-API bereitstellt und vom Anatomie-Modell (Gattung, Lebewesen, Organe) abgegrenzt bleibt. Das Framework zerlegt cache-bewusste Suchalgorithmen in austauschbare, verteilte (über den Code gestreute) Entwurfsentscheidungen eines großen Entwurfsraums (*design space*) [IZA⁺18, IZH⁺18] und setzt sie durch dynamische Zusammensetzung mittels Permutation (keine Laufzeit-Adaption) wieder zusammen. Strukturell verschiedene Verfahren—von baumartigen Indizes (Tries, B⁺-Bäume) bis zu nicht-baumartigen Hash-Tabellen—werden als Punkte desselben Entwurfsraums hinter einem einheitlichen Interface ausgedrückt; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Entwurfsentscheidungen. Algorithmen des Standes der Technik werden als explizite Konfigurationen (Punkte im Entwurfsraum) rekonstruiert und auf drei Granularitäten gemessen—einzelne Entwurfsentscheidungen, einzelne Interface-Operationen und ganze Algorithmen—unter bekannten Workload-Framework-Lastprofilen (YCSB und weitere, von diesem ein Framework integrierte Frameworks), wobei jedes Modul wahlweise mit oder ohne Mess-Observer für Debug, Messung oder Release gebaut werden kann. Als Prüfling trägt zu dieser Arbeit der PRT-ART bei, ein Probst-Redirect-Tree-/ART-Hybrid mit acht eigenen Bausteinschichten; ein Mess-Treiber mit profilbewusstem Workload-Routing bewertet ihn unter drei verpflichtenden Messreihen gegen dreißig rekonstruierte Stand-der-Technik-Verfahren (acht etablierte Referenzverfahren mit Bausteinen und weitere); Fernziel ist die Auslieferung der besten Rekombination als eigenständige Binary.

Inhaltsverzeichnis

1	Einleitung	3
1.1	Motivation	3
1.2	Problemstellung	4
1.3	Forschungsfragen	4
1.4	Zielsetzung und Beiträge	5
1.5	Aufbau der Arbeit	7
2	Suchbäume und Grundlagen	9
2.1	Konzept und Überblick über den Stand der Technik	9
2.2	Hardware: historische Anpassung an die Suchleistung	11
2.3	Software-Mittel und Zusammenführung zur Abstraktion	14
3	Mess-System mit PRT-ART als Demonstration	23
3.1	Belange der Aufgabenstellung	23
3.2	Stand der Technik als Achsen-Instanzen	23
3.3	Das Mess-System	30
3.4	PRT-ART als Demonstrator	32
3.5	Schärfung durch Heuristiken	33
4	Implementierung und konkrete Algorithmusbestandteile	35
4.1	Drei-Repository-Architektur	35
4.2	PRT-ART im Detail	36
4.3	Achsen-Implementierung und Mess-System-Ebenen	37
5	Evaluation	41
5.1	Methodik	41
5.2	Ergebnisse und Analyse	42
6	Fazit und Ausblick	45
6.1	Beantwortung der Forschungsfragen	45
6.2	Limitierungen	46
6.3	Ausblick	47
	Literaturverzeichnis	49
	Abbildungsverzeichnis	63
	Tabellenverzeichnis	67

A	Vollständige Messreihen	69
A.1	Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)	69
A.2	Bias-Bruch-Matrix	69
A.3	Latenz-Surfaces je Schnittstellen-Funktion	69
A.4	Achsen-Austauschbarkeit	71
A.5	Limitierungen	83
B	Code-Struktur	91
C	Glossar	93
D	Bausteine-Matrix	97
D.1	T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)	97
D.2	T1 Cache-Speicher-Traversierung (Fanout-Lookup)	100
D.3	T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)	101
D.4	T3 Pfadkompression	101
D.5	T4 Knotentypen (Node-Type-Achse)	102
D.6	T5 Speicher-Layout (Memory Layout)	104
D.7	T6 Allokator-Achse (T6)	105
D.8	T7 Prefetch-Strategie	107
D.9	T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)	109
D.10	T9 Serialisierung (Kodierung und Kompression)	110
D.11	T10 Telemetrie und Messinstrumentierung (axis_11)	111
D.12	T11 Wert-Handle (Value-Handle)	112
D.13	T12 Instruktionssatz-Architektur (ISA)	113
D.14	T13 Index-Organisation	114
D.15	T14 I/O-Dispatch-Achse	115
D.16	T15 Migrations-Strategie-Achse	117
D.17	T16 Filter-Achse	118
D.18	T17 Pufferung Q1 (Operations-Puffer)	119
D.19	T18 Pufferung Q2 (Flush-Policy)	121
D.20	Build-PG Seitentyp-Achse (PAGE-TYPE)	123
D.21	Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse	124
D.22	Build-HW — Plattform-Hardware-Strategie	125
D.23	A-Korpus Allokator-Korpus (A01-A23)	126
E	Architekturentscheidungen	131
F	Vergleichsinterfaces: <code>std::map</code> und <code>std::vector</code>	133
F.1	<code>std::map</code> (geordnet-assoziativ)	133
F.2	<code>std::vector</code> (Sequenz)	136
F.3	Zusammengesetzte Funktionen und Konformitäts-Teilmenge	138

1 Einleitung

1.1 Motivation

Suchbäume und Indexstrukturen tragen moderne datenintensive Systeme — von In-Memory-Datenbanken über Dateisysteme bis zur Sprachverarbeitung. Zugleich werden heutige Prozessoren von der Speicher-Hierarchie dominiert: nicht die Zahl der Operationen, sondern ihr Cache-Verhalten bestimmt die Laufzeit. Aus diesem Spannungsfeld erwächst die Leitfrage der Arbeit. Konkret bilden Suchalgorithmen und

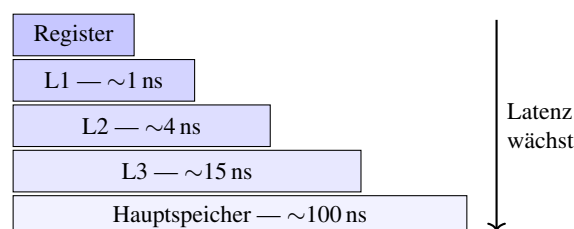


Abbildung 1.1: Die „Cache-Wand“: die Zugriffslatenz wächst über die Speicher-Hierarchie um rund zwei Größenordnungen — nicht die Operationszahl, sondern das Cache-Verhalten bestimmt die Laufzeit.

Indexstrukturen das Rückgrat moderner Hauptspeicher-Datenbanken; trie-basierte Indizes und cache-line-optimierte Verarbeitung stehen im Zentrum dieser Arbeit. Seit dem Adaptive Radix Tree (ART) [LKN13] von 2013 hat sich die Forschung ausdifferenziert: Trie-Höhen-Optimierung (HOT [BZP⁺18]), hybride B⁺-Strukturen (Masstree [MKM12], B²-Baum [SSL⁺21]), succinct-Repräsentationen (CoCo-Trie [BFTV24], SuRF [ZLL⁺18]) und selbstoptimierende Ansätze (START [FJKN20]).

Parallel dazu ist ein eigener Forschungskorpus zu Cache-Bewusstheit, Prefetching und Allokationsstrategien entstanden. Heuristiken wie Cache-Sensitive Search Trees, cache-oblivious Layouts und hierarchisches Prefetching adressieren jeweils einen separat betrachteten Aspekt der Suchalgorithmus-Architektur, wobei der Fokus meist auf empirisch gefundenen Verbesserungen und Messungen liegt. Weil so vieles auf cache-line-optimiertem Verhalten beruht, liegt eine Segmentierung der Suchalgorithmus-Bestandteile in *Achsen* nahe — eine rückwärtsgerichtete Abstraktion bekannter Bausteine in wiederverwendbare Module über die wichtigsten Arbeiten des Standes der Technik hinweg.

Diese Bausteine lassen sich als austauschbare Entwurfsdimensionen eines großen Entwurfsraums (*design space*) auffassen [IZA⁺18, IZH⁺18]. Strukturell verschiedene Suchverfahren — von baumartigen Indizes bis zu nicht-baumartigen Hash-Tabellen, samt Spezialeigenschaften wie Cache-Lines — werden so als Punkte desselben Entwurfsraums vergleichbar; nicht-baumartige Verfahren belegen dabei ein eingeschränktes Subset der Dimensionen über demselben Suchalgorithmus-Interface, als wären sie der einzige Fächer einer Baum-Wurzel. Als Prüfling für die Stand-der-Technik-Zugehörigkeit dient PRT-ART, ein Probst-Redirect-Tree-/ART-Hybrid und eine abstrakte, unvollständige Ansammlung neuartiger Suchalgorithmus-Bestandteile.

1.2 Problemstellung

Cache-Bewusstheit ist in heutigen Suchstrukturen entweder fest verdrahtet oder ganz ignoriert; vor allem aber lässt sich nicht *trennen*, welche einzelne Entwurfsentscheidung welchem Lastmuster nützt — das *Trennbarkeits-Problem*, der rote Faden dieser Arbeit. Bestehende cache-bewusste Index-Entwürfe optimie-

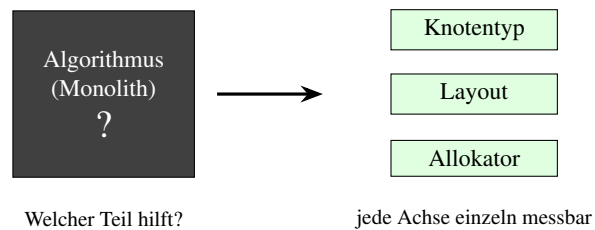


Abbildung 1.2: Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar.

ren ein oder wenige Aspekte isoliert und berichten Ergebnisse, die sich über Veröffentlichungen hinweg schwer vergleichen lassen, da jede Arbeit eigene Datenstrukturen, Workloads und Mess-Methodiken verwendet. Die Gewichtungen der Verbesserungen einzelner Algorithmus-Bestandteile bleiben dabei untrennbar: Ein gemeldeter Leistungsgewinn lässt sich nicht eindeutig auf einen einzelnen Bestandteil (Knotentyp, Layout, Value-Ablage, Prefetching) zurückführen, weil die Bestandteile in den Original-Entwürfen verflochten bleiben. Es fehlt ein gemeinsamer Rahmen, der (i) solche Entwürfe in austauschbare Achsen zerlegt, (ii) publizierte Algorithmen als explizite Konfigurationen rekonstruiert und (iii) sowohl einzelne Achsen als auch ganze Algorithmen auf einer einheitlichen, fairen Basis vermessen kann.

1.3 Forschungsfragen

Aus der Aufgabenstellung und dem Ziel der Cache-Line-Optimierung ergeben sich — aufgrund der Allgegenwärtigkeit des Einflusses der Cache-Line-Einstellungen — eine Hauptfrage (FF0) und vier messbare Teilfragen (FF1–FF4); ergänzende Aspekte sind als *Teilfragen* zugeordnet.

FF0 (Hauptfrage) Wie stark verbessern *aktive, messgetriebene* cache-bewusste Entscheidungen — Seitentyp, Speicher-Layout, Value-Ablage und Prefetching — die Leistung trie-basierter Suchstrukturen gegenüber *passiven, statisch ausgelegten* Varianten auf unterschiedlichen CPU-Plattformen (Hybrid-CPU, Sapphire Rapids)?

FF1 (Komposition) Wie lässt sich ein Suchalgorithmus-Entwurf kanonisch in orthogonale Achsen zerlegen und in einer Permutationsmatrix systematisieren, sodass die Stand-der-Technik-Verfahren möglichst aus ihrem Originalcode bias-frei als Profil-Konfigurationen rekonstruierbar und beliebig permutierbar werden? *Teilfrage:* Welche Achsen-Kopplungen (Knotentyp $\times$ SIMD $\times$ Pfadkompression) begrenzen die Isolation einzelner Achsen, und wie verhält sich dieselbe achsen-konstante Struktur im Cache-Update- und Cache-Line-Füllverhalten, wenn allein die ISA-/SIMD-Achse (SSE2/AVX2/AVX-512, NEON/SVE2, RVV) variiert wird?

FF2 (Mess-Methodik) Wie muss eine reproduzierbare Mess-Pipeline mit Hardware-Zähler-Erhebung gestaltet sein, damit jede Achsen-Permutation fair (unter sonst gleichen Bedingungen, also gleicher

Compiler- und Flag-Basis) in drei Messreihen (A–C) über drei Granularitäten (Micro-, Makro- und Gesamt-Benchmarking) gemessen wird, und unter welchen plattform-kalibrierten Kriterien gilt eine Plattform als erfolgreich verifiziert? *Teilfrage*: Wie lassen sich widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine Cache-Line bei CSS/CSB⁺ gegenüber bis zu sechzehn bei Hankins/Patel) durch achsen-isolierte, unter sonst gleichen Bedingungen permutierte Messung bias-frei nachmessen, statt sie aus inkompatiblen Original-Setups zu übernehmen?

FF3 (Prüfling-Vergleich) Gewinnt der Prüfling PRT-ART gegenüber den acht Rang-1-Entwürfen im Stand der Technik (*State of the Art*, kurz SOTA; ART, HOT, Masstree, CoCo-Trie, START, B²-Baum, Wormhole, SuRF)¹ und der Rang-2/3-SOTA *nicht* durch asymptotische Neuheit, sondern durch bessere Mikroarchitektur-Eigenschaften (höhere Cache-Line-Auslastung, weniger LLC- und dTLB-Misses, geringere Branch-Kosten, kleinere heiße Suchpfade) bei Exact-Lookup, Prefix-Lookup und Prefix-Enumeration — gemessen unter den drei Pflicht-Messreihen A (Prüfling vs. Stand der Technik), B (systematische Variation der Entwurfsentscheidungen) und C (Merge/Regression alt gegen neu)? *Teilfrage*: Welche lokale Seitendarstellung (8-/16-Bit-Punktseite gegenüber -Kompaktseite) ist je Verzweigungspunkt am günstigsten, und nach welchen plattform-kalibrierten Umschaltregeln — einschließlich der Reihenfolge $\text{Array}_{256} \rightarrow \text{Array}_{65535} \rightarrow \text{Vector}\langle u8, u8 \rangle \rightarrow \text{Vector}\langle u16, u16 \rangle$ je Füllstand und Suchtyp — soll adaptiv gewechselt werden?

FF4 (Compile-Time vs. Laufzeit) Welche Achsen-Eigenschaften müssen zur Übersetzungszeit fixiert werden, um plattform-optimierte, provenienz-nachweisbare Binaries je Permutation zu erzeugen, und welche können bzw. müssen zur Laufzeit gewählt werden (bit-codierter Permutations-Identifizier, adaptive Layout-Selection), ohne die wissenschaftliche Vergleichbarkeit (gleiche Compiler- und Flag-Basis) zu verletzen? Wird es durch die Messung möglich, ISA+OS optimierte Binaries für eine Suchalgorithmus-Rekombination zu erzeugen, deren heuristische Optimierung perfekt auf ihre Laufzeitumgebung zugeschnitten ist?

1.4 Zielsetzung und Beiträge

Die Zielsetzung folgt der Aufgabenstellung in vier Schritten: cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zerlegen, Stand-der-Technik-Entwürfe als explizite Konfigurationen rekonstruieren, cache-line-bewusstes Verhalten sowohl je Bestandteil als auch für den ganzen Algorithmus auf gemeinsamer CPU-Basis messen und auswerten sowie den besten Gesamtalgorithmus nach einer autonomen Heuristik-Profil-Auswertung und Heuristik-Funktions-Modellierung als eigenständige Binary ausliefern. Das zugrunde liegende Architekturmodell — die drei Ebenen (Gattungs-Interface, Gattungs-Unterklasse, Achsen-Organe), die vier Subsysteme und die ABI-stabile Auslieferung der Suchalgorithmus-*Lebewesen* — entwickelt Kapitel 2; das Konzept der Achsen-Architektur selbst ist aus einer Vorarbeit des Autors übernommen (zur Herkunft siehe Abschnitt 2.3), der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine mess-orientierte Realisierung. Die zentralen Beiträge sind:

¹Die *Ränge* ordnen die Stand-der-Technik-Veröffentlichungen nach Zentralität für diese Arbeit: Rang-1 umfasst die zehn wichtigsten Kern-Arbeiten — darunter die acht eigenständigen Suchentwürfe sowie ARTSync (P08) [LABN16] und LOUDS (P09) [Jac89], die als Organe eingehen —, Rang-2 die cache-bewussten Layout- und B⁺-Vorarbeiten und Rang-3 die Prefetching-, Synchronisations- und TU-Dresden-Hardware-Arbeiten.

- Eine ABI-stabile Modul-Schnittstelle: eine einheitliche C-Modulgrenze (`comdare_create_anatomy() → IAnatomyBase`) exportiert je Permutation genau ein Lebewesen (Suchalgorithmus-Binary); dahinter liegt eine dreischichtige Abstraktionskette (`CacheEngine → ExecutionEngine → SearchEngine`) mit variadisch templatierten Hybrid-API-Spezialisierungen (1 Parameter $\Rightarrow$ `std::vector`-API; 2 Parameter $\Rightarrow$ `std::map`-API; $N > 2 \Rightarrow$ `std::map<K, std::tuple<V1...VN>>`).
- Eine neunzehn-Achsen-Permutationsmatrix mit Boost.MP11-Compile-Time-Typlisten-Bausteinen (kein Runtime-Tag, kein `std::variant`; der `std::variant`-Katalog besteht nur als historische Elf-Achsen-Vorstufe) und Compile-Time-Fallback zwischen Prüfling-Stack und Stand-der-Technik-Stack.
- Dreißig SOTA-Algorithmus-Profile als persistierte XML-Konfigurationen (8 eigenständige Rang-1 + 22 Rang-2/3), die der `ExperimentDriver` automatisch in vorkompilierte C++23-Module überführt; perspektivisch soll der Profilsatz jede analysierte Veröffentlichung mit mindestens einem Algorithmus-Profil abdecken.
- PRT-ART als Prüfling mit acht eigenen Bausteinschichten (4+1+2-Pool-Allokator, optimistisches Lock-Coupling mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, Density-Tracker, Hypothesen-Metriken H1/H2/H3, Inline/External/ChainRef-ValueHandles, Signaling-Bit-Serialisierung); im Mess-Apparat führt der Prüfling davon sechs Schlüsselstellen als eigene Organe — die Build-Achse Seitentyp sowie Pfadkompression (Redirect), pfadorientiertes Prefetch, OLC-Nebenläufigkeit, Telemetrie und den ChainRef-Value-Handle-Slot —, während MultiLevel-Layout, Pool-Allokator und Serialisierung im Mess-Pfad derzeit über den Cache-Engine-Standard laufen (Abschnitt 3.4).
- Ein Mess-Treiber mit Defined-/Full-Modus-Unterscheidung, profilbewusstem Workload-Routing und drei Granularitäten — Micro-, Makro- und Gesamt-Benchmarking (einzelne Bestandteile, Interface-Operationen, ganze Algorithmen).
- Eine bias-freie Vergleichs-Matrix, die *alle* Lastprofile gegen *alle* rekonstruierten Lebewesen fährt und so dem methodischen Heimspiel-Bias der Einzelveröffentlichungen begegnet.
- Eine generische Plattform-Probe, die Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) je Zielplattform ermittelt und als feste Größen in eine systemoptimierte Binary einfließen lässt; die beiden Produktionsmaschinen werden dafür unter zwei Betriebssystemen (immutables Talos und root-Linux mit vollem Hardware-Zähler-Zugriff) vermessen.
- *Als Zielsetzung und Ausblick:* die Auslieferung des besten Gesamtalgorithmus unter dem Messaspekt des besten Cache-aware-Verhaltens und der Beschleunigung unter Cache-Line-Anpassung — automatisch ausgewählt aus den feingliedrigen Messergebnissen — als eigenständige, ABI-stabile Binary hinter dem einheitlichen Suchalgorithmus-Interface; die Mess-Pipeline liefert die dafür nötige Rangbildung bereits, und ein erstes Inkrement der automatischen Auswahl und des Binary-Versands ist als eigenständiges Werkzeug (`best_binary_selector`) implementiert; die heuristik-gesteuerte Vollautomatisierung bleibt Ausblick.

1.5 Aufbau der Arbeit

Der Aufbau folgt dem Weg vom Allgemeinen zum Speziellen: Kapitel 2 legt — konzeptionell und bildhaft — die Grundlagen (Suchbäume, Hardware, Software-Mittel der Abstraktion); Kapitel 3 begründet daraus das Mess-System als Lösung und demonstriert es an PRT-ART; Kapitel 4 vertieft die Implementierung im Detail; Kapitel 5 wertet aus; Kapitel 6 schließt mit Fazit und Ausblick. Die Anhänge vertiefen begleitend: die Messreihen-Dokumentation (Anhang A), die Code-Struktur (Anhang B), das Glossar (Anhang C), die vollständige Bausteine-Matrix (Anhang D), die Architekturentscheidungen (Anhang E) und die Vergleichs-Interfaces (Anhang F).

2 Suchbäume und Grundlagen

Dieses Kapitel führt vom vertrauten Gegenstand — den Suchbäumen — über die Hardware, die ihre Leistung prägt, zu den Software-Mitteln, mit denen sich ihre Entwurfsentscheidungen abstrahieren lassen. Alles bleibt *bildhaft und konzeptionell*; die systematischen Kataloge folgen in Kapitel 3.

2.1 Konzept und Überblick über den Stand der Technik

Wir kartieren zunächst die Landschaft der Suchstrukturen — vergleichsbasierte Bäume, digitale und präfixbasierte Verfahren, Hashing, räumliche und flache Strukturen — und stellen *jeden* bekannten Algorithmus bildhaft im Detail dar. Hier wird die Abstraktion bereits konzeptionell verortet: jeder Algorithmus ist ein Punkt in einem gemeinsamen Entwurfsraum. Suchstrukturen lassen sich danach einordnen, mit

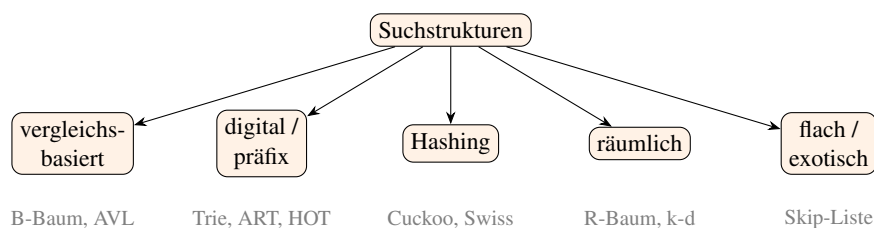


Abbildung 2.1: Die Landschaft der Suchstrukturen: fünf Klassen, die das vorgeschlagene Achsen-System gemeinsam erfasst.

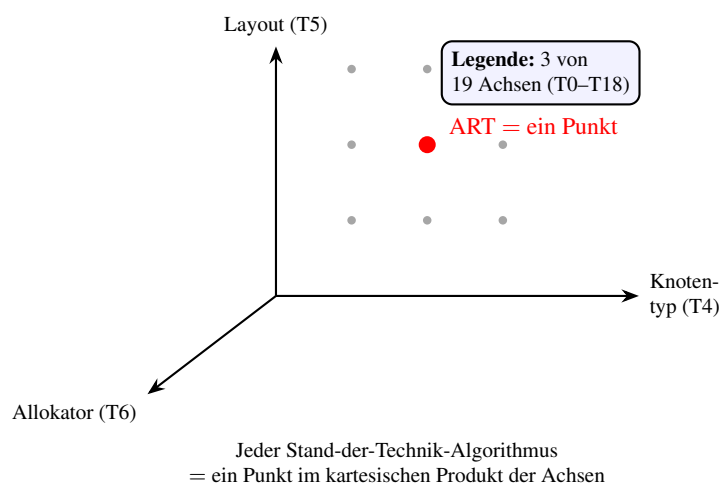


Abbildung 2.2: Der Entwurfsraum: die Achsen spannen ein kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der neunzehn Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat neunzehn Dimensionen.

welchem Mechanismus sie einen Schlüssel auf einen Datensatz abbilden [Knu98]. *Vergleichsbasierte*

Strukturen lokalisieren den Schlüssel über Ordnungsvergleiche und setzen eine totale Ordnung voraus — von den selbstbalancierenden AVL- [AVL62] und Rot-Schwarz-Bäumen [Bay72, GS78] über die block- und externspeicherorientierten B-/B⁺-Bäume [BM72, Com79], deren Knoten typischerweise einer Cache- oder Speicherblock-Größe entsprechen [GL01, HP03], bis zum hauptspeicher-nahen T-Baum [LC86], der knotenlokale Wertebereiche mit der AVL-Balance verbindet und so die Zahl der Zeiger-Indirektionen je Suchschritt senkt. *Digitale* Strukturen verzweigen dagegen nach den Bytes des Schlüssels statt durch Vergleich, sodass die Suchpfadlänge von der Schlüssellänge und nicht von der Schlüsselanzahl abhängt: der *Trie* [Bri59, Fre60] als Grundform, in dem jeder Wurzelpfad einen Schlüssel buchstabiert und gemeinsame Präfixe geteilt werden, die pfadkomprimierten Radix-/PATRICIA-Bäume [Mor68], die dünn belegte Ein-Kind-Ketten kollabieren, und der *Adaptive Radix Tree* (ART) [LKN13], der die Knotenrepräsentation an den tatsächlichen Verzweigungsgrad anpasst und dieser Arbeit durchgängig als Referenzentwurf dient. Jenseits der baumartigen Verfahren bildet *Hashing* Schlüssel direkt über eine Hashfunktion ab und kennt keine inhärente Ordnung, weshalb Bereichsabfragen teuer sind [Knu98]; Cuckoo-Hashing [PR04] und SIMD-beschleunigte Tabellen wie SwissTable [Goo18] treten dieser Arbeit als nicht-baumartige Vergleichsgröße zur Seite. *Räumliche* Strukturen (k-d-Baum [Ben75], Quadtree [FB74], R-Baum [Gut84]) partitionieren einen mehrdimensionalen Schlüsselraum und liegen außerhalb des Umfangs, ebenso wie die theoretischen „Exoten“ — der van-Emde-Boas-Baum [EB75] und der y-fast Trie [Wil83] mit $O(\log \log u)$ über einem Universum der Größe u oder der die Vergleichsschranke unterschreitende Fusion Tree [FW93] — sowie das flache, sortierte Array mit Binär- oder Interpolationssuche [Knu98] — streng genommen das einzelne Blatt eines Baumes — und seine String-Verallgemeinerung, das Suffix-Array [MF02]. Diese Arbeit konzentriert sich auf die baumartigen Strukturen der vergleichs- und der digitalbasierten Klasse; das entwickelte Prinzip ist jedoch nicht an Bäume gebunden, denn so verschieden die Klassen äußerlich wirken, teilen sie doch eine gemeinsame innere *Anatomie* aus orthogonalen Teilentscheidungen, die Abschnitt 2.3 als Achsensatz freilegt (Abbildung 2.1).

Quer zu dieser Suchalgorithmus-Sicht steht die etablierte Untergliederung der *Container* der C++-Standardbibliothek [ISO24]: Sequenz-Container (`std::vector`) ordnen nach Einfügeposition, geordnet-assoziative (`std::map`, `std::set`) nach Schlüsselordnung, ungeordnete (`std::unordered_map`) bilden über eine Hashfunktion ab. Diese genormten Schnittstellen-Verträge bilden die gemeinsame Basis, gegen die sich strukturell verschiedene Verfahren einheitlich vermessen lassen — die geordnete Map als gemeinsamer Nenner vergleichs- wie digitalbasierter Suchstrukturen, der `std::vector` als Sequenz-Sicht und die Hash-Tabelle als ungeordnete Gegenprobe; ihre Einordnung in das Begriffsgerüst dieser Arbeit (Gattung, Gattungs-Unterklasse, Achse) entwickelt Abschnitt 2.3.

Der so umrissene Entwurfsraum wird im Stand der Technik systematisch ausgemessen. Die Analyse stützt sich auf 33 tiefenanalysierte Suchalgorithmus-Veröffentlichungen (P01–P33) und einen parallelen Allokator-Korpus aus 23 Arbeiten (A01–A23), deren Achsen-Belegung jeweils als XML-Profil in der `comdare-cache-engine` persistiert ist; die vollständigen Bausteine- und Allokator-Matrizen führt der Anhang. Der Kernbeitrag ist dabei die *Sezierung*: Jeder Gesamt-Algorithmus — ein „Lebewesen“ — wird in seine orthogonalen Organe, die Achsen, zerlegt, und die so gewonnenen Organ-Algorithmen werden je Achse einsortiert, mit kurzem Verweis auf das Herkunfts-Lebewesen, einer web-verifizierten Provenienz-Map über rund 110 Organ-Algorithmen folgend; die systematische, achsenweise Sezierung erfolgt in Kapitel 3 (Abschnitt 3.2). Jeder Stand-der-Technik-Algorithmus erscheint so als ein Punkt im kartesischen Produkt der Achsen (Abbildung 2.2). Dass das Achsen-Gerüst auch nicht-baumartige Verfah-

ren trägt, belegt eine reine Hash-Tabelle als Gegenprobe innerhalb der SearchAlgorithm-Gattung — als zwei wählbare T0-Bausteine, von denen je Permutation genau einer gewählt wird und die beide permutiert werden: ein Open-Addressing-Hash nach Knuth (S14, implementiert) und die SwissTable [Goo18] (als eigener Baustein vorgesehen). Die Gegenprobe erfüllt das Punktanfrage-Subset des `std::map`-Interface (`find/insert/erase`), belegt ein limitiertes Achsen-Subset — Allokation, Prefetch, Nebenläufigkeit, Speicher-Layout und ISA-SIMD bleiben geteilt, die trie-spezifischen Achsen entfallen — und erfordert so keine eigene Gattung.

Die Veröffentlichungen entstammen sechs thematischen *Herkunfts-Clustern*, die hier nur der ganzheitlichen Einordnung jedes Lebewesens dienen, bevor es in seine Organe zerlegt wird. Die *Trie-Familie* umfasst ART (P01) [LKN13], HOT (P02) [BZP⁺18], Masstree (P03) [MKM12], CoCo-Trie (P04) [BFTV24], START (P05) [FJKN20] sowie die succinct-Tries LOUDS (P09) [Jac89] und SuRF (P10) [ZLL⁺18]. Die *Hybrid- und B⁺-Familie* vereint den B²-Baum (P06) [SSL⁺21], Wormhole (P07) [WNJ19] und die cache-sensitiven CSS-, CSB⁺- und Hankins-Bäume (P11–P13) [RR99, RR00, HP03]. Die *Layout-Theorie* reicht von Samuel (P14) [SPB05] und Graefe/Larson (P15) [GL01] über die cache-oblivious Entwürfe von Bender et al. (P16/P17) [BDFC00, BDFC05, BDFC02, BCDFC02, BCD⁺02] bis zu den layout-invarianten Bäumen von Saikkonen/Soisalon-Soininen (P18/P19) [SSS08, SSS16]. Das *Prefetching* umfasst *B-Trees Are Back* (P20) [MBL25], die Prefetching-B⁺-Bäume von Chen et al. (P21/P22) [CGM01, CGM02], Khan (P23) [Kha11], Naderan-Tahan (P24) [NTSA16], Mahling (P25) [MWR25] sowie Zhang (P26/P27) [ZSZ⁺24, ZGH⁺25] und Kuehn (P28) [Kue23]. Die *Synchronisation* trägt das optimistische Lock-Coupling von ART (P08) [LABN16], RCU (P29) [McK01] und Hazard-Pointer (P30) [Mic04a]; die hardware-nahe *TU-Dresden-Linie* schließlich Ungethüm (P31) [UHL⁺17], Schmidt (P32) [SKK⁺25] und VAMPIR (P33) [BH23]. Der parallele *Allokator-Korpus* (A01–A23) spannt von den Klassikern `dlmalloc` [Lea96], Hoard [BMBW00], Slab [Bon94, BA01] und Michaels lock-freier Allokation [Mic04b] über die mehrkern-skalierbaren `mimalloc` [LZM19], `jemalloc` [Eva06], `TCMalloc` [GM05], `snmalloc` [LDPB19], `scalloc` [AKLS15], `rpmalloc` [Jan17] und `lrmalloc` [LR18] bis zu den cache-aware/vorhersagbaren CAMA [HBHR11], `NUMALloc` [YZZ⁺23], `StarMalloc` [RFP⁺24b] und `PIM-malloc` [VIA26]. Abbildung 2.3 verdichtet die wichtigsten rekonstruierten Suchstrukturen zu einer Galerie — je Verfahren seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet; alle erscheinen so als Punkte im gemeinsamen Entwurfsraum.

2.2 Hardware: historische Anpassung an die Suchleistung

Die Leistung von Suchstrukturen wird seit jeher von der Hardware getrieben. Wir führen die historische Einführung cache-naher Hardware ein — Cache-Hierarchie, Cache-Line, Adressübersetzung, Kohärenz — mit Fokus auf die *statischen und übersetzungszeit-relevanten* Eigenschaften, die später die Abstraktion tragen. Moderne Prozessoren schalten mehrere Cache-Ebenen (L1, L2 und — nicht auf jeder Plattform — L3) zwischen die Register und den Hauptspeicher [HP19]; ein Zugriff, der sein Datum nahe findet, ist günstig, während ein Fehlzugriff bis zum Hauptspeicher einige hundert Zyklen kostet [Dre07], auf Mehr-Sockel-Systemen zusätzlich abhängig vom besitzenden NUMA-Knoten. Die Übertragungseinheit zwischen den Ebenen ist die *Cache-Line* (Abbildung 2.4); ihre Größe ist keine universelle Konstante, sondern je Mikroarchitektur fest verdrahtet — 64 Byte auf x86-64 und gängigen

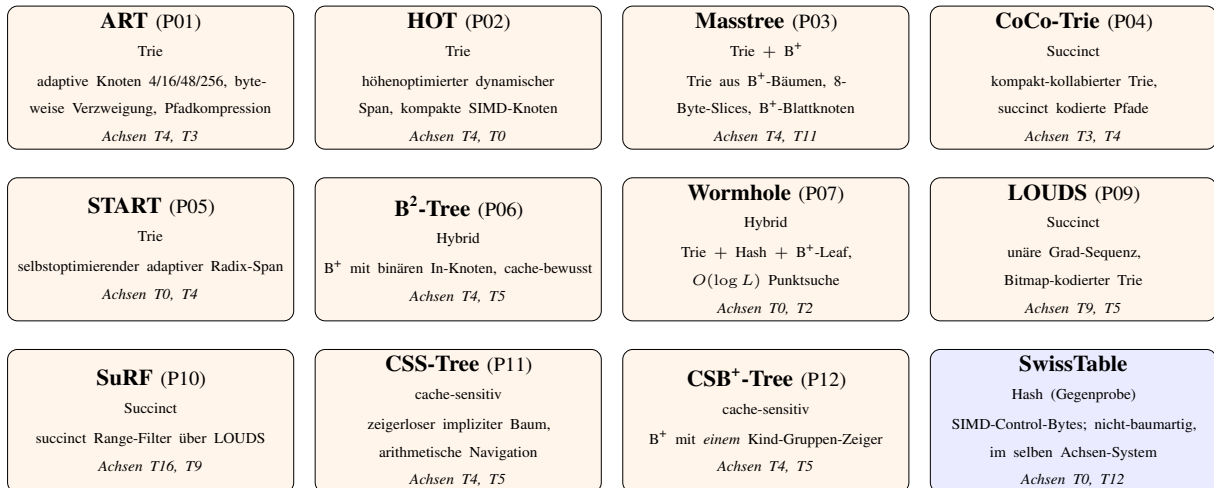
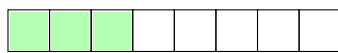


Abbildung 2.3: Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.



64-Byte-Cache-Line:

nur 3 von 8 Einheiten genutzt $\Rightarrow$ niedrige Auslastung

Abbildung 2.4: Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.

ARM-Kernen, 128 Byte etwa auf Apple-Silicon-L2, schmalere 32 Byte auf älteren oder eingebetteten Kernen [HP19, Int24a, Adv23, App24]. Programme können sie zur Laufzeit *auslesen* — über `CPUID` (x86), `CTR_EL0` (ARM) oder `riscv_hwprobe` (RISC-V) [Arm24, Int24b] —, aber nicht verändern; auch die Tiefe der Hierarchie variiert (einfache RISC-V-Kerne enden bei L2, manche AMD-Prozessoren bringen einen gestapelten 3D-V-Cache-L3 mit). Für ein cache-line-bewusstes Layout ist die je Zielplattform ermittelte Line-Größe daher ein *Eingabeparameter der Übersetzung*, keine feste Annahme — der Transfer selbst erfolgt dabei nicht streng blockweise am Stück, denn *critical-word-first* liefert zuerst das angefragte Wort und DDR5 unterteilt den Speicherkanal in zwei unabhängige Sub-Kanäle [KAS21]. Eben dieses cache-line-orientierte Layout ist der historische Leitgedanke der einschlägigen Forschung: Der CSS-Baum [RR99] führt den Cache-Line-Knoten ein, der CSB⁺-Baum [RR00] das zusammenhängende Geschwister-Cluster, Hankins und Patel [HP03] vermessen den Effekt der Knotengröße und Samuel et al. [SPB05] machen den Knoten prozessor-bewusst; der Adaptive Radix Tree [LKN13] staffelt mit Node4/16/48/256 die Knotengröße cache-line-orientiert, und Schmidt et al. [SKK⁺25] vermessen Stride-gegen Sequenz-Zugriffe auf High-Bandwidth-Memory. Auf der Speicher-Layout-Ebene stehen damit konkret das dichte und das cache-line-ausgerichtete Array-of-Structs, das spaltenorientierte Struct-of-Arrays, das bit-gepackte LOUDS [Jac89] und das SIMD-gekachelte AoSoA. Die mehrstufige Hierarchie selbst ist überdies Gegenstand software-/hardware-kooperativer Prefetch-Verfahren wie dem hierarchischen Instruktionen-Prefetcher von Zhang et al. [ZGH⁺25], der als Prefetching-Instanz und nicht als datenstruktur-spezifischer Baum-Datenprefetch eingeht.

Neben der Daten-Cache-Hierarchie existiert eine zweite Lokaltätsgrenze: der *Translation Lookaside*

Buffer (TLB) cacht Virtueller-zu-Physisch-Abbildungen, und ein TLB-Miss löst einen teuren Page-Walk aus [HP19]. Prozessoren trennen aus Lokalisierungsgründen den Instruktions- vom Daten-TLB (dTLB) und legen ihn mehrstufig aus [Int24b, Dre07]; für Suchstrukturen ist der dTLB die zweite, eigenständige Grenze neben der Cache-Line, denn große oder weit gestreute Knoten erhöhen den dTLB-Druck unabhängig von der Cache-Auslastung, weshalb große Seiten (Huge-Pages, 2 MiB/1 GiB) den TLB-Reach vergrößern. Im Stand der Technik tritt der dTLB vorwiegend als *Mess-Ziel* auf; algorithmenseitig adressieren ihn Huge-Pages, die mehrere Allokatoren (mimalloc, jemalloc, TCMalloc, snmalloc) unterstützen und die Schmidt et al. [SKK⁺25] mit 2-MiB-Seiten empirisch nutzen — eine *eigene* dTLB-Optimierungs-Achse bleibt in den Suchalgorithmus-Papern jedoch unterbelegt, eine Beobachtung, auf die die Forschungslücke in Kapitel 3 (Abschnitt 3.1) zurückkommt.

Auch das Schreib- und Update-Verhalten der Caches ist überwiegend mikroarchitektonisch fixiert. Schreibzugriffe folgen *Write-Through* oder *Write-Back*, orthogonal dazu legt die Allokations-Politik (Write-Allocate bzw. No-Write-Allocate) die Behandlung eines Schreib-Fehlzugriffs fest; ARMv8 macht diese Wahl über Seiten-Attribute explizit [Arm24], während x86 im Normalfall Write-Back mit Write-Allocate verwendet [Int24b]. Welche Line verdrängt wird, entscheidet eine heuristische Ersetzungsstrategie von LRU/Pseudo-LRU bis zur zwei-Bit-sparsamen *Re-Reference Interval Prediction* [JTJE10, QJP⁺07]; wie viele Ebenen eine Line zugleich führen, regelt die Inklusions-Politik, und in Mehrkern-Systemen sichert ein invalidierungsbasiertes Kohärenzprotokoll (MESI mit den Varianten MOESI und MESIF) die Konsistenz [HP19]. Als konkrete Instanz dieser fixierten Eigenschaften variiert etwa die L3-Inklusions-Politik zwischen den Generationen — inklusiv bei frühen Intel-Client-Kernen, nicht-inklusiv bei Intel-Server ab Skylake-SP und überwiegend victim-artig bei AMD Zen [Int24a, Adv23]. Da all dies zur Laufzeit nicht frei wählbar ist, fließt es als *vermessene* Plattform-Eigenschaft in die systemoptimierte Binary ein; seine Wirkung — insbesondere das *Cache-Line-Ping-Pong* bei konkurrierenden Schreibzugriffen auf dieselbe Line — motiviert die Telemetrie- und Nebenläufigkeits-Achsen (Abschnitt 2.3, Kapitel 4).

Diese Eigenschaften spannen das Spektrum auf, in dem sich eine Datenstruktur verorten lässt: *cache-bewusst* (cache-aware), wenn ihr Layout auf konkrete Cache-Parameter abgestimmt ist, und *cache-oblivious*, wenn sie über alle Ebenen hinweg gute Lokalität erreicht, ohne sie zu kennen [FLPR99]. Beide Philosophien sind im Stand der Technik instanziiert: Bender et al. begründen den cache-oblivious B-Baum über das van-Emde-Boas-Layout [BDFO05], aufbauend auf dem Order-Maintenance-/Packed-Memory-Primitiv [BCD⁺02], der cache-oblivious Traversierung geordneter Mengen [BDFO02] und dem nahezu optimalen Baum-Layout fester Topologie [BDFO02, BDFO00]; Saikkonen und Soisalon-Soininen halten eine cache-sensitive Layout-Invariante auch unter Updates, und Graefe/Larson [GL01] vergleichen beide Ansätze über die CPU-Cache-Ebenen, während sämtliche praktischen Trie- und B⁺-Indizes cache-bewusst ausgelegt sind. Die vorliegende Arbeit verfolgt einen dritten, dazwischenliegenden Weg: Die tatsächlichen Cache-Eigenschaften (Line-Größe, Hierarchie-Tiefe, Topologie) werden je Zielplattform *ermittelt* und gehen als feste Größen in eine systemoptimierte Binary ein, während die *konfigurierbaren* cache-nahen Einstellungen (Prefetch-Distanz, Stapel- und Inline-Schwellen, Pool-Budgets) als Achsen-Parameter gegen eine bereits geladene Binary permutiert werden — abgestuft nach dem, was Hardware und Betriebssystem erlauben, von voll laufzeit-konfigurierbar (mutables Linux mit MSR-Zugriff) über übersetzungsstatisch (immutable Betriebssysteme) bis fixiert (read-only-Cache-Register auf ARM und RISC-V). So wird cache-oblivious-Verhalten um *gemessenes* Lokalisierungswissen ergänzt; das *Warum* dieser Konstruktion entwickelt Abschnitt 2.3.

2.3 Software-Mittel und Zusammenführung zur Abstraktion

Mit welchen Software-System-Mitteln — Übersetzungszeit-Metaprogrammierung, statische Nutzung der Hardware aus Abschnitt 2.2 — lässt sich die Abstraktion *umsetzen*? Hier führen wir die bekannten Verfahren bildhaft zu *unseren* Achsen zusammen und definieren alle Bestandteile, die konzeptionellen Spezialkonstrukte (Hüllen, besondere Entwurfsmuster), die Begrifflichkeiten und die Grundlagen wissenschaftlichen Messens. Die zentrale Idee ist, einen Algorithmus nicht als Monolith zu behandeln, sondern als *Komposition orthogonaler Achsen* — jede Achse eine Teilentscheidung eines cache-bewussten Index (wie Seiten angeordnet werden, wie Knoten verzweigen, wie Speicher alloziert wird). Ein konkreter Algorithmus ist dann ein Punkt im kartesischen Produkt aller Achsen, ein Stand-der-Technik-Entwurf eine explizite Konfiguration. Entscheidend ist die *modulare Austauschbarkeit*: Eine neue Achsen-Variante lässt sich isoliert untersuchen — ihr Effekt je Achse *und* für den ganzen Algorithmus messen —, ohne die übrigen neu zu implementieren; genau das löst das Trennbarkeits-Problem, da der Beitrag jedes Bestandteils bei sonst fixierten Achsen isoliert messbar wird. Eine anschauliche Metapher fasst dies zusammen: ein Suchalgorithmus als *Lebewesen*, dessen *Organe* die Achsen sind, und das Mess-System als seine medizinische Diagnostik (Abbildung 2.5).

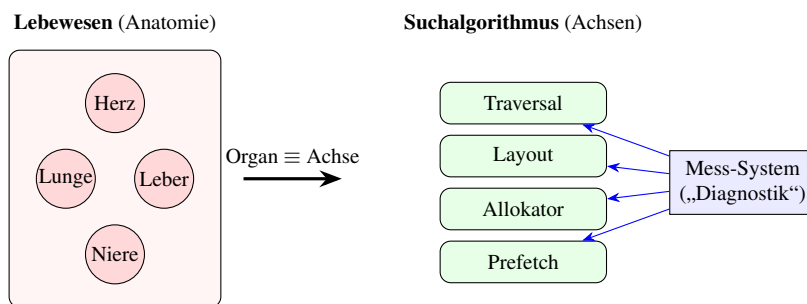


Abbildung 2.5: Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Mess-System übernimmt die Rolle der medizinischen Diagnostik und vermisst jedes Organ einzeln.

Die Permutation der Achsen gleicht dann einem genetischen Experiment: Der Austausch von Organen zwischen Lebewesen erzeugt neue Komposit-Algorithmen mit bisher unbekanntem Cache-Line-Verhalten. Die Organe selbst werden aus dem Stand der Technik gewonnen — die Entwurfsbausteine der bekannten Verfahren (Abschnitt 2.1) werden in orthogonale Achsen zerlegt und in einer gemeinsamen Achsen-Matrix gesammelt, sodass jede Achse die Beiträge des gesamten Korpus erntet (Abbildung 2.6).

Die so gewonnene *Anatomie* ist mehr als die Aufzählung der Organe: Sie ist ihre *Verdrahtung*. Jedes Organ exportiert ein generalisiertes Interface, über das die übrigen seine Dienste nutzen, ohne dessen konkrete Belegung zu kennen — so stellt das Allokations-Organ ein gemeinsames Allokations-Interface bereit, das Knoten-Layout, Wert-Ablage und Serialisierung für ihre Speicheranforderungen verwenden (Abbildung 2.7). Software-technisch ist dies die *Feature-Interaktion* feature-orientierter Software-Produktlinien [ABKS13, BO92]: eine Achse entspricht einem Feature, ein Lebewesen einer Feature-Komposition, die Anatomie der Verdrahtung zwischen den Features. Welche Achse welche nutzt, zeigt der Inter-Organ-Nutzungs-Graph (Abbildung 2.8).

Setzt man diese paarweisen Kopplungen über alle Organe zusammen, entsteht ein gerichteter Inter-Organ-Nutzungs-Graph.

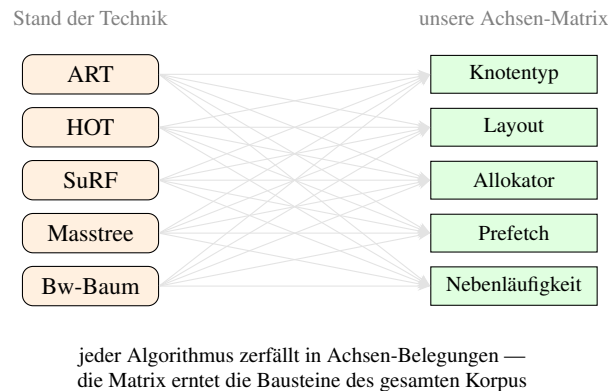


Abbildung 2.6: Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 2.2 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).

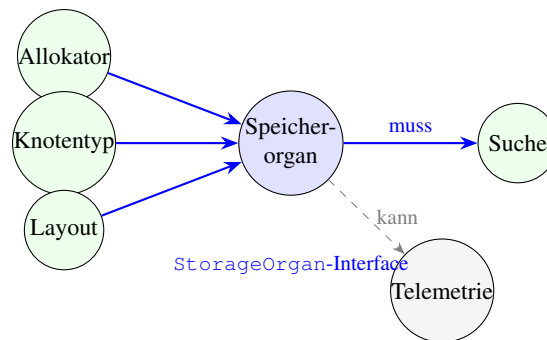


Abbildung 2.7: Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes `StorageOrgan-Interface`, über das die Suche läuft (durchgezogen, *verpflichtend*); weitere Organe wie die Telemetrie können es über optionale Beobachter-Hooks nutzen (gestrichelt, *optional*), müssen aber nicht.

Diese Organe sind in *eine* Architektur eingebettet — eine einzige, in sich geschlossene Hierarchie ohne Parallel-Bäume. An ihrer Wurzel steht die `ExecutionEngine` als gemeinsame, ausmessbare Abstraktion über allem Messbaren; darunter lassen sich drei Ebenen unterscheiden: die *Gattung* als nach außen sichtbares Algorithmus-Interface (das *Prüf-Dock*), darunter die *Gattungs-Unterklasse* mit festem Achsensatz, darunter die *Organe* selbst (Abbildung 2.9). „Lebewesen“ und `SearchAlgorithm` bezeichnen denselben Gegenstand — Metapher und technischer Begriff —; die Anatomie ist kein zweites Modell, sondern der *Körper* dieses Lebewesens: die feste Komposition seiner neunzehn Achsen-Organen. Auf der obersten Ebene stehen genau drei Gattungen. Die `SearchAlgorithm`-Gattung — das schlüsselwert-orientierte Prüf-Dock und der Fokus dieser Arbeit — trägt als einzige Unterklasse das volle Neunzehn-Achsen-„Säugetier“; die `Container`-Gattung (schlüsselloser Speicher) fächert in die Unterklassen `Set`, `Sequence`, `Adapter` und `View` auf; die `Graph`-Gattung trägt einen eigenen, andersartigen Organsatz (Abbildung 2.10). Nicht-baumartige Verfahren erfordern so keine eigene Gattung, sondern belegen ein eingeschränktes Achsen-Subset derselben Gattung.

Der eigentliche Beitrag liegt damit weniger in einer einzelnen Struktur als in einem *Werkzeug*: Statt das Vergleichsproblem direkt zu lösen, baut diese Arbeit zuerst die „Schaufel“ — den Permutations-

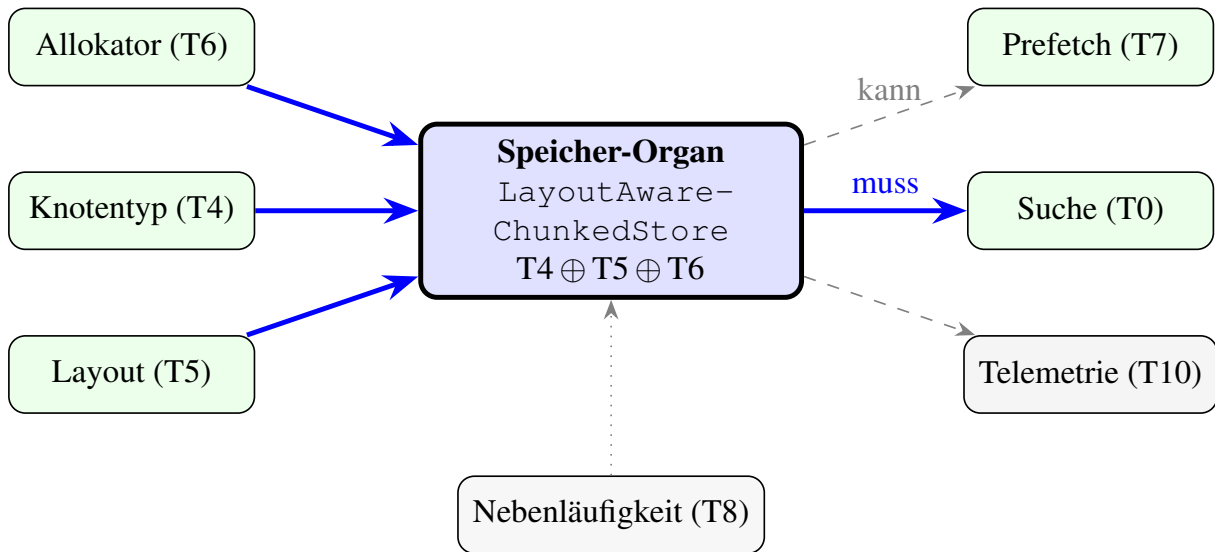


Abbildung 2.8: Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knotentyp (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und Telemetrie (T10) lesen den Store nur über *requires*-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist im Code (noch) nicht verdrahtet (gepunktet).

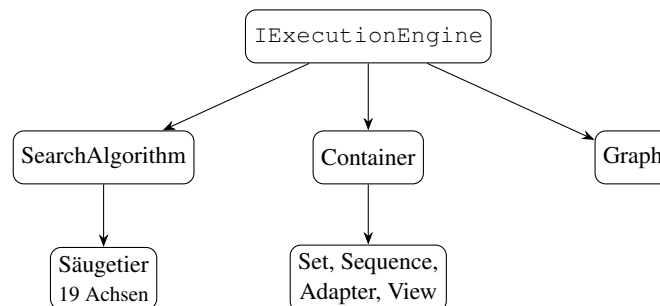


Abbildung 2.9: Das Drei-Ebenen-Modell: Gattung (Ebene 1, Außen-Interface) → Tier-Unterklasse (Ebene 2, fester Achsen-Satz) → Organe/Achsen (Ebene 3). Die SearchAlgorithm-Gattung trägt das „Säugetier“ mit den neunzehn Achsen; die Container-Gattung die Unterklassen Set/Sequence/Adapter/View.

und Mess-Apparat —, mit der sich beliebige Achsen-Kombinationen erzeugen und fair vermessen lassen. Wissenschaftlich ordnet sich dieser Apparat in das Software-Engineering der *Komposition* ein: die invasive Software-Komposition aus Gray-Box-Komponenten [Aßm03], die generative Programmierung [CE00] und die feature-orientierten Software-Produktlinien [ABKS13, BO92]. Das Achsen-Konzept selbst ist aus einer Vorarbeit des Autors (Marke Comdare / BEP Venture UG, ursprünglich zur Deduplikations-Optimierung der UltiHash-Datenbank) übernommen; der Beitrag dieser Arbeit ist seine Übertragung auf cache-bewusste Suchstrukturen und seine design-pattern-getriebene, mess-orientierte Realisierung. Software-technisch ist der Apparat eine Komposition benannter Entwurfsmuster und zweier neuer Übersetzungszeit-Muster (Abbildung 2.11).

Die klassischen Muster sind dabei keine Zierde, sondern die Verkabelung des Apparats: *Strategy* kapselt

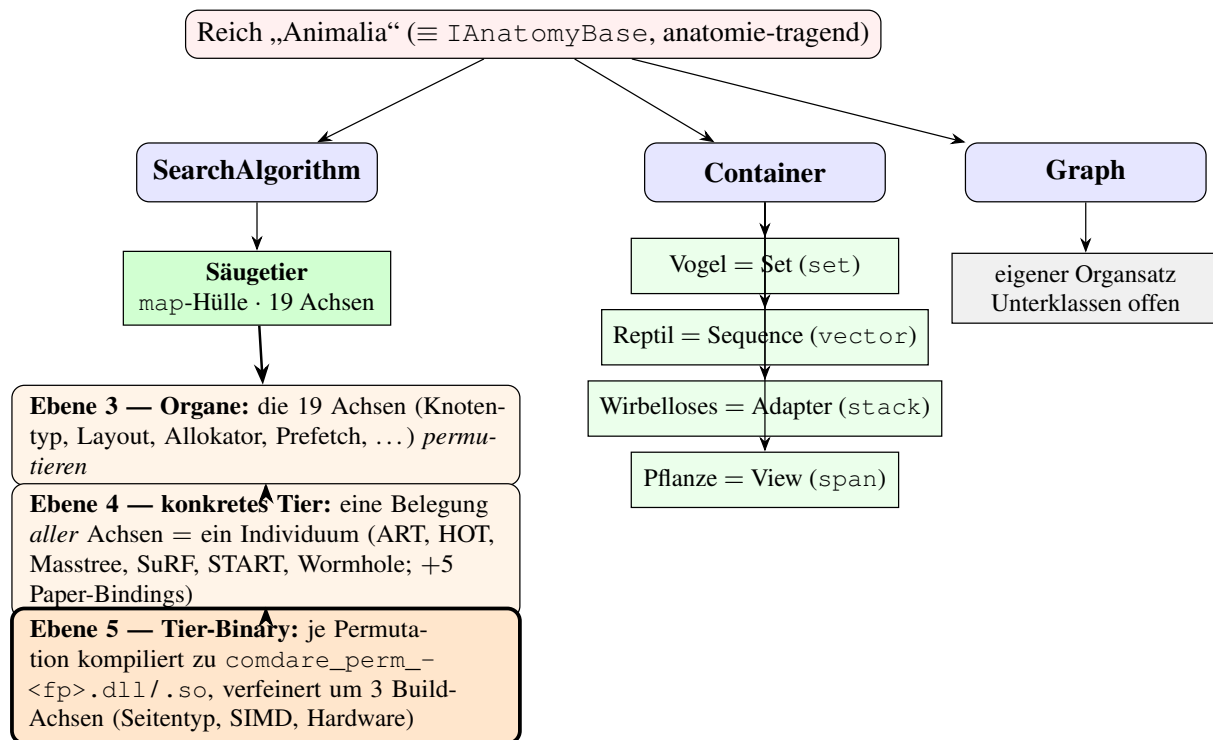


Abbildung 2.10: Die Gattungs-/Tier-Hierarchie über fünf Ebenen. `SearchAlgorithm`, `Container` und `Graph` sind die drei *Gattungen* (Ebene 1, Außen-Interface). Jede Gattung beherbergt ihre *eigenen* Tier-Unterklassen (Ebene 2): `SearchAlgorithm` ausschließlich das „Säugetier“ (map-Hülle, volle Neunzehn-Achsen-Anatomie), `Container` jede *nicht*-Säugetier-Unterklasse (`Vogel`/`Reptil`/`Wirbelloses`/`Pflanze` = `Set`/`Sequence`/`Adapter`/`View`), `Graph` noch offene. Der Säugetier-Ast ist hier bis zum Individuum ausgezogen: die neunzehn Achsen-Organe (Ebene 3) permutieren zu einer konkreten Achsen-Komposition = einem konkreten Tier (Ebene 4, z. B. ART, HOT, Masstree), das je Permutation als eigenständige Tier-Binary kompiliert wird (Ebene 5, zusätzlich identitäts-verfeinert durch drei Build-Achsen). *Jedes Tier-Binary ist damit ein eigenes Tier*; die Hierarchie reicht von der Gattung bis zur einzelnen kompilierten Permutation. Vgl. das verdichtete Modell in Abbildung 2.9.

jede der neunzehn Achsen als austauschbaren Algorithmus, *Abstract Factory*, *Composite* und *Iterator* bilden und durchlaufen den Permutations-Baum, *Adapter* bindet Fremd-Code hinter der stabilen Engine-Schnittstelle an, und *Observer* sowie *Decorator* (nicht-intrusive Mess-Hüllen) erheben die Messwerte, während ein *Memento* (Copy-on-Write) den Zustand zwischen Warmlauf und Messung sichert [GHJV94]. Den statischen Polymorphismus liefert das *Curiously Recurring Template Pattern* (CRTP) samt *Template-Method*-Concept-Prüfungen; concept-geprüftes `if constexpr(resolve_baustein)` wählt je Achse zwischen Prüfling- und Cache-Engine-Standard, und *Policy-based Design* fasst die Komposition als Tupel von Achsen-Policies [Ale01]. C++23-Module strukturieren dabei die Übersetzungseinheiten, garantieren für sich genommen aber noch keine portable ABI-Stabilität [Xu24] — diese stellt erst die C-stabile Modulgrenze her. Über die etablierten Muster hinaus benennt die Arbeit zwei Konstruktionen, für die eine Web-Recherche keine etablierte Bezeichnung fand: die *lazy-spärliche kartesische Materialisierung*, die den Produktraum von über 10^{14} Achsen-Kombinationen nur arithmetisch zählt und je Pfad mixed-radix einzeln on-demand erzeugt, statt den Compiler mit dem vollen Typ-Baum zu sprengen; und das *typgetriebene Quelltext-Emittieren*, das aus einer compile-zeitlichen Achsen-Komposition

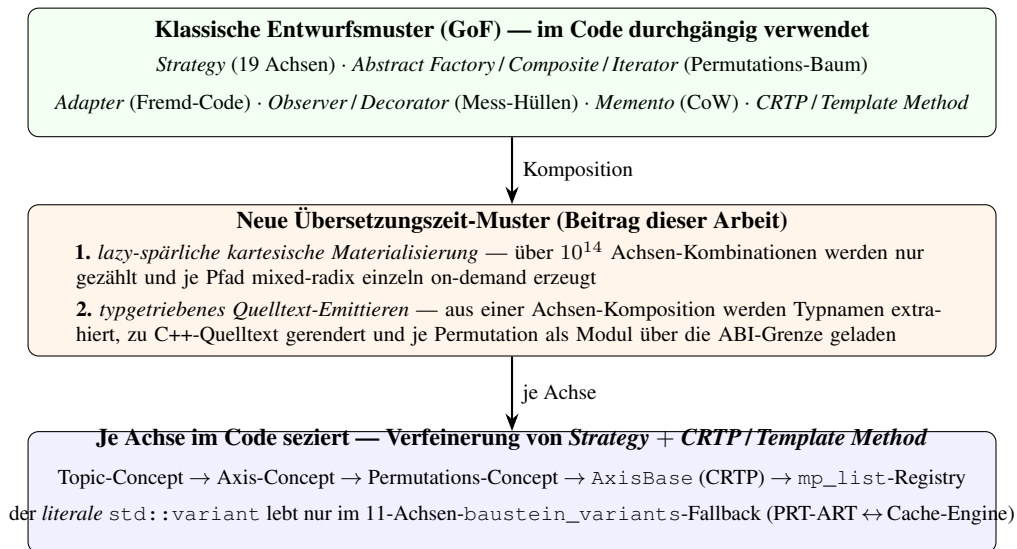


Abbildung 2.11: Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → mp_list-Registry) als Verfeinerung von *Strategy/CRTP* — wobei der *literale* `std::variant` nicht hier, sondern nur im elf-achsigen `baustein_variants`-Fallback-Modell auftritt. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Abschnitt 2.3).

voll-qualifizierte Typnamen extrahiert, zu C++-Quelltext rendert und je Permutation als eigenständiges Modul über die ABI-Grenze lädt — so wird der Prüfling PRT-ART per Metaprogrammierung in Matrix-Schichten gegen die Cache-Engine kompiliert. Diese eine Architektur wird über eine ABI-stabile *Hülle* nach außen sichtbar. Das Lebewesen `SearchAlgorithm` und seine ABI-Laufzeit-Sicht — die „SearchEngine“ — sind nur zwei Sichten desselben Gegenstands über die Modulgrenze, kein paralleles Konstrukt: Die Schichtung `CacheEngine` → `ExecutionEngine` → `SearchEngine` trennt die Rollen — die `CacheEngine` als Achsen-Bausteine-Bibliothek, die `ExecutionEngine` als Wurzel der Ausführungs- und Mess-Werkzeuge (OS-Primitiven; im Experiment-Modus der Mess-Aggregator über genau eine Beobachter-Schnittstelle) und die `SearchEngine` als die über genau diese Schnittstelle vermessene Such-API-Sicht (Abbildung 2.12). Achsenlose *Viren* stehen als Geschwister unter derselben Wurzel, tragen aber keine Anatomie.

Konkret ist die Modulgrenze eine C-stabile Schnittstelle, die je Permutation genau ein Lebewesen exportiert; ihre variadische Hybrid-API spezialisiert sich nach Parameterzahl, und komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert (Abbildung 2.13). Wie diese und die übrigen Code-Interfaces im Detail zusammenhängen, fasst die UML-Landkarte in Abbildung 2.15 zusammen.

Damit strukturell verschiedene Suchalgorithmen *fair* verglichen werden, bildet die Arbeit sie alle auf eine einzige, gut verstandene Schnittstelle ab: den C++-Vertrag der geordneten Map `std::map<Key, Value>` mit Punktanfrage (`find`), Einfügen/Aktualisieren (`insert`, `emplace`), Löschen (`erase`) und ordnungsbasierter Navigation (`lower_bound`/`upper_bound`, Bereichs-Iteration) [ISO24]. Die Achsen beschreiben das *innere* Verhalten hinter dieser festen Schnittstelle,

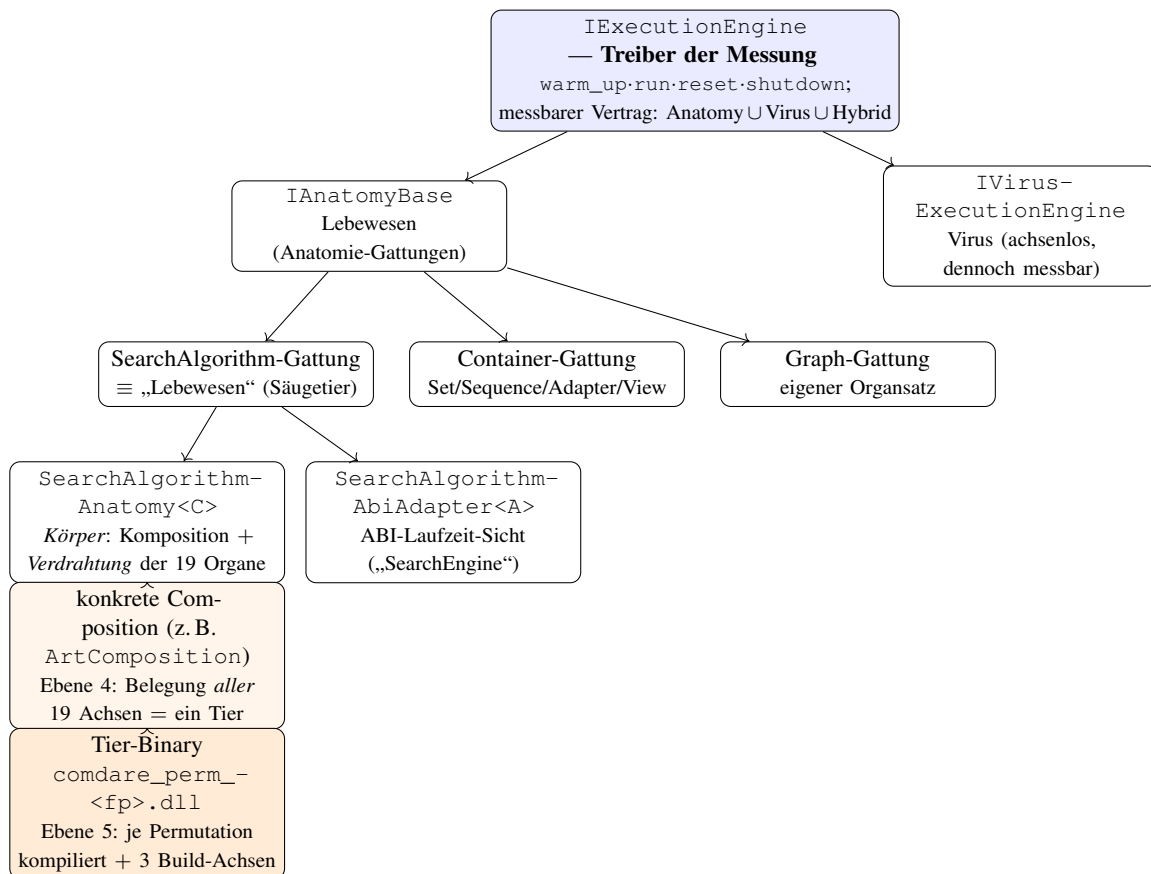


Abbildung 2.12: Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht IExecutionEngine als *Treiber der Messung* — der universelle messbare Vertrag (warm_up/run/reset/shutdown) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Gattungen* (SearchAlgorithm, Container, Graph) und die achsenlosen *Viren*. SearchAlgorithm ist das fokale Lebewesen (Säugetier); seine Anatomie ist sein *Körper* — die feste Komposition und Verdrahtung der 19 Organe —, und „SearchEngine“ nur dessen ABI-Laufzeit-Sicht. Der Säugetier-Ast ist bis zum Individuum ausgezogen: eine konkrete Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5). Container (schlüsselloser Speicher) und Graph tragen eigene Organsätze.

```

// C-stabile Modulgrenze: exportiert genau EIN Lebewesen je Permutation
extern "C" IAnatomyBase* comdare_create_anatomy();

// Variadische Hybrid-API je Parameterzahl (Schlüssel K, Werte V):
// 1 Parameter -> std::vector<V>
// 2 Parameter -> std::map<K, V>
// N > 2       -> std::map<K, std::tuple<V1, ..., VN>>
template<class... Ts> class SearchEngine; // ABI-Laufzeit-Sicht des Lebewesens

```

Abbildung 2.13: Die ABI-stabile C-Modulgrenze und die variadische Hybrid-API. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert.

sodass der Vergleich zweier konformer `std::map`-Implementierungen der zentrale Mess-Akt wird; parallel dient die Sequenz-Hülle `std::vector` (indizierter Zugriff, Größenänderung, zusammenhängender Speicher) als Container-Vergleichsfall, um zu prüfen, ob ein Achsen-Komposit auch die Container- und nicht nur die Suchsemantik trägt. Hinter der festen Außen-Schnittstelle unterscheidet das System mehrere *Ebenen der Dynamik*, die festlegen, *wann* eine Achsen-Entscheidung fällt: die oberste Trennung Übersetzungszeit gegen Laufzeit; der Lebewesen-Typ, der nur eine Teilmenge der Achsen belegt; die zur Übersetzungszeit in eine Binary einkompilierten Achsenalgorithmen (statische Rekombination); die dynamischen Sub-Achsen, deren Laufzeit-Parameter (Prefetch-Distanz, Stapelgröße, Inline-Schwelle, Pool-Budget) ohne Neuübersetzung variiert werden; und die statischen Sub-Achsen als übersetzungszeitliche Auswahl unter Teil-Algorithmen unterschiedlicher Komplexität. Gegenüber der statischen Entwurfsraum-Aufzählung des *Data Calculator* [IZH⁺18] und dem klassischen *Policy-based Design* [Ale01] führt diese Taxonomie compile- und laufzeitvariable Achsen-Ebenen ausdrücklich gemeinsam.

Diese Bausteine sind nur so aussagekräftig wie ihre Vermessung. Wissenschaftliches Messen unterliegt etablierten Gütekriterien [HB15]: *Reproduzierbarkeit* (feste Seeds, deterministische Reihenfolge, dokumentierte Konfiguration), *Validität* (intern kausal isolierte Effekte, Konstrukt- und externe Übertragbarkeit), *Fairness* (identische Schlüssel, Workloads und Warmup; fehlende Fähigkeiten als N/A statt verdeckt emuliert) und die *Streuung* als Teil der Aussage (Perzentile über HDR-Histogramme statt arithmetischer Mittel, mit Signifikanzprüfung). Auf diesen Kriterien bauen die verwendeten Mess-Muster auf: eine *Zwei-Phasen-Operationsschleife* trennt einen verworfenen Warmlauf von der eigentlichen Messung; die Erhebung erfolgt *hybrid* — isolierte Achsen-Algorithmen direkt gegeneinander, ein zusammengesetztes Lebewesen zentral über eine ABI-stabile Per-Achsen-Beobachter-Schnittstelle —; und ein *Konformitäts-Gatter* prüft vor jeder Messung den `std::map`-Vertrag, damit nur Vergleichbares verglichen wird. Diese Konzepte bilden die *Definitions-Klassen* des Messens; ihre technischen *Instanzen* — Hardware-Performance-Zähler, OTF2-/VAMPIR-Profilng, die TU-Dresden-Hardware-Messmethodik — sowie das *Warum* ihrer Wahl führt das Mess-System in Kapitel 3 aus.

Was diese Mess-Muster konkret erheben, sind drei gegenläufige Komponenten — *Latenz*, *Durchsatz* und *Speicherverbrauch* —, gegen die sich jede Achsen-Permutation als Trade-off-Profil aufspannen lässt: dieselbe Permutation, die in Abbildung 2.2 ein Punkt im Entwurfsraum ist, erscheint hier als Punkt im Inneren des Trade-off-Dreiecks (Abbildung 2.14). Eine vierte, für cache-bewusste Suchstrukturen eigentlich zentrale Mess-Komponente — das cache-bewusste Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (die PMC-Erfassung ist standardmäßig abgeschaltet) und bleibt daher als ausstehende vierte Mess-Komponente markiert.

Damit sind die Software-Mittel beisammen: die Achsen als Organe, die *eine* Architektur mit ihrer messbaren Wurzel und die wissenschaftlichen Mess-Muster. Kapitel 3 setzt genau diese Bausteine zum Mess-System zusammen und demonstriert es am Prüfling PRT-ART.

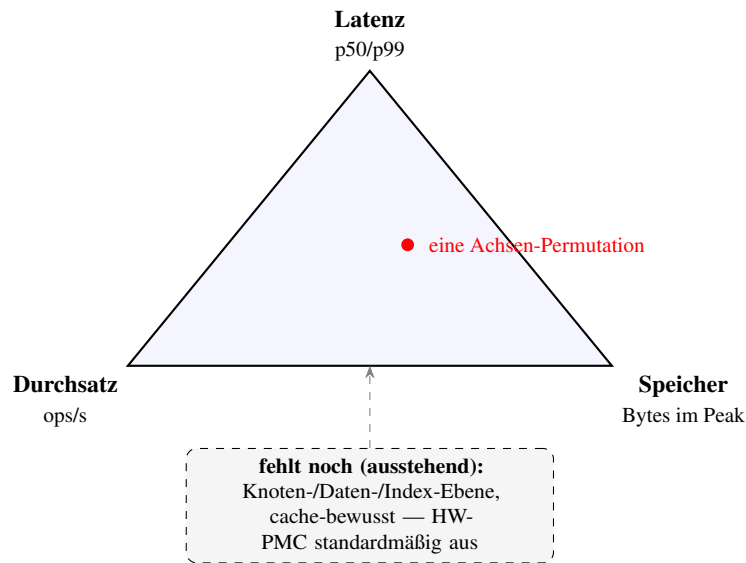


Abbildung 2.14: Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 2.2, wo dieselbe Permutation ein Punkt ist). Die für cache-bewusste Suchstrukturen eigentlich zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (gestrichelt).

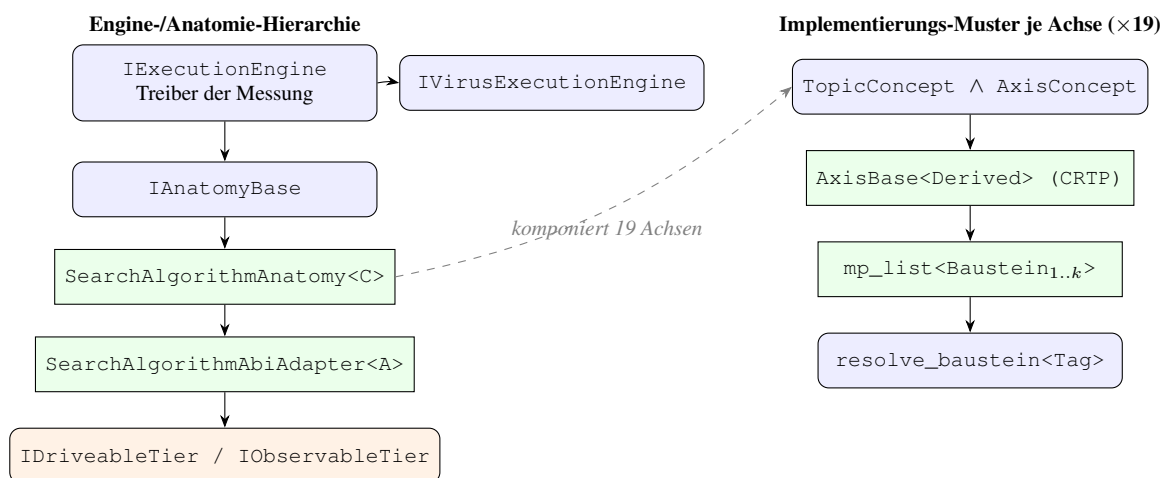


Abbildung 2.15: UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel `IExecutionEngine` als *Treiber der Messung* (Anatomy $\cup$ Virus $\cup$ Hybrid) über `IAnatomyBase` und der achsenlosen `IVirusExecutionEngine`, darunter der *Körper* `SearchAlgorithmAnatomy<C>` (Komposition und Verdrahtung der neunzehn Achsen) und seine ABI-Laufzeit-Sicht `SearchAlgorithmAbiAdapter<A>` („SearchEngine“), die über die Treib-/Beobachter-Verträge `IDriveableTier`/`IObservableTier` gemessen wird; rechts das je Achse wiederholte Implementierungs-Muster — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die `mp_list`-Baustein-Familie und die Übersetzungszeit-Auflösung `resolve_baustein`, die je Achse zwischen Prüfling-Slot und Cache-Engine-Standard wählt.

3 Mess-System mit PRT-ART als Demonstration

Auf den Grundlagen aus Kapitel 2 aufbauend führt dieses Kapitel die eigentlichen Belange der Aufgabe ein und entwickelt das Mess-System als deren *Lösung* — demonstriert am Prüfling PRT-ART.

3.1 Belange der Aufgabenstellung

Die Aufgabe verlangt, cache-bewusste Suchalgorithmen so zu zerlegen und zu vermessen, dass der Beitrag jeder Entwurfsentscheidung isoliert sichtbar wird. Wir machen diese Belange hier vollständig im Text verständlich — ohne Rückgriff auf das Aufgabenblatt — im Kontext der Grundlagen aus Kapitel 2. Im Kern verlangt die Aufgabe vier Schritte: cache-bewusste Suchalgorithmen in austauschbare Entwurfsbestandteile zu *zerlegen*, Stand-der-Technik-Entwürfe als explizite Konfigurationen zu *rekonstruieren*, ihr cache-line-bewusstes Verhalten je Bestandteil wie für den ganzen Algorithmus auf gemeinsamer Basis zu *messen* und daraus den besten Gesamtalgorithmus abzuleiten und systemoptimiert *auszuliefern*. Der cache-aware-Fokus ist dabei keine Randbedingung, sondern die Leitgröße: Nicht die asymptotische Komplexität, sondern das Cache-Line-Verhalten unter realer Last entscheidet über die Leistung (Kapitel 2), und genau dieses Verhalten soll je Achse isoliert messbar werden.

Der Stand der Technik (Kapitel 2) legt dabei die zentrale Lücke offen: Jede der untersuchten Arbeiten optimiert nur wenige Achsen isoliert, und im untersuchten Korpus belegt *kein einziges Verfahren alle neunzehn Achsen gemeinsam*. Manche Achsen sind ausschließlich mit Baseline- oder Lehrbuch-Varianten besetzt (das Mapping zwischen Algorithmus- und Cache-Traversierung), andere treten nur als *Mess-Ziel* auf (der dTLB), die räumliche Such-Klasse fehlt ganz, und Achsen wie Ein-/Ausgabe, Migration, Filter und Pufferung sind zwar je einzeln, aber nie gemeinsam mit den Such-Achsen in einem Entwurf belegt. Hinzu kommt eine methodische Lücke: Ergebnisse werden in nicht vergleichbaren Settings berichtet, und die häufige Wahl szenariospezifischer, oft günstig zugeschnittener Lastprofile erschwert einen fairen Quervergleich. Es fehlt eine bias-reduzierte Vermessung *aller* Lastprofile gegen *alle* Verfahren auf einer gemeinsamen Basis. Was fehlt — und was diese Arbeit beiträgt — ist daher die systematische, orthogonale Achsen-Zerlegung dieser Entwürfe samt einer *permutierten Mess-Architektur*, die sie als explizite Konfigurationen rekonstruiert und auf gemeinsamer Basis vermisst; denn cache-aware Algorithmen sind ohne eine Aufteilung der das Caching maßgeblich bestimmenden Eigenschaften nicht differenzierbar. Welche der katalogisierten Instanzen das Framework übernimmt und welche es als überholt ersetzt, entwickelt der folgende Abschnitt 3.2 dialektisch (These–Antithese–Synthese); das Mess-System selbst (Abschnitt 3.3) ist die Lösung, die diese Lücke schließt.

3.2 Stand der Technik als Achsen-Instanzen

Die bekannten Verfahren erscheinen nun systematisch als Achsen-Kompositionen; die vollständigen Kataloge liegen hier. Diese Aneignung verläuft *dialektisch* (These–Antithese–Synthese): je Achse wird

ein fremdes Organ *übernommen* (These) oder eine überholte Annahme *ersetzt* (Antithese), woraus der eigene Beitrag als Synthese hervorgeht. Leitend ist der „dritte Weg“ aus Abschnitt 2.2 — gemessenes Lokaliätswissen statt fest verdrahteter oder ignorierte Cache-Bewusstheit —, der jede Achse als orthogonales, permutierbares und einzeln messbares Organ explizit macht. Bei vielen Achsen ist die Antithese keine überholte Fremd-Instanz, sondern die implizite Annahme, die Entscheidung gar nicht als wählbar zu betrachten; die Synthese besteht dann darin, sie überhaupt als Achse explizit zu machen (in der Ersetzen-Spalte als „—“ markiert). Tabelle 3.1 fasst diese Aneignung je Kompositions-Achse zusammen.

Tabelle 3.1: Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in Achsen-Reihenfolge T0–T18; Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware-/Messung/Engineering)

Achse		übernommen (These)	ersetzt (Antithese)	Synthese-Kern
T0	Search-Algorithm (A)	ART, HOT, START, SuRF, Wormhole; k-äre/Interpolations-Suche	Monolith „Struktur ist Verfahren“	Such-Paradigma als permutierbare Achse
T1	Cache-Traversal (B)	Cache-Line-Stride (Chen), cache-oblivious (Bender), HBM (Schmidt)	implizite Lehrbuch-Linearsuche	Algorithmus- und Cache-Sicht getrennt
T2	Mapping (B)	lineare 1:1-Abbildung; Hash-Redirect, Permutations-Index	—	Direct- vs. Pool-Relative-Mapping explizit
T3	Path-Compression (A)	byte-weise (ART), Patricia (HOT), Macro-Node (CoCo), Multi-Byte (START)	unkomprimierter Rohpfad	Kompressions-Granularität wählbar
T4	Node-Type (A)	ART Node4/16/48/256	feste Knotengröße	adaptive Knotenklasse + Storage-Delegation
T5	Memory-Layout (B)	AoS-aligned, SoA, LOUDS-bitgepackt, CSS/CSB ⁺ , Ao-SoA	—	vier Layout-Dimensionen permutierbar
T6	Allocator (C)	A01–A23 (Hoard, mimalloc, jemalloc, TCMalloc, ...) + RCU/Hazard	algorithmus-fest verdrahteter Allokator	Allokation als gemeinsames Interface-Organ (sieben Sub-Familien)
T7	Prefetch (D)	Distance-Estimator (ART), Hardware (Wormhole), Stride (Chen), adaptiv	Prefetch nicht als Achse isoliert	Path-Oriented Bundle-Prefetch (eigen)
T8	Concurrency (E)	OLC (ART), RCU, Hazard-Pointer; Lock-/Wait-Free	grobes Sperren als einzige Annahme	Pattern $\perp$ Reclamation als zwei Sub-Achsen
T9	Serialization (F)	Raw-Binary, Varint (ART), succinct (LOUDS/FST), LZ77	—	Kodierung/Kompression wählbar
T10	Telemetry (F)	Density-Tracker, Insert-Zähler, HdrHistogram, Pfad-Zähler	All-Node-Zähler (Cache-Line-Ping-Pong)	kohärenz-schonende Leaf-Only-Telemetry
T11	Value-Handle (F)	Inline, External-Pool, Immutable-Shared (RCU), Versioned	implizit fest eingebetteter Wert	Platzierung, Ownership und Versionierung (+ Chain-Ref)
T12	ISA (F)	Vendor-Spec x86-64/AArch64/Power/RISC-V (+ SIMD)	—	Ziel-ISA als Compile-Time-Achse

Achse	übernommen	ersetzt	Synthese-Kern
T13 Index-Organization (B)	Clustered, IOT, Non-Clustered, Heap	—	Storage-Order als eigene Achse
T14 I/O-Dispatch (F)	Buffered, mmap, Direct-IO	mmap als Anti-Pattern	Persistenz von der Suchstruktur entkoppelt
T15 Migration (F)	Hot/Cold, Multi-Tier NVM, LeCaR-ML	Tiering nie mit Such-Achsen kombiniert	Migrationslogik als messbare Achse
T16 Filter (F)	Bloom, Cuckoo, SuRF-Range, Xor	Filter nie mit Such-Achsen integriert	Filter als orthogonales Organ
T17 Queuing-q1 (F)	Disruptor-Ring, Delta-Chain, CoW, Epoch-RCU, SPSC/MPMC	Operations-Puffer nie als Achse	Puffer mit Zugriffsmuster-Klassen
T18 Queuing-q2 (F)	Eager/Lazy, Watermark (LSM), Timed	Flush-Policy implizit	adaptive EWMA-Flush-Policy (eigen)

Der systematische Kern ist die achsenweise *Sezierung*: Jeder Gesamt-Algorithmus wird in seine Organe zerlegt, und je Achse werden die beitragenden Organ-Algorithmen aufgeführt, mit kurzem Verweis auf das Herkunfts-Lebewesen. Das Framework führt *neunzehn Kompositions-Achsen* (T0–T18) sowie drei *Build-Achsen* (Seitentyp, SIMD-Erweiterung, Hardware-Profil); viele Achsen sind in Sub-Achsen verfeinert (etwa SA1–SA4 oder AA1–AA7) und liefern anderen Achsen verteilte Interfaces für deren Detail-Operationen. Tabelle 3.2 gibt den Überblick, welche Veröffentlichungen je Achse ein Organ beitragen; die thematischen Herkunfts-Cluster der Vorarbeiten erscheinen dabei *aufgelöst* in ihren Organ-Beiträgen.

Die *Such-Achse* (T0) trägt die meisten Organe — als Original-Bindungen ART [LKN13], HOT [BZP⁺18], START [FJKN20], SuRF [ZLL⁺18] und Wormhole [WNJ19], dazu generische Such-Methoden wie k-äre Suche [SGL09], Interpolationssuche [PIA78], Eytzinger-Layout-Suche [KM17a], Skip-Liste [Pug90], B-Baum [BM72] und Open-Addressing-Hashing [Knu98]. Die Traversierungs-Achsen trennen Algorithmus- und Cache-Sicht: T1 reicht von der Cache-Line-Wanderung über Stride-Prefetch [CGM01] und HBM-Nähe [SKK⁺25] bis zur cache-oblivious-Rekursion [FLPR99], während T2 (Mapping) über lineare 1:1-Abbildung, Hash-Redirect [WNJ19] und Permutations-Index [MKM12] eine dünn besetzte Achse bleibt. Die *Pfadkompression* (T3) instanziiert die byte-weise Variante [LKN13], den Patricia-Split [Mor68, BZP⁺18], die Macro-Node [BFTV24] und das Multi-Byte-Rewiring [FJKN20], und die *Knotentyp-Achse* (T4) bündelt die adaptiven Layouts von Node4/16/48/256 [LKN13] über HOT-Compound [BZP⁺18] und die CSS-/CSB⁺-/Hankins-Knoten bis zur PRT-ART-Familie.

Die *Speicher-Layout-Achse* (T5) reicht vom cache-line-ausgerichteten AoS über zeigerfreie Anordnung [RR99], Geschwister-Cluster [RR00] und eingebetteten Baum [SSL⁺21] bis zum bit-gepackten LOUDS [Jac89]. Die *Allokator-Achse* (T6) ist mit dem Korpus A01–A23 am dichtesten besetzt und in sieben Sub-Achsen verfeinert — von der Free-List-Topologie über Thread-Lokalität/NUMA-Affinität [UHL⁺17] und die Allokationsstrategie bis zur Reklamation (Epoch, RCU [McK01], Hazard-Pointer [Mic04a], QSBR); der Code stellt 25 Allokator-Implementierungen zum 23-Arbeiten-Korpus bereit (einzelne Arbeiten steuern mehrere Varianten bei), von denen Tabelle 3.3 die zehn lauffähigen Profile mit Lizenz und Workload-Affinität listet. Die *Wert-Achse* (T11) unterscheidet Inline-Wert [LKN13], externen Pool [WNJ19], versionierten Zeiger [MKM12] und immutable

Tabelle 3.2: Achsen-Sezierung der Suchverfahren: neunzehn Kompositions-Achsen (T0–T18) und drei Build-Achsen; je Achse die ein Organ beitragenden Quellen.

Achse	Sub-Achsen	Beitragende Quellen
T0 search_algo	SA1–SA4	P01,P02,P05,P07,P10; k-ary (TUD 2009), Interpol., Eytzinger, Skip-List, B-Baum, Hash
T1 cache_traversal	CT1–CT2	Cache-Line-Default; P17,P21,P32
T2 mapping	MP1–MP2	P03,P07; CE-Baseline
T3 path_compression	PC1–PC3	P01,P02,P04,P05,P09
T4 node_type	NT1–NT3	P01,P02,P03,P04,P05,P06,P11,P12,P13
T5 memory_layout	HM1–HM4	P06,P09,P11,P12,P32; Default
T6 allocator	AA1–AA7	A01–A23; P29,P30
T7 prefetch	PF1–PF3	P07,P21,P23,P25,P26,P27
T8 concurrency	CC1–CC2	P08,P29,P30; Dijkstra,Courtois,Herlihy,Michael/Scott
T9 serialization	SR1–SR3	P01,P09,P10; LZ77
T10 telemetry	TM1–TM3	P01,P02,P16,P26,P28; HdrHistogram (Gil Tene)
T11 value_handle	VH1–VH3	P01,P03,P07,P29
T12 isa	IS1–IS3	x86-64/AArch64/Power/RISC-V (Vendor-Spec)
T13 index_organization	io1–io3	Bayer/McCreight,Comer,Oracle
T14 io_dispatch	IO1–IO3	Stonebraker,Crotty; Baselines
T15 migration_policy	MG1–MG3	Anti-Caching,NVM-Tiering,LeCaR; None
T16 filter	FT1–FT3	Bloom,Cuckoo,P10,Xor
T17 queuing_q1	QS1–QS6	Disruptor, Bw-Tree, Driscoll, P29, Pugh, LSM, Lamport, Michael/Scott
T18 queuing_q2	FS1–FS4	Eager/Lazy/Watermark/Kafka/RocksDB
Build page_type	PG1–PG3	P01,P02,P04,P05,P06,P10,P11,P12,P13,P20; PRT-ART (2026)
Build simd_extension	SE1–SE3	SSE2/AVX2/AVX-512/NEON/SVE2/RVV; GPU: CUDA
Build general_hardware	HW1–HW4	HW-Strategie je Paper (Tab. 3.4)

Shared-Ref [McK01], die *Serialisierungs-Achse* (T9) Raw-Binary, succinct LOUDS/FST und Varint [LKN13].

Tabelle 3.3: Allokator-Profil mit Lizenz und Workload-Affinität

Profil	Familie	Lizenz	expected_workload
hoard	A01	Apache-2.0	YCSB_A
michael_lockfree	A03	BSD-3	YCSB_B
mimalloc	A04	MIT	YCSB_A
jemalloc	A05	BSD-2	YCSB_B
tcmalloc	A06	BSD-3	YCSB_C
snmalloc	A07	MIT	YCSB_A
scalloc	A08	BSD-3	YCSB_A
rpmalloc	A10	Public Domain	YCSB_C
lrmalloc	A11	BSD-3	YCSB_B
dlmalloc	A20	CC0	YCSB_C

expected_workload = YCSB-Workload-Klasse (A/B/C).

Auf der hardware-nahen Seite reicht die *Prefetch-Achse* (T7) von keinem Prefetch über software-festes [CGM01], adaptiv-distanziertes [Kha11] und fill-buffer-bewusstes [MWR25] bis zu pfadorientiertem [ZSZ⁺24] und hierarchischem Bündel-Prefetch [ZGH⁺25]; die explizite Hardware-Prefetch-Instruktion stammt aus Wormhole [WNJ19]. Die *ISA-Achse* (T12) und die Build-SIMD-Erweiterung führen die Vendor-Spezifikationen (x86-64, AArch64, Power, RISC-V mit SSE2/AVX2/AVX-512, NEON/S-

VE2, RVV). Zwei übergreifende Strategie-Achsen halten fest, was ein Algorithmus aktiv *nutzt* bzw. wie er Arbeit *verteilt* — die Hardware-Strategie und die Scheduling-Strategie —, die Tabelle 3.4 je Paper zuordnet. Die *Telemetrie-Achse* (T10) instanziiert Density-Tracker [LKN13], Insert-Zähler [BZP⁺18], HdrHistogram und Pfad-Lese-Zähler [ZSZ⁺24]; die kohärenz-schonenden Leaf-Only-Varianten gehen auf Kuehn [Kue23] zurück, während der Zähler in allen Knoten als *Anti-Pattern* (Cache-Line-Ping-Pong) gilt.

Tabelle 3.4: In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profile je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; der Vollkatalog folgt im Anhang).

Paper	Hardware-Strategie	Scheduling-Strategie
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (Bit-Vektor), L1-aware, CAS	thread-per-core
P03 Masstree	scalar, L1-aware, CAS	thread-per-core + work-stealing
P05 START	AVX2 (Multi-Byte-Span), L1-aware, CAS	thread-per-core
P20 B-Trees Are Back	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (Instr.-Bundle), all-cache, CAS	thread-per-core + Co-Routine
P28 Kuehn	PREFETCH (scalar Counter), L1-aware, CAS	thread-per-core + Sampling
P31 Ungethuem	AVX-512 (HBM), HBM-aware, NUMA-aware, CAS	hybrid (P-/E-Kerne)
P32 Schmidt	AVX-512 (SIMD-Stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)	thread-per-core + 2-SIMD-Worker-Limit + hybrid + Co-Routine + micro-batch

Die *Nebenläufigkeits-Achse* (T8) gliedert sich in Muster und Sperr-Modus: vom seriellen Nullpunkt über grobes Sperren [Dij65], Leser-/Schreiber-Sperren [CHP71] und optimistisches Lock-Coupling [LABN16] zu lock-freien [MS96], wait-freien [Her91] sowie RCU- [McK01] und Hazard-Pointer-Verfahren [Mic04a]. Die *I/O-Achse* (T14) instanziiert gepufferte [Sto81], direkte und memory-mapped Ein-/Ausgabe [CLP22], die *Migrations-Achse* (T15) Hot/Cold-Trennung und mehrstufige RAM→NVM→SSD-Migration, und die *Pufferungs-Achsen* (T17/T18) reichen vom Disruptor-Ringpuffer [TFB⁺11] über die Bw-Tree-Delta-Kette [LLS13], den Copy-on-Write-Puffer [DSST89] und den Tombstone-Puffer [OCGO96] bis zu lock-freien SPSC- [Lam77] und MPMC-Queues [MS96]. Die *Index-Organisations-Achse* (T13) unterscheidet geclusterte Anordnung [BM72], Heap-Ablage und nicht-geclusterten Sekundärindex [Com79], die *Filter-Achse* (T16) den Bloom- [Blo70], Cuckoo- [FAKM14], SuRF- [ZLL⁺18] und Xor-Filter [GL20]. Auch die *Lasten* erscheinen als Instanzen. Die in der Literatur verwendeten Workloads entstammen *mehreren*, untereinander nicht vergleichbaren Benchmark-Frameworks; Tabelle 3.5 ordnet sie den nutzenden Veröffentlichungen zu. Am verbreitetsten ist der Yahoo! Cloud Serving Benchmark (YCSB) [CST⁺10]; daneben treten der Index-Benchmark SOSD [KMR⁺19], die TPC-Familie [Tra24], die CPU-Suite SPEC CPU [Sta17], die Scale-out-Suite CloudSuite [FAK⁺12] sowie — für den Allokator-Korpus — mimalloc-bench [Lc21] und

die Klassiker Larson, threadtest und shbench auf. Da diese Frameworks unvergleichbare Lasten erzeugen, abstrahiert die Arbeit sie zu einem gemeinsamen Satz von Lastprofilen.

Tabelle 3.5: Benchmark-Frameworks und nutzende Veröffentlichungen

Framework	Typ	Nutzende Veröffentlichungen
YCSB [CST ⁺ 10]	KV-Workload	P02, P03, P07, P20, P26
SOSD [KMR ⁺ 19]	Index-Benchmark	P05
TPC-C / TPC-DS [Tra24]	OLTP / analytisch	P19
Real-String-Korpora	String-Datensätze	P04, P06, P10
RocksDB-Integration	Real-System	P10 (SuRF)
SPEC CPU [Sta17]	CPU-Suite	P23, P24, P27, A18
CloudSuite [FAK ⁺ 12]	Scale-out-Server	P24, P27
IBM DB2	Produktions-DBMS	P22
mimalloc-bench [Lc21]	Allokator-Suite	A04, A07, A09, A10, A18
Larson / threadtest / shbench	Allokator-Klassiker	A01, A03, P08, P30
LADDIS	Kernel-Allokator	A02
AggSum / Memory-Stream	Bandbreite	P32
Custom-Microbenchmarks	per Paper (Zipf/Uniform)	P01, P04, P06, P11–P14, P21, P28

Aus dieser Analyse destilliert die Arbeit *vierzehn* distinkte Lastprofile (LP01–LP14, Tabelle 3.6): Sie geben den heterogenen Lasten ein gemeinsames, uint64-basiertes Operations-Modell — der Op-Mix nennt die Anteile von Einfügen, Lookup, Löschen, Scan und Read-Modify-Write über definierten Schlüssel-Verteilungen — und sind je aus konkreten Veröffentlichungen abgeleitet.

Tabelle 3.6: Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)

LP	Kennung	Op-Mix	Verteilung	neg%	Abgeleitet aus
LP01	bulk-insert-dense	100/0/0/0/0	uniform-seq	0	P01, P02, P03, P05, P12
LP02	bulk-insert-sparse	100/0/0/0/0	uniform-rand	0	P01,P02,P08
LP03	insert-value-length-sweep	100/0/0/0/0	uniform-seq	0	P01,P08
LP04	static-read-positive	0/100/0/0/0	uniform	0	P02, P05, P11, P12, P16, P25
LP05	static-read-zipfian	0/100/0/0/0	zipfian(.99)	0	P10,P20,P25,P28
LP06	static-read-negsweep	0/100/0/0/0	uniform	0–100	P04,P06,P22
LP07	static-read-string-corpus	0/100/0/0/0	real-corpus	50	P04,P06,P10
LP08	range-scan-heavy	0/0/0/100/0	uniform-seq	0	P01,P10,P12,P32
LP09	insert-lookup-5050	50/50/0/0/0	uniform	0	P03,P29
LP10	delete-heavy-post-build	50/0/50/0/0	uniform-rand	0	P01,P08
LP11	mixed-ycsb-oltp	25/50/0/0/25	zipfian(.99)	0	P10,P19,P20
LP12	concurrent-rmw-stress	25/9/36/9/9	uniform	0	P07,P29
LP13	concurrent-read-scaling	0/100/0/0/0	uniform	0	P07,P08,P25,P28
LP14	dynamic-realistic-trace	9/91/0/0/0	tpcc-nonuniform	0	P19,P20

Op-Mix = Anteile von Einfügen/Lookup/Löschen/Scan/RMW; neg% = Anteil negativer Lookups.

Für jeden SOTA-Algorithmus und jeden Allokator hält ein Profil-XML die Achsen-Belegung, eine Schlüssel-Wert-Signatur und einen optionalen `<expected_workload>`-Tag fest, der die Mess-Heuristik überschreibt (Tabelle 3.7). Drei der 33 Korpus-Arbeiten besitzen bewusst kein eigenständiges Profil, sondern gehen nur seziert ein: das optimistische Lock-Coupling von ART [LABN16] als Nebenläufigkeits-Organ, die succinct LOUDS-Repräsentation [Jac89] als Speicher-Layout-Organ (u. a. in SuRF [ZLL⁺18]) und das Profiling-Werkzeug VAMPIR [BH23] als Mess-Technologie; die übrigen 30

tragen je ein eigenes Profil-XML.

Tabelle 3.7: SOTA-Algorithmus-Profile mit Workload-Affinität

Profil	Paper-Ref.	expected_workload
art	P01 (Leis 2013)	YCSB_C
hot	P02 (Binna 2018)	YCSB_C
masstree	P03 (Mao 2012)	YCSB_A
coco_trie	P04 (Boffa 2024)	YCSB_C
start	P05 (Fent 2020)	YCSB_C
b2tree	P06 (Schmeisser 2022)	YCSB_E
wormhole	P07 (Wu 2019)	YCSB_A
surf	P10 (Zhang 2018)	YCSB_A
css_tree	P11 (Rao/Ross 1999)	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	YCSB_E
hankins	P13 (Hankins 2003)	YCSB_C
samuel	P14 (Samuel 2005)	YCSB_C
graefe	P15 (Graefe 2001)	YCSB_C
bender_treelayout	P16 (Bender 2002)	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	YCSB_E
btreesareback	P20 (Mueller 2025)	YCSB_E
chen_prefetch	P21 (Chen 2001)	YCSB_A
chen_fractal	P22 (Chen 2002)	YCSB_A
khan_adaptive	P23 (Khan 2010)	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	YCSB_A
mahling	P25 (Mahling 2025)	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	YCSB_A
zhang_asplos	P27 (Zhang 2025)	YCSB_A
kuehn	P28 (Kuehn 2023)	YCSB_C
rcu	P29 (McKenney 2001)	YCSB_B
hazard_pointers	P30 (Michael 2004)	YCSB_B
ungethuem	P31 (Ungethuem 2017)	YCSB_C
to_stride	P32 (Schmidt 2025)	YCSB_C

expected_workload = YCSB-Workload-Klasse (A/B/C/E/F).

Der Kern-Befund dieser Lastprofil-Analyse ist ein systematischer *Bias*: Jede Veröffentlichung wählt das Lastprofil, unter dem ihr eigener Entwurf am besten abschneidet. Statische, read-only-orientierte Tries und Filter rahmen „build-once, 100 % positive Lookups“ und meiden Aktualisierungen; komprimierte Verfahren machen den Negativ-Anfrage-Anteil zur Primärachse; zipfian-zentrierte Arbeiten begünstigen cache-bewusste Reordering-Layouts; und schreib- bzw. nebenläufigkeits-lastige Arbeiten treten umgekehrt nur dort an, wo statische Strukturen versagen. Vierzehn der Veröffentlichungen liefern überhaupt kein eigenes Index-Lastprofil. Erst wenn *alle* Lastprofile gegen *alle* Lebewesen-Binaries laufen — der Workload als dynamische Achse (Kapitel 5) —, lässt sich der „Sieger nur im selbstgewählten Heimprofil“ sichtbar machen und bias-reduzierte Vergleichbarkeit gewinnen.

Schließlich erscheinen auch die Güte-Kriterien wissenschaftlichen Messens (Abschnitt 2.3) als konkrete Technologien. *Hardware-Performance-Zähler* (PMC, über `perf`) [Mel10] sind nahezu universell und liefern Cache-Miss-Raten, dTLB-Misses, Branch-Misses und IPC/CPI; *Profiler* wie OTF2/VAMPIR [BH23] und VTune [Rei05] erheben trace-basierte Laufzeitprofile, *Simulatoren* wie gem5 [BBB⁺11] liefern hardware-genaue Schätzungen, wo reale Zähler fehlen, und das *HdrHistogramm* [Ten12] erhebt die Latenz-

Perzentile (p50/p95/p99). Auf der Erhebung baut die Auswertung auf: Statt arithmetischer Mittel werden Median und Perzentile berichtet, Unterschiede mit dem verteilungsfreien *Mann-Whitney-U-Test* [MW47] (mit Holm-Korrektur [Hol79]) auf Signifikanz geprüft und über das Effektmaß *Cliff's δ* [Cli93] quantifiziert, den Empfehlungen für wissenschaftliches Benchmarking [HB15] folgend. Eine eigene methodische Linie bildet die hardware-nahe Messmethodik der TU Dresden (Habich-Gruppe): Ungethüm et al. [UHL⁺17] systematisieren die Hardware-Optimierungen für Datenbank-Maschinen, Schmidt et al. [SKK⁺25] vermessen Stride- gegen Sequenz-Zugriffe auf HBM, und VAMPIR [BH23] profiliert nicht-funktionale Eigenschaften heterogener Speicher. Aus all diesen Technologien ergibt sich der Satz der Mess-Kategorien — Cache-Line-Auslastung, Cache-Misses, dTLB-Misses, Speicher-Fußabdruck, Branch-Misses, IPC/CPI, Latenz, Durchsatz und Energie —, deren zeit- und observer-basierte Teilmenge die eigene Apparatur bereits auflöst, während die zählerbasierten PMC-Kategorien an Hardware-Zähler-Zugriff gebunden bleiben; die vollständige Mess-Architektur entwickeln das folgende Mess-System (Abschnitt 3.3) und Kapitel 5.

3.3 Das Mess-System

Das Mess-System trennt vier Subsysteme und prüft in drei Stufen, die die drei Pflicht-Messreihen speisen (nicht eins-zu-eins, Tabelle 3.8); den Permutationsraum reduziert es gezielt. Vier formal getrennte Subsysteme mit unterschiedlichen Verantwortlichkeiten bilden das Modell: der `messung_driver` (der Auswertungs-Orchestrator, der je Pflicht-Messreihe A/B/C die Pipeline durchläuft), der `CacheEngineBuilder` (ein autonomes Plattform-Ausmess-System, das die Sieben-Phasen-Pipeline über seinen `ExperimentDriver` ausführt), die `CacheEngine` (die Achsen-Bausteine-Bibliothek mit den Standard-Implementierungen je Achse) und der Prüfling PRT-ART (der einige Achsen-Organe selbst stellt und für die übrigen Cache-Engine-Dienste konsumiert). Die `ExecutionEngine` ist dabei kein fünftes Subsystem, sondern die gemeinsame messbare Wurzel (das `IExecutionEngine-Interface`), über die diese vier verbunden sind. Definierend ist die *bidirektionale* Cache-Engine $\leftrightarrow$ Prüfling-Beziehung: Die Cache-Engine instantiiert den Prüfling je Permutations-Konfiguration, und der Prüfling ruft zur Laufzeit Cache-Engine-Dienste (Telemetrie, Prefetch, Heuristik) während seiner eigenen Lookups auf. Diese Symmetrie trägt die *Multi-Prüfling*-Fähigkeit, mit der Messreihe A den Prüfling gegen viele SOTA-Adapter in einer einzigen Pipeline vergleicht (Abbildung 3.1).

Der Builder orchestriert eine *Drei-Stufen-Prüfung*: **Stufe 1** (nur Cache-Engine) nutzt die Standard-Variante je Achse, **Stufe 2** (nur Prüfling) ersetzt die überschriebenen Achsen durch die Prüfling-Varianten und behält sonst die Standards, und **Stufe 3** (Full Join, nicht-redundant) kombiniert Standard- und alle Prüfling-Varianten. Jede Stufe kann bis zu tausende nicht-redundante Permutations-Binaries erzeugen, die zur Laufzeit als ABI-stabile C++23-Module geladen werden; der vorliegende Mess-Lauf materialisiert davon eine spärliche, gedeckelte Teilmenge (Abbildung 3.2).

Diese Stufen bilden *nicht* eins-zu-eins auf die drei Pflicht-Messreihen ab, sondern speisen sie: Die Standard-Binaries der Stufe 1 (samt der rekonstruierten SOTA-Profile) und die Prüfling-Binaries der Stufe 2 bilden *gemeinsam* die Vergleichs-Messreihe A; der nicht-redundante Full Join der Stufe 3 ist die Grundlage der Variations-Messreihe B; und die Regressions-Messreihe C ist an keine einzelne Stufe gebunden, sondern läuft build-übergreifend über aufeinanderfolgende Stände derselben Konfiguration

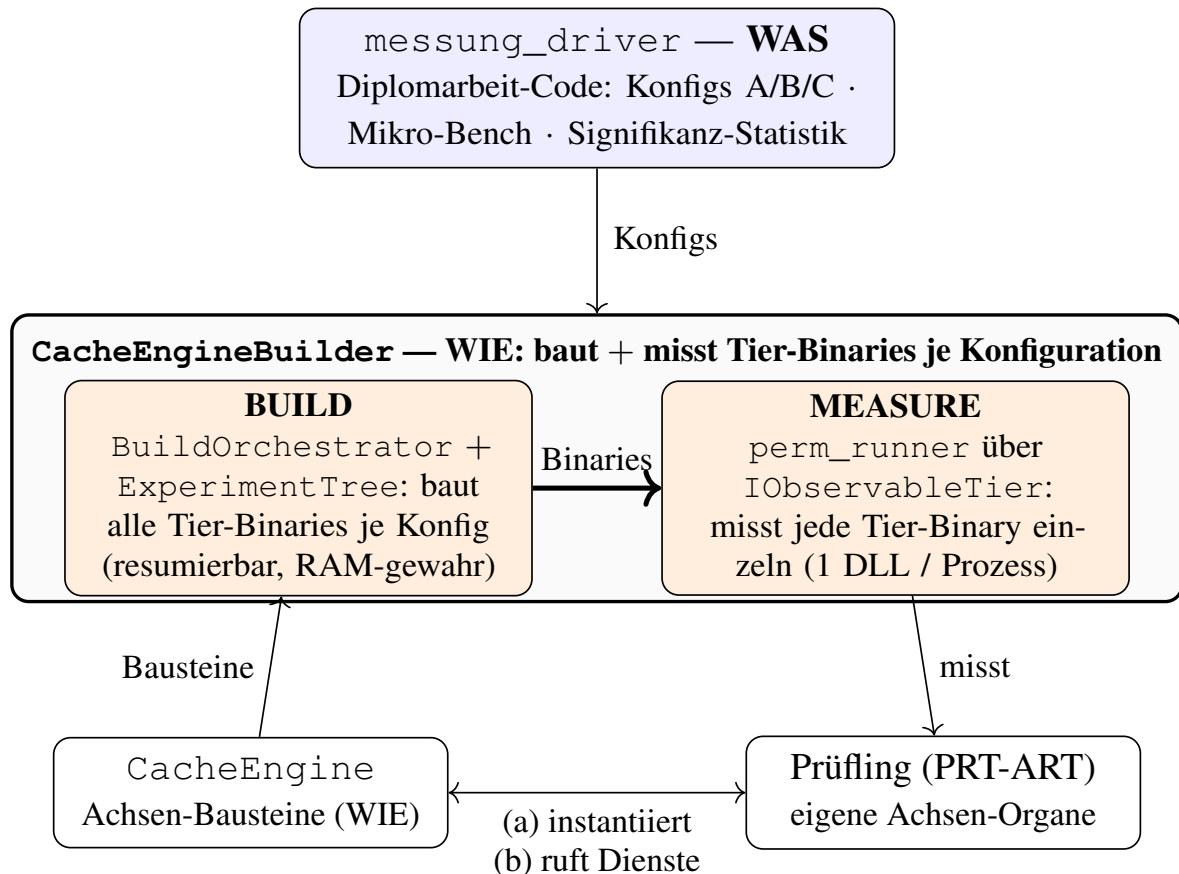


Abbildung 3.1: Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (WAS, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (WIE, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **BUILD** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **MEASURE** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) trägt die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel.

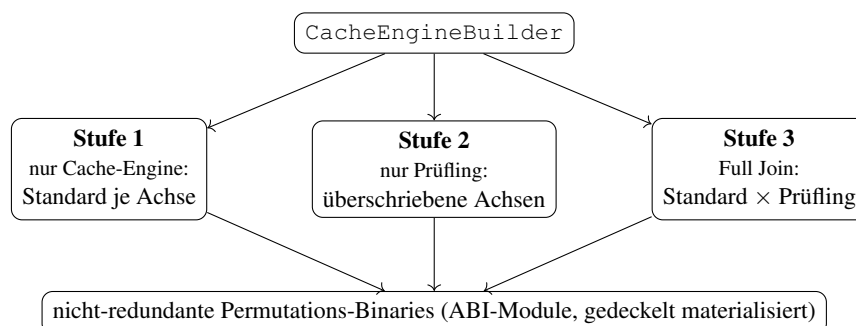


Abbildung 3.2: Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8.

(Tabelle 3.8).

Tabelle 3.8: Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen

Prüf-Stufe	Inhalt	speist Messreihe
Stufe 1 (nur Cache-Engine)	Standard-Bausteine + SOTA-Profil	A (Baseline auch für C)
Stufe 2 (nur Prüfling)	überschriebene Achsen = Prüfling-Varianten	A
Stufe 3 (Full Join)	Standard $\times$ Prüfling, nicht-redundant	B
build-übergreifend	alt gegen neu derselben Konfiguration	C

Die drei Prüf-Stufen bilden nicht eins-zu-eins, sondern in dieser Zuordnung auf die drei Pflicht-Messreihen A/B/C ab; Reihe C ist build-/versionsübergreifend (nicht an eine einzelne Prüfstufe gebunden).

Die drei Messreihen — alle vom `messung_driver` über die `ExperimentDriver`-Bibliothek ausgeführt — sind damit: *Reihe A* (PRT-ART vs. Stand der Technik) mit den Modi `A_defined` (8 Rang-1-Profil) und `A_full` (alle 30 SOTA-Profil); *Reihe B* (systematische Variation der Achsen-Entscheidungen unter sonst gleichen Bedingungen, um den Beitrag jeder Achse zu isolieren); und *Reihe C* (konkrete Merge-Punkte zwischen PRT-ART-Bausteinen und Cache-Engine-Permutationen, etwa PRT-ART-OLC + `tcmalloc` + ART-Page). Jede Reihe wird in drei Granularitäten erhoben: *Micro-Benchmarking* vergleicht jeden Entwurfsbestandteil isoliert über das gemeinsame Kategorie-Interface, *Makro-Benchmarking* misst die einzelnen `std::map`-Operationen, und *Gesamt-Benchmarking* fährt die YCSB-Lastprofile über jede Rekombination. Genau diese drei-granulare Methodik ist die Antwort auf das Trennbarkeits-Problem: Indem jeder Bestandteil zunächst isoliert (Micro) vermessen wird, bevor er im Operations- und Gesamtkontext erscheint, werden die Beiträge getrennt und isoliert messbar, statt im Gesamtdurchsatz vermengt zu bleiben.

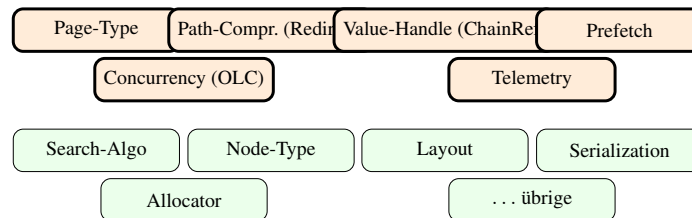
Den so aufgespannten Raum reduziert das System gezielt. Mit der Erweiterung auf neunzehn Achsen plus rund vierundsechzig Sub-Achsen wächst er auf über 10^{14} Permutationen; drei Mechanismen dämpfen die Explosion: der `<expected_workload>`-Profil-Filter routet nur kompatible Workloads, der `Defined`-Modus führt nur die deklarierten Tupel aus (rund 100–1000 je Reihe, die ins Manuskript einfließen), und der `Full`- bzw. `Full-Sampled`-Modus (etwa jede tausendste Permutation) läuft nur auf dem ZIH-Cluster unter Beachtung der vom Plattform-Probe gemeldeten Achsen-Kompatibilität.

3.4 PRT-ART als Demonstrator

PRT-ART ist der Prüfling, der zeigt, dass das System trägt: er stellt einige Organe selbst und greift für die übrigen auf die Standard-Bausteine zurück. PRT-ART ist ein *Prüfling*: ein abstraktes Lebewesen, das einige Achsen-Organe selbst stellt und für die übrigen über einen `Compile-Time-Fallback` (`resolve_baustein.hpp`) auf die Cache-Engine-Standardbausteine zurückgreift, wo es eine Achse nicht überschreibt; die hybride `PrtArtSearchEngine<Ts...>`-API bleibt davon unberührt, und die Laufzeit-Anbindung erfolgt über den `Execution-Engine-Adapter`, nicht über eine eigene Such-Engine-Schicht. Konkret überschreibt PRT-ART im Mess-Apparat die Build-Achse `Page-Type` sowie die Organe `Pfadkompression` (`Redirect`), `Wert-Handle` (`ChainRef-Slot`), `Prefetch`, `Concurrency-Pattern` und `Telemetry` — `Path-Oriented-(Bundle-)Prefetch`, optimistisches `Lock-Coupling` mit reservierten Wert-Blöcken sowie die Hypothesen-Metriken `H1/H2/H3` —; seine weiteren eigenen Bausteine, der `4+1+2-Pool-Allokator` (`Free-List-Sub-Achse`) und das `MultiLevel-Layout`, liegen im Prüfling-Repo und laufen im Mess-Pfad

derzeit über den Cache-Engine-Standard (Anbindung als Folgeschritt E6); für die übrigen Achsen verwendet er bestehende SOTA-Bausteine über die Permutations-Konfiguration wieder. So zeigt der Prüfling, dass das Mess-System trägt: Ein neuartiges Lebewesen entsteht aus wenigen eigenen Organen plus dem Standard-Baukasten (Abbildung 3.3) und ist über dasselbe Prüf-Dock messbar wie jedes rekonstruierte SOTA-Verfahren.

PRT-ART (Prüfling): ein Lebewesen aus 19 Achsen-Organen + Build-Achse Page-Type



orange = im Apparat überschriebener PRT-ART-Slot (inkl. Build-Achse Page-Type) grün = Cache-Engine-Standard (Compile-Time-Fallback)

Abbildung 3.3: PRT-ART als Demonstrator im M-Modell: Der Prüfling überschreibt im Mess-Apparat sechs Slots — die Build-Achse Page-Type sowie die fünf Achsen-Organen Pfadkompression (Redirect), Wert-Handle (ChainRef), Prefetch, OLC-Nebenläufigkeit und Telemetrie — und bezieht die übrigen vierzehn Kompositions-Achsen über den Compile-Time-Fallback aus der Cache-Engine; seine weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad derzeit über den Cache-Engine-Standard (Anbindung: Folgeschritt E6). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen.

3.5 Schärfung durch Heuristiken

Aus den Messreihen lässt sich — als Ausblick-Mechanismus — je Last- und Datentyp-Profil die beste Achsen-Komposition ableiten und als wiederverwendbares Profil ablegen. *Dies ist ein Ausblick-*

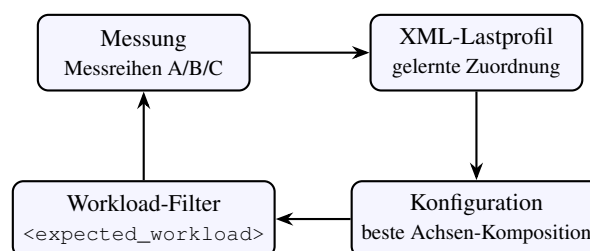


Abbildung 3.4: Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt.

Mechanismus, keine im Rahmen dieser Arbeit umgesetzte Funktion. Über die reine Rangbildung hinaus ist eine *Heuristik-Extraktion* als Folge-Phase nach **Persist** vorgesehen, die aus den gesammelten Messungen automatisch ableitet, welche Achsen-Konfiguration je Workload und Datensatz die beste ist. Der angestrebte Mechanismus ist eine ML-Klassifikation in drei Schritten: Zunächst werden je Permutation Merkmale gewonnen — die Lastprofil- und Datensatz-Merkmale (Workload-Typ, Schlüssel-Verteilung, Alphabet- und Präfix-Charakteristik) sowie die Achsen-Belegung —, mit der je Lastmuster gemessenen

Bestleistung als Zielgröße; darüber lernt ein *ML-Klassifikator* die Abbildung von Lastprofil-/Achsen-Merkmalen auf die beste Konfiguration; und dieser liefert je Workload-/Datensatz-Klasse die empfohlene Achsen-Komposition, ohne den gesamten Permutationsraum erneut zu fahren. Das Ergebnis wird als wiederverwendbares *XML-Lastprofil* persistiert — symmetrisch zu den Messreihen-XML-Konfigurationen — und kann ohne erneute Vollvermessung in den `<expected_workload>`-Filter zurückgespeist werden; damit schließt sich der Kreis zwischen Messung, gelernter Heuristik und konfigurationsleitender Auswahl (Abbildung 3.4). Dieser *measure* $\rightarrow$ *profile* $\rightarrow$ *config* $\rightarrow$ *filter*-Kreislauf ist die klassische, etablierte Form des *empirischen, messgetriebenen Autotunings*: Er steht in der Linie von Autotuning-Frameworks wie OpenTuner [AKV⁺14] und den messen-im-Kreis-Autotunern FFTW [FJ98] und ATLAS [WPD01]; die lastprofil-getriebene Konfigurationswahl mit Workload-Filter findet ihre Datenbank-Entsprechung in OtterTune [APGZ17] und im Database Tuning Advisor mit Workload-Kompression [ACK⁺04], die online-laufzeitnahe Variante in Active Harmony [TCH02]; die Bezeichnung „Software-in-the-Loop“ selbst entstammt der X-in-the-Loop-Validierungstaxonomie [TMS⁺16]. Die konkrete Umsetzung dieses Auswahl- und Auslieferungs-Mechanismus bleibt zukünftiger Arbeit vorbehalten (Kapitel 6). Zunächst vertieft Kapitel 4 die konkrete Implementierung der hier eingeführten Bausteine.

4 Implementierung und konkrete Algorithmusbestandteile

Dieses Kapitel steigt in das Detail: zuerst die Repository-Architektur, dann jede Schlüsselstelle von PRT-ART einzeln und bildhaft, schließlich die Achsen-Implementierung und die Ebenen des Mess-Systems.

4.1 Drei-Repository-Architektur

Die Implementierung verteilt sich auf drei Git-Repositorien mit klar getrennten Verantwortlichkeiten:

- `comdare-cache-engine` — **wie** gemessen wird: die Bausteine-Bibliothek und der Builder (XML-Config-Parser, `generate_module_from_profile`-Codegenerierung, Permutations-Schleife, Modul-Loader und der sieben-phasige `ExperimentDriver`), die ABI-Header, 30 SOTA-Profilen unter `algorithm_profiles/sota` und die Test-Suites; der Quellbaum folgt einem Pitchfork-Layout (`apps/`, `libs/`, `libs/common`).
- `comdare-prt-art` — **der Prüfling**: die hybride `PrtArtSearchEngine` (Vector/Map/Tuple-API), ihre ABI-Adapter und ihre Bausteine (4+1+2-Allokator-Pools — sieben Pools: vier Größenklassen-Pools A–D, ein Rest-Pool R und zwei reservierte Wert-Pools —, OLC mit reservierten Blöcken, MultiLevel-Layout, pfadorientiertes Prefetch, DensityTracker, Hypothesen-Metriken H1/H2/H3); Pool-, Prefetch- und Layout-Komponenten sind dabei derzeit Deskriptor-Skelette ohne echte Speicheranbindung.
- `probst-Diplomarbeit-cache-engine` — **was** getestet wird: der `messung_driver`, die Experiment-Konfigurationen, die Auswertungs-Pipeline und das Manuskript.

Quer dazu trennt die Terminologie zwei Engine-Begriffe, die nicht kurzgeschlossen werden dürfen. Die `CacheEngine` (`cache-engine/libs`) ist die *Bibliothek der Achsen-Bausteine*: Sie stellt je Achse die austauschbaren Entwurfsbestandteile als Concept-/CRTP-Komponenten bereit und enthält *keine* Mess-Logik. Die `ExecutionEngine` hingegen ist die quer zu allen Achsen liegende *Ausführungs- und Mess-Infrastruktur*: Als abstrakte messbare Wurzel (`IExecutionEngine`, Abschnitt 3.3) liefert der konkrete Laufzeit-Adapter `SearchAlgorithmAbiAdapter<A>` über die `SearchAlgorithmAnatomy` die OS-Primitiven (cache-line-ausgerichtete Allokation, NUMA-konformes Read-Pinning, kohärenz-bewusstes Schreiben); die Mess- und Observer-Kopplungen des Adapters sind über `COMDARE_MEASUREMENT_ON` gegated, und den Mess-Aggregator führt der `ExperimentDriver` host-seitig (eine CMake-Option `COMDARE_EXPERIMENT_MODE` existiert, verdrahtet dieses Gating derzeit aber nicht). Kurz: Die `CacheEngine` liefert die *Bausteine*, die `ExecutionEngine` die *Werkzeuge*; diese Trennung hält den Produktiv-Pfad mess-frei.

4.2 PRT-ART im Detail

Aufbauend auf dem bekannten Stand der Technik beschreiben wir jede Schlüsselstelle von PRT-ART einzeln und bildhaft. PRT-ART steuert mehrere eigene Schlüsselstellen bei, je an ihrer Achse verortet (zwei davon auf T10). Auf der *Allokator-Achse* ersetzt ein *4+1+2-Pool-Allokator* (vier Größenklassen-Pools, ein Rest-Pool plus zwei reservierte Wert-Pools) die algorithmus-fest verdrahtete Allokation durch eine cache-line-ausgerichtete, fragmentierungsarme Free-List. Auf der *Nebenläufigkeits-Achse* — selbst gegliedert in Pattern (8.1) und Locking-Mode (8.2) — trägt PRT-ART *optimistisches Lock-Coupling (OLC) mit reservierten Wert-Blöcken* bei, koordiniert über einen Concurrency-Manager, der die Disziplinen OLC, HTM, STM, Lock-Free, RCU und Hazard Pointers führt; die Speicherfreigabe ist dabei eine Sub-Achse des Allokators (epoch-, RCU-, Hazard-Pointer- und QSBR-Strategien je Permutation wählbar). Auf der *Prefetch-Achse* liefert ein *pfadorientiertes Bundle-Prefetch* die heißen Suchpfade vorab, auf der *Layout-Achse* ein *MultiLevel-Layout*. Besonders cache-kritisch ist die *Telemetrie-Achse*: Per-Node-Zähler verursachen auf Mehr-Kern-Systemen Cache-Line-Ping-Pong und verfälschen genau jene Messungen, die sie erheben. Der Kuehn-Forschungslinie [Kue23] folgend bietet PRT-ART daher cache-kohärenzbewusste Strategien — *Leaf-Only-Zähler* (Hauptvariante), *Sampling* (jeder N -te Blattzugriff) und *Offline-Recompute* (Bottom-up-Aggregation vor dem Reordering) —, während der klassische Per-Node-Zähler nur als gekennzeichnetes Anti-Pattern erhalten bleibt. Den DensityTracker und die Hypothesen-Metriken H1/H2/H3 ergänzt der Prüfling als eigene Telemetrie-Organe. Diese Schlüsselstellen sind *Bausteine des Prüfling-Repos*; im Mess-Apparat überschrieben sind davon derzeit Prefetch, Nebenläufigkeit und Telemetrie (zusammen mit der Build-Achse Seitentyp, Pfadkompressions-Redirect und ChainRef-Wert-Handle), während 4+1+2-Pool-Allokator und MultiLevel-Layout im Mess-Pfad über den Cache-Engine-Standard laufen, bis ihre Anbindung (Folgeschritt E6) landet (Abschnitt 3.4). Auf eine Galerie verdichtet zeigt Abbildung 4.1 die sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse verortet.

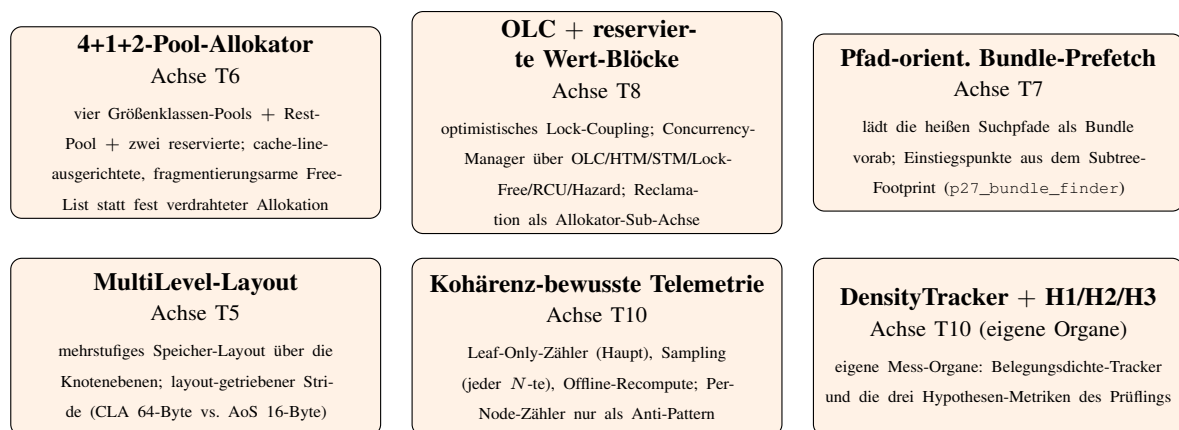


Abbildung 4.1: Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenzbewusste Telemetrie (T10) samt DensityTracker und den Hypothesen-Metriken H1/H2/H3. Die Kacheln zeigen die *Bausteine des Prüfling-Repos*; im Apparat überschrieben sind derzeit T7/T8/T10, die Bausteine zu T5/T6 laufen dort noch über den Cache-Engine-Standard (E6). Die übrigen vierzehn Kompositions-Achsen bezieht der Prüfling über den Compile-Time-Fallback aus der Cache-Engine.

4.3 Achsen-Implementierung und Mess-System-Ebenen

Jede Achse wird als generalisiertes Interface implementiert; daneben liegen die Ebenen des Mess-Systems von der Codegenerierung bis zur Sieben-Phasen-Pipeline. Jede Achse ist ein Topic-Verzeichnis, das ein zweischichtiges *Concept* (ein breites Topic-Concept und ein enges Achsen-Concept) sowie eine durch eine *requires*-Klausel abgesicherte *CRT-Baseklasse* enthält. Die Prüfling-gegen-Standard-Auswahl wird für die elf PRT-ART-Fallback-Achsen zur Übersetzungszeit über `resolve_baustein.hpp` (Concept-geprüftes `if constexpr` über Tag-Spezialisierungen) aufgelöst, sodass der Hot-Path weder `virtual`-Aufrufe noch Laufzeit-Switches enthält — die zero-cost abstraction aus Abschnitt 2.3. Jeder SOTA-Entwurf und jeder Allokator wird über einen dünnen *Adapter* integriert, der die Original-Implementierung hinter dem Engine-ABI bereitstellt: Allokator-Vendors werden über per-Vendor-Include-Shims (`#if COMDARE_AXIS_06_USE_<X>` mit Forward-Stubs), SOTA-Bausteine über `constexpr`-Enable-Flags samt `mp_filter` (`COMDARE_AXIS_*_USE_-/ENABLE_-Flag-Familie`) übersetzungszeitlich zu- oder abgeschaltet, jeweils mit sicherem Fallback (etwa `std::malloc` oder einer Re-Implementierung), sodass der Build eigenständig lauffähig bleibt — `COMDARE_HAVE_<X>` besteht nur noch als historisches Detektions-Define; die Integration des hierarchischen Prefetching [ZGH⁺25] liefert zusätzlich einen Call-Graph-Analyzer zur Übersetzungszeit (`p27_bundle_finder`, ein C++23-Port des Original-Werkzeugs `hp_soft`, der Funktionen mit Subtree-Footprint oberhalb eines Schwellwerts als Bundle-Einstiegspunkte identifiziert).

Der Builder überführt jedes Profil in ein vorkompiliertes C++23-Modul; jede Permutation ist durch eine bit-gepackte *Permutations-Flag*-Struktur identifiziert, die mit der N-Phasen-Verfeinerung von neun Bänken (rund 50 Bit) auf einen breiteren *Permutations-Identifizier* mit vierzehn Bänken (82 Bit) und Sub-Bank-Bitfeldern wächst; die neunzehn Kompositions-Achsen identifiziert der aktive Mess-Pfad über den string-serialisierten Achsen-Pfad samt Fingerprint, und ein Constraint-Filter schließt ungültige Kombinationen vor jeder Modulerzeugung aus. Über diese Bausteine läuft die *Sieben-Phasen-Mess-Pipeline* (Abbildung 4.2) des `ExperimentDriver` je Reihe: (1) **Enumerate** (XML-Configs parsen, Permutationsliste aufbauen), (2) **Codegen** (ein `comdare_perm_<fp>.cpp` je Permutation), (3) **Compile**, (4) **Load** (LoadLibrary/dlopen aller Module), (5) **Execute** (`create_instance` + profilbewusstes `run_workload`), (6) **Measure** (binäre Mess-Records aggregieren) und (7) **Persist** (`measurements.csv/.json`); zwei opt-in-Phasen kompilieren fehlende Module hot und verifizieren den ABI-Vertrag je Modul. Die nachgelagerte Auswertungs-Werkzeugkette ist eine Folge kleiner Programme — `sample_data_generator` (deterministische Testdaten ohne reale Hardware) → `messung_driver` (steuert die Reihen) → `binary_to_csv` (deserialisiert die binären Mess-Records) → `csv_to_latex` (booktabs-Tabellen mit Algorithmus-Steckbriefen je Permutations-ID) → `diagram_generator` (A4-bewusste TikZ-Plots) → `latex_to_pdf` —; die Mess-Kopplungen sind über das `COMDARE_MEASUREMENT_ON`-Gating vom Produktiv-Pfad getrennt, sodass dieser keinen Mess-Overhead trägt, und Continuous Integration (GitLab CI plus gespiegelter GitHub-Workflow) läuft über alle drei Repositorien. Die neunzehn Kompositions-Achsen sind nicht abstrakt, sondern je ein konkretes *Strategie-Concept* mit mehreren austauschbaren Bausteinen, aus denen eine konkrete Composition je Slot genau einen wählt. Abbildung 4.3 fasst alle neunzehn als Galerie zusammen — je Achse der Kurzname, die Sub-Achsen-Familie und eine repräsentative Auswahl der Bausteine; die vollständige Bausteine-Matrix mit allen Herkunfts-Quellen führt Anhang D, die SOTA-Beiträge je Achse Tabelle 3.2.

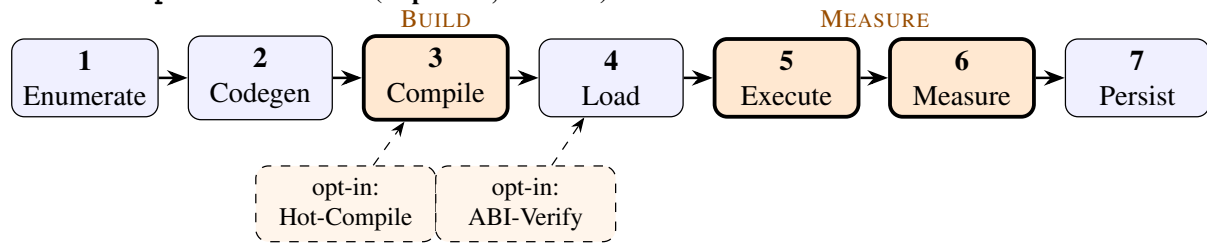
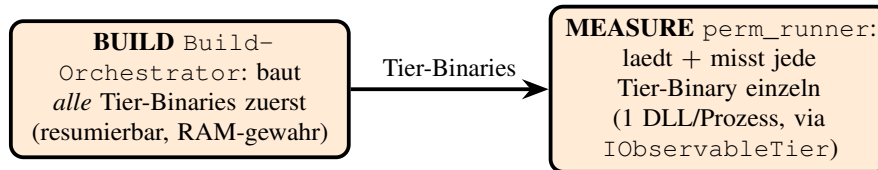
Pfad A — ExperimentDriver (in-process, 7 Phasen)**Pfad B — BuildOrchestrator + perm_runner (out-of-process, empfohlen)**

Abbildung 4.2: Die Mess-Pipeline in zwei Ausprägungen. **Pfad A** (oben) ist die in-process Sieben-Phasen-Pipeline des `ExperimentDriver` je Messreihe (`Enumerate` → `Codegen` → `Compile` → `Load` → `Execute` → `Measure` → `Persist`; zwei opt-in-Phasen kompilieren fehlende Module hot und verifizieren den ABI-Vertrag). Hervorgehoben sind die eigentlich substantziellen Phasen: `COMPILE` (der *Build* der Tier-Binaries) und `EXECUTE/MEASURE` (die *Messung*); `Codegen/Load/Persist` sind reines Plumbing. **Pfad B** (unten, der empfohlene Weg) trennt beide Aufgaben out-of-process: der `BuildOrchestrator` baut *zuerst alle* Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst `perm_runner` jede Binary einzeln (eine DLL je Prozess) über die Beobachter-Schnittstelle `IObservableTier`.

Damit ist der Apparat vollständig beschrieben — von den Repositorien über den Prüfling bis zu den neunzehn Achsen und ihren Mess-System-Ebenen. Kapitel 5 legt die Methodik offen, mit der dieser Apparat bewertet wird.

T0 Suchalgorithmus SA1-SA4 Array256, OriginalART/HOT, KAry, Interpol., Hash ... (21: 17 Basis + 4 per-K) <i>Std. Array256</i>	T1 Cache-Travers. CT1-CT2 Linear-Fanout, Hash- Lookup, Binär-Fanout <i>Std. Linear</i>	T2 Slot-Mapping MP1-MP2 Direkt-Platzierung, Pool-relativ <i>Std. Direkt</i>	T3 Pfadkompression PC1-PC3 Keine, Byte-weise (ART), Patricia <i>Std. Keine</i>	T4 Knotentyp NT1-NT3 Node4 / 16 / 48 / 256 <i>Std. Node256</i>
T5 Speicher-Layout HM1-HM4 AoS, Cache-Line-aligned, SoA, Bitmap (LOUDS), AoSoA <i>Std. Cache-Line</i>	T6 Allokator AA1-AA7 Hoard, Slab, Mimalloc, Jemalloc, TCMalloc, Buddy ... (25) <i>Std. Std-Malloc</i>	T7 Prefetch PF1-PF3 Keine, Distanz-Schätzer (ART), Hardware, Pfad-orient. <i>Std. Keine</i>	T8 Nebenläufigkeit CC1-CC2 Keine, Blocking, R/W, OLC, Lock-free, RCU, Hazard <i>Std. Keine</i>	T9 Serialisierung SR1-SR3 Roh-Binär, VarLen (LEB128), Succinct, LZ4/Delta <i>Std. Roh-Binär</i>
T10 Telemetrie TM1-TM3 Leaf-Only-Zähler, Density- Tracker, HDR-Histogramm <i>Std. Leaf-Only</i>	T11 Wert-Handle VH1-VH3 Inline, Extern-Pool, RCU- Shared, MVCC, Chain-Ref <i>Std. Inline</i>	T12 ISA IS1-IS3 Amd64 (SSE2), AArch64 (NEON), PowerPC, RISC-V <i>Std. Amd64</i>	T13 Index-Org. io1-io3 Clustered, Heap, IOT (embedded), Non-Clustered <i>Std. Heap</i>	T14 I/O-Dispatch IO1-IO3 In-Memory, Buffered, Direct (O_DIRECT), Mmap <i>Std. In-Memory</i>
T15 Migration MG1-MG3 Keine, Hot/Cold, Tier- basiert, Adaptiv (ML) <i>Std. Keine</i>	T16 Filter FT1-FT3 Bloom, Cuckoo, Range- SuRF (LOUDS), Xor <i>(kein None)</i>	T17 Op.-Puffer QS1-QS6 Kein, Append, FIFO/LIFO, Ring, Priority ... (15) <i>Std. Kein-Puffer</i>	T18 Flush-Policy FS1-FS4 Eager, Lazy, Watermark, Timed, Adaptiv-LSM <i>Std. Lazy</i>	Lesart Jede Achse = ein Concept mit austauschbaren Bausteinen (Strategy); eine Composition wählt je Slot genau einen.

Abbildung 4.3: Galerie der neunzehn Kompositions-Achsen (T0–T18): je Achse das Implementierungskonzept — Kurzname, Sub-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein Concept-Interface mit mehreren Strategie-Bausteinen (Strategy-Muster); eine konkrete Composition belegt je Slot genau einen. Der Standard („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag; bei den großen Achsen T0/T6/T17 ist die Gesamt-Bausteinanzahl in Klammern angegeben. Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D.

5 Evaluation

Die Evaluation legt zuerst die Methodik offen und wertet darauf aufbauend die Messreihen aus.

5.1 Methodik

Drei *Hypothesen* leiten die Auswertung:

H1 (Page-Type-Kosten) Die mittleren Zugriffskosten je Page-Type-Variante unterscheiden sich systematisch nach Workload: dichte Page-Typen gewinnen erwartungsgemäß unter Zipf-Verteilung (YCSB-A), bereichsoptimierte Pages unter Bereichs-Scans (YCSB-E).

H2 (Code-Qualität pro Quelle) Der vom Original-Quell-Repository geerbte Code-Qualitäts-Score korreliert messbar mit dem erreichten Durchsatz.

H3 (Inline- vs. External- vs. Chain-Verteilung) Die beobachtete Verteilung der drei ValueHandle-Varianten korreliert mit der Page-Dichte: dichte Pages bevorzugen Inline, spärliche Pages External und PRT-ART-Heuristiken ChainRef.

Diese *Evaluations-Hypothesen* sind nicht zu verwechseln mit den gleichnamigen PRT-ART-Telemetrie-Metriken H1–H3 (Abschnitt 3.4): Erstere sind prüfbare Aussagen über den Entwurfsraum, Letztere Mess-Organen des Prüflings, die unter anderem die Daten für diese Prüfung liefern.

Phase 5 leitet den YCSB-Workload je Permutation aus dem Traversal-Tag des Profils ab (Tabelle 5.1); Module ohne Profil verwenden den globalen Default-Workload. Als Datengrundlage dienen reale String-Datensätze unterschiedlicher Charakteristik (Tabelle 5.2), jeder mit einer Reproduzierbarkeits-Akte aus Quelle, Prüfsumme, Zeilenzahl, Vorverarbeitung und Seed-Regel für die synthetischen Miss-, Update- und Delete-Schlüssel.

Tabelle 5.1: Profil-bewusstes Workload-Routing

Traversal-Tag	YCSB-Workload	Charakteristik
RANGE_SCAN / RANGE_HOT_PATH	YCSB-E	Bereichsabfragen
HOT_PATH / PRTART_HOT_PATH	YCSB-A	50/50 Lesen/Aktualisieren, Zipf
STANDARD (Default)	YCSB-C	nur Lesen

Die Messungen zielen auf eine lokale Workstation zur Entwicklungs-Verifikation und den TU-Dresden-ZIH-Cluster (Barnard-CPU-Knoten mit Intel Sapphire Rapids, Capella GPU) für die vollständigen Sweeps, plus die Rang-3-Plattformen (Ryzen 9 9950X3D, Intel i9 14900KS und eine HBM-Plattform). Ein `IPlatformProbe` (Phase 1) meldet je Plattform die lauffähige Teilmenge der Achsen (etwa AVX-512 nur dort, wo verfügbar, HBM-aware nur auf HBM-Hardware); je Plattform werden ein Scalar-, ein AVX2- und — auf Barnard — ein AVX-512-Build vorbereitet, und auf Hybrid-CPUs werden P-

Tabelle 5.2: Reale Datensätze und ihre Charakteristik

Datensatz	Charakteristik
url	sehr lange Schlüssel, lange gemeinsame Präfixe, mittleres Alphabet
dna	kleines Alphabet, hohe Wiederholung, tiefe Präfix-Entropie
protein	biologische Sequenzen, kleines bis mittleres Alphabet
xml	strukturierte Strings mit wiederkehrenden Präfixen
tpcds-id	datenbanknahe IDs, sortiert und unsortiert
trec-terms	Wörterbuch-/Suchindex-Terme mit definierten Miss-Schlüsseln

und E-Cores getrennt vermessen. Auf den Produktions-Zielmaschinen wird zudem jede Messung unter zwei *Betriebssystem-Regimes* erhoben, um Produktions-Treue von Mess-Tiefe zu trennen: Ein *immutable* Betriebssystem (Talos) spiegelt den Produktiv-Einsatz und erzwingt boot-/übersetzungsstatische Cache-Einstellungen (Abschnitt 2.2); ein *root-Linux mit vollem Hardware-Zähler-Zugriff* (perf/MSR) macht hingegen die zählerbasierten Mess-Kategorien (Cache-, dTLB- und Branch-Misses, IPC/CPI, Abschnitt 3.2) und die laufzeit-konfigurierbaren cache-nahen Achsen-Parameter (Prefetch-Distanz, Hardware-Prefetcher) überhaupt erst zugänglich. Die zeit- und observer-basierten Kategorien laufen unter beiden Regimes identisch und sichern die Vergleichbarkeit.

Faire Vergleichbarkeit ist über feste Regeln gesichert: Jeder Vergleich trennt einen gemeinsamen Minimalmodus (Common-Denominator: externe Werte-Handles, keine PRT-ART-Spezialpfade) vom PRT-ART-Native-Modus; alle Verfahren erhalten identische, normalisierte Byte-Schlüssel sowie identische Miss-, Update- und Delete-Mengen, mit je Plattform und Workload protokolliertem Warmup, Randomisierung und Thread-Pinning. Fehlende Fähigkeiten einer Baseline — etwa Bereichs-Scans einer Hash-Tabelle — werden als N/A ausgewiesen statt verdeckt emuliert; für jede Fremdbibliothek werden Compiler, Flags, ISA-Pfad, Allokator und Commit-Hash mitgeloggt, und jede Konfiguration wird in mehreren getrennten Läufen mit über HDR-Histogramme bestimmten Perzentilen vermessen.

5.2 Ergebnisse und Analyse

Zum Zeitpunkt der Erstellung sind Methodik und Auswertungs-Pipeline vorhanden und auf Beispieldaten verifiziert; eine erste reale Smoke-Messreihe (43 Permutationen, echte DLL-Messung auf der Entwicklungsmaschine) dokumentiert Anhang A samt ihren Limitierungen (Abschnitt A.5), die vollständigen Läufe der Pflicht-Messreihen auf den Zielplattformen stehen aus (Abschnitt 6.2). Pro Reihen-Lauf schreibt der `messung_driver` unter `Code/_runs/<date>/<spec_id>/` die Dateien `measurements.csv` — eine Zeile je Permutation mit den Spalten des kanonischen Mess-Records: `op_count`, `total_cycles`, L1-/L2-/L3-Cache-Misses, dTLB-Misses, Kohärenz-Invalidierungen, Energie sowie die Speicher-Felder (`bytes_allocated`, `bytes_in_use_peak`, Fragmentierung); Branch-Misses und Durchsatz werden als Schema-Erweiterung des Records ergänzt (der Durchsatz liegt bislang im Kommando-Pfad vor) —, `measurements.json` und optionale binäre Records; die Auswertung reichert sie über `binary_to_csv` an, rendert `csv_to_latex`-Steckbriefe je Permutations-ID (in der Reihe `A_defined` entspricht das je einem SOTA-Profil) und `diagram_generator`-Diagramme (A4-bewusste TikZ-Balkendiagramme, Scatter-Plots, Heatmaps), die das Manuskript über `\input{tikz/<spec_id>/...}` und `\input{tabellen/<spec_id>...}` aus den

sprachneutralen Verzeichnissen einbindet. Die drei Messreihen werden dann ausgewertet: Reihe A (PRT-ART vs. Stand der Technik, mit den Evaluations-Hypothesen H1–H3 aus Abschnitt 5.1), Reihe B (systematische Variation der Cache-Engine-Permutationen, gespeist aus dem Full Join der Stufe 3) und Reihe C (Merge/Regression alt gegen neu, build-übergreifend; Tabelle 3.8).

Den Kern der Auswertung bildet die *Achsen-Sensitivitätsanalyse*, die unmittelbar das Trennbarkeits-Problem der Aufgabenstellung beantwortet: Im Stand der Technik sind die Gewichtungen der Verbesserungen *einzelner* Algorithmus-Bestandteile bislang nicht trennbar, weil jede Veröffentlichung ihren Nutzen in einer eigenen, inkompatiblen Messumgebung belegt. Indem das Achsen-Framework jeden Bestandteil als austauschbare Achse über einer sonst konstanten Struktur isoliert, wird der Beitrag jeder Entwurfsentscheidung erstmals unter sonst gleichen Bedingungen separierbar und messbar zurechenbar. Anhand der Records je Permutation quantifiziert die Analyse auf der Micro-Granularität, welche Achse am meisten zur Performance-Varianz beiträgt; Makro- und Gesamt-Benchmarking ergänzen die Operations- und die YCSB-Lastprofil-Ebene, und aus der Rangbildung über die drei Granularitäten folgt die Empfehlung von Standard-Konfigurationen je Workload-Klasse. Damit schließt sich der Bogen zur Aufgabenstellung: Die Sensitivitätsanalyse liefert genau die getrennte Gewichtung der einzelnen Bestandteile, deren Fehlen das Ausgangsproblem war. Die Ergebnisse werden schließlich vor dem Leitmotiv der Arbeit diskutiert — dem Übergang vom heuristischen zum informierten Cache-Management, also der Frage, ob telemetriegetriebene, automatisch adaptierte Konfigurationen statisch gewählte übertreffen. Kapitel 6 zieht daraus das Fazit.

6 Fazit und Ausblick

Dieses Kapitel zieht Bilanz: Es beantwortet die Forschungsfragen FF0–FF4 aus Abschnitt 1.3, benennt die Grenzen des erreichten Standes und skizziert die nächsten Schritte.

6.1 Beantwortung der Forschungsfragen

FF0 (Hauptfrage). Wie stark aktive, messgetriebene cache-bewusste Entscheidungen ihre passiven, statisch ausgelegten Gegenstücke übertreffen, lässt sich abschließend erst nach den End-to-End-Läufen je Plattform beziffern (Abschnitt 6.3). Vorbereitet ist die Apparatur, die genau diese Differenz isoliert: dieselbe achsen-konstante Struktur wird einmal aktiv (messgetriebene Seitentyp-, Layout-, Prefetch- und Value-Ablage-Entscheidung) und einmal passiv (statisch ausgelegt) permutiert und unter identischer Compiler- und Flag-Basis auf Hybrid-CPU und Sapphire-Rapids gegenübergestellt.

FF1 (Komposition). Die neunzehn Permutations-Achsen (Search-Algorithm, Cache-Traversal, Mapping, Path-Compression, Node-Type, Memory-Layout, Allocator, Prefetch, Concurrency, Serialization, Telemetry, Value-Handle, ISA, Index-Organization, I/O-Dispatch, Migration, Filter und die beiden Queuing-Achsen) spannen einen Konfigurationsraum von rund 10^{14} Permutationen auf. Stand-der-Technik-Entwürfe werden über Algorithmus-Profil-XMLs eindeutig rekonstruiert: 30 SOTA-Profilen (8 Rang-1 + 22 Rang-2/3) zeigen, dass die Achsen-Zerlegung P01–P33 trägt. Die variadische API-Spezialisierung (1/2/N Parameter auf `std::vector/std::map/Tuple`) macht sowohl algorithmus- als auch container-orientierte Muster auf einer Engine ausdrückbar. Die nicht voll-orthogonalen Achsen-Kopplungen (Knotentyp $\times$ SIMD $\times$ Pfadkompression) werden dabei als solche markiert und bei der Permutation explizit eingeschränkt. Die Teilfrage nach dem Cache-Update- und Cache-Line-Füllverhalten unter *allein* variiertem ISA-/SIMD-Achse ist methodisch beantwortet — die ISA-Achse (T12) ist bei sonst achsen-konstanter Struktur isoliert permutierbar —; die empirische Antwort folgt mit den End-to-End-Läufen (Abschnitt 6.3).

FF2 (Mess-Methodik). Die Messung ist in einen Produktiv-Modus (`COMDARE_MEASUREMENT_ON` abgeschaltet, kein Overhead) und einen Experiment-Modus geteilt: Der `ResultAggregator` wird host-seitig im `ExperimentDriver` geführt (Phasen 5–7), und der Experiment-Modus aktiviert die Mess-Hooks und die Observer-Kopplung im ABI-Adapter. Profil-bewusstes Workload-Routing wählt den Workload je Permutation aus dem Traversal-Tag und sichert so faire Vergleiche zwischen bereichsabfrage- und punktzugriffsorientierten Entwürfen. Jede Messreihe wird in drei Granularitäten erhoben — Micro-, Makro- und Gesamt-Benchmarking —, sodass sowohl einzelne Entwurfsbestandteile als auch ganze Algorithmen vergleichbar werden. Widersprüchliche Literaturbefunde zur optimalen Knoten-/Cache-Line-Größe (eine gegenüber bis zu sechzehn Cache-Lines) werden so nicht aus inkompatiblen Original-Setups übernommen, sondern achsen-isoliert und unter sonst gleichen Bedingungen nachgemessen. Als erfolgreich verifiziert gilt eine Plattform dabei erst, wenn das Konformitäts-Gatter den `std::map`-

Vertrag je Modul bestätigt und die Plattform-Probe die gemeldete Achsen-Teilmenge deckt; die plattform-kalibrierten Schwellwerte selbst werden mit den realen Läufen festgelegt.

FF3 (Prüfling-Vergleich). Der konkrete Vergleich von PRT-ART gegen die 30 SOTA-Profile steht nach den ersten Mess-Läufen aus (Abschnitt 6.3). Die Architektur ist vollständig vorbereitet: das PRT-ART-Profil, die drei Pflicht-Messreihen-Templates, der `ExperimentDriver` mit Auto-Pickup der SOTA-Profile und der ABI-konforme Adapter mit dem `std::map`-Kernvertrag (Konformitäts-Teilmenge, Anhang F); der Vollausbau auf alle siebzehn Vertragsoperationen samt Notify-Hooks ist als Folgeschritt ausgewiesen. Auch die günstigste lokale Seitendarstellung je Verzweigungspunkt (Punkt- gegenüber Kompaktseite) und ihre plattform-kalibrierten Umschaltregeln sind Teil des PRT-ART-Profils.

FF4 (Compile-Time vs. Laufzeit). Die Trennung ist im Build-System verankert: plattform-spezifische Achsen (ISA, Hardware-Konstanten) werden zur Übersetzungszeit fixiert und erzeugen provenienz-nachweisbare Binaries je Permutation (SHA256-validierte Original-Bodies — deren Voll-Implementierung Abschnitt 6.2 als offen ausweist —, `get_compiler()`-Kennung), während Layout- und Seiten-Selektion über einen bit-codierten Permutations-Identifizier zur Laufzeit wählbar bleiben, ohne die gemeinsame Compiler- und Flag-Basis zu verletzen. Die vorwärtsgerichtete Teilfrage, ob die Messung selbst ISA- und betriebssystem-optimierte Binaries für eine Rekombination zuschneiden kann, greift der Ausblick als Heuristik-Automation auf (Abschnitt 6.3); die Betriebssystem-Dimension ist über die zwei Mess-Regimes aus Kapitel 5 bereits im Messplan verankert.

6.2 Limitierungen

- **Modul-Bodies:** der adhoc-Codegen des aktiven Mess-Pfads emittiert je Permutation reale Achsen-Library-Bodies (die siebzehn voll qualifizierten Achsen-Typen samt Umbrella-Include); nur der Legacy-Profil-Codegen erzeugt weiterhin ABI-konforme Skelett-Module, und als Lücken bleiben der PRT-ART-Prüfling-Stub sowie die ehrlich auf null gesetzten Storage-/Prefetch-Achsen der Pool-Familien offen.
- **Hardware-Validierung ausstehend:** die Konfiguration ist lokal grün, aber der vollständige `ctest`-Lauf und die End-to-End-messung_driver-Läufe über die BlockAO-Plattformen (AVX2/AVX-512, ARM NEON/SVE) stehen aus.
- **Ergebnis-Kapitel:** aktuell methodisch; konkrete Messungen folgen. Eine erste reale Smoke-Messreihe (43 Permutationen) liegt in Anhang A vor, ersetzt die Pflicht-Messreihen aber nicht.
- **Hardware-Zähler standardmäßig aus:** die PMC-Erfassung (L1-/L2-/L3-, dTLB-Misses, Kohärenz-Invalidierungen, Energie/RAPL; Branch-Misses und IPC sind als Erweiterung des Zähler-Satzes geplant, werden derzeit nicht erfasst) ist default-abgeschaltet; in der bisherigen Smoke-Reihe stehen die Hardware-Spalten auf null — die cache-bewusste Knoten-/Daten-/Index-Messkomponente steht damit noch aus (Abbildung 2.14).
- **Smoke-Reihe mit Apparat-Vorbehalten:** die bisher realen Messungen variieren sieben der neunzehn Achsen in zwei Teilblöcken (27 SIMD×Layout×Allokator-Kombinationen plus 16 Kombinationen über Node-Breite, Pfadkompression, interne Suche und Telemetrie) und approximieren

Achsen-Effekte teils über Wall-Clock-Proxys; einzelne Effekte sind Apparat- statt Organ-Artefakte („Beschattung“). Die vollständige Limitierungs-Tabelle führt Abschnitt A.5.

6.3 Ausblick

Kurzfristig: der End-to-End-Lauf der drei Pflicht-Messreihen auf realer Hardware, die Generierung der Mess-Diagramme und die Schließung der verbleibenden Modul-Body-Lücken (PRT-ART-Stub, honest-0-Storage-/Prefetch-Achsen der Pool-Familien).

Heuristik-Automation und Binary-Delivery. Der in der Aufgabenstellung (Teilaufgabe 7) geforderte Mechanismus, der aus den Messergebnissen automatisch die beste Konfiguration ableitet, ist bewusst noch *nicht* implementiert und bleibt einer Folge-Phase vorbehalten. Er setzt drei Komponenten voraus, von denen die zweite bereits vorliegt: (i) einen *Heuristik-/ML-Klassifikator*, der aus den Lastprofil- und Achsen-Features (Operationsmix, Schlüssel-Charakteristik, Hardware-Kennung) auf die empfohlene Achsen-Konfiguration abbildet; (ii) den bereits implementierten *XML-Parser* (Import und Export samt Extraktor) für die als XML-Lastprofil-Ergebnis abgelegten, je Architekturfokus wiederverwendbaren Heuristik-Richtlinien; und (iii) einen *Binary-Packager*, der den so ausgewählten Gesamtalgorithmus als eigenständige Binary hinter dem einheitlichen Suchalgorithmus-Interface ausliefert. Voraussetzung für alle drei ist, dass die End-to-End-Messreihen zuvor vollständig vorliegen — erst die belastbare Datenbasis über alle Permutationen und Lastprofile macht eine messgetriebene, automatisch gesteuerte Auswahl sinnvoll; bis dahin werden die Konfigurations-Empfehlungen des Auswertungs-Schemas aus Abschnitt 5.2 nach Vorliegen der Daten zunächst manuell abgeleitet.

Mittel- bis langfristig: ein GPU-Adapter (Grace-Hopper-Cluster, ZIH), die PIM-Allokator-Integration (A16) und die formale Verifikation ausgewählter Permutations-Klassen (StarMalloc-Stil, A13). Die Drei-Repository-Architektur erlaubt es künftigen Studierenden, weitere Prüfling-Algorithmen analog zu PRT-ART einzubringen, ohne die Cache-Engine-Bausteine-Bibliothek zu modifizieren — ein zentrales Designprinzip für die Skalierbarkeit der Forschungsinfrastruktur. Über die hier behandelten baumartigen Suchstrukturen hinaus ist das Anatomie-Prinzip über höherwertige Abstraktionen auf die weiteren Strukturklassen aus Abschnitt 2.1 erweiterbar — hash-basierte, räumliche sowie mengen- und sequenz-orientierte Strukturen —, sodass derselbe Mess- und Permutationsapparat ohne Neubau auf ganz neue Klassen angewendet werden kann; darin liegt ein wesentlicher Hebel für künftigen experimentellen Fortschritt.

Literaturverzeichnis

- [ABKS13] APEL, Sven ; BATORY, Don ; KÄSTNER, Christian ; SAAKE, Gunter: *Feature-Oriented Software Product Lines: Concepts and Implementation*. 1. Springer, 2013
- [ACK⁺04] AGRAWAL, Sanjay ; CHAUDHURI, Surajit ; KOLLÁR, Lubor ; MARATHE, Arunprasad P. ; NARASAYYA, Vivek R. ; SYAMALA, Manoj: Database Tuning Advisor for Microsoft SQL Server 2005. In: *30th International Conference on Very Large Data Bases (VLDB)*, 2004, S. 1110–1121
- [Adv23] ADVANCED MICRO DEVICES. *Software Optimization Guide for the AMD Zen Microarchitecture (Family 19h)*. Advanced Micro Devices, Santa Clara, CA, USA. 2023
- [AKLS15] AIGNER, Christoph ; KIRSCH, Christoph ; LIPPAUTZ, Michael ; SOKOLOVA, Ana: Fast, Multicore-Scalable, Low-Fragmentation Memory Allocation. In: *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2015, S. 513–531
- [AKV⁺14] ANSEL, Jason ; KAMIL, Shoaib ; VEERAMACHANENI, Kalyan ; RAGAN-KELLEY, Jonathan ; BOSBOOM, Jeffrey ; O'REILLY, Una-May ; AMARASINGHE, Saman: OpenTuner: An Extensible Framework for Program Autotuning. In: *23rd International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2014, S. 303–316
- [Ale01] ALEXANDRESCU, Andrei: *Modern C++ Design: Generic Programming and Design Patterns Applied*. 1. Addison-Wesley, 2001
- [AMH08] ABADI, Daniel J. ; MADDEN, Samuel R. ; HACHEM, Nabil: Column-Stores vs. Row-Stores: How Different Are They Really? In: *ACM SIGMOD International Conference on Management of Data*, 2008, S. 967–980
- [APGZ17] AKEN, Dana V. ; PAVLO, Andrew ; GORDON, Geoffrey J. ; ZHANG, Bohan: Automatic Database Management System Tuning Through Large-scale Machine Learning. In: *ACM SIGMOD International Conference on Management of Data*, 2017, S. 1009–1024
- [App24] APPLE INC. *Apple Silicon CPU Optimization Guide*. Apple Inc., Cupertino, CA, USA, Version 4. 2024
- [Arm24] ARM LIMITED. *Arm Architecture Reference Manual for A-profile Architecture (DDI 0487)*. Arm Limited, Cambridge, UK. 2024
- [Aßm03] ASSMANN, Uwe: *Invasive Software Composition*. 1. Springer, 2003
- [AVL62] ADELSON-VELSKY, Georgy ; LANDIS, Evgenii: An Algorithm for the Organization of Information. In: *Doklady Akademii Nauk SSSR* 146 (1962), S. 263–266

- [BA01] BONWICK, Jeff ; ADAMS, Jonathan: Magazines and Vmem: Extending the Slab Allocator to Many CPUs and Arbitrary Resources. In: *Proceedings of the USENIX Annual Technical Conference (ATC)* USENIX, 2001
- [Bay72] BAYER, Rudolf: Symmetric Binary B-Trees: Data Structure and Maintenance Algorithms. In: *Acta Informatica* 1 (1972), Nr. 4, S. 290–306
- [BBB⁺11] BINKERT, Nathan ; BECKMANN, Bradford ; BLACK, Gabriel ; REINHARDT, Steven K. ; SAIDI, Ali ; BASU, Arkaprava ; HESTNESS, Joel ; HOWER, Derek R. ; KRISHNA, Tushar ; SARDASHTI, Somayeh ; SEN, Rathijit ; SEWELL, Korey ; SHOAIB, Muhammad ; VAISH, Nilay ; HILL, Mark D. ; WOOD, David A.: The gem5 Simulator. In: *ACM SIGARCH Computer Architecture News* 39 (2011), Nr. 2, S. 1–7
- [BCD⁺02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin ; ZITO, Jack: Two Simplified Algorithms for Maintaining Order in a List. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 152–164
- [BCDFC02] BENDER, Michael A. ; COLE, Richard ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Scanning and Traversing: Maintaining Data for Traversals in a Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 139–151
- [BDFC00] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *Proceedings of the 41st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 2000, S. 399–409
- [BDFC02] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Efficient Tree Layout in a Multilevel Memory Hierarchy. In: *Algorithms — ESA 2002, 10th Annual European Symposium* Bd. 2461, Springer, 2002, S. 165–173
- [BDFC05] BENDER, Michael A. ; DEMAINE, Erik D. ; FARACH-COLTON, Martin: Cache-Oblivious B-Trees. In: *SIAM Journal on Computing* 35 (2005), Nr. 2, S. 341–358
- [Ben75] BENTLEY, Jon L.: Multidimensional Binary Search Trees Used for Associative Searching. In: *Communications of the ACM* 18 (1975), Nr. 9, S. 509–517
- [BFTV24] BOFFA, Antonio ; FERRAGINA, Paolo ; TOSONI, Francesco ; VINCIGUERRA, Giorgio: CoCo-trie: Data-aware Compression and Indexing of Strings. In: *Information Systems* 120 (2024), S. 102316
- [BH23] BERTHOLD, Johannes ; HABICH, Dirk: VAMPIR/OTF2 Profiling and Performance Analysis for Database Workloads. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, 2023. – Poster presentation
- [Blo70] BLOOM, Burton H.: Space/Time Trade-offs in Hash Coding with Allowable Errors. In: *Communications of the ACM* 13 (1970), Nr. 7, S. 422–426

- [BM72] BAYER, Rudolf ; MCCREIGHT, Edward M.: Organization and Maintenance of Large Ordered Indexes. In: *Acta Informatica* 1 (1972), Nr. 3, S. 173–189
- [BMBW00] BERGER, Emery D. ; MCKINLEY, Kathryn S. ; BLUMOF, Robert D. ; WILSON, Paul R.: Hoard: A Scalable Memory Allocator for Multithreaded Applications. In: *Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*, ACM, 2000, S. 117–128
- [BO92] BATORY, Don ; O'MALLEY, Sean: The Design and Implementation of Hierarchical Software Systems with Reusable Components. In: *ACM Transactions on Software Engineering and Methodology* 1 (1992), Nr. 4, S. 355–398
- [Bon94] BONWICK, Jeff: The Slab Allocator: An Object-Caching Kernel Memory Allocator. In: *Proceedings of the USENIX Summer 1994 Technical Conference* USENIX, 1994, S. 87–98
- [Bri59] DE LA BRIANDAIS, René: File Searching Using Variable Length Keys. In: *Western Joint Computer Conference*, 1959, S. 295–298
- [BZP⁺18] BINNA, Robert ; ZANGERLE, Eva ; PICHL, Martin ; SPECHT, Günther ; LEIS, Viktor: HOT: A Height Optimized Trie Index for Main-Memory Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 521–534
- [CE00] CZARNECKI, Krzysztof ; EISENECKER, Ulrich W.: *Generative Programming: Methods, Tools, and Applications*. 1. Addison-Wesley, 2000
- [CGM01] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Improving Index Performance through Prefetching. In: *Proceedings of the 2001 ACM SIGMOD International Conference on Management of Data*, ACM, 2001, S. 235–246
- [CGM02] CHEN, Shimin ; GIBBONS, Phillip B. ; MOWRY, Todd C.: Fractal Prefetching B+-Trees. In: *Proceedings of the 2002 ACM SIGMOD International Conference on Management of Data*, ACM, 2002, S. 157–168
- [CHP71] COURTOIS, Pierre-Jacques ; HEYMANS, Frans ; PARNAS, David L.: Concurrent Control with “Readers” and “Writers”. In: *Communications of the ACM* 14 (1971), Nr. 10, S. 667–668
- [Cli93] CLIFF, Norman: Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions. In: *Psychological Bulletin* 114 (1993), Nr. 3, S. 494–509
- [CLP22] CROTTY, Andrew ; LEIS, Viktor ; PAVLO, Andrew: Are You Sure You Want to Use MMAP in Your Database Management System? In: *12th Conference on Innovative Data Systems Research (CIDR 2022)*, Chaminade, CA, USA, January 9-12, 2022, www.cidrdb.org, 2022
- [Com79] COMER, Douglas: The Ubiquitous B-Tree. In: *ACM Computing Surveys* 11 (1979), Nr. 2, S. 121–137

- [CST⁺10] COOPER, Brian F. ; SILBERSTEIN, Adam ; TAM, Erwin ; RAMAKRISHNAN, Raghu ; SEARS, Russell: Benchmarking Cloud Serving Systems with YCSB. In: *1st ACM Symposium on Cloud Computing (SoCC)*, 2010, S. 143–154
- [Des14] DESROCHERS, Cameron. *moodycamel::ConcurrentQueue: A Fast Multi-Producer, Multi-Consumer Lock-Free Queue for C++*. <https://github.com/cameron314/concurrentqueue>. 2014
- [Dij65] DIJKSTRA, Edsger W.: Solution of a Problem in Concurrent Programming Control. In: *Communications of the ACM* 8 (1965), Nr. 9, S. 569
- [DPT⁺13] DEBRABANT, Justin ; PAVLO, Andrew ; TU, Stephen ; STONEBRAKER, Michael ; ZDONIK, Stan: Anti-Caching: A New Approach to Database Management System Architecture. In: *Proceedings of the VLDB Endowment* 6 (2013), Nr. 14, S. 1942–1953
- [Dre07] DREPPER, Ulrich: What Every Programmer Should Know About Memory / Red Hat, Inc. 2007. – Forschungsbericht. White paper
- [DSST89] DRISCOLL, James R. ; SARNAK, Neil ; SLEATOR, Daniel D. ; TARJAN, Robert E.: Making Data Structures Persistent. In: *Journal of Computer and System Sciences* 38 (1989), Nr. 1, S. 86–124
- [EB75] VAN EMDE BOAS, Peter: Preserving Order in a Forest in Less Than Logarithmic Time. In: *16th Annual Symposium on Foundations of Computer Science (FOCS)*, 1975, S. 75–84
- [Eva06] EVANS, Jason: A Scalable Concurrent malloc(3) Implementation for FreeBSD. In: *Proceedings of BSDCan 2006* BSDCan, 2006
- [FAK⁺12] FERDMAN, Michael ; ADILEH, Almutaz ; KOCBERBER, Onur ; VOLOS, Stavros ; ALISAFEE, Mohammad ; JEVDJIC, Djordje ; KAYNAK, Cansu ; POPESCU, Adrian D. ; AILAMAKI, Anastasia ; FALSAFI, Babak: Clearing the Clouds: A Study of Emerging Scale-out Workloads on Modern Hardware. In: *17th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2012, S. 37–48
- [FAKM14] FAN, Bin ; ANDERSEN, David G. ; KAMINSKY, Michael ; MITZENMACHER, Michael: Cuckoo Filter: Practically Better Than Bloom. In: *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT '14)*, ACM, 2014, S. 75–88
- [FB74] FINKEL, Raphael A. ; BENTLEY, Jon L.: Quad Trees: A Data Structure for Retrieval on Composite Keys. In: *Acta Informatica* 4 (1974), Nr. 1, S. 1–9
- [FJ98] FRIGO, Matteo ; JOHNSON, Steven G.: FFTW: An Adaptive Software Architecture for the FFT. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* Bd. 3, 1998, S. 1381–1384
- [FJKN20] FENT, Philipp ; JUNGMAIR, Michael ; KIPF, Andreas ; NEUMANN, Thomas: START: Self-Tuning Adaptive Radix Tree. In: *36th IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2020, S. 147–153

- [FLPR99] FRIGO, Matteo ; LEISERSON, Charles E. ; PROKOP, Harald ; RAMACHANDRAN, Sridhar: Cache-Oblivious Algorithms. In: *40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999, S. 285–297
- [Fre60] FREDKIN, Edward: Trie Memory. In: *Communications of the ACM* 3 (1960), Nr. 9, S. 490–499
- [FW93] FREDMAN, Michael L. ; WILLARD, Dan E.: Surpassing the Information Theoretic Bound with Fusion Trees. In: *Journal of Computer and System Sciences* 47 (1993), Nr. 3, S. 424–436
- [GHJV94] GAMMA, Erich ; HELM, Richard ; JOHNSON, Ralph ; VLISSIDES, John: *Design Patterns: Elements of Reusable Object-Oriented Software*. 1. Addison-Wesley, 1994
- [GL01] GRAEFE, Goetz ; LARSON, Per-Åke: B-Tree Indexes and CPU Caches. In: *Proceedings of the 17th IEEE International Conference on Data Engineering (ICDE)*, IEEE, 2001, S. 349–358
- [GL20] GRAF, Thomas M. ; LEMIRE, Daniel: Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters. In: *ACM Journal of Experimental Algorithmics* 25 (2020), Nr. 1, S. 1.5:1–1.5:16
- [GM05] GHEMAWAT, Sanjay ; MENAGE, Paul: TCMalloc: Thread-Caching Malloc / Google. 2005. – Forschungsbericht. Design documentation
- [GMUW09] GARCIA-MOLINA, Hector ; ULLMAN, Jeffrey D. ; WIDOM, Jennifer: *Database Systems: The Complete Book*. 2. Upper Saddle River, NJ, USA : Pearson Prentice Hall, 2009
- [Goo18] GOOGLE. *Abseil Swiss Tables* (flat_hash_map). <https://abseil.io/about/design/swisstable>. 2018
- [GS78] GUIBAS, Leo J. ; SEDGEWICK, Robert: A Dichromatic Framework for Balanced Trees. In: *19th Annual Symposium on Foundations of Computer Science (FOCS)*, 1978, S. 8–21
- [Gut84] GUTTMAN, Antonin: R-Trees: A Dynamic Index Structure for Spatial Searching. In: *ACM SIGMOD International Conference on Management of Data*, 1984, S. 47–57
- [HB15] HOEFLER, Torsten ; BELLI, Roberto: Scientific Benchmarking of Parallel Computing Systems: Twelve Ways to Tell the Masses When Reporting Performance Results. In: *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015
- [HBHR11] HERTER, Jörg ; BACKES, Peter ; HAUPENTHAL, Florian ; REINEKE, Jan: CAMA: A Predictable Cache-Aware Memory Allocator. In: *23rd Euromicro Conference on Real-Time Systems (ECRTS)*, 2011, S. 23–32
- [Her91] HERLIHY, Maurice: Wait-Free Synchronization. In: *ACM Transactions on Programming Languages and Systems* 13 (1991), Nr. 1, S. 124–149

- [HKT⁺21] HUNTER, A. H. ; KENNELLY, Chris ; TURNER, Paul ; GOVE, Darryl ; MOSELEY, Tipp ; RANGANATHAN, Parthasarathy: Beyond malloc Efficiency to Fleet Efficiency: A Hugepage-Aware Memory Allocator. In: *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021
- [Hol79] HOLM, Sture: A Simple Sequentially Rejective Multiple Test Procedure. In: *Scandinavian Journal of Statistics* 6 (1979), Nr. 2, S. 65–70
- [HP03] HANKINS, Richard A. ; PATEL, Jignesh M.: Effect of Node Size on the Performance of Cache-Conscious B+-Trees. In: *Proceedings of the 2003 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, ACM, 2003, S. 120–131
- [HP19] HENNESSY, John L. ; PATTERSON, David A.: *Computer Architecture: A Quantitative Approach*. 6th. Morgan Kaufmann, 2019
- [Int24a] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Optimization Reference Manual*. Intel Corporation, Santa Clara, CA, USA. 2024
- [Int24b] INTEL CORPORATION. *Intel 64 and IA-32 Architectures Software Developer's Manual, Volume 3A: System Programming Guide*. Intel Corporation, Santa Clara, CA, USA. 2024
- [ISO24] ISO/IEC. *ISO/IEC 14882:2024 — Programming Languages — C++*. International Organization for Standardization, Geneva, Switzerland. 2024
- [IZA⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; ATHANASSOULIS, Manos ; DAYAN, Niv ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi ; MAAS, Lukas M. ; QIN, Wilson ; WASAY, Abdul ; SUN, Yiyou: The Periodic Table of Data Structures. In: *IEEE Data Engineering Bulletin* 41 (2018), Nr. 3, S. 64–75
- [IZH⁺18] IDREOS, Stratos ; ZOUMPATIANOS, Kostas ; HENTSCHEL, Brian ; KESTER, Michael S. ; GUO, Demi: The Data Calculator: Data Structure Design and Cost Synthesis from First Principles, and Learned Cost Models. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 535–550
- [Jac89] JACOBSON, Guy J.: Space-Efficient Static Trees and Graphs. In: *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, IEEE, 1989, S. 549–554
- [Jan17] JANSSON, Mattias. *rpmalloc: A Public Domain License Per-thread Memory Allocator*. GitHub Repository. 2017
- [JTJE10] JALEEL, Aamer ; THEOBALD, Kevin B. ; JR., Simon C. S. ; EMER, Joel: High Performance Cache Replacement Using Re-Reference Interval Prediction (RRIP). In: *37th Annual International Symposium on Computer Architecture (ISCA)*, 2010, S. 60–71
- [KAS21] KOYLE, Neal ; AYYAPUREDDI, Sujeet ; SEIBERT, Mike: Micron DDR5: Client Module Features / Micron Technology, Inc. 2021 (CCM004676576390-11566, Rev. A). – White Paper. Online-Ressource

- [Kha11] KHAN, Minhaj A.: Data Cache Prefetching With Dynamic Adaptation. In: *The Computer Journal* 54 (2011), Nr. 5, S. 815–823
- [KM17a] KHUONG, Paul-Virak ; MORIN, Pat: Array Layouts for Comparison-Based Searching. In: *ACM Journal of Experimental Algorithmics* 22 (2017), S. 1.3:1–1.3:39
- [KM17b] KHUONG, Paul-Virak ; MORIN, Pat: Array Layouts for Comparison-Based Searching. In: *ACM Journal of Experimental Algorithmics* 22 (2017)
- [KMR⁺19] KIPF, Andreas ; MARCUS, Ryan ; VAN RENEN, Alexander ; STOIAN, Mihail ; KEMPER, Alfons ; KRASKA, Tim ; NEUMANN, Thomas: SOSD: A Benchmark for Learned Indexes. In: *NeurIPS Workshop on Machine Learning for Systems*, 2019
- [Kno65] KNOWLTON, Kenneth C.: A Fast Storage Allocator. In: *Communications of the ACM* 8 (1965), Nr. 10, S. 623–625
- [KNR11] KREPS, Jay ; NARKHEDE, Neha ; RAO, Jun: Kafka: A Distributed Messaging System for Log Processing. In: *6th International Workshop on Networking Meets Databases (NetDB)*, 2011, S. 1–7
- [Knu98] KNUTH, Donald E.: *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2nd. Addison-Wesley, 1998
- [Kue23] KUEHN, Lena: Towards Data-Based Cache Optimization of B+-Trees. In: *Proceedings of the 19th International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2023, S. 8:1–8:8
- [LABN16] LEIS, Viktor ; ANDERSEN, Michael ; BIRKEDAL, Erik ; NEUMANN, Thomas: The ART of Practical Synchronization. In: *Proceedings of the International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2016, S. 8:1–8:8
- [Lam77] LAMPORT, Leslie: Concurrent Reading and Writing. In: *Communications of the ACM* 20 (1977), Nr. 11, S. 806–811
- [LC86] LEHMAN, Tobin J. ; CAREY, Michael J.: A Study of Index Structures for Main Memory Database Management Systems. In: *12th International Conference on Very Large Data Bases (VLDB)*, 1986, S. 294–303
- [Lc21] LEIJEN, Daan ; CONTRIBUTORS. *mimalloc-bench: Suite for Benchmarking Malloc Implementations*. <https://github.com/daanx/mimalloc-bench>. 2021
- [LDPB19] LIETAR, Paul ; DREYER, Derek ; PARKINSON, Adam ; BORNAT, Richard: snmalloc: A Message Passing Allocator. In: *Proceedings of the 2019 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, ACM, 2019, S. 122–135
- [Lea96] LEA, Doug. *A Memory Allocator*. Design article and C implementation. 1996
- [LJY25] LI, Ruihao ; JOHN, Lizy K. ; YADWADKAR, Neeraja J.: Old is Gold: Optimizing Single-Threaded Applications with Exgen-Malloc. In: *IEEE Computer Architecture Letters* 24 (2025), Nr. 2

- [LKN13] LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas: The Adaptive Radix Tree: ARTful Indexing for Main-Memory Databases. In: *29th IEEE International Conference on Data Engineering (ICDE)*, 2013, S. 38–49
- [LLS13] LEVANDOSKI, Justin J. ; LOMET, David B. ; SENGUPTA, Sudipta: The Bw-Tree: A B-tree for New Hardware Platforms. In: *29th IEEE International Conference on Data Engineering (ICDE 2013), Brisbane, Australia, April 8-12, 2013*, IEEE Computer Society, 2013, S. 302–313
- [LNOM08] LINDHOLM, Erik ; NICKOLLS, John ; OBERMAN, Stuart ; MONTRYM, John: NVIDIA Tesla: A Unified Graphics and Computing Architecture. In: *IEEE Micro* 28 (2008), Nr. 2, S. 39–55
- [LR18] LEITE, Ricardo ; ROCHA, Rui: LRMalloc: A Modern and Competitive Lock-Free Dynamic Memory Allocator. In: *Proceedings of the 13th International Conference on Parallel Processing and Applied Mathematics (VECPAR)*, Springer, 2018, S. 188–200
- [LYTL19] LI, Tianlin ; YAO, Yiping ; TANG, Wenjie ; LIN, Zhongwei: HMalloc: A Hybrid, Scalable, and Lock-Free Memory Allocator for Multi-Threaded Applications. In: *2019 IEEE 25th International Conference on Parallel and Distributed Systems (ICPADS)*, 2019, S. 406–409
- [LZM19] LEIJEN, Daan ; ZORN, Benjamin ; DE MOURA, Leonardo: Mimalloc: Free List Sharding in Action. In: *Proceedings of the 17th Asian Symposium on Programming Languages and Systems (APLAS)*, Springer, 2019, S. 172–192
- [LZS⁺23] LUO, Xuchuan ; ZUO, Pengfei ; SHEN, Jiacheng ; GU, Jiazhen ; WANG, Xin ; LYU, Michael R. ; ZHOU, Yangfan: SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory. In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023
- [MBL25] MÜLLER, Marcus ; BENSON, Lawrence ; LEIS, Viktor: B-Trees Are Back: Engineering Fast and Pageable Node Layouts. In: *Proceedings of the ACM on Management of Data* 3 (2025), Nr. 1
- [McK01] MCKENNEY, Paul E.: Read-Copy Update. In: *Proceedings of the 2001 Ottawa Linux Symposium (OLS)*, 2001, S. 338–367
- [Me110] DE MELO, Arnaldo C.: The New Linux perf Tools. In: *Linux Kongress*, 2010, S. 1–42
- [MF02] MANZINI, Giovanni ; FERRAGINA, Paolo: Engineering a Lightweight Suffix Array Construction Algorithm. In: *Algorithms — ESA 2002, 10th Annual European Symposium Bd. 2461*, Springer, 2002, S. 698–710
- [Mic04a] MICHAEL, Maged M.: Hazard Pointers: Safe Memory Reclamation for Lock-Free Objects. In: *IEEE Transactions on Parallel and Distributed Systems* 15 (2004), Nr. 6, S. 491–504
- [Mic04b] MICHAEL, Maged M.: Scalable Lock-Free Dynamic Memory Allocation. In: *Proceedings of the ACM SIGPLAN 2004 Conference on Programming Language Design and Implementation (PLDI)*, ACM, 2004, S. 35–46

- [MKM12] MAO, Yandong ; KOHLER, Eddie ; MORRIS, Robert T.: Cache Craftiness for Fast Multicore Key-Value Storage. In: *7th ACM European Conference on Computer Systems (EuroSys)*, 2012, S. 183–196
- [Mor68] MORRISON, Donald R.: PATRICIA — Practical Algorithm To Retrieve Information Coded in Alphanumeric. In: *Journal of the ACM* 15 (1968), Nr. 4, S. 514–534
- [MPU⁺23] MINERVINI, Francesco ; PALOMAR, Oscar ; UNSAL, Osman ; REGGIANI, Enrico ; QUIROGA, Josue ; MARIMON, Joan [u. a.]: Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications. In: *ACM Transactions on Architecture and Code Optimization* 20 (2023), Nr. 2
- [MS96] MICHAEL, Maged M. ; SCOTT, Michael L.: Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In: *Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, ACM, 1996, S. 267–275
- [MW47] MANN, Henry B. ; WHITNEY, Donald R.: On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. In: *The Annals of Mathematical Statistics* 18 (1947), Nr. 1, S. 50–60
- [MWR25] MAHLING, Fabian ; WEISGUT, Marcel ; RABL, Tilmann: Fetch Me If You Can: Evaluating CPU Cache Prefetching and Its Reliability on High Latency Memory. In: *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN)*, ACM, 2025
- [NR21] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: Brief Announcement: Crystalline: Fast and Memory Efficient Wait-Free Reclamation. In: *35th International Symposium on Distributed Computing (DISC)* Bd. 209, 2021, S. 60:1–60:4
- [NR24] NIKOLAEV, Ruslan ; RAVINDRAN, Binoy: A Family of Fast and Memory Efficient Lock-and Wait-Free Reclamation. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. PLDI
- [NTSA16] NADERAN-TAHAN, Mahmood ; SARBAZI-AZAD, Hamid: Why Does Data Prefetching Not Work for Modern Workloads? In: *The Computer Journal* 59 (2016), Nr. 2, S. 244–259
- [OCGO96] O’NEIL, Patrick E. ; CHENG, Edward ; GAWLICK, Dieter ; O’NEIL, Elizabeth J.: The Log-Structured Merge-Tree (LSM-Tree). In: *Acta Informatica* 33 (1996), Nr. 4, S. 351–385
- [PIA78] PERL, Yehoshua ; ITAI, Alon ; AVNI, Haim: Interpolation Search - A Log Log N Search. In: *Communications of the ACM* 21 (1978), Nr. 7, S. 550–553
- [PR04] PAGH, Rasmus ; RODLER, Flemming F.: Cuckoo Hashing. In: *Journal of Algorithms* 51 (2004), Nr. 2, S. 122–144
- [Pug90] PUGH, William W.: Skip Lists: A Probabilistic Alternative to Balanced Trees. In: *Communications of the ACM* 33 (1990), Nr. 6, S. 668–676

- [QJP⁺07] QURESHI, Moinuddin K. ; JALEEL, Aamer ; PATT, Yale N. ; JR., Simon C. S. ; EMER, Joel: Adaptive Insertion Policies for High Performance Caching. In: *34th Annual International Symposium on Computer Architecture (ISCA)*, 2007, S. 381–391
- [Rei05] REINDERS, James: *VTune Performance Analyzer Essentials: Measurement and Tuning Techniques for Software Developers*. 1. Hillsboro, OR, USA : Intel Press, 2005. – ISBN 978–0–9743649–5–7
- [RFP24a] REITZ, Antonin ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the ACM on Programming Languages* 8 (2024), Nr. OOPSLA2
- [RFP⁺24b] REITZ, Chris ; FROMHERZ, Aymeric ; PROTZENKO, Jonathan ; CHEN, Boniface ; FIEDLER, Talia ; BICHHAWAT, Ananya ; ALHANAHNAH, Dalia ; OU, Elaine ; BHARGAVAN, Karthik-ya: StarMalloc: Verifying a Modern, Hardened Memory Allocator. In: *Proceedings of the 2024 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, ACM, 2024. – Also available as arXiv:2403.09435
- [RIS21] RISC-V INTERNATIONAL. *RISC-V V Vector Extension, Version 1.0*. https://docs.riscv.org/reference/isa/extensions/vector/_attachments/riscv-v-spec.pdf. 2021
- [RLK⁺18] VAN RENEN, Alexander ; LEIS, Viktor ; KEMPER, Alfons ; NEUMANN, Thomas ; HASHIDA, Takushi ; OE, Kazuichi ; DOI, Yoshiyasu ; HARADA, Lilian ; SATO, Mitsuru: Managing Non-Volatile Memory in Database Systems. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 1541–1555
- [RR99] RAO, Jun ; ROSS, Kenneth A.: Cache Conscious Indexing for Decision-Support in Main Memory. In: *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, 1999, S. 78–89
- [RR00] RAO, Jun ; ROSS, Kenneth A.: Making B+-Trees Cache Conscious in Main Memory. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, ACM, 2000, S. 475–486
- [SBB⁺17] STEPHENS, Nigel ; BILES, Stuart ; BOETTCHER, Matthias ; EYOLE, Mbou ; GABRIELLI, Giacomo ; HORSNELL, Matt ; MAGKLIS, Grigorios ; MARTINEZ, Alejandro ; PREMILLIEU, Nathanael ; REID, Alastair ; RICO, Alejandro ; WALKER, Paul: The ARM Scalable Vector Extension. In: *IEEE Micro* 37 (2017), Nr. 2, S. 26–39
- [SDF⁺00] SRINIVASAN, Jagannathan ; DAS, Souripriya ; FREIWALD, Chuck ; CHONG, Eugene I. ; JAGANNATH, Mahesh ; YALAMANCHI, Aravind ; KRISHNAN, Ramkumar ; TRAN, Anh-Tuan ; DEFAZIO, Samuel ; BANERJEE, Jayanta: Oracle8i Index-Organized Table and its Application to New Domains. In: *26th International Conference on Very Large Data Bases (VLDB)*, 2000, S. 285–296

- [SGL09] SCHLEGEL, Benjamin ; GEMULLA, Rainer ; LEHNER, Wolfgang: k-Ary Search on Modern Processors. In: *Proceedings of the Fifth International Workshop on Data Management on New Hardware (DaMoN 2009)*, ACM, 2009, S. 52–60
- [SKK⁺25] SCHMIDT, Lennart ; KÜHN, Roland ; KRAUSE, Matti ; TEUBNER, Jens ; LEHNER, Wolfgang ; HABICH, Dirk: To Stride or Not to Stride the Memory Access? In: *Proceedings of the 3rd Workshop on Disruptive Memory Systems (DIMES)*, ACM, 2025, S. 19–26
- [SPB05] SAMUEL, Curt ; PEDERSEN, Torben B. ; BONNET, Philippe: Processor-Conscious Indexing. In: *Proceedings of the 2005 ACM SIGMOD International Conference on Management of Data*, ACM, 2005, S. 235–246
- [SSL⁺21] SCHMEISSER, Josef ; SCHÜLE, Maximilian E. ; LEIS, Viktor ; NEUMANN, Thomas ; KEMPER, Alfons: The B²-Tree: Cache-Friendly String Indexing within B-Trees. In: *Datenbanksysteme für Business, Technologie und Web (BTW)*, 2021, S. 39–58
- [SSS08] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Binary Trees. In: *Fifth IFIP International Conference on Theoretical Computer Science (TCS 2008)* Bd. 273, Springer, 2008
- [SSS16] SAIKKONEN, Riku ; SOISALON-SOININEN, Eljas: Cache-Sensitive Memory Layout for Dynamic Binary Trees. In: *The Computer Journal* 59 (2016), Nr. 5, S. 630–649
- [Sta17] STANDARD PERFORMANCE EVALUATION CORPORATION. *SPEC CPU 2017 Benchmark Suite*. <https://www.spec.org/cpu2017/>. 2017
- [Sto81] STONEBRAKER, Michael: Operating System Support for Database Management. In: *Communications of the ACM* 24 (1981), Nr. 7, S. 412–418
- [ȚCH02] ȚĂPUȘ, Cristian ; CHUNG, I-Hsin ; HOLLINGSWORTH, Jeffrey K.: Active Harmony: Towards Automated Performance Tuning. In: *ACM/IEEE Conference on Supercomputing (SC)*, 2002, S. 1–11
- [Ten12] TENE, Gil. *HdrHistogram: A High Dynamic Range (HDR) Histogram*. <https://github.com/HdrHistogram/HdrHistogram>. 2012
- [TFB⁺11] THOMPSON, Martin ; FARLEY, Dave ; BARKER, Michael ; GEE, Patricia ; STEWART, Andrew: Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads / LMAX. 2011. – Technical Paper. Version 1.0
- [TMS⁺16] TIBBA, Ghizlane ; MALZ, Christoph ; STOERMER, Christoph ; NAGARAJAN, Natarajan ; ZHANG, Licong ; CHAKRABORTY, Samarjit: Testing Automotive Embedded Systems under X-in-the-Loop Setups. In: *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016, S. 1–8
- [Tra24] TRANSACTION PROCESSING PERFORMANCE COUNCIL. *TPC Benchmark C and TPC-DS Specifications*. Transaction Processing Performance Council, San Francisco, CA, USA. 2024

- [UHL⁺17] UNGETHUEM, Sigmund ; HABICH, Dirk ; LEHNER, Wolfgang [u. a.]: Hardware Acceleration for Database Systems: A Comprehensive Survey. In: *ACM Computing Surveys* 50 (2017), Nr. 1, S. 3:1–3:35
- [VIA26] VIA RESEARCH: PIM-malloc: A Fast and Scalable Dynamic Memory Allocator for Processing-In-Memory Systems. In: *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2026. – Accepted to HPCA 2026; also available as arXiv:2505.13002 (2025)
- [VRM⁺18] VIETRI, Giuseppe ; RODRIGUEZ, Liana V. ; MARTINEZ, Wendy A. ; LYONS, Steven ; LIU, Jason ; RANGASWAMI, Raju ; ZHAO, Ming ; NARASIMHAN, Giri: Driving Cache Replacement with ML-based LeCaR. In: *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2018
- [Vyu11] VYUKOV, Dmitry. *Bounded MPMC Queue*. <https://www.1024cores.net/home/lock-free-algorithms/queues/bounded-mpmc-queue>. 2011
- [Wil64] WILLIAMS, J. W. J.: Algorithm 232: Heapsort. In: *Communications of the ACM* 7 (1964), Nr. 6, S. 347–348
- [Wil83] WILLARD, Dan E.: Log-Logarithmic Worst-Case Range Queries are Possible in Space $\Theta(N)$. In: *Information Processing Letters* 17 (1983), Nr. 2, S. 81–84
- [WNJ19] WU, Xingbo ; NI, Fan ; JIANG, Song: Wormhole: A Fast Ordered Index for In-Memory Data Management. In: *14th ACM European Conference on Computer Systems (EuroSys)*, 2019
- [WPD01] WHALEY, R. C. ; PETITET, Antoine ; DONGARRA, Jack J.: Automated Empirical Optimization of Software and the ATLAS Project. In: *Parallel Computing* 27 (2001), Nr. 1–2, S. 3–35
- [Xu24] XU, Chuanqi: Modules ABI Requirement / ISO/IEC JTC1/SC22/WG21. 2024 (P3092R0). – Forschungsbericht. Online-Ressource
- [YZZ⁺23] YANG, Hanmei ; ZHAO, Xin ; ZHOU, Jin ; WANG, Wei ; KUNDU, Sandip ; WU, Bo ; GUAN, Hui ; LIU, Tongping: NUMAlloc: A Faster NUMA Memory Allocator. In: *2023 ACM SIGPLAN International Symposium on Memory Management (ISMM)*, 2023
- [ZDL⁺13] ZAHARIA, Matei ; DAS, Tathagata ; LI, Haoyuan ; HUNTER, Timothy ; SHENKER, Scott ; STOICA, Ion: Discretized Streams: Fault-Tolerant Streaming Computation at Scale. In: *24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013, S. 423–438
- [ZGH⁺25] ZHANG, Tingji ; GROT, Boris ; HE, Wenjian ; LV, Yashuai ; QU, Peng ; SU, Fang ; WANG, Wenxin ; ZHANG, Guowei ; ZHANG, Xuefeng ; ZHANG, Youhui: Hierarchical Prefetching: A Software-Hardware Instruction Prefetcher for Server Applications. In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 2, ACM, 2025, S. 529–544

- [ZGV⁺24] ZHOU, Zhuangzhuang ; GOGTE, Vaibhav ; VAISH, Nilay ; KENNELLY, Chris ; XIA, Patrick ; KANEV, Svilen ; MOSELEY, Tipp ; DELIMITROU, Christina ; RANGANATHAN, Parthasarathy: Characterizing a Memory Allocator at Warehouse Scale. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, S. 192–206
- [ZL77] ZIV, Jacob ; LEMPEL, Abraham: A Universal Algorithm for Sequential Data Compression. In: *IEEE Transactions on Information Theory* 23 (1977), Nr. 3, S. 337–343
- [ZLL⁺18] ZHANG, Huanchen ; LIM, Hyeontaek ; LEIS, Viktor ; ANDERSEN, David G. ; KAMINSKY, Michael ; KEETON, Kimberly ; PAVLO, Andrew: SuRF: Practical Range Query Filtering with Fast Succinct Tries. In: *ACM SIGMOD International Conference on Management of Data*, 2018, S. 323–336
- [ZSZ⁺24] ZHANG, Qian ; SONG, Haoyun ; ZHOU, Kaiyan ; WEI, Jianhao ; XIAO, Chuqiao: A Prefetching Indexing Scheme for In-Memory Database Systems. In: *Future Generation Computer Systems* 156 (2024), S. 179–190

Abbildungsverzeichnis

1.1	Die „Cache-Wand“: die Zugriffslatenz wächst über die Speicher-Hierarchie um rund zwei Größenordnungen — nicht die Operationszahl, sondern das Cache-Verhalten bestimmt die Laufzeit.	3
1.2	Das Trennbarkeits-Problem: als Monolith bleibt unklar, welche Entwurfsentscheidung wirkt; erst die Zerlegung in orthogonale Achsen macht jeden Beitrag isoliert messbar. .	4
2.1	Die Landschaft der Suchstrukturen: fünf Klassen, die das vorgeschlagene Achsen-System gemeinsam erfasst.	9
2.2	Der Entwurfsraum: die Achsen spannen ein kartesisches Produkt auf; ein konkreter Algorithmus ist ein Punkt darin. Zur Veranschaulichung sind drei der neunzehn Achsen gewählt — das reale Speicherorgan aus Knotentyp (T4), Speicher-Layout (T5) und Allokator (T6); der volle Entwurfsraum hat neunzehn Dimensionen.	9
2.3	Galerie der wichtigsten rekonstruierten Suchstrukturen: je Verfahren (mit Paper-ID) seine Herkunfts-Klasse, die charakteristische Struktur-Idee und die Achsen, an denen es seinen Hauptbeitrag leistet. Trie-, Hybrid-, succinct- und cache-sensitive Verfahren stehen neben einer nicht-baumartigen Hash-Gegenprobe (SwissTable) — alle als Punkte im selben Achsen-Entwurfsraum. Die vollständige Achsen-Sezierung führt Tabelle 3.2.	12
2.4	Cache-Line-Auslastung: eine Cache-Line wird stets vollständig geladen; werden nur wenige ihrer Bytes genutzt, sinkt die effektive Bandbreite — cache-bewusstes Layout hebt die Auslastung.	12
2.5	Die Anatomie-Brücke: wie ein Lebewesen aus zusammenwirkenden Organen besteht, ist ein Suchalgorithmus eine Komposition von Achsen-Organen; das Mess-System übernimmt die Rolle der medizinischen Diagnostik und vermisst jedes Organ einzeln. . . .	14
2.6	Synthese Stand der Technik → Achsen: die Entwurfsbausteine der bekannten Verfahren werden in orthogonale Achsen zerlegt; jede Achse sammelt die Beiträge des gesamten Korpus — die Umkehrung von Abbildung 2.2 (dort ist ein Algorithmus ein Punkt, hier eine Achse ein Sammelbecken).	15
2.7	Achse = Organ, Anatomie = Verdrahtung: das Speicher-Organ (Komposition aus Knotentyp, Layout und Allokator) exportiert ein generalisiertes <code>StorageOrgan</code> -Interface, über das die Suche läuft (durchgezogen, <i>verpflichtend</i>); weitere Organe wie die Telemetrie <i>können</i> es über optionale Beobachter-Hooks nutzen (gestrichelt, <i>optional</i>), müssen aber nicht.	15

- 2.8 Inter-Organ-Nutzungs-Graph („welche Achse nutzt welche“): Es gibt genau *einen* kanonischen Hub — das Speicher-Organ `LayoutAwareChunkedStore`, das Knoten-typ (T4), Layout (T5) und Allokator (T6) komponiert. *Verpflichtend* (durchgezogen) konsumiert der Store diese drei Achsen, und die Suche (T0) läuft über ihn. Alle übrigen Kopplungen sind *optional* (gestrichelt): Prefetch (T7) und Telemetrie (T10) lesen den Store nur über `requires`-gegate Beobachter-Hooks — eine Achse *kann* fremde Interfaces nutzen, *muss* aber nicht. Die Kopplung der Nebenläufigkeit (T8) an den Store ist im Code (noch) nicht verdrahtet (gepunktet). 16
- 2.9 Das Drei-Ebenen-Modell: Gattung (Ebene 1, Außen-Interface) → Tier-Unterklasse (Ebene 2, fester Achsen-Satz) → Organe/Achsen (Ebene 3). Die `SearchAlgorithm`-Gattung trägt das „Säugetier“ mit den neunzehn Achsen; die Container-Gattung die Unterklassen `Set/Sequence/Adapter/View`. 16
- 2.10 Die drei Gattungen und die Tier-Hierarchie bis zum kompilierten Binary 17
- 2.11 Entwurfsmuster als Bausteine der Architektur: oben die im Code durchgängig verwendeten klassischen GoF-Muster mit ihrer jeweiligen Rolle, in der Mitte die zwei in dieser Arbeit benannten neuen Übersetzungszeit-Muster, unten die code-seziert *Drei-Stufen-Concept-Kette* je Achse (Topic-Concept → Axis-Concept → CacheEngine-Permutations-Concept → CRTP-Basis → `mp_list`-Registry) als Verfeinerung von *Strategy/CRTP* — wobei der *literale* `std::variant` nicht hier, sondern nur im elf-achsigen `baustein_variants`-Fallback-Modell auftritt. Ihre Komposition bildet die design-pattern-getriebene, mess-orientierte Architektur (Abschnitt 2.3). 18
- 2.12 Die *eine* Architektur als einzige Hierarchie ohne Parallel-Bäume. An der Wurzel steht `IExecutionEngine` als *Treiber der Messung* — der universelle messbare Vertrag (`warm_up/run/reset/shutdown`) über alle drei Engine-Arten Anatomy, Virus und Hybrid; unter ihm verzweigen die anatomie-tragenden *Gattungen* (`SearchAlgorithm`, Container, Graph) und die achsenlosen *Viren*. `SearchAlgorithm` ist das fokale Lebewesen (Säugetier); seine Anatomie ist sein *Körper* — die feste Komposition *und Verdrahtung* der 19 Organe —, und „SearchEngine“ nur dessen ABI-Laufzeit-Sicht. Der Säugetier-Ast ist bis zum Individuum ausgezogen: eine konkrete Achsen-Composition (Ebene 4) wird je Permutation als eigenständige Tier-Binary kompiliert (Ebene 5). Container (schlüsselloser Speicher) und Graph tragen eigene Organsätze. 19
- 2.13 Die ABI-stabile C-Modulgrenze und die variadische Hybrid-API. Komplexe Schlüssel werden von einer Fingerprint-Komponente auf 16-Byte-Binärstrings fester Länge reduziert. 19
- 2.14 Das Mess-Trade-off-Dreieck: jede Achsen-Permutation spannt sich gegen die drei gegenläufigen Mess-Komponenten Latenz, Durchsatz und Speicherverbrauch auf und ist ein Punkt in seinem Inneren (vgl. den Entwurfsraum in Abbildung 2.2, wo dieselbe Permutation ein Punkt ist). Die für cache-bewusste Suchstrukturen eigentlich zentrale vierte Komponente — das Verhalten auf Knoten-, Daten- und Index-Ebene über Hardware-Performance-Zähler — ist im aktuellen Implementierungsstand noch nicht erhoben (gestrichelt). 21

- 2.15 UML-Landkarte der Code-Interfaces: links die Engine-/Anatomie-Hierarchie — die messbare Wurzel `IExecutionEngine` als *Treiber der Messung* (`Anatomy ∪ Virus ∪ Hybrid`) über `IAnatomyBase` und der achsenlosen `IVirus-ExecutionEngine`, darunter der *Körper* `SearchAlgorithmAnatomy<C>` (Komposition und Verdrahtung der neunzehn Achsen) und seine ABI-Laufzeit-Sicht `SearchAlgorithmAbiAdapter<A>` („SearchEngine“), die über die Treiber-/Beobachter-Verträge `IDriveableTier/IObservableTier` gemessen wird; rechts das je Achse wiederholte Implementierungs-Muster — ein zweischichtiges Concept, eine requires-gesicherte CRTP-Basis, die `mp_list`-Baustein-Familie und die Übersetzungszeit-Auflösung `resolve_baustein`, die je Achse zwischen Prüfling-Slot und Cache-Engine-Standard wählt. 21
- 3.1 Das M-Modell mit Fokus auf die Build- und Mess-Phasen. Die äußere `messung_driver`-Schleife (WAS, Diplomarbeit-Code) speist die Konfigurationen der drei Messreihen A/B/C ein; der `CacheEngineBuilder` (WIE, Bibliothek) zerfällt in die beiden eigentlich interessanten Phasen: **BUILD** (`BuildOrchestrator/ExperimentTree` baut je Konfiguration eine eigenständige Tier-Binary, resumierbar und RAM-gewahr) und **MEASURE** (`perm_runner` treibt und misst jede Tier-Binary einzeln über die Beobachter-Schnittstelle `IObservableTier`). Die `CacheEngine` liefert die Standard-Achsen-Bausteine, der Prüfling PRT-ART seine eigenen Organe; ihre bidirektionale Beziehung ((a) instantiiert, (b) ruft Dienste) trägt die Multi-Prüfling-Fähigkeit. Die `ExecutionEngine` bleibt die gemeinsame messbare Wurzel. 31
- 3.2 Die Drei-Stufen-Prüfung des Builders: jede Stufe erzeugt nicht-redundante Permutations-Binaries; ihre Zuordnung zu den Messreihen A/B/C zeigt Tabelle 3.8. 31
- 3.3 PRT-ART als Demonstrator im M-Modell: Der Prüfling überschreibt im Mess-Apparat sechs Slots — die Build-Achse Page-Type sowie die fünf Achsen-Organe Pfadkompression (Redirect), Wert-Handle (ChainRef), Prefetch, OLC-Nebenläufigkeit und Telemetrie — und bezieht die übrigen vierzehn Kompositions-Achsen über den Compile-Time-Fallback aus der Cache-Engine; seine weiteren eigenen Bausteine (4+1+2-Pool-Allokator, MultiLevel-Layout) laufen im Mess-Pfad derzeit über den Cache-Engine-Standard (Anbindung: Folgeschritt E6). So entsteht ein vollständiges, mess-fähiges Lebewesen aus wenigen eigenen plus vielen Standard-Organen. 33
- 3.4 Die Heuristik-Schleife: aus den Messreihen wird je Lastprofil die beste Achsen-Komposition gelernt, als XML-Lastprofil abgelegt und über den Workload-Filter in die nächste Messung zurückgeführt. 33

4.1	Galerie der sechs eigenen Schlüsselstellen von PRT-ART, je an ihrer Achse: ein 4+1+2-Pool-Allokator (T6), optimistisches Lock-Coupling mit reservierten Wert-Blöcken (T8), pfad-orientiertes Bundle-Prefetch (T7), ein MultiLevel-Layout (T5) und cache-kohärenz-bewusste Telemetrie (T10) samt DensityTracker und den Hypothesen-Metriken H1/H2/H3. Die Kacheln zeigen die <i>Bausteine des Prüfling-Repos</i> ; im Apparat überschrieben sind derzeit T7/T8/T10, die Bausteine zu T5/T6 laufen dort noch über den Cache-Engine-Standard (E6). Die übrigen vierzehn Kompositions-Achsen bezieht der Prüfling über den Compile-Time-Fallback aus der Cache-Engine.	36
4.2	Die Mess-Pipeline in zwei Ausprägungen. Pfad A (oben) ist die in-process Sieben-Phasen-Pipeline des <code>ExperimentDriver</code> je Messreihe (Enumerate → Codegen → Compile → Load → Execute → Measure → Persist; zwei opt-in-Phasen kompilieren fehlende Module hot und verifizieren den ABI-Vertrag). Hervorgehoben sind die eigentlich substanziellen Phasen: COMPILE (der <i>Build</i> der Tier-Binaries) und EXECUTE/MEASURE (die <i>Messung</i>); Codegen/Load/Persist sind reines Plumbing. Pfad B (unten, der empfohlene Weg) trennt beide Aufgaben out-of-process: der <code>BuildOrchestrator</code> baut <i>zuerst alle</i> Tier-Binaries (resumierbar, RAM-gewahr), danach lädt und misst <code>perm_runner</code> jede Binary einzeln (eine DLL je Prozess) über die Beobachter-Schnittstelle <code>IObservableTier</code>	38
4.3	Galerie der neunzehn Kompositions-Achsen (T0–T18): je Achse das Implementierungskonzept — Kurzname, Sub-Achsen-Familie und eine repräsentative Auswahl der austauschbaren Bausteine. Jede Achse ist ein <code>Concept</code> -Interface mit mehreren Strategie-Bausteinen (<i>Strategy</i> -Muster); eine konkrete Composition belegt je Slot genau einen. Der <i>Standard</i> („Std.“) ist eine abgeleitete Vergleichs-Baseline, kein Code-Flag; bei den großen Achsen T0/T6/T17 ist die Gesamt-Bausteinanzahl in Klammern angegeben. Die vollständige Matrix mit Herkunfts-Quellen führt Anhang D.	39
A.1	Zyklen je Lebewesen-Permutation (v5_pipeline_demo)	71
A.2	Zyklen je Lebewesen-Permutation (cartesian_smoke43)	76
A.3	Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)	85
A.4	Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	86
A.5	Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	87
A.6	Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	88
A.7	Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	89
A.8	Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)	90

Tabellenverzeichnis

3.1	Dialektische Aneignung der Stand-der-Technik-Instanzen je Kompositions-Achse (in Achsen-Reihenfolge T0–T18; Cluster-Kürzel: A Trie/Knoten, B Hybrid/Layout, C Allokation, D Prefetching, E Synchronisation, F Hardware/Messung/Engineering)	24
3.2	Achsen-Sezierung der Suchverfahren: neunzehn Kompositions-Achsen (T0–T18) und drei Build-Achsen; je Achse die ein Organ beitragenden Quellen.	26
3.3	Allokator-Profil mit Lizenz und Workload-Affinität	26
3.4	In dieser Arbeit zugewiesene Ziel-Hardware- und Scheduling-Profil je rekonstruiertem SOTA-Verfahren (Auswahl; die Profile sind Rekonstruktions-Ziele dieser Arbeit, nicht zwingend Original-Paper-Eigenschaften; der Vollkatalog folgt im Anhang).	27
3.5	Benchmark-Frameworks und nutzende Veröffentlichungen	28
3.6	Lastprofil-Katalog (14 Profile, aus der 33-Paper-Analyse abgeleitet)	28
3.7	SOTA-Algorithmus-Profil mit Workload-Affinität	29
3.8	Zuordnung der Drei-Stufen-Prüfung zu den Pflicht-Messreihen	32
5.1	Profil-bewusstes Workload-Routing	41
5.2	Reale Datensätze und ihre Charakteristik	42
A.1	Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)	69
A.2	Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)	70
A.3	Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)	70
A.4	Achsen-Austauschbarkeit: search_algo (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	71
A.5	Achsen-Austauschbarkeit: node_type (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: Vorbehalt)	72
A.6	Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 640 Geschwister-Paare; Konfundierung: Vorbehalt)	76
A.7	Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)	82
A.8	Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)	83
F.1	Vollständige <code>std::map</code> -Schnittstelle (C++23)	133
F.2	Vollständige <code>std::vector</code> -Schnittstelle (C++23)	136
F.3	Zusammengesetzte Funktionen („unter der Haube“)	138

A Vollständige Messreihen

Dieser Anhang enthält die vollständigen Messreihen der vorkonfigurierten Lebewesen-Permutationen. Tabelle A.1 listet die je Permutation tabellierten Kennzahlen (Operationen; Taktzyklen approximiert; Cache-Misses L1–L3 derzeit 0/nicht erhoben, vgl. Limitierung 1 in Abschnitt A.5); Abbildung A.1 stellt die Taktzyklen grafisch gegenüber.

Permutation	Ops	Zyklen	L1	L2	L3
ArtComposition_0	2000	825053	0	0	0
HotComposition_1	2000	740176	0	0	0
StartComposition_2	2000	805750	0	0	0
SurfComposition_3	2000	873976	0	0	0
WormholeComposition_4	2000	696350	0	0	0
MasstreeComposition_5	2000	770871	0	0	0

Tabelle A.1: Messreihe v5_pipeline_demo: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)

A.1 Kartesische Smoke-Messreihe (43 Permutationen, echte DLL-Messung)

Die folgende Messreihe entstand aus dem vollständigen Ketten-Durchlauf: die gebauten Permutations-DLLs (43 Kombinationen in zwei Teilblöcken: 27 kartesische SIMD×Layout×Allokator-Kombinationen plus 16 weitere über Node-Breite, Pfadkompression, interne Suche und Telemetrie (2⁴) — in Summe sieben variierte der neunzehn Achsen) wurden am Prüf-Dock gemessen, als binäre Mess-Records persistiert, via Pipeline-Stufe 03 in das 16-Spalten-Schema konvertiert und automatisch hier eingebunden. Tabelle A.2 und Abbildung A.2 zeigen die Ergebnisse.

A.2 Bias-Bruch-Matrix

Tabelle A.3 fasst die je Suchverfahren und Lastprofil gemessene Median-Latenz (ns/op, nearest-rank) über alle dynamischen Einstellungen und Wiederholungen zusammen; berücksichtigt sind ausschließlich die als two_phase_valid markierten Läufe. Die Matrix macht sichtbar, bei welchen Lastprofilen das jeweilige Verfahren seinen systematischen Bias bricht.

A.3 Latenz-Surfaces je Schnittstellen-Funktion

Die folgenden sechs Heatmaps stellen die Latenz-Oberfläche (Suchalgorithmus × Lastprofil) je Schnittstellen-Funktion dar: die Gesamt-Latenz (ns_per_op) sowie die p50-Latenzen der Einzelopera-

Permutation	Ops	Zyklen	L1	L2	L3
avx2_aos_jemalloc	1000	8025	0	0	0
avx2_aos_mimalloc	1000	7995	0	0	0
avx2_aos_std	1000	7995	0	0	0
avx2_hybrid_jemalloc	1000	315164	0	0	0
avx2_hybrid_mimalloc	1000	243930	0	0	0
avx2_hybrid_std	1000	230759	0	0	0
avx2_soa_jemalloc	1000	423930	0	0	0
avx2_soa_mimalloc	1000	400634	0	0	0
avx2_soa_std	1000	398265	0	0	0
scalar_aos_jemalloc	1000	6975	0	0	0
scalar_aos_mimalloc	1000	7005	0	0	0
scalar_aos_std	1000	6975	0	0	0
scalar_hybrid_jemalloc	1000	1366319	0	0	0
scalar_hybrid_mimalloc	1000	1279365	0	0	0
scalar_hybrid_std	1000	1356329	0	0	0
scalar_soa_jemalloc	1000	440835	0	0	0
scalar_soa_mimalloc	1000	394200	0	0	0
scalar_soa_std	1000	438960	0	0	0
sse4_aos_jemalloc	1000	16845	0	0	0
sse4_aos_mimalloc	1000	14160	0	0	0
sse4_aos_std	1000	14324	0	0	0
sse4_hybrid_jemalloc	1000	1281180	0	0	0
sse4_hybrid_mimalloc	1000	1275404	0	0	0
sse4_hybrid_std	1000	1279350	0	0	0
sse4_soa_jemalloc	1000	397095	0	0	0
sse4_soa_mimalloc	1000	392294	0	0	0
sse4_soa_std	1000	396780	0	0	0
pa_compact_lazy_binary_leaf_only	1000	348105	0	0	0
pa_compact_lazy_binary_sampling	1000	375600	0	0	0
pa_compact_lazy_linear_leaf_only	1000	445815	0	0	0
pa_compact_lazy_linear_sampling	1000	423090	0	0	0
pa_compact_none_binary_leaf_only	1000	360765	0	0	0
pa_compact_none_binary_sampling	1000	346980	0	0	0
pa_compact_none_linear_leaf_only	1000	398939	0	0	0
pa_compact_none_linear_sampling	1000	391875	0	0	0
pa_wide_lazy_binary_leaf_only	1000	309300	0	0	0
pa_wide_lazy_binary_sampling	1000	311805	0	0	0
pa_wide_lazy_linear_leaf_only	1000	445605	0	0	0
pa_wide_lazy_linear_sampling	1000	492495	0	0	0
pa_wide_none_binary_leaf_only	1000	307110	0	0	0
pa_wide_none_binary_sampling	1000	311925	0	0	0
pa_wide_none_linear_leaf_only	1000	445140	0	0	0
pa_wide_none_linear_sampling	1000	478635	0	0	0

Tabelle A.2: Messreihe cartesian_smoke43: vorkonfigurierte Lebewesen-Permutationen (Pfad A, run_workload)

Suchverfahren	cyto_P04_avg0	cyto_P04_avg100	cyto_P04_avg25	cyto_P04_avg50	cyto_P04_avg75	ih	ih	hp_balanced_5000	hp_bulk_insert	hp_concurrent_mnw	hp_dedup_heavy	hp_dynamic_trace	hp_index_opt	hp_range_scan	hp_read_uniform	ks-0.a	ks-0.b	ks-0.c	ks-0.d	ks-0.e	ks-0.f
eytzinger	4663	9367	5755	6924	8129	175317	12536	97590	235805	91687	134523	19093	49297	33	4651	98559	12663	4730	16086	7794	8205
interpolation	3650	7202	4557	5444	6339	153540	11475	86204	207310	88743	115834	17004	42893	91811	3607	85569	11425	3704	15762	99954	6283
k_ary	2817	5473	3467	4113	4824	92827	7134	52027	125172	51927	76934	10594	26662	23	2802	51911	7208	2851	9141	4187	4820
linear_scan	11506	23486	14487	17282	20557	48913	12990	30947	63537	96624	88800	14029	25348	168720	11385	31086	13001	11652	21425	166863	21763

Tabelle A.3: Bias-Bruch-Matrix (Median ns/op, Suchverfahren x Lastprofil)

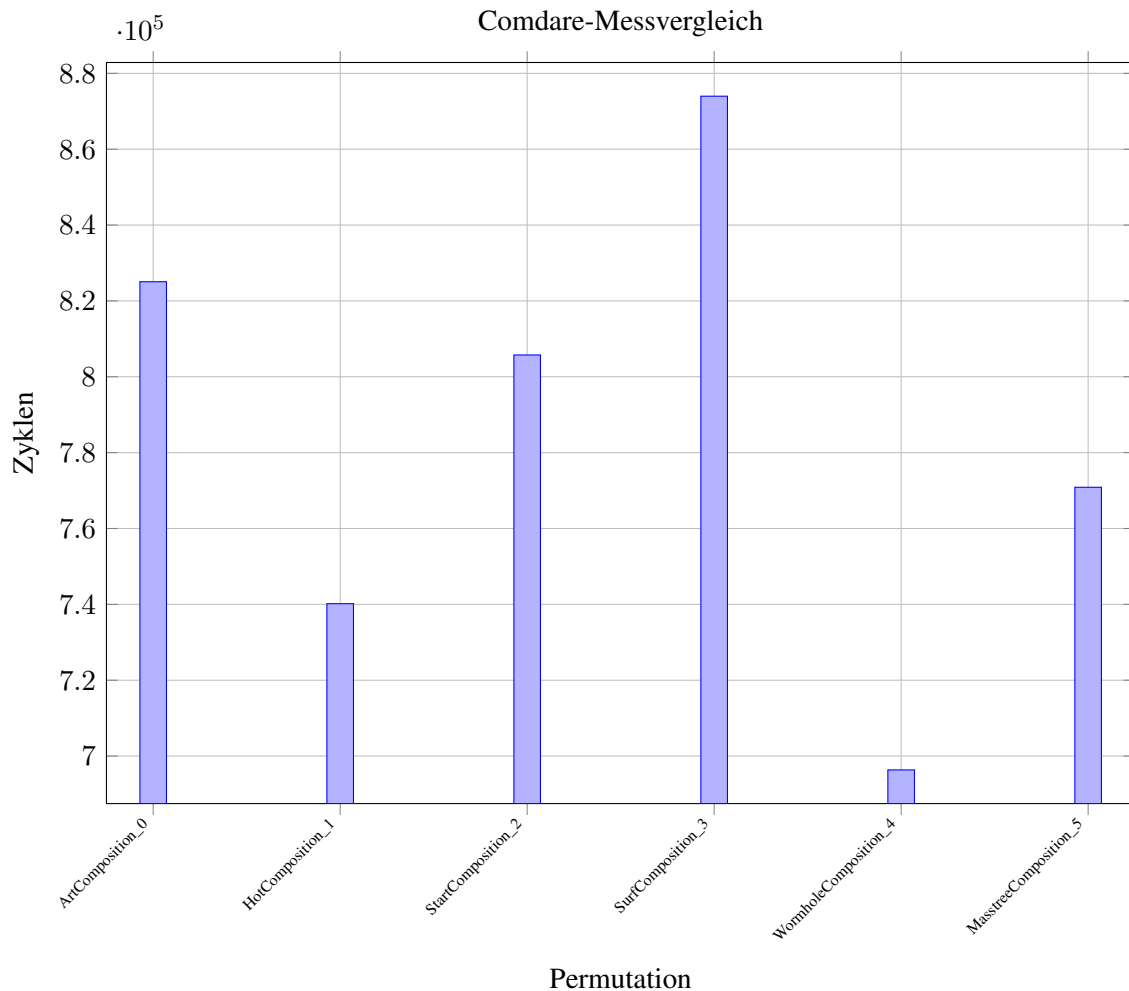


Abbildung A.1: Zyklen je Lebewesen-Permutation (v5_pipeline_demo)

tionen `insert`, `lookup`, `erase`, `scan` und `rmw`.

A.4 Achsen-Austauschbarkeit

Die vier folgenden Langtabellen belegen die Austauschbarkeit je variabler Achse anhand der Geschwister-Paar-Differenzen (Median rel. Δ ns/op bezüglich v): `search_algo` (Tabelle A.4), `node_type` (Tabelle A.5), `memory_layout` (Tabelle A.6) und `prefetch` (Tabelle A.7). Am wenigsten konfundiert sind `search_algo` und `prefetch`.

Tabelle A.4: Achsen-Austauschbarkeit: `search_algo` (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
eytzinger	interpolation	ns_per_op	-1493	-0.145	0.574	1680	verschiedener Organ-Pfad
eytzinger	interpolation	insert	0	-0.103	0.399	960	verschiedener Organ-Pfad
eytzinger	interpolation	lookup	-200	-0.146	0.436	1360	verschiedener Organ-Pfad
eytzinger	interpolation	erase	0	-0.148	0.344	160	verschiedener Organ-Pfad
eytzinger	interpolation	scan	0	+1177.000	309.000	11	verschiedener Organ-Pfad

Fortsetzung nächste Seite

Tabelle A.4 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
eytzing	interpolation	rmw	0	-0.171	0.403	240	verschiedener Organ-Pfad
eytzing	k_ary	ns_per_op	-6145	-0.420	0.145	1680	verschiedener Organ-Pfad
eytzing	k_ary	insert	-40500	-0.428	0.166	960	verschiedener Organ-Pfad
eytzing	k_ary	lookup	-1500	-0.357	0.125	1360	verschiedener Organ-Pfad
eytzing	k_ary	erase	0	-0.357	0.218	160	verschiedener Organ-Pfad
eytzing	k_ary	scan	0	-1.000	0.000	11	verschiedener Organ-Pfad
eytzing	k_ary	rmw	0	-0.374	0.167	240	verschiedener Organ-Pfad
eytzing	linear_scan	ns_per_op	2587	+0.189	2.026	1680	verschiedener Organ-Pfad
eytzing	linear_scan	insert	-77100	-0.755	0.151	960	verschiedener Organ-Pfad
eytzing	linear_scan	lookup	5600	+1.347	0.777	1360	verschiedener Organ-Pfad
eytzing	linear_scan	erase	0	+0.146	0.383	160	verschiedener Organ-Pfad
eytzing	linear_scan	scan	0	+2758.000	107.000	11	verschiedener Organ-Pfad
eytzing	linear_scan	rmw	0	+1.587	0.842	240	verschiedener Organ-Pfad
interpolation	k_ary	ns_per_op	-6981	-0.340	0.275	1680	verschiedener Organ-Pfad
interpolation	k_ary	insert	-28100	-0.372	0.187	960	verschiedener Organ-Pfad
interpolation	k_ary	lookup	-900	-0.289	0.235	1360	verschiedener Organ-Pfad
interpolation	k_ary	erase	0	-0.243	0.211	160	verschiedener Organ-Pfad
interpolation	k_ary	scan	0	-1.000	0.000	80	verschiedener Organ-Pfad
interpolation	k_ary	rmw	0	-0.278	0.256	240	verschiedener Organ-Pfad
interpolation	linear_scan	ns_per_op	4390	+0.338	2.065	1680	verschiedener Organ-Pfad
interpolation	linear_scan	insert	-55300	-0.735	0.178	960	verschiedener Organ-Pfad
interpolation	linear_scan	lookup	6200	+1.596	1.277	1360	verschiedener Organ-Pfad
interpolation	linear_scan	erase	0	+0.395	0.446	160	verschiedener Organ-Pfad
interpolation	linear_scan	scan	0	+1.777	1.149	80	verschiedener Organ-Pfad
interpolation	linear_scan	rmw	0	+2.000	1.488	240	verschiedener Organ-Pfad
k_ary	linear_scan	ns_per_op	8960	+1.322	3.446	1680	verschiedener Organ-Pfad
k_ary	linear_scan	insert	-18200	-0.586	0.322	960	verschiedener Organ-Pfad
k_ary	linear_scan	lookup	7800	+2.815	1.095	1360	verschiedener Organ-Pfad
k_ary	linear_scan	erase	0	+0.774	0.409	160	verschiedener Organ-Pfad
k_ary	linear_scan	rmw	0	+3.254	1.326	240	verschiedener Organ-Pfad

Tabelle A.5: Achsen-Austauschbarkeit: node_type (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node16	node256	ns_per_op	1010	+0.126	0.592	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node256	insert	0	-0.033	0.441	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node256	lookup	500	+0.207	0.493	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.5 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node16	node256	erase	0	-0.025	0.354	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node256	scan	0	+0.339	0.419	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node256	rmw	0	+0.188	0.473	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	ns_per_op	6164	+0.573	0.943	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	insert	12900	+0.993	0.540	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	lookup	0	+0.058	0.349	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	erase	0	+1.085	0.749	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	scan	0	+0.275	0.677	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node4	rmw	0	+0.127	0.455	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node48	ns_per_op	-160	-0.027	0.325	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node48	insert	0	-0.135	0.407	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.5 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node16	node48	lookup	0	+0.037	0.245	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node48	erase	0	-0.151	0.225	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node48	scan	0	+0.052	0.183	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node16	node48	rmw	0	+0.018	0.257	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	ns_per_op	4066	+0.268	0.947	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	insert	6700	+1.065	1.332	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	lookup	0	-0.024	0.326	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	erase	0	+1.436	0.754	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	scan	0	-0.010	0.447	49	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node4	rmw	0	+0.042	0.411	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node48	ns_per_op	-1410	-0.132	0.382	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.5 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
node256	node48	insert	0	-0.063	0.352	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node48	lookup	-300	-0.140	0.317	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node48	erase	0	-0.100	0.278	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node48	scan	0	-0.299	0.368	49	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node256	node48	rmw	0	-0.114	0.386	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	ns_per_op	-5104	-0.360	0.567	1680	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	insert	0	-0.586	0.262	960	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	lookup	0	-0.012	0.264	1360	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	erase	0	-0.596	0.103	160	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	scan	0	-0.151	0.230	40	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
node4	node48	rmw	0	-0.084	0.277	240	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

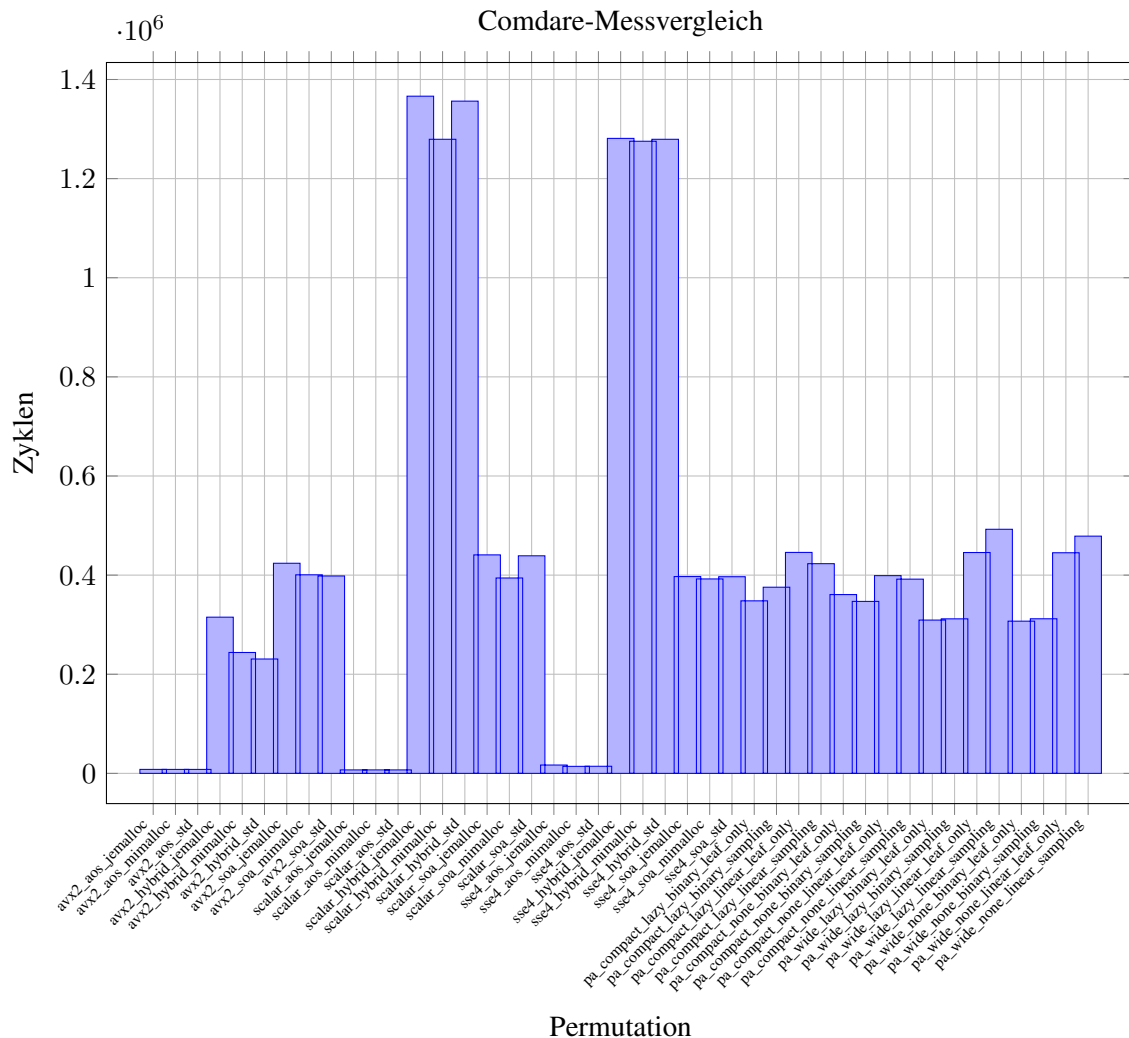


Abbildung A.2: Zyklen je Lebewesen-Permutation (cartesian_smoke43)

Tabelle A.6: Achsen-Austauschbarkeit: memory_layout (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 640 Geschwister-Paare; Konfundierung: Vorbehalt)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ aos_strict	memory_layout_ aoso	ns_per_op	1138	+0.092	0.309	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung,
memory_layout_ aos_strict	memory_layout_ aoso	insert	0	+0.211	0.326	768	Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung,
memory_layout_ aos_strict	memory_layout_ aoso	lookup	0	+0.000	0.255	1088	Apparat-Artefakt moeglich verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung,

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ aos_strict	memory_layout_ aoso	erase	0	+0.209	0.244	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ aoso	scan	0	-0.045	0.351	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ aoso	rmw	0	+0.097	0.257	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	ns_per_op	1081	+0.077	0.223	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	insert	0	+0.230	0.259	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	lookup	0	+0.000	0.124	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	erase	0	+0.229	0.213	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	scan	0	+0.020	0.096	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ cache_line_aligned	rmw	0	+0.099	0.139	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ packed_bitmap	ns_per_op	-45	-0.008	0.225	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ packed_bitmap	insert	0	-0.009	0.234	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ aos_strict	memory_layout_ packed_bitmap	lookup	0	+0.000	0.265	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ packed_bitmap	erase	0	+0.005	0.220	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ packed_bitmap	scan	0	-0.099	0.254	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ packed_bitmap	rmw	0	+0.000	0.306	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	ns_per_op	1170	+0.096	0.303	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	insert	0	+0.277	0.284	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	lookup	0	+0.000	0.148	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	erase	0	+0.253	0.302	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	scan	0	-0.014	0.186	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aos_strict	memory_layout_ soa	rmw	0	+0.107	0.185	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aosoa	memory_layout_ cache_line_aligned	ns_per_op	-221	-0.025	0.265	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_aosoa	memory_layout_cache_line_aligned	insert	0	-0.030	0.275	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_line_aligned	lookup	0	-0.020	0.243	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_line_aligned	erase	0	-0.038	0.259	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_line_aligned	scan	0	+0.010	0.358	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_cache_line_aligned	rmw	0	-0.014	0.317	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	ns_per_op	-1439	-0.105	0.241	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	insert	0	-0.201	0.176	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	lookup	0	+0.000	0.168	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	erase	0	-0.171	0.147	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	scan	0	-0.019	0.200	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_aosoa	memory_layout_packed_bitmap	rmw	0	-0.085	0.141	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ aoso	memory_layout_ soa	ns_per_op	98	+0.015	0.272	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aoso	memory_layout_ soa	insert	0	+0.010	0.280	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aoso	memory_layout_ soa	lookup	0	+0.020	0.257	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aoso	memory_layout_ soa	erase	0	+0.020	0.256	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aoso	memory_layout_ soa	scan	0	+0.025	0.412	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ aoso	memory_layout_ soa	rmw	0	+0.027	0.300	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	ns_per_op	-1157	-0.110	0.326	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	insert	0	-0.209	0.239	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	lookup	0	+0.000	0.416	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	erase	0	-0.179	0.234	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	scan	0	-0.085	0.285	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ cache_line_aligned	memory_layout_ packed_bitmap	rmw	0	-0.097	0.363	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	ns_per_op	-32	-0.005	0.198	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	insert	0	-0.007	0.209	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	lookup	0	+0.000	0.185	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	erase	0	+0.005	0.209	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	scan	0	-0.043	0.144	32	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ cache_line_aligned	memory_layout_ soa	rmw	0	-0.014	0.188	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ packed_bitmap	memory_layout_ soa	ns_per_op	1763	+0.157	0.356	1344	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ packed_bitmap	memory_layout_ soa	insert	2700	+0.288	0.337	768	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ packed_bitmap	memory_layout_ soa	lookup	0	+0.020	0.216	1088	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ packed_bitmap	memory_layout_ soa	erase	0	+0.251	0.246	128	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Fortsetzung nächste Seite

Tabelle A.6 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
memory_layout_ packed_bitmap	memory_layout_ soa	scan	0	+0.009	0.294	35	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich
memory_layout_ packed_bitmap	memory_layout_ soa	rmw	0	+0.127	0.260	192	verschiedener Organ-Pfad; Vorbehalt: Q2-Schritt-4 search_organ_- Beschattung, Apparat-Artefakt moeglich

Tabelle A.7: Achsen-Austauschbarkeit: prefetch (Geschwister-Paar-Diffs, Median rel. Δ ns/op bzgl. v ; 480 Geschwister-Paare; Konfundierung: am wenigsten konfundiert)

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
prefetch_distance_ estimator	prefetch_hardware	ns_per_op	-13	-0.002	0.088	1680	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_hardware	insert	0	-0.002	0.081	960	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_hardware	lookup	0	+0.000	0.081	1360	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_hardware	erase	0	-0.004	0.076	160	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_hardware	scan	0	-0.005	0.121	43	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_hardware	rmw	0	+0.000	0.092	240	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	ns_per_op	-18	-0.003	0.075	1680	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	insert	0	-0.010	0.087	960	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	lookup	0	+0.000	0.064	1360	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	erase	0	-0.009	0.054	160	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	scan	0	-0.014	0.071	43	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_none	rmw	0	-0.007	0.093	240	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	ns_per_op	50	+0.007	0.123	1680	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	insert	0	+0.005	0.128	960	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	lookup	0	+0.000	0.118	1360	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	erase	0	-0.001	0.127	160	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	scan	0	-0.010	0.156	43	verschiedener Organ-Pfad
prefetch_distance_ estimator	prefetch_path_ oriented	rmw	0	+0.014	0.165	240	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	ns_per_op	-10	-0.003	0.103	1680	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	insert	0	-0.002	0.118	960	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	lookup	0	+0.000	0.084	1360	verschiedener Organ-Pfad

Fortsetzung nächste Seite

Tabelle A.7 – Fortsetzung

v	v'	Funktion	Median abs. Δ (ns)	Median rel. Δ	IQR rel.	n	Diagnose
prefetch_hardware	prefetch_none	erase	0	+0.000	0.098	160	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	scan	0	+0.000	0.119	44	verschiedener Organ-Pfad
prefetch_hardware	prefetch_none	rmw	0	+0.000	0.138	240	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	ns_per_op	36	+0.005	0.081	1680	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	insert	0	+0.003	0.076	960	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	lookup	0	+0.000	0.070	1360	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	erase	0	+0.003	0.070	160	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	scan	0	-0.003	0.053	44	verschiedener Organ-Pfad
prefetch_hardware	prefetch_path_ oriented	rmw	0	+0.012	0.123	240	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	ns_per_op	77	+0.009	0.116	1680	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	insert	0	+0.010	0.116	960	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	lookup	0	+0.000	0.109	1360	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	erase	0	+0.004	0.090	160	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	scan	0	-0.005	0.107	42	verschiedener Organ-Pfad
prefetch_none	prefetch_path_ oriented	rmw	0	+0.026	0.115	240	verschiedener Organ-Pfad

A.5 Limitierungen

Tabelle A.8 führt alle nicht-gefixten Vorbehalte der Messreihen auf, damit kein Befund still verfällt; Zeile 1 betrifft die fehlende PMC-Kernmetrik (Cache-Misses).

Tabelle A.8: Ehrliche Limitierungen: nicht-gefixte Vorbehalte (kein Befund verfällt still)

#	Vorbehalt	Status / Konsequenz
1	Cache-Misses (Kernmetrik): L1/L2/L3 + dTLB + Coherence + Energy = 0 / nicht erhoben	NullPmcSource, available=false; Intel-PCM-Windows-Treiber + Linux-PAPI ausstehend (#26/P4); Cache-Verhalten nur indirekt über Wall-Clock-Proxy (seg_memory_layout_ns/ns_per_op).
2	15 gepinnte Achsen = 0 Austauschbarkeits-Belege	Nur 4 Achsen variieren (search_algo, node_type, memory_layout, prefetch). Gepinnt (je 1 Wert): cache_traversal, mapping, path_compression, allocator, concurrency, serialization, telemetry, value_handle, isa, index_organization, io_dispatch, migration_policy, filter, queuing_q1, queuing_q2.
3	Observer-Proben-Zähler (*_find/*_probe/*_get) by-design ≈ 1	Erwartetes Verhalten, kein Phantom: je Lookup eine Probe. Keine Aussage über interne Iterationen.
4	Honest-0 inaktiver Sub-Features gepinnter Strategien	concurrency/migration/io/value_handle melden 0 (Feature inaktiv), nicht Mess-Fehler.
5	Wall-Clock nicht bit-reproduzierbar	Seed steuert Keys (deterministisch), NICHT das CPU-Timing; Wall-Clock variiert lauf-zu-lauf.
6	RC-Dimension nur nominal	K1: $\times 18 \rightarrow \times 3$ degeneriert; RC-Achse liefert keine volle Ausprägungs-Breite.

Fortsetzung nächste Seite

Tabelle A.8 – Fortsetzung

#	Vorbehalt	Status / Konsequenz
7	2/21 Scan-Profile = No-Op (ausgeschlossen)	ycsb_e und lp_range_scan führen keinen echten Scan aus; bei scan-Diffs ausgeschlossen (19 valide Lastprofile).
8	Insert-Profile messen Upserts; stat_*-Spalten enthalten Load-Phase	Insert-Pfad ist faktisch Upsert; die stat_*-Zähler akkumulieren die Lade-Phase mit.
9	prefetch misst Key-Werte als Pseudo-Adressen (K9)	Prefetch-Achse interpretiert Schlüssel-Werte als Adressen; kein echter Speicher-Prefetch.
10	node_type + memory_layout: Q2-Schritt-4-Beschattung	Apparat-Artefakt möglich (search_organ_-Beschattung); search_algo + prefetch sind am wenigsten konfundiert.
11	memory_layout-Effekt teils sub-noise	Teil der Layout-Differenzen liegt unter dem Mess-Rauschen → PMC extern-gated (#26).
12	Sekundäre Indirektions-Organen (cache_traversal/mapping): uint16-Schlüssel-Sättigung	Der uint64-Primärschlüssel wird beim Auto-Koppeln in die schmalere slot_index/key_type-Felder der GEPINNTEN cache_traversal-/mapping-Organen verengt (low-bits); bei records > Typbreite kollidieren deren Observer-Einträge. Betrifft NICHT den Primär-Lookup (uint64); reine Observer-Limitierung gepinnter Achsen (vgl. Zeile 2/4).
13	Kapsel-interne (verschlucktes) OOM in einer Mess-Op	Nach aussen geworfenes OOM entwertet die Messung jetzt hart (two_phase_valid=false). Ein vom Tier-Code INTERN per catch(...) gefangenes OOM (z.B. defensiver Memento-Verzicht) ist von aussen nicht beobachtbar → konservativ als Restvorbehalt dokumentiert, nicht erzwungen.
14	Timer-Overhead (clock::now()) nicht kalibriert	Die Wall-Clock-Segmentzeiten enthalten den nicht subtrahierten Eigen-Overhead der std::chrono::steady_clock::now()-Aufrufe (eine additive, lauf-stabile ns-Komponente je gemessenem Segment). Es findet KEINE Leer-Schleifen-Kalibrierung statt → dokumentierter Vorbehalt; betrifft den absoluten ns-Nullpunkt, nicht die relativen Achsen-Vergleiche (G5-Audit, MESS-MINOR-7).
15	Entwurfs-Etiketten = Namenskonvention, kein Mess-Einfluss	Einzelne Pattern-Etiketten im Code/Doku sind Benennungs-/Entwurfsvorbehalte ohne Mess-Wirkung: „hybrider Visitor“ (real ein Memento, kein accept()/Host-Besuch), sowie Interpreter-/Template-/Decorator-Etiketten und reserve-/if constexpr-Hygiene. Reine Naming-Konvention → als Vorbehalt dokumentiert, kein Einfluss auf Messwerte (G5-Audit, PATTERN-MINOR-1..7).



Abbildung A.3: Heatmap: Gesamt-Latenz (ns/op) (Suchalgorithmus x Workload)

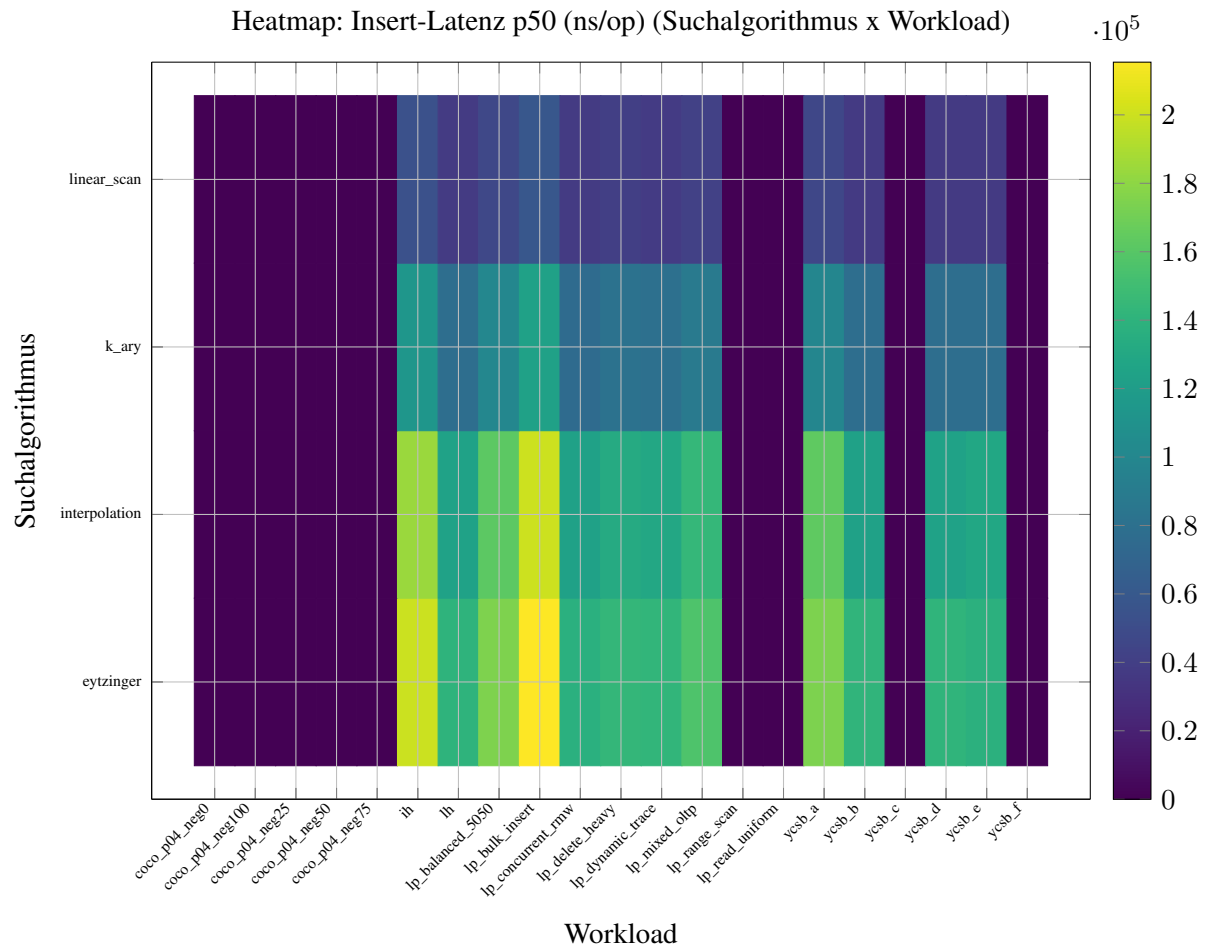


Abbildung A.4: Heatmap: Insert-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

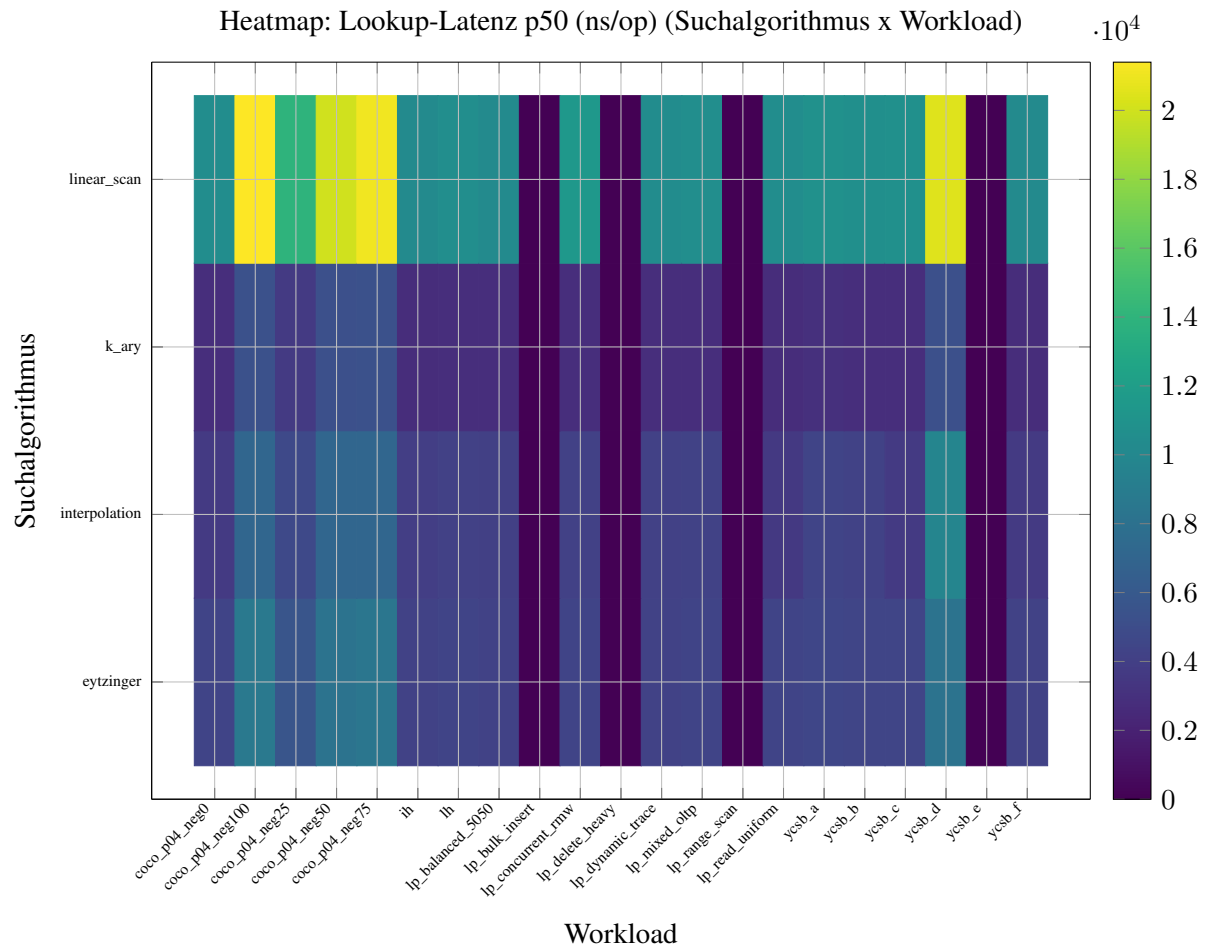


Abbildung A.5: Heatmap: Lookup-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

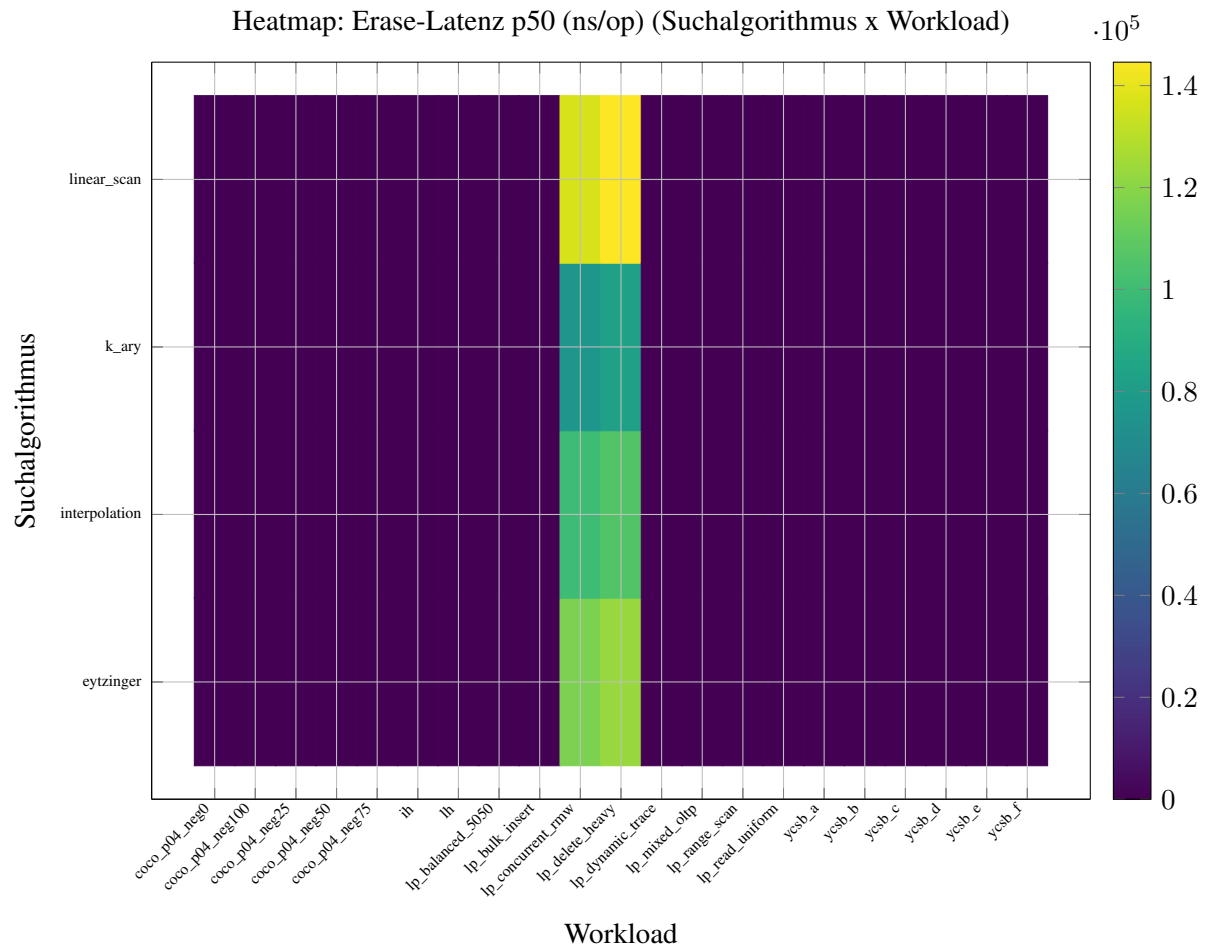


Abbildung A.6: Heatmap: Erase-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

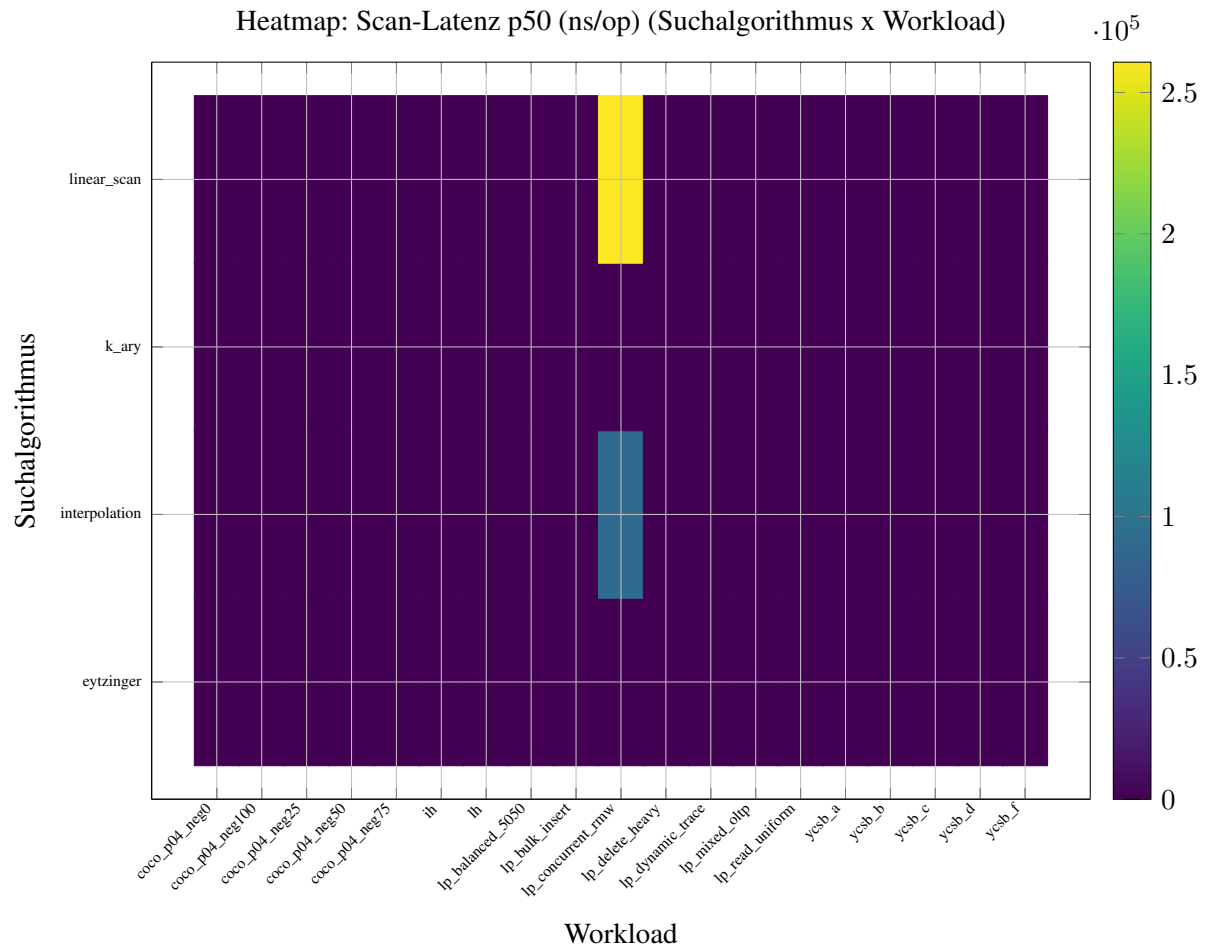


Abbildung A.7: Heatmap: Scan-Latenz p50 (ns/op) (Suchalgorithmus x Workload)



Abbildung A.8: Heatmap: RMW-Latenz p50 (ns/op) (Suchalgorithmus x Workload)

B Code-Struktur

C Glossar

Dieses Glossar fasst die in der Arbeit verwendeten Abkürzungen und zentralen Begriffe zusammen.

ABI Application Binary Interface — die binär-stabile Schnittstelle, über die der Builder die Permutations-Module (Lebewesen) lädt.

Achse Eine orthogonale Teilentscheidung („Organ“) eines Suchalgorithmus; das Framework führt neunzehn Kompositions-Achsen.

Adaptiv vs. dynamisch vs. messgetrieben Begriffsabgrenzung für die Anpassung des Algorithmus-Verhaltens. *Adaptiv*: Anpassung anhand laufzeit-beobachteter Datenmerkmale (z. B. adaptive Knotengrößen in ART). *Dynamisch*: die Laufzeit-Auswahl des fertig kompilierten Permutations-Moduls über den Permutations-Identifizier (Modul-Ladung) — alle neunzehn Achsen sind je Binary compile-time-gebacken; hinzu kommen fünf laufzeit-steuerbare Resource-Control-Achsen (Concurrency, Prefetch, Allocator, Traversal, Value-Handle), keine Laufzeit-Wahl einzelner Achsen-Strategien. *Messgetrieben* (*measurement-driven*): Auswahl der Konfiguration auf Basis empirisch erhobener Messreihen — das Leitmotiv dieser Arbeit; der zugehörige Auswahl-Mechanismus ist in einem ersten Inkrement implementiert (`best_binary_selector`: Ranking und Auslieferung der besten Permutation je Metrik aus den Mess-CSVs), die vollautomatische, lastprofilübergreifende Auswahl bleibt Ausblick.

Anatomie Der *Körper* eines Lebewesens: die feste Komposition seiner neunzehn Achsen-Organen und vor allem deren *Verdrahtung* — die generalisierten Interfaces, über die ein Organ die Dienste der übrigen nutzt (etwa das gemeinsame Allokations-Interface). Kein zweites Modell neben dem Suchalgorithmus, sondern dessen Innenstruktur (vgl. Abschnitt 2.3).

ART Adaptive Radix Tree [LKN13]; Referenzentwurf mit adaptiven Knotengrößen (Node4/16/48/256).

Autonome Heuristik-Profil-Auswertung (*autonomous profile-driven heuristic evaluation*) Der angestrebte Mechanismus, der aus den Messreihen je Lastprofil automatisch die beste Achsen-Konfiguration ableitet und als wiederverwendbare Heuristik-Richtlinie ablegt (Ziel-Mechanismus: ML-Klassifikation von Lastprofil-/Achsen-Features auf Konfigurationen). Erste Bausteine sind implementiert — die automatische Best-Binary-Auswahl aus Messreihen (`best_binary_selector`) und die Extraktion/Ablage als XML-Lastprofil-Ergebnis —; die ML-Klassifikation selbst ist weiterhin Ausblick (vgl. Abschnitt 6.3).

AVX, AVX-512 x86-SIMD-Erweiterungen (256- bzw. 512-Bit).

CLU Cache-Line-Utilization — Anteil genutzter Bytes je geladener Cache-Line.

CPI, IPC Cycles per Instruction bzw. Instructions per Cycle.

CRTP Curiously Recurring Template Pattern — C++-Idiom für statischen Polymorphismus.

Drei-Ebenen-Modell Die Schichtung der einen Architektur: *Gattung* (Ebene 1, das äußere Algorithmus-Interface — SearchAlgorithm, Container, Graph), *Gattungs-Unterklasse* (Ebene 2, fester Achsen-Satz) und *Organe/Achsen* (Ebene 3). Siehe Abschnitt 2.3.

dTLB Daten-Translation-Lookaside-Buffer; cacht Daten-Adressübersetzungen.

Gattung Ein nach außen sichtbares Interface (SearchAlgorithm, Container, Graph) — die oberste Ebene (Ebene 1) des Frameworks; siehe *Drei-Ebenen-Modell*.

Gattungs-Unterklasse Die zweite Ebene unter einer Gattung mit festem Achsen-Satz (Code-Begriff: *Tier-Unterklasse*, *AnatomyGenus*). In der biologischen Lesart das „Säugetier“ = SearchAlgorithm (volle Neunzehn-Achsen-Anatomie); unter Container die Unterklassen Set, Sequence, Adapter und View („Vogel“, „Reptil“, „Wirbelloses“, „Pflanze“).

GoF „Gang of Four“ — die kanonischen Entwurfsmuster nach Gamma et al. [GHJV94].

HBM High-Bandwidth Memory.

HOT Height Optimized Trie [BZP⁺18].

Lebewesen Ein vollständiger Suchalgorithmus als Komposition aller Achsen (metaphorischer Code-Begriff; technischer Identifier: SearchAlgorithm-Instanz).

LOUDS Level-Ordered Unary Degree Sequence — succinct Baum-Kodierung (Jacobson 1989).

MESI, MOESI, MESIF Cache-Kohärenzprotokolle (Grund-Protokoll bzw. AMD-/Intel-Varianten).

NINE Non-Inclusive-Non-Exclusive — Inklusions-Politik des L3 in Intel-Server-Prozessoren.

NUMA Non-Uniform Memory Access.

OLC Optimistic Lock Coupling — optimistische Nebenläufigkeits-Technik (ARTSync, P08) [LABN16].

Organ Metaphorischer Begriff für eine *Achse* (Ebene 3 des Drei-Ebenen-Modells); ein Suchalgorithmus besitzt neunzehn Organe.

PMC (Hardware) Performance Monitoring Counter.

Prüf-Dock Die Builder-Seite, die genau eine Gattung lädt, durch ihr Interface treibt, vermisst und persistiert.

Prüfling Der testweise einzugliedernde neue Suchalgorithmus (hier PRT-ART); wird vom Builder gegen die Kernbestandteile der Cache-Engine kompiliert.

PRT-ART Probst-Redirect-Tree-/ART-Hybrid — der Prüfling dieser Arbeit.

Rang Zentralitäts-Einordnung der Stand-der-Technik-Veröffentlichungen (Rang-1/2/3); erklärt in Abschnitt 1.3.

RCU Read-Copy-Update — Reklamations-Schema (McKenney, P29) [McK01].

RRIP, DRRIP (Dynamic) Re-Reference Interval Prediction — cache-Ersetzungs-Heuristik [JTJE10].

SIMD Single Instruction, Multiple Data.

SOSD Search On Sorted Data — Index-Benchmark [KMR⁺19].

SOTA State of the Art — der Stand der Technik.

SuRF Succinct Range Filter [ZLL⁺18].

SVE, NEON, RVV ARM- (SVE, NEON) bzw. RISC-V- (RVV) SIMD-Erweiterungen.

TLB Translation Lookaside Buffer.

XML-Lastprofil-Ergebnis (*XML load-profile result*) Das in der Aufgabenstellung (Teilaufgabe 7) geforderte Persistenz-Format, in dem die für einen Gesamtalgorithmus extrahierte, je Architekturfokus wiederverwendbare Heuristik-Richtlinie unter Referenz der angewendeten Lastprofile abgelegt wird; dient als Eingabe der autonomen Heuristik-Profil-Auswertung. Import-Parser und Export-Writer samt Extraktor sind implementiert (round-trip-fähiges Schema `comdare_load_profile`); als Ausblick verbleibt die darauf aufsetzende autonome Heuristik-Auswertung (ML-Klassifikation).

YCSB Yahoo! Cloud Serving Benchmark [CST⁺10].

D Bausteine-Matrix

Dieser Anhang enthält die vollständige Bausteine-Matrix, auf die der Stand der Technik (Kapitel 3.2) verweist. Jede Achse ist dabei ein *Organ* des Lebewesens: Ein konkreter Suchalgorithmus (technisch `SearchAlgorithm`, metaphorisch ein Lebewesen) ist die feste Komposition seiner neunzehn Achsen, von denen jede einem Organ entspricht; die je Achse aufgeführten Baustein-Varianten sind die austauschbaren Ausprägungen dieses Organs, und ihre Permutation gleicht einem genetischen Experiment am Lebewesen. Die *Anatomie* dieses Lebewesens ist dabei die Verdrahtung *zwischen* den Organen: Achsen-Algorithmen nutzen die Interfaces anderer Achsen (so stellt die Allokations-Achse ein gemeinsames Allokations-Interface für die übrigen Organe bereit). Je Achse sind der Zweck, die *statischen* Sub-Achsen (übersetzungszeitlich wählbare Varianten-Familien) und die *dynamischen* Sub-Achsen (Laufzeit-Parameter) sowie die konkreten Baustein-Varianten mit ihrer Quelle (Paper-Kennung oder Code/Baseline) aufgeführt. Die Angaben sind aus dem Code der `comdare-cache-engine` und gegen die Paper-Provenienz-Map (Doc 18) verifiziert.

D.1 T0 Suchalgorithmen-Achse T0 (Such-Strukturen und Such-Methoden)

Diese Achse ist das *Such-Organ* (T0) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2). Systematische Permutation von 21 konkreten Suchalgorithmen und Such-Strukturen (S01–S21: 17 Basis-Bausteine — Array-basiert, sortierte Vektoren, Original-Paper-Wrapper, Such-Methoden wie k-ary/Interpolation, sowie geordnete Strukturen wie Skip-Liste/B-Baum — plus 4 opt-in per-K-k-ary-Wrapper K2/K4/K8/K16, standardmäßig abgeschaltet) zur messbaren Quantifizierung des Einflusses der Such-Paradigma-Wahl auf Performance-Charakteristiken im einheitlichen `std::map`-Interface. Sub-Achsen klassifizieren nach Zugriffsdichte (SA1 dense, SA2 sparse, SA3/SA4 multilevel), ermöglichen Compliance-Nachweise über Paper-Mixins (Habich-Anforderung: `is_original`-Markierung pro Funktion/Wrapper), und identifizieren SIMD-fähige sowie store-traversierbare Varianten.

Statische Sub-Achsen

ID	Name	Beschreibung
SA1	Direkt-adressiert hohe Dichte (Dense Access)	Array-Familie: direkt-adressiert ($O(1)$ lookup), hohe Füllungsdichte ≥ 50 %. Umfasst <code>Array256SearchAlgo</code> (256-Slot compact array, ART Node256 Re-Impl) und <code>Array65535SearchAlgo</code> (uint16-basiert, mid-density 25-50 % direkt-adressiert).
SA2	Sparse/Geordnet (Sparse Access)	Sortierte Vektoren, geordnete Such-Methoden und tree/hash-Strukturen: niedrige Füllungsdichte, Range-Scan-Support, variable $O(\log n)$ - $O(1)$ Lookup. Umfasst <code>VectorU8U8SearchAlgo</code> , Original-Paper-Wrapper (ART/HOT/START/Wormhole/SuRF), Such-Methoden (k-ary, Interpolation, Eytzinger, Hash), sowie Strukturen (SkipList, BST, B-Baum).

ID	Name	Beschreibung
SA3	Multi-Byte-Diskriminatoren (Multilevel Access)	Mehrbyteige Such-Strukturen mit indizierten Sub-Diskriminatoren (z.B. START mit Cost-DP-Mehrebenen). <code>VectorU16U16SearchAlgo</code> (u16-Keys, sorted lower_bound) und <code>OriginalStartSearchAlgo</code> (Paper P05) gehören hier.
SA4	Direkt-adressiert mittlere Dichte (Direct Multibyte)	Teilt die Direkt-Adressierungs-Zugriffsmuster (wie SA1) mit Multi-Byte-Granularität (uint16-Größe) und mittlerer Füllungsdichte 25–50 % (im Gegensatz zu SA1 ≥ 50 %). Beispiel: prt-art Array65535 (REV 6 §5.17). Diese Subachse wurde hinzugefügt, um die SA1-Abstraktion nicht zu brechen, aber die Density-Variante zu erfassen.

Bausteine

Variante	Beschreibung	Quelle
Array256SearchAlgo (S01)	ART Node256 direkt-adressierte Re-Implementierung: <code>std::array<std::optional<u64>, 256></code> mit $O(1)$ Lookup/Insert/Erase. Cache-Line-aligned, SIMD-fähig, kein Heap-Allocation. Baseline für dichte Zugriffe.	Code/Baseline (Leis ICDE 2013 ART Paper P01)
VectorU8U8SearchAlgo (S02)	Sparse sortiertes Key-Value-Vektor-Paar: <code>std::vector<u8> keys + std::vector<u64> values</code> mit <code>std::lower_bound</code> . $O(\log n)$ Lookup, $O(n)$ Insert/Erase. Konzept-äquivalent zu HOT-k-Constrained. SIMD-fähig (Bitmask-Scan).	Code/Baseline (Binna SIGMOD 2018 HOT Paper P02)
VectorU16U16SearchAlgo (S03)	Sparse sortiertes u16-Vektor-Paar: <code>std::vector<u16> keys + std::vector<u64> values</code> . Vereinfachtes START-Konzept (Multi-Byte-Diskriminatoren ohne Cost-DP). $O(\log n)$ Lookup, $O(n)$ Insert/Erase.	Code/Baseline (Fent ICDEW 2020 START Paper P05)
Array65535SearchAlgo (S09)	Mid-Density direkt-adressiert uint16-basiert (25–50 % Füllungsdichte). Aus prt-art Legacy-Code (REV 6 §5.17) migriert. $O(1)$ Lookup aber mit Speicher-Overhead vs. SA1 denser Arrays.	Code/Baseline (prt-art F.6-Migration)
OriginalArtSearchAlgo (S04)	ART Paper-Bindung (<code>unodb::db::ist_original_module()=true</code> (4/4 Funktionen original). Paper-Mixin via SHA256-validierte Tool-Codegenerierung. Habich-Compliance: <code>get_compiler()=gcc-9.5</code> , all insert/lookup/erase/clear marked original.	P01 (Leis/Kemper/Neumann ICDE 2013, Apache-2.0 unodb)
OriginalHotSearchAlgo (S05)	HOT Paper-Bindung (<code>HOTRowex::is_original_module()=false</code> (2/4 original). insert/lookup original, aber erase/clear Lücken (HotRowex ist append-only; Re-Impl fallback). Paper P02.	P02 (Binna et al. SIGMOD 2018, ISC ISC)
OriginalStartSearchAlgo (S06)	START Paper-Bindung (sosd-Adapter): <code>is_original_module()=false</code> (2/4 original). insert/lookup original, erase/clear Lücken (kein remove im sosd-Adapter). Paper P05, Multi-Byte Diskriminatoren.	P05 (Fent/Jungmair/Kipf/Neumann ICDEW 2020, MIT)
OriginalWormholeSearchAlgo (S07)	Wormhole Paper-Bindung: <code>is_original_module()=false</code> (3/4 original). put/get/del original, clear Lücke. Hybrid Hash+Trie+B+ Index. GPL-3.0-lizenziert (blockiert Original-Code-Linking für Thesis).	P07 (Wu/Ni/Jiang EuroSys 2019, GPL-3.0)
OriginalSurfSearchAlgo (S08)	SuRF Paper-Bindung (FST mit LOUDS-Bitmap): <code>is_original_module()=false</code> (1/4 original, nur lookup). Succinct Range Filter, Multi-Header-Bibliothek. Paper P10.	P10 (Zhang/Lim/Andersen SIGMOD 2018, Apache-2.0)

Variante	Beschreibung	Quelle
KArySearchAlgo (S10)	k-ary Search Such-METHODE (Schlegel/Gemulla/Lehner DaMoN 2009): K-Wege-Partition statt Halbierung bei der Binärsuche. K ist iterable_aspect ($K \in \{2,4,8,16\}$). $O(\lceil \log_{K+1} n \rceil)$ Iterationen, parallele ILP-Vergleiche. Array-Familie aber kein treues dediziertes Traversal-Organ (SA2-Klassifikation). C++23-Re-Impl, <code>is_original=false</code> .	Schlegel et al. DaMoN 2009 [SGL09]
InterpolationSearchAlgo (S11)	Interpolationssuche Such-METHODE (Perl/Itai/Avni CACM 1978): verteilungsbewusste Positionsschätzung statt blinder Halbierung. $O(\log \log N)$ avg bei Gleichverteilung, $O(N)$ worst-case. Nicht SIMD-vektorisierbar (datenabhängiger Index). C++23-Re-Impl.	Perl/Itai/Avni CACM 1978 [PIA78]
EytzingerSearchAlgo (S12)	Eytzinger/BFS-Layout branch-free Suche (Khuong/Morin JEA 2017): Cache-Layout-Such-METHODE mit branch-free Kern. Keys in BFS-Ordnung des impliziten Suchbaums. Fuer große Arrays laut Paper schnellste Allzweck-Layout. C++23-Re-Impl (Experiment-Harness ohne Standard-Lizenz).	Khuong/Morin JEA 2017 [KM17b]
SkipListSearchAlgo (S13)	Skip-Liste probabilistische geordnete STRUKTUR (Pugh CACM 1990): $O(\log n)$ erwartet search/insert/erase ohne explizites Rebalancing. Jeder Knoten hat 1..kMaxLevel Forward-Zeiger (Münzwurf Probability 0.5). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, value-semantisch). Lehrbuch-Algorithmus, C++23-Re-Impl.	Pugh CACM 1990 [Pug90]
HashSearchAlgo (S14)	Open-Addressing-Hashtabelle UNGEORDNET (Knuth TAOCP 3 §6.4): Fibonacci-Hash, Linear Probing, Tombstone-Erase bei Resize. $O(1)$ erwartet lookup/insert bei <code>load_factor < 0.7</code> . Keine Range-Scan-Unterstützung (distinkte Klassifikation). Korrekte Tombstone-Markierung zur Probe-Ketten-Integrität. C++23-Re-Impl.	Knuth TAOCP 3 §6.4 [Knu98]
LinearScanSearchAlgo (S15)	Unsortierter linearer Scan Baseline (Leis ICDE 2013 ART Node4): $O(n)$ lookup auf unsortiertem KV-Array. Einfachste Vergleichs-Nullpunkt-Struktur: insert $O(1)$ -amortisiert, erase $O(1)$ via swap-and-pop. Store-traversierbar (Array-Familie). Keine Range-Scan-Unterstützung.	Code/Baseline (Leis ICDE 2013 ART Paper P01 Node4)
BinarySearchTreeSearchAlgo (S16)	Unbalancierter Binärer Suchbaum (Knuth TAOCP 3 §6.2.2 Hibbard-Deletion): deterministische Vergleichs-Baum-Baseline (vs. SkipList probabilistisch). Index-basiert (<code>std::vector<Node></code> mit uint32 indices, Free-List). Hibbard-erase (3 Fälle: Blatt/1 Kind/2 Kinder via In-Order-Nachfolger). $O(n)$ worst-case bei sortiertem Input.	Knuth TAOCP 3 §6.2.2 [Knu98]
BTreeSearchAlgo (S17)	Balancierter Block-orientierter Mehrwege-B-Baum (Bayer/McCreight Acta Inf. 1972 / CLRS Kap. 18): $t=4$, daher [3..7] Keys pro Knoten, bis zu 8 Kinder. IMMER balanciert ($O(\log n)$ guaranteed). Index-basiert, <code>alignas(64)</code> -Knoten (Cache-Line-Alignment). Top-Down Splitten beim Insert, Borrow/Merge beim Erase. Das DB-Index-Arbeitspferd.	Bayer/McCreight Acta Inf. 1972 / CLRS [BM72]

Beitragende Quellen: P01, P02, P05, P07, P10, k-ary (Schlegel et al. 2009), Interpolation (Perl et al. 1978), Eytzinger (Khuong/Morin 2017), Skip-List (Pugh 1990), Hashing/BST (Knuth TAOCP 3), B-Baum (Bayer/McCreight 1972) [LKN13, BZP⁺18, FJKN20, WNJ19, ZLL⁺18, SGL09, PIA78, KM17b, Pug90, Knu98, BM72].

D.2 T1 Cache-Speicher-Traversierung (Fanout-Lookup)

Diese Achse ist das *Cache-Traversierungs-Organ* (T1) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Die Achse `axis_03b` definiert drei distinkte Strategien zur Schlüssel-Auflösung in kleinen Fanout-Arrays (typisch: B+-Tree oder Trie Inner Nodes). Sie spannt ein Spektrum von linearer Suche ($O(N)$, cache-freundlich, klein), über multiplikatives Hashing mit Open-Addressing ($O(1)$ amortisiert, Knuth-Fibonacci), bis zur sortierten Binärsuche ($O(\log N)$, geordnete Traversierung, B+-Tree-Standard). Alle drei sind Lehrbuch-Baselines ohne Original-Code-Linking, implementiert als reine C++23-Re-Implementierungen.

Statische Sub-Achsen

ID	Name	Beschreibung
CT1	Lineare Zugriffe	Linear-scan-basierte Lookup-Methoden auf kleinen, unsortierten Fanout-Arrays. Charakterisiert durch $O(N)$ -Auflösung mit einfachen Vergleichen, typisch für Nodes mit $\text{Fanout} < 32$ in B+-Trees. Sub-Achse für <code>LinearFanout</code> und <code>BinarySearchFanout</code> (beide: <code>axis_tag = linear_access_tag</code>).
CT2	Hash-basierte Zugriffe	Hash-Lookup mit Open-Addressing und linearem Probing. $O(1)$ amortisierte Auflösung, charakterisiert durch Fibonacci-Multiplikator ($11400714819323198485 \approx 2^{64}/\varphi$), Power-of-2 Bucket-Count, dynamisches Rehashing bei Load-Factor > 0.7 . Nur Hash Lookup-Wrapper. Sub-Achse: <code>axis_tag = hash_access_tag</code> .

Bausteine

Variante	Beschreibung	Quelle
<code>LinearFanout</code>	Unsortierte Fanout-Traversal via <code>std::find_if</code> und linearem Scan. $O(N)$ Lookup, kein Hash-Overhead, cache-freundlich für kleine N (< 32). Klassische B-Tree-Baseline (Bayer/McCreight 1972). Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)
<code>HashLookup</code>	Fibonacci-Hash open-addressing mit linearem Probing (Knuth TAOCP Vol. 3, §6.4). $O(1)$ amortisiert, dynamisches Rehashing (verdoppelt Capacity bei $\text{load} \geq 0.7$), iterable Laufzeit-Kapazität (<code>kIterableInitialCapacities: {8, 16, 64, 256, 1024}</code>). Standalone C++23-Re-Impl, keine Original-Code-Bindung. Gehört zu Sub-Achse CT2.	Code/Baseline — Knuth 1998 (The Art of Computer Programming Vol.3)
<code>BinarySearchFanout</code>	Sortierte Fanout-Traversal via <code>std::lower_bound</code> . $O(\log N)$ Lookup bei sortierter Ordnung, bewahrt Schlüssel-Sequenzierung für Range-fähige Routing-Layer. Klassische B+-Tree-Inner-Node-Suche (Bayer/McCreight 1972), dritte distinkte Strategie der Achse. C++23-Re-Impl (R7.2, 2026-05-29), <code>is_original=false</code> . Gehört zu Sub-Achse CT1.	Code/Baseline — Bayer/McCreight 1972 (Acta Informatica)

Beitragende Quellen: Bayer/McCreight 1972, Knuth TAOCP Vol. 3 [BM72, Knu98].

Verwandte Literatur: P17, P21, P32 [BDFC05, CGM01, SKK⁺25].

D.3 T2 Slot-zu-Offset-Mapping (Traversal-Sublayer)

Diese Achse ist das *Mapping-Organ* (T2) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2). Vermittlung zwischen logischen Slot-Indizes und physischen Speicher-Offsets in Trie-Inner-Nodes und Speicher-Pools. Die Achse bietet zwei grundsätzlich verschiedene Zuordnungsstrategien: direkte absolute Offsets (lineare Packing) oder pool-relative Offsets mit automatischem Rebase-Mechanismus zur Unterstützung von Speicher-Umallokation und Persistent-Data-Structures.

Statische Sub-Achsen

ID	Name	Beschreibung
MP1	Direct-Access-Mapping (MP1)	Direkter Slot-zu-Absolute-Offset-Mapping ohne Indirection. Wrapper-Familie für klassisches Array-of-Pointers-Layout wie in B-Tree Inner-Nodes (Bayer/McCreight 1972). Einfache Cache-Locality, $O(N)$ Lookup per <code>std::find_if</code> .
MP2	Pool-Relative-Mapping (MP2)	Slot-zu-Pool-Relativem-Offset-Mapping mit Based-Pointer-Arena-Pattern. Alle Offsets relativ zu dynamischem <code>pool_base_address</code> gespeichert. Ermöglicht $O(1)$ Rebase bei Speicher-Umallokation — zentrale Eigenschaft für Persistent-Data-Structures (Driscoll et al. 1989) und CustomAligned Structure-Pattern in prt-art.

Bausteine

Variante	Beschreibung	Quelle
DirectPlacement	Wrapper für direktes Slot-zu-Absolute-Offset-Mapping (lineares Packing). <code>std::vector<pair<slot_index_type, offset_type>></code> ; <code>resolve_offset/register_slot</code> via <code>linear std::find_if</code> . Verwendungskontext: prt-art <code>Inode::placement_page_ + Mapping-Table</code> . Speicher: 16-bit Slot-Index + <code>size_t</code> Offset pro Entry. Keine Pool-Basis-Abhängigkeit.	CE-Baseline
PoolRelative	Wrapper für Pool-Relatives-Offset-Mapping mit $O(1)$ -Rebase-Fähigkeit. <code>std::vector<pair<slot_index_type, relative_offset>></code> + mutable <code>pool_base_address</code> . Bei Pool-Umallokation: <code>rebase(new_pool_base)</code> setzt nur <code>pool_base_neu</code> — alle relativen Offsets bleiben gültig. Verwendet bei prt-art CustomAligned Structure + Versioning-Szenarien (Persistent Data Structures nach Driscoll/Sarnak/Sleator/Tarjan 1989).	CE-Baseline

Beitragende Quellen: — (CE-Baseline, eigene Wrapper).

Verwandte Literatur: Bayer/McCreight 1972, Driscoll et al. 1989, P03, P07 [BM72, DSST89, MKM12, WNJ19].

D.4 T3 Pfadkompression

Diese Achse ist das *Pfadkompressions-Organ* (T3) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2).

Definiert die Kompressions-Granularität und -Strategie für Schlüsselpfade in Trie-Strukturen. Steuert, wie innere Knoten redundante Byte-Sequenzen oder signifikante Bits gemeinsamer Präfixe speichern und navigieren, um Speicher- und Cache-Effizienz zu optimieren.

Statische Sub-Achsen

ID	Name	Beschreibung
pc1	Kompressions-Granularität (PC1)	Definiert die minimale Einheit der Pfad-Kompression: Bit-weise (Patricia), Byte-weise (ART) oder keine Kompression (raw path). Bestimmt die Trie-Höhe und navigationsoptimale Codeblöcke im Traversal.
pc2	Sprung-Strategie (PC2)	Definiert das Navigations-Pattern bei inneren Knoten: Linear über 8 Bytes (ByteWise), Single-Bit-Vergleich (Patricia) oder direkter Sprung ohne Zwischenschritte. Beeinflusst Branch-Dichte und Prefetch-Verhalten.
pc3	Decode-Komplexität (PC3)	Klassifiziert die Laufzeit-Komplexität beim Extrahieren von Präfix-Informationen: $O(1)$ für direkte Extraktion, $O(\log n)$ für struktur-basierte Suche, $O(n)$ für sequenzielle Prüfung. Wichtig für Operationen wie <code>common_prefix_len()</code> und Schlüssel-Vergleiche.

Bausteine

Variante	Beschreibung	Quelle
PathCompression None	Keine Pfad-Kompression; rohe Schlüsselpfade werden vollständig und unkomprimiert in Knoten gespeichert. Dient als Mess-Baseline mit <code>compression_ratio=1.0</code> . Verwendet als Kontrollfall zur Quantifizierung der Speicher- und Höhen-Einsparungen komprimierender Strategien.	Code/Baseline
ByteWisePath Compression	Byte-für-Byte-Kompression (ART-Stil); speichert bis zu 7 Präfix-Bytes pro Knoten in einem kompakten <code>u64</code> -Wort mit Längenetikett im High-Byte. Reduktion der Trie-Höhe um typischerweise Faktor 2 (<code>compression_ratio=0.5</code>). Ursprung: Leis et al., ART ICDE 2013 (P01). Verwendet Goldstandard-Operationen <code>common_prefix_len()</code> und <code>cut()</code> für Navigations-Optimierung.	P01
PatriciaPath Compression	Single-Bit-Split-Pfadkompression (Patricia-Trie); navigiert bitweise statt byteweise, reduziert Trie-Höhe um typischerweise Faktor 3 (<code>compression_ratio=0.3</code>). Ursprung: Morrison PATRICIA JACM 1968; modern implementiert in HOT (Binna SIGMOD 2018, P02) und Wormhole (Wu EuroSys 2019, P07). F15-operative Primitiven: <code>key_split_bit()</code> für MSB-first Bit-Extraktion, <code>path_descend_scan()</code> für goldstandard-konforme <code>_scan</code> -Signatur mit schlüssel-abhängiger variabler Tiefe.	P02

Beitragende Quellen: P01, P02, P07, Morrison 1968 [LKN13, BZP⁺18, WNJ19, Mor68].

Verwandte Literatur: P04, P05, P09 [BFTV24, FJKN20, Jac89].

D.5 T4 Knotentypen (Node-Type-Achse)

Diese Achse ist das *Knotentyp-Organ* (T4) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Die Achse `axis_04_node_type` definiert die vier Knotenformate des Adaptive Radix Tree (ART) als austauschbare Varianten: `Node4`, `Node16`, `Node48` und `Node256`. Jeder Knotentyp bietet eine andere Balance zwischen Speicher, Cache-Effizienz und Suchgeschwindigkeit. Diese Achse bildet die Basis für adaptive Baum-Reorganisationen und ist zentral für das Design der Suchalgorithmen.

Statische Sub-Achsen

ID	Name	Beschreibung
NT1	Kapazitätsklasse (NT1)	Klassifizierung nach Fanout-Größe und Slotanzahl: klein (Node4 mit 4 Slots, linear search), mittel (Node16 mit 16 Slots, SIMD-binary-search) bis maximal (Node256 mit 256 Slots, direct-addressed).
NT2	Zugriffsmuster (NT2)	Art der Schlüsselsuche innerhalb eines Knotens: linearer Scan (Node4), binäre Suche mit SIMD (Node16), indirekte Indizierung (Node48) oder direkte Adressierung (Node256).
NT3	Kompaktheit (NT3)	Speicherlayout und Dichte: spärlich und cache-aligned (Node4), dicht mit Indirection (Node48) oder maximal direkt (Node256), mit Auswirkungen auf Speicherverbrauch und Cache-Line-Misses.

Bausteine

Variante	Beschreibung	Quelle
Node4NodeType	Kleiner ART-Knoten mit 4 Slots und linearer Suche. Ideal für spärlich besetzte obere Baumebenen, passt komplett in eine Cache-Line (64 Byte). Implementiert als CE-Reimplementierung der ART-Spezifikation mit <code>node_find_scan()</code> als linear scan über sortierte Schlüssel.	P01
Node16NodeType	Mittlerer ART-Knoten mit 16 Slots, optimiert für SIMD-basierte binäre Suche (SSE2 PCMPSTRM / AVX2 VPCMPSTRM). Balance zwischen Speicher und Suchgeschwindigkeit; größte Kapazität ohne Indirection. CE-Re-Impl der Leis-ICDE-2013-Spezifikation.	P01
Node48NodeType	Großer ART-Knoten mit 48 Slots, nutzt Indirection: 256-Byte ChildIndex (Byte→Slot-Mapping, 0=leer) plus 48er ChildArray. Trade-off zwischen Speicher (vs Node256 direct) und Cache-Misses (vs Node4 linear). Doppelter Indirektions-Zugriffsmuster (Index + Child).	P01
Node256NodeType	Maximaler ART-Knoten mit 256 Slots, direkt adressiert ohne Index. O(1) Lookup (kein Scan, kein Index-Touch). Hoher Speicherverbrauch (256 Pointer), aber optimale Suchgeschwindigkeit für dicht besetzte Knoten. Default für untere Baumebenen.	P01
NodeTypeSlotStore	Storage-Organ: <code>node_type</code> -getriebener boundierter Slot-Speicher. Erstreckt sich über <code>std::array</code> mit Kapazität <code>N::max_capacity()</code> über einen gemeinsamen uint64-Key. Erfüllt das StorageOrgan-Concept und ist Drop-in-Substrat für ComposedSearch. Baseline-Variante ohne Layout/Allocator-Integration.	Code/Baseline
ComposedStore	3-Achsen-Storage-Organ: $\text{node_type}(N) \oplus \text{layout}(L) \oplus \text{allocator}(A)$. Unboundierter vector über Allocator A; layout fließt als statisches <code>cache_line_size</code> und via optionale <code>organ_scan_field_sum()</code> ein. Real messbar (F15-Brücke).	Code/Baseline
NodeChunkedStore	3-Achsen-Storage-Organ mit Laufzeitwirkung der <code>node_type</code> -Achse: Slots liegen in Chunks der Kapazität <code>N::max_capacity()</code> . Chunk-Count = Zahl der node-großen Allokationen; damit ist die <code>node_type</code> -Achse REAL messbar (Node4 viele Chunks, Node256 wenige). Beweis für Delegation.	Code/Baseline
LayoutAwareChunkedStore	Layout-ehrende Variante von NodeChunkedStore: speichert Records am layout-getriebenen Stride (<code>eff_stride</code>). Jeder Record belegt <code>eff_stride</code> Bytes (Key 0..8, Value 8..16, Rest Padding). Macht <code>memory_layout</code> -Achse echt messbar, OOB-sicher für scan-Methoden.	Code/Baseline
ObservableNodeType	ObservableAxis-Hülle um eine <code>node_type</code> -Strategie. Reicht die statische API durch <code>(max_capacity/name/topic_tag)</code> + bietet <code>observe_node_find()</code> als Instanz-Driver. Trackt <code>find_count</code> , <code>keys_stored</code> , <code>queries_run</code> , <code>last_checksum</code> . Format-divergente Lookup-Latenz messbar (Pfad B).	Code/Baseline

Beitragende Quellen: P01 [LKN13].

D.6 T5 Speicher-Layout (Memory Layout)

Diese Achse ist das *Speicher-Layout-Organ* (T5) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Diese Achse klassifiziert die räumliche Organisation von Datensätzen im Speicher und beeinflusst damit Cache-Effizienz, SIMD-Vectorisierbarkeit und False-Sharing-Charakteristiken. Sie erfasst orthogonal vier Design-Dimensionen: (1) Alignment-Strategien (Cache-Line-Padding vs. Strict-Packing), (2) Datenorganisation (zeilenorientiert AoS vs. spaltenorientiert SoA vs. Hybrid AoSoA), (3) Packing-Dichte (dense vs. succinct bit-packed), (4) Stride-Muster (sequenziell vs. interleaved). Die Layout-Wahl ist ein primärer Treiber für Cache-Lokalität und SIMD-Fähigkeit bei Index-Traversal.

Statische Sub-Achsen

ID	Name	Beschreibung
HM1	Alignment-Strategie	Entscheidung zwischen Cache-Line-gepaddetem Layout (False-Sharing-Vermeidung) und Strict-Packing (Speicherkompaktheit). Bestimmt Stride-Overhead bei sequenzieller Traversal.
HM2	Daten-Organisation	Grundlegende Speicher-Ordnung: Array-of-Structs (AoS, zeilenorientiert), Struct-of-Arrays (SoA, spaltenorientiert), oder Hybrid AoSoA (Block-basierte Mischung). Kritisch für Cache-Line-Ausnutzung bei Feld-selektiven Scans.
HM3	Packing-Dichte	Dichtegrad der Speichernutzung: von Standard-Records (8–64 Byte) bis zu kompaktem Bit-Packing (z.B. LOUDS-Bitmaps mit ~1 Bit pro Slot). Beeinflusst Decode-Latenz und L1/L2-Auslastung.
HM4	Stride-Muster	Muster der Speicheradressierung bei Traversal: Sequenzielle Strides (aufeinanderfolgende Records), Interleaved (z.B. um Hot-Cold-Daten zu trennen) oder Cache-Aware-Layouts (z.B. Eytzinger BFS). Optimiert Prefetch-Effizienz.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
CacheLineSize	Cache-Line-Größe (Unterachse KF-5)	Runtime-Parameter für die Zielplattform-Cache-Zeile: 64 Byte (Standard x86-64/ARM64), 128 Byte (Azure, manche Power ISAs) oder 32 Byte (embedded). Wird zur Compile-Time über <code>CacheLineConfig-NTTP</code> gewählt und beeinflusst Alignment/Padding-Overhead aller alignierten Layouts.

Bausteine

Variante	Beschreibung	Quelle
<code>AoSStrictMemoryLayout</code>	Array-of-Structs, kompakt gepackt ohne Cache-Line-Alignment. Vorteil: minimale Speichernutzung (record_size-Strides, kompakt). Nachteil: False-Sharing bei concurrent Writes, schlechte Cache-Hit-Quote bei schmalen Feldscans. Typisch für Wormhole-Index.	Code/Baseline

Variante	Beschreibung	Quelle
CacheLineAligned MemoryLayout	AoS mit 64-Byte-Alignment per Record. Vermeidet False-Sharing durch Padding (Stride = round_up(record_size, 64)). Standard für ART/HOT/Masstree/START. Operativ: Bei record_size=48 (Mess-Build) entsteht 16-Byte-Overhead pro Record → messbar höhere Latenz als aos_strict, echter Cache-Effekt.	Code/Baseline
SoAMemoryLayout	Struct-of-Arrays: Alle Felder spaltenweise hintereinander. Vorteil: Kontiguierliche Feldscans (z.B. nur Keys ohne Values) nutzen maximal Cache-Lines (16× weniger Cache-Lines als AoS bei Einfeld-Scan). Ideal für SIMD-Vectorization und OLAP-Indizes. Vorteil beschrieben in Abadi et al. SIGMOD 2008.	Abadi et al. 2008 [AMH08]
PackedBitmap MemoryLayout	Bit-packed succinct Layout (LOUDS / SuRF FST). Speichert n Einträge mit $\sim n \cdot \lg(\sigma)$ Bits statt bytes. Extreme Kompaktheit (32× weniger Cache-Lines als AoS), aber höherer Decode-Overhead per Lookup. Fundament: Jacobson LOUDS FOCS 1989 (P09). Moderne Impl: SuRF Apache-2.0.	P09 (Jacobson LOUDS FOCS 1989)
AoSoAMemoryLayout	Array-of-Structures-of-Arrays: Hybrid-Layout mit Block-Breite W=8 (AVX2-Lanes). Innerhalb eines Blocks: SoA (Felder kontiguierlich). Über Blocks: AoS (Blocks sind gestrided). Vereint SIMD-Vectorisierbarkeit (SoA innerhalb mit räumlicher Lokalität (AoS über). Standard in HPC/SIMD-Engines. Konvention; kein einzelnes Ursprungspaper, aber ISPC-Idiom.	Code/Baseline (HPC/SIMD-Konvention)

Beitragende Quellen: P09, Abadi et al. 2008 [Jac89, AMH08].

Verwandte Literatur: P06, P11, P12, P32 [SSL⁺21, RR99, RR00, SKK⁺25].

D.7 T6 Allokator-Achse (T6)

Diese Achse ist das *Allokations-Organ* (T6) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2). Diese Achse stellt die Speicherverwaltungsstrategie für die Cache-Engine bereit. Sie definiert 25 konkrete Allocator-Wrapper für verschiedene Algorithmen und Techniken (von klassischen Baselines wie Buddy und Slab über moderne Research-Allokatoren wie `mimalloc`, `jemalloc`, `snmalloc` bis zu spezialisierten Varianten für NUMA, PIM und formal verifizierte Systeme). Alle Wrapper erfüllen standardisierte Concepts und sind orthogonal nach 7 Compile-Time-Varianten-Familien (Freelist-Topologie, Size-Class-Schema, Thread-Lokalität, Synchronisation, Allokationspolicy, Reklamation, Fragmentierungsstrategie) strukturierbar.

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Wie sind freie Speicherblöcke organisiert? Beispiele: Per-Page-Sharding (<code>mimalloc</code> A04), Per-Thread-Caching (<code>snmalloc</code> A07), Buddy-Tree (A19), Single-Global-Free-List (<code>dlmalloc</code> A20), Spans (<code>scalloc</code> A08), hybride Strategien (<code>hmalloc</code> A15).
AA2	Size-Class-Schema	Wie werden Block-Größen klassifiziert? Beispiele: Slab pro Objektgröße (A02), <code>dlmalloc</code> Bins (A20), <code>jemalloc</code> Größenklassen (A05), <code>scalloc</code> Spans (A08), Buddy Power-of-2 (A19), formal verifizierte Bins (StarMalloc A13), <code>std::pmr</code> pools (A22).

ID	Name	Beschreibung
AA3	Thread-Lokalität	Wie lokal pro Thread/Core ist die Allokation? Beispiele: tcmalloc thread-local-caches (A06), rpmalloc thread-heaps (A10), snmalloc message-passing (A07), tcmalloc-warehouse hyperscale (A14), Hoard per-heap-locks (A01).
AA4	Synchronisation	Welcher Concurrency-Mechanismus? Beispiele: Lock-Free CAS (Michael A03, LRMalloc A11), Per-Heap-Locks (Hoard A01), Single-Threaded (Exgen A18), Wait-Free (Crystalline A17).
AA5	Allokationspolicy	Wie werden Allokationsanfragen geroutet? Beispiele: NUMA-Origin-Awareness (NUMAlloc A09), Cache-Set-Awareness (CAMA A12), NUCA-Awareness (TCMalloc-Warehouse A14), PIM-Awareness (PIM-malloc A16), polymorphe Memory-Resource (std::pmr A22).
AA6	Reklamation	Wie werden gelöschte Blöcke zurückgegeben? Beispiele: Magazine-Cache (Slab A02), Page-Heap-Release (TCMalloc A06), Lock-Free-Reclamation (Crystalline A17), Deferred-Free (mimalloc A04), Vmem-Magazine-Extension (A23).
AA7	Fragmentierungsstrategie	Wie wird Fragmentierung vermieden? Beispiele: dlmalloc Coalescing (A20), Object-Coloring (Slab A02), Buddy-Splitting (A19), Page-Local-Sharding (mimalloc A04), Span-Pooling (scalloc A08).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierbarkeit. Berger et al. (PPoPP 2000). Synchronisierungskonzept via Per-Heap-Locks + lokale Heap-Caches pro Thread.	P— (OSS, Apache-2.0)
SlabAllocator (A02)	Object-Caching-Kernel-Allokator mit Magazine-Cache-Strategie. Bonwick (USENIX 1994, 2001). Klassisches Lehrbuch-Paradigma ohne eigenständigen OSS-Code.	P— (Baseline/Textbook)
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allokator mit Hazard-Pointers. Maged Michael (PLDI 2004). Re-Implementierung mit IBM-Patent-Hintergrund.	P— (MIT Re-Impl)
MimallocAllocator (A04)	Free-List-Sharding mit deferred-free und page-local management. Leijen/Zorn/de Moura (MSR-TR-2019-18, APLAS 2019). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_ original)
JemallocAllocator (A05)	Arena-basierte Größenklassen mit per-CPU Arenas und tcache Layer. Evans (BSDCan 2006). Standard-Allokator von FreeBSD und Facebook. Original Code-Linking (is_ original=2/2).	P— (OSS, BSD-2-Clause, is_ original)
TCMallocAllocator (A06)	Thread-Caching Malloc mit zentralem Page-Heap. Ghemawat/Menage (Google ca. 2005) + TEMERAIRE (OSDI 2021). Google-Produktions-Standard.	P— (OSS, Apache-2.0)
SnmallocAllocator (A07)	Message-Passing-Allokator mit asymmetrischen Heap-Operationen. Paul Liétar et al. (ISMM 2019). Original Code-Linking (is_ original=2/2).	P— (OSS, MIT, is_ original)
ScallocAllocator (A08)	Span-basierte Free-List mit Virtual Spans für Multi-Core-Skalierbarkeit. Aigner/Kirsch/Lippautz/Sokolova (OOPSLA 2015). Low-Fragmentierung via Span-Pooling.	P— (OSS, BSD-3-Clause)
NUMAllocAllocator (A09)	NUMA-Origin-Awareness mit schnellerer Speicherallokation. UTSASRG (ISMM 2023). Optimierte für Multi-Socket-Architekturen.	P— (OSS, vermutlich Apache-2.0)
RPMallocAllocator (A10)	Per-Thread Span-Caching ohne Dependency zu anderen Allokator-Bibliotheken. Mattias Jansson (2017). Original Code-Linking (is_ original=2/2).	P— (OSS, Public-Domain/ MIT, is_ original)

Variante	Beschreibung	Quelle
LRMallocAllocator (A11)	Lock-Free Allocator mit Hazard-Pointers für Multi-Core-Skalierbarkeit. Leite/Rocha (VECPAR 2018, LNCS 11333). Original Code-Linking (is_original=2/2).	P— (OSS, MIT, is_original)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator mit vorhersehbarem Verhalten. Herter/Backes/Haupenthal/Reineke (ECRTS 2011, Saarland). Keine eigenständige OSS-Implementierung.	P— (Baseline/Research)
StarMallocAllocator (A13)	Formal verifizierte härtete Memory-Allocator mit F*/Steel. Inria-Prosecco: Reitz/Fromherz/Protzenko et al. (OOPSLA 2024). Mathematisch verifizierter Allocator.	P— (OSS, Apache-2.0)
TCMallocWarehouse Allocator (A14)	TCMalloc-Hyperscale-Erweiterung mit Huge Page-Unterstützung für Warehouse-Scale-Computing. TEMERAIRE (Hunter OSDI 2021) + Warehouse-Charakterisierung (Zhou ASPLOS 2024). NUCA-aware TCMalloc-Variante.	P— (OSS, Apache-2.0)
HMallocAllocator (A15)	Hybrid Lock-Free Allocator mit skalierbare Speicherverwaltung. Li/Yao/Tang/Lin (IEEE ICPADS 2019). Keine eigenständige OSS-Implementierung.	P— (Research Baseline)
PIMMallocAllocator (A16)	Processing-In-Memory Allocator für heterogene HW mit schnellem lokalen Speicher. VIA-Research (HPCA 2026, arXiv:2505.13002). Spezialisiert auf PIM-Hardware.	P— (OSS, MIT)
CrystallineReclamation (A17)	Wait-free Safe-Memory-Reclamation-Familie (Nikolaev/Ravindran DISC 2021 / PLDI 2024). Reclamation-Policy, kein eigenständiger Allokator.	P— (Research)
ExgenAllocator (A18)	Single-Threaded Specialized Allocator mit Optimierungen für Single-Threaded Workloads. UT Austin (IEEE CAL 2025, arXiv:2510.10219). „Old is Gold“ Prinzip.	P— (Research, Konfidenz medium)
BuddyAllocator (A19)	Buddy-System mit Power-of-2-Splitting für einfache Fragmentierungskontrolle. Knuth (TAoCP Vol.1, 1968). Klassisches Lehrbuch-Paradigma.	P— (Baseline/Textbook)
DlmallocAllocator (A20)	Doug Lea Malloc mit Bins-basierter Klassifizierung. Lea (1987-2012). Original Code-Linking (is_original=2/2).	P— (OSS, Public-Domain/CC0, is_original)
PtMalloc2Allocator (A21)	ptmalloc2 (glibc malloc) mit expliziter Wrapper-Kontrolle. Wolfram Gloger / Ulrich Drepper (1990er-2000er). Lizenz-blockiert (LGPL-2.1+).	P— (OSS, LGPL-2.1+, blockiert Linking)
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis) — portable Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen.	P— (Standard-Library)
PmrResourceAllocator (A22b)	std::pmr::memory_resource (polymorphe Memory-Ressource). Halpern (N3916, C++17). Abstrakte Schnittstelle ohne verhaltens-distinkte Default-Implementierung.	P— (Standard-Library, C++17)
PoolResourceAllocator (A22c)	std::pmr::unsynchronized_pool_resource mit eigenem Size-Class-Pool. Halpern (N3916, C++17). Verhaltens-distinkt ohne Vendor-Linking (F15-operativ).	P— (Standard-Library, C++17, F15-operativ)
VmemMagazinesAllocator (A23)	Vmem + Magazines als Erweiterung des Slab-Allocators für Multi-Core-Skalierung. Bonwick (USENIX ATC 2001). Keine eigenständige OSS-Implementierung.	P— (Baseline/Textbook)

Beitragende Reclamation-Quellen: P29, P30 [McK01, Mic04a]; vollständiger Allokator-Korpus A01–A23 siehe Abschnitt „A-Korpus“.

D.8 T7 Prefetch-Strategie

Diese Achse ist das *Prefetch-Organ* (T7) des Lebewesens SearchAlgorithm (vgl. Kapitel 3.2).

Die Prefetch-Achse bestimmt die Strategie zum vorausschauenden Laden von Knoten und Adressen entlang des erwarteten Such-Pfads in einer Trie-Datenstruktur. Sie unterstützt vier konkrete Strategien mit unterschiedlichen Triggermechanismen und Heuristiken: keine Prefetch (Mess-Baseline), distanzgeschätzte Software-Prefetch auf Basis von Knoten-Dichte und Cache-Latenz (ART-Standard), explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard) und pfadorientierte Prefetch mit Bundle-Granularität (PRT-ART Diplomarbeit).

Statische Sub-Achsen

ID	Name	Beschreibung
PF1	Triggermechanismus	Art der Kontrolle über den Prefetch-Moment: ob kein Mechanismus (Baseline), durch Compiler-Hinweise via Distance-Estimator, durch explizite Hardware-Prefetch-Instruktionen oder durch pfadorientierte Vorhersage
PF2	Distance-Heuristik	Berechnungsweise für die Prefetch-Distanz (wie weit voraus geladen wird): linear, adaptiv basierend auf Knoten-Dichte und Tier-Latenz, oder keine Heuristik
PF3	Granularität	Prefetch-Größe pro Trigger: Cache-Line (64 Byte, einzeln), Page (4K, Betriebssystem-Fenster) oder Bundle (mehrere Cache-Lines, Trie-pfadorientiert)

Bausteine

Variante	Beschreibung	Quelle
NonePrefetch	Keine Prefetch-Strategie (Baseline). Dient als Vergleichs-Referenz für Mess-Reihen. Trigger: keine; Heuristik: keine; Granularität: N/A. <code>is_active()</code> gibt false zurück.	Code/Baseline
DistanceEstimator Prefetch	Distanzgeschätzte Software-Prefetch (ART-Standard nach Leis ICDE 2013). Schätzt Cache-Distance zur nächsten Node basierend auf Knoten-Dichte [%] und Tier-Latenz [cycles] mit Heuristik: <code>estimate = clamp(1 + sparseness*8 + latency_factor, [1,16])</code> . Trigger: Compiler-Hinweise via <code>__builtin_prefetch</code> ; Heuristik: adaptiv (Dichte + Latenz); Granularität: Cache-Line. Setzt auf ART-Pattern (unodb/libart).	P01 (ART/unodb, Leis ICDE 2013)
HardwarePrefetch	Explizite Hardware-Prefetch-Instruktionen (Wormhole-Standard nach Wu EuroSys 2019). Nutzt <code>PREFETCHT0/T1/T2/NTA</code> auf Hash-Anchor-Chain ohne Software-Heuristik; CPU schätzt Distance autonom. Trigger: explizite Hardware-Instruktion; Heuristik: Hardware-managed; Granularität: Hardware-bestimmt. Lizenz GPL-3.0 blockiert <code>is_original-Linking</code> .	P07 (Wormhole, Wu EuroSys 2019)
PathOriented Prefetch	Pfadorientierte Prefetch-Heuristik (PRT-ART Diplomarbeit 2026). Pre-loadet alle Knoten entlang vorhergesagtem Trie-Pfad bevor Suche startet. Tracker verfolgt Suchpfad und empfiehlt nächste Adresse via linearer Schritt-Extrapolation. V11.1 Hot-Path-Hints: interpretiert erste 8 Byte eines Schlüssels als uint64-Adresse. Trigger: pfadorientierte Vorhersage; Heuristik: linear (Schritt-Extrapolation); Granularität: Bundle (mehrere Cache-Lines). Verwandte Literatur: Chen/Gibbons/Mowry SIGMOD 2001 (pB+-Trees Prefetching) + Mowry 1994.	P21 (verwandt: Chen/Gibbons/Mowry SIGMOD 2001); Diplomarbeit 2026

Beitragende Quellen: P01, P07, P21 [LKN13, WNJ19, CGM01].

Verwandte Literatur: P23, P25, P26, P27 [Kha11, MWR25, ZSZ⁺24, ZGH⁺25].

D.9 T8 Nebenläufigkeits-Synchronisations-Achse (Concurrency Axis T8)

Diese Achse ist das *Nebenläufigkeits-Organ* (T8) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Diese Achse regelt Synchronisationsmuster und sichere Speicherreclamation für nebenläufige Zugriffe auf Baum-/Indexstrukturen. Die Achse stellt compile-time wählbare Synchronisations-Patterns (None/Blocking/Reader-Writer/Optimistic/Lock-Free/Wait-Free) bereit sowie orthogonale Reclamation-Schemata (RCU/Hazard-Pointer) für sichere Speicherfreigabe in lock-freien Strukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
CC1	Synchronisations-Pattern (CC1)	Compile-time Varianten-Familie der Synchronisationsmuster: None (single-threaded Baseline), Blocking (pessimistic Mutex), ReaderWriter (shared_mutex), Optimistic (lock-free Versionsvalidierung), LockFree (CAS-Loops), WaitFree (gebundene Schritte). Jede Familie hat distinkte Laufzeit-Charakteristiken (0-Overhead bis Mutex-Overhead).
CC2	Reclamation-Schema (CC2)	Orthogonale Compile-time Varianten-Familie für sichere Speicherfreigabe in lock-freien Strukturen: RCU (deferred grace-period, via userspace-rcu), Hazard-Pointer (per-thread in-use-Marken). Kombinierbar mit lock-freien Patterns, optional bei Blocking/Optimistic.

Bausteine

Variante	Beschreibung	Quelle
NoneConcurrency	Keine Synchronisation, single-threaded Baseline. Beide <code>acquire()</code> und <code>release()</code> sind no-ops (volatile Dummy). Vergleichs-Nullpunkt für Concurrency-Overhead-Messung (F15).	Code/Baseline
Blocking Concurrency	Pessimistisches Mutex-Locking mit <code>std::mutex</code> . Globaler coarse-grained Lock. Serialisiert alle Zugriffe. Obergrenze des Locking-Overheads. Paper: Dijkstra CACM 1965.	Code/Baseline
ReaderWriter Concurrency	Single-Writer/Multi-Reader via <code>std::shared_mutex</code> . Lock-Sharing bei read-lastigen Workloads. Mittelgewicht zwischen pessimistischen und optimistischen Mustern. Paper: Courtois et al. CACM 1971.	Code/Baseline
OlcOptimistic Concurrency	Optimistic Lock-Coupling (ART-Sync): Versions-Counter pro Knoten, lock-freie Reader mit Retry-on-Conflict. Billig bei niedrigen Konflikten. Paper: Leis DaMoN 2016 / DEBULL 2019 (P08, <code>is_original=true</code> , Apache-2.0, unodb).	P08
LockFree Concurrency	Lock-free via Compare-And-Swap (<code>std::atomic</code> CAS-Loops). System-weiter Fortschritt. Generisches CAS-Pattern ohne Blockierung. Paper: Michael/Scott PODC 1996 (Anker-Paper).	Code/Baseline
WaitFree Concurrency	Wait-free: Jeder Thread schließt in beschränkter Schrittzahl ab (stärkste Garantie). OHNE Retry-Loop wie LockFree. Realisiert via Sequence-Lock / Fetch-Add. Paper: Herlihy TOPLAS 1991 (Theorie-Anker).	Code/Baseline

Variante	Beschreibung	Quelle
RcuConcurrency	Read-Copy-Update (McKenney OLS 2001): Lock-freie Reader via per-Thread Nesting-Zähler (RELAXED-Order, KEIN Memory-Barrier auf Read-Side). Reclamation-Schema orthogonal zu Pattern. Lokale Re-Impl (P29 userspace-rcu LGPL-2.1, is_original=false).	P29
HazardPointer Concurrency	Hazard Pointers (Michael TPDS 2004, P30): Pro-Thread markierte in-use-Zeiger verhindern vorzeitige Speicherfreigabe (ABA-safe). Reclamation-Schema. Lokale Re-Impl (P30 NO LICENSE, is_original=false). SEQ_CST-Store auf acquire().	P30
OlcReservedBlocks Concurrency	OLC + Cache-Line-Reservierte Value-Blocks (Multi-Writer Cache-Coherence-Storm-Vermeidung). Spezialisierung des Optimistic-Patterns mit zusätzlicher atomarer Block-ID-Reservierung (fetch_add). F.6 Doku 19 (prt-art-Optional, nicht ist-original).	Code/Baseline

Beitragende Quellen: Dijkstra 1965, Courtois et al. 1971, P08, Michael/Scott 1996, Herlihy 1991, P29, P30 [Dij65, CHP71, LABN16, MS96, Her91, McK01, Mic04a].

D.10 T9 Serialisierung (Kodierung und Kompression)

Diese Achse ist das *Serialisierungs-Organ* (T9) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2). Achse 10 bestimmt, wie Daten innerhalb des Cache-Engine-Systems kodiert und komprimiert werden: vom Raw-Binary-Speicherlayout über variable Längenkodierung bis zur Succinct-Bit-Packing. Die Wahl der Serialisierungsstrategie beeinflusst direkt CPU-Kosten (Delta-Encoding, Bit-Manipulation, LEB128-Varint), Speicher-Overhead und IO-Bandbreitennutzung — kritisch für LSM-Trees und Read-Heavy-Workloads mit Approx-Filter (SuRF/LOUDS).

Statische Sub-Achsen

ID	Name	Beschreibung
SR1	Byte-Order und Wortformat	Wie werden Bytes innerhalb eines Datensatzes angeordnet? Raw (memcpy Roh-Layout), Varlen (LEB128-Varint mit Signalisierungsbits), oder Packed (Bit-für-Bit-Packing)?
SR2	Dichte (Speicherauslastung)	Speicher-Overhead pro Datensatz: Sparse (vollständige Records), Dense (mit Padding), oder Succinct (Information-theoretisch dicht, $n \cdot H + o(n)$ bits)?
SR3	Kompression und Vorverarbeitung	Block-Kompression (LZ4/Snappy, LZ77-basiert) oder Delta-Encoding (Zigzag-Transform für negative Deltas)? Oder keine Kompression?

Bausteine

Variante	Beschreibung	Quelle
RawBinary Serialization	Roh-Byte-Speicherlayout (memcpy). Baseline ohne Transformation. Minimaler CPU-Aufwand. Typ. für In-Memory-Workloads ohne Kompression. Variante: <code>serialisierte_scan()</code> liest 32-Bit Felder + Order-sensitive Checksum.	Code/Baseline
VarLen Serialization	Variable-Längen-Kodierung mit Signalisierungsbits (LEB128-VarInt). Standard für ART (Leis ICDE 2013 P01). Kleine Werte = wenige Bytes. Signalisierungsbits markieren Byte-Länge. Datenabhängige Schleife (1-5 Bytes). FNV-1a-Mix der Varint-Bytes.	P01

Variante	Beschreibung	Quelle
Succinct Serialization	Bit-für-Bit-Packing mit minimaler Bitbreite (LOUDS/SuRF). Erreicht Information-theoretisches Minimum $n \cdot H + o(n)$ bits. Read-Only nach Build. Optimal für Approx-Filter + Read-Heavy. Höchster Per-Element-CPU-Aufwand (echte Bit-Manipulation). Variante: <code>serialize_scan()</code> mit 64-Bit Akkumulator.	P10,P09
Compressed Serialization	Block-Kompression (LZ4/Snappy) auf Roh-Binary aufgesetzt. Trade-off: CPU-Kosten vs IO/Memory-Bandbreite. Variante: Delta-Encoding gegen Vorgänger + Zigzag-Transform (negatives Delta $\rightarrow$ kleine unsigned). Typisch für LSM-Trees. Mittlerer CPU-Aufwand.	Code/LZ77 (Ziv/Lempel 1977) [ZL77]

Beitragende Quellen: P01, P09, P10, Ziv/Lempel 1977 [LKN13, Jac89, ZLL⁺18, ZL77].

D.11 T10 Telemetrie und Messinstrumentierung (axis_11)

Diese Achse ist das *Telemetrie-Organ* (T10) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2). Definiert Strategien zur Messung von Index-Operationen (Insert, Lookup, Erase) mit verschiedenen Overhead-Profilen. Unterstützt Density-Tracking für adaptive Node-Typ-Wahl, globale Insert-Counter für Durchsatz-Messungen, HDR-Histogramme für Latenz-Quantile und leaf-only-Counter zur Vermeidung von Cache-Line-False-Sharing in den Messzyklen.

Statische Sub-Achsen

ID	Name	Beschreibung
TM1	Scope (Geltungsbereich)	Gültigkeitsbereich der Zählerverwaltung: leaf-only (nur Blatt-Knoten), per-node (alle Knoten), oder global (einzelner Akkumulator für die ganze Struktur). Compile-Time-Wahl, beeinflusst Overhead durch Konkurrenz-Druck auf gemeinsame Cache-Linien.
TM2	Metrik-Typ (Messgröße)	Art der gemessenen Metrik: counter (Zähler für Ereignisse), histogram (Verteilung mit Quantilen), oder density (Slot-Belegung pro Knoten). Beeinflusst den Speicher-Footprint und die Mess-Granularität.
TM3	Overhead-Level (Messlast)	Größenordnung des Mess-Overheads: low (atomare Operationen), medium (per-node-Datenstrukturen), high (Histogramm-Logging). Bestimmt die Eignung für verschiedene Workload-Szenarien (Benchmarking vs. Produktion).

Bausteine

Variante	Beschreibung	Quelle
LeafOnly Counter	Zähler NUR in Blatt-Knoten (vermeidet Cache-Line-Ping-Pong durch Verzicht auf Inner-Node-Updates). Niedriger Overhead bei leaf-only Scope. Entspricht dem kohärenz-schonenden Zähler-Konzept aus Kuehn DaMoN 2023, obwohl das Original-Paper kein Leaf-Counter-Algorithmus ist — CE-Baseline für False-Sharing-Vermeidung.	P28 (Kuehn DaMoN 2023, Kontext; Code/Baseline)
DensityTracker	Per-Node Density/Slot-Occupancy-Tracking für adaptive Node-Type-Wahl ($4 \rightarrow 16 \rightarrow 48 \rightarrow 256$). Kontextuelle Telemetrie zur ART-Struktur (Leis ICDE 2013), misst Slot-Belegung aller Knoten. Höherer Overhead als LeafOnly, liefert aber Adaption-Signal für dynamische Rebalancing-Entscheidungen.	P01 (ART/Leis ICDE 2013, Kontext; Code/Baseline)

Variante	Beschreibung	Quelle
InsertCounter	Globaler atomarer Insert-Zähler mit minimalem Overhead (1 atomic increment pro Insert). Kontextuelle Telemetrie zur HOT-Struktur (Binna SIGMOD 2018), global Scope. Ausreichend für Durchsatz-Mess-Reihen, keine Laten-Granularität.	P02 (HOT/Binna SIGMOD 2018, Kontext; Code/Baseline)
Latency Histogram	HDR-Histogramm zur Lookup-Latenz-Verteilung (p50/p95/p99 Quantile). Basiert auf HdrHistogram (Gil Tene), CC0-1.0 Lizenz. Höchster Overhead (pro Lookup), liefert aber vollständige Latenz-Statistik für SLA-orientierte und research-grade Messreihen. Einziger is_original-Kandidat der Achse.	HdrHistogram (github.com/HdrHistogram/HdrHistogram_c, CC0-1.0, is_original)

Beitragende Quellen: HdrHistogram (Gil Tene) [Ten12].

Verwandte Literatur: P01, P02, P28 [LKN13, BZP⁺18, Kue23].

D.12 T11 Wert-Handle (Value-Handle)

Diese Achse ist das *Wert-Handle-Organ* (T11) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2). Die Wert-Handle-Achse definiert, wie Datenwerte (Values) im Cache-Index gespeichert und zugegriffen werden: inline im Node-Slot, extern in einem Pool, mittels immutable-shared References (RCU-Style), versioned Pointer (MVCC), oder verkettete externe Referenzen (Multi-Value). Diese Achse charakterisiert die Speicherlayout-Strategie für Values und bestimmt damit die Zugriffslatenz (direkt vs. indirekt) und Concurrency-Semantik (Snapshot vs. Versioning).

Statische Sub-Achsen

ID	Name	Beschreibung
VH1	Speicherort (Storage-Location)	Compile-Time Varianten für die physische Platzierung des Value: inline im Node-Slot (kombiniert Key/Value in einem Slot ohne Indirektion) oder extern in einem separaten Pool (Node hält nur Offset/Pointer).
VH2	Eigentümer-Semantik (Ownership)	Compile-Time Varianten für Ownership und Sharing-Strategie: owned (exclusive per Key), shared (immutable reference-counted via RCU-style), weak (borrowing ohne Increment).
VH3	Versionierung (Versioning)	Compile-Time Varianten für Concurrency-Kontrolle: keine Versionierung (simple Ownership), Version-Tags in Pointer (MVCC für Reader-Snapshot-Isolierung).

Bausteine

Variante	Beschreibung	Quelle
InlineValueHandle	Value wird direkt im Node-Slot eingebettet (keine Indirektion). Standard für kleine Values wie <code>uint64_t</code> . Minimale Zugriffslatenz (1 direkter Read), aber Node-Speicher wird durch größere Values beansprucht. Quelle: ART (Leis ICDE 2013, P01).	P01
ExternalPoolValueHandle	Value extern in Pool gespeichert; Node hält nur Pool-Offset. Ermöglicht variable Value-Größen und kompaktere Nodes, kostet aber 1 zusätzlichen Cache-Miss pro Lookup (Pointer-Chasing). Quelle: Wormhole (Wu EuroSys 2019, P07).	P07

Variante	Beschreibung	Quelle
ImmutableSharedRef ValueHandle	Value via reference-counted Pointer (RCU-Style Snapshot-Sharing). Optimal für Reader-Concurrency: Leser sehen stabile Snapshots, Writer erstellen neue immutable Snapshots. Requires RefCount-Acquire/Release pro Zugriff. Quelle: RCU (McKenney OLS 2001, P29).	P29
VersionedPointer ValueHandle	Pointer mit Version-Tag im oberen Byte (MVCC). Ermöglicht Multi-Version-Concurrency-Control: Reader bekommt Snapshot-Version zur Sichtbarkeitskontrolle, Writer erstellt neue Version. Tombstone-Erkennung via Versions-Bit. Quelle: Masstree (Mao EuroSys 2012, P03) und SMART (OSDI 2023).	P03
ChainRefValue Handle	Multi-Value verkettete externe Referenz (Linked-List). Node hält Offset zum Chain-Head im Pool; jeder Chain-Knoten hält (value_offset, next_offset). Teuerste Variante mit 2 abhängigen Derefs pro Value-Zugriff. Quelle: prt-art (lokale Re-Impl, F.6 Migration).	Code/Baseline

Beitragende Quellen: P01, P03, P07, P29, SMART (OSDI 2023) [LKN13, MKM12, WNJ19, McK01, LZS⁺23].

D.13 T12 Instruktionssatz-Architektur (ISA)

Diese Achse ist das *ISA-Organ* (T12) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2).

Achse T12 spezifiziert die Ziel-CPU-Instruktionssatz-Architektur als Compile-Time-Konstante und ermöglicht damit Cross-Platform-Optimierungen für vier dominante Serverarchitekturen: x86-64 (Intel/AMD), AArch64 (ARM), Power-ISA (IBM) und RISC-V (Open-Source). Jede Variante definiert eine hardware-spezifische SIMD-Baseline (SSE2, NEON, VSX, skalar) und charakterisiert damit die Vektorbeschleunigungspotenziale für Cache-Engine-Operationen wie `simd_field_sum` (Akkumulation von 32-bit-Wort-Feldern).

Statische Sub-Achsen

ID	Name	Beschreibung
IS1	Wortbreite (Word-Size)	CPU-Datenpfadbreite: 32-bit, 64-bit oder 128-bit. In der Cache-Engine auf 64-bit standardisiert für alle vier ISA-Profilen.
IS2	CPU-Hersteller/Familie (Vendor)	Instruktionssatz-Familie: <code>intel_amd</code> (x86-64), <code>arm</code> (AArch64), <code>ibm</code> (Power-ISA), <code>open</code> (RISC-V). Bestimmt SIMD-Extension-Kompatibilität: SSE2/AVX2/AVX-512 (x86-64), NEON/SVE/SVE2 (AArch64), VSX (Power), RVV (RISC-V).
IS3	ZIH-Cluster-Zugehörigkeit	Produktionsmaschinen an TU Dresden: Barnard (AMD EPYC 7763 Zen 3, x86-64), Capella (Intel Xeon, x86-64), GraceHopper (NVIDIA Grace Hopper ARMv9 Neoverse V2, AArch64), N/A (Forschungsboards, RISC-V/Power). Ermöglicht Hardware-spezifische Experimente mit echten Produktionscluster-Profilen.

Bausteine

Variante	Beschreibung	Quelle
Amd64Isa	x86-64 / Intel 64 ISA (AMD/Intel Server+Desktop). Dominante Serverplattform mit SSE2-SIMD-Baseline (4 uint32-Lanes pro 128-bit-Register). Implementiert echte SSE2-Vektorisierung via <code>_mm_loadu_si128</code> + <code>_mm_add_epi32</code> auf nativen x86-64-Hosts; skalarer Fallback auf Nicht-x86-Build-Hosts. ZIH-Präsenz: Barnard (AMD EPYC 7763 Zen 3), Capella (Intel Xeon).	Vendor-Spec (AMD 2000 / Intel fortlaufend)
Aarch64Isa	ARMv8-A 64-bit ISA (ARM Holdings). Wachsende Serverrelevanz (Apple M-Series, AWS Graviton, Ampere Altra). SIMD-Baseline: NEON (128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert NEON-Breite ohne ARM-Intrinsics auf x86-Build-Hosts); echte NEON-Vektorisierung auf nativen ARMv8-Hosts. ZIH-Präsenz: GraceHopper (NVIDIA Grace Hopper, Neoverse V2 + Hopper GPU).	Vendor-Spec (ARM Ltd., ab 2011/2013)
PowerPcIsa	Power ISA (POWER9/10, ppc64le Little-Endian). IBM Power Systems (AC922, IC922, S1022). SIMD-Baseline: VSX (Vector-Scalar Extension, 128-bit, 4 uint32-Lanes). Cache-Engine-Implementierung: 4-fach entrollter Skalar-Fallback (modelliert VSX-Breite ohne Power-Intrinsics auf x86-Build-Hosts). Relevant für HPC-Workloads mit NVLink-GPU-Anbindung (AC922). ZIH-Präsenz: nicht primär, aber für Forschungs-Vergleiche verfügbar.	Vendor/Standards-Spec (IBM/ OpenPOWER, v3.0 2017, v3.1 2020)
RiscVIsa	RISC-V RV64GC Open-Source ISA (RV64 General + Compressed). Wachsende Bedeutung (SiFive, Alibaba Xuantie, Andes, ETH Zürich Cores). SIMD-Baseline: KEINE (RVV ist optional, nicht Teil von RV64GC baseline). Cache-Engine-Implementierung: bewusst plain skalarer Pfad (nicht entrollt) – modelliert fehlende Vektoreinheit der Basis-ISA und erzeugt damit reale, langsamere Laufzeit gegenüber x86-64 und ARM. ZIH-Präsenz: embedded + Forschungs-Boards.	Standards-Spec (RISC-V Intl.; UCB TR ab 2011)

Beitragende Quellen: —

D.14 T13 Index-Organisation

Diese Achse ist das *Index-Organisations-Organ* (T13) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Klassifizierung der Datenspeicherung und Indexstrukturierung nach drei orthogonalen Dimensionen: Storage-Order (Heap/Clustered/Ungeordnet), Index-Count (keine/einzeln/mehrfach) und Data-Embedding (separate Speicherung vs. in Leaf-Pages eingebettet). Diese Achse modelliert fundamentale DBMS-Entwurfsentscheidungen, die Cache-Verhalten, Zugriffsmuster und I/O-Charakteristiken auf Basis von Index-Order vs. Storage-Order-Alignment entscheidend beeinflussen.

Statische Sub-Achsen

ID	Name	Beschreibung
<code>iol_storage_order</code>	Storage-Order	Anordnung der Datensätze im Speicher: ungeordnet (Insert-Order im Heap), sortiert nach Primary-Key (Clustered), oder mit Pointer-Indirektion (Non Clustered). Definiert die sequenzielle oder zufällige Zugriffsmuster bei Range-Scans.

ID	Name	Beschreibung
io2_index_count	Index-Multiplizität	Anzahl unterstützter Indizes pro Tabelle: keine (Heap-Baseline), Single-Index (Clustered, MySQL InnoDB Primärschlüssel), oder Multiple Secondary-Indizes (NonClustered, SQL Server NONCLUSTERED INDEX).
io3_data_embedding	Daten-Einbettung	Speicherplatzierung der Daten relativ zum Index: getrennt im Row-Store (Clustered/NonClustered mit Pointer-Hop), oder direkt in Index-Leaf-Pages eingebettet (IOT/Trie-Style wie ART/HOT).

Bausteine

Variante	Beschreibung	Quelle
ClusteredIndex Organization	Speicher-Reihenfolge entspricht Index-Reihenfolge (1 Primary-Key). Optimiert für Range-Scans entlang des Primary-Key mit sequenziellen Disk-Reads (cache-freundlich). Standard: MySQL InnoDB, PostgreSQL CLUSTER, SQL Server CLUSTERED INDEX. Zugriffsmuster: sequenzielle Forward-Iteration ohne Predicate-Evaluation, O(n) Summen-Scan.	Bayer & McCreight, Acta Informatica 1972 (10.1007/BF00288683)
HeapIndex Organization	Storage-Reihenfolge = Insert-Order, KEIN Index (Baseline). Lookup mittels O(n) Full-Scan mit Predicate-Evaluation pro Record (datenabhängiger Branch, kein früher Abbruch). Standard: PostgreSQL Heap-Tabellen, HBase. Zugriffsmuster: ungeordneter sequenzieller Scan mit Vergleichslast und Branch-Misprediction.	Garcia-Molina, Ullman & Widom, Database Systems (Textbook Baseline)
IotIndexOrganization	Daten direkt in Index-Leaf-Pages eingebettet (Oracle IOT, SQL Server CLUSTERED INDEX style). Spart Pointer-Indirektion vs. NonClustered. Äquivalent zu B+Tree-Leaves mit eingebetteten Daten oder Trie-Knoten (ART/HOT/Wormhole). Zugriffsmuster: sequenziell wie Clustered, aber mit Zusatzlast durch Daten im Leaf (zwei Felder pro Record).	Srinivasan et al., VLDB 2000 (Oracle8i IOT) [SDF ⁺ 00]
NonClusteredIndex Organization	Index-Reihenfolge != Storage-Reihenfolge, mehrere Secondary-Indizes pro Tabelle. Lookup über Index + Pointer-Hop zur Daten-Row (Random-Stride, Cache-Misses). Standard: SQL Server NONCLUSTERED INDEX, PostgreSQL CREATE INDEX. Zugriffsmuster: Index-navigation gefolgt von zufälligen Storage-Hops (LCG-determiniert, REAL cache-Miss-Latenz durch TLB/L2-Invalidation).	Comer, ACM Computing Surveys 1979 (The Ubiquitous B-Tree, 10.1145/356770.356776)

Beitragende Quellen: Bayer & McCreight 1972, Comer 1979, Srinivasan et al. 2000, Garcia-Molina, Ullman & Widom (Textbook) [BM72, Com79, SDF⁺00, GMUW09].

D.15 T14 I/O-Dispatch-Achse

Diese Achse ist das *I/O-Dispatch-Organ* (T14) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2). Modellierung und Auswahl von I/O-Strategien für die Persistierung und den Speicherzugriff im Index: Speicherung im RAM-only (Baseline), gepuffert via OS-Page-Cache (Standard), direkter I/O mit `O_DIRECT`-Flag (NVMe-Optimierung), oder Memory-Mapped File I/O (Persistent Memory). Diese Achse

ermöglicht die Entkopplung der Such- und Baum-Strukturen von der Persistierungs-Implementierung und gestattet A/B-Messungen der Dispatch-Overhead-Profile unter identischen Workloads.

Statische Sub-Achsen

ID	Name	Beschreibung
IO1	Persistierungsmodell	Klassifiziert die Speicherart: RAM-only (keine Persistierung, reine Mess-Baseline), oder Disk/Persistent-Memory (Datei-gesicherte Indizes mit verschiedenen Caching-Strategien).
IO2	Caching-Strategie	Charakterisiert die OS-Puffer-Nutzung: Buffered (OS Page-Cache, read-ahead + write-back, Standard), Direct I/O (<code>O_DIRECT</code> , Page-Cache-Bypass, 512B/4KB-Alignment), oder Memory-Mapped (file-backed <code>mmap</code> mit automatischer OS-Pagination und page-fault-getriebenem Zugriff).
IO3	Schreib-Dauerhaftigkeit	Legt die Konsistenz-Garantien fest: keine <code>fsync</code> (höchste Performance, keine Absicherung), <code>fsync</code> nach Batch (Kompromiss, Batch-Commits), oder atomare Schreibvorgänge (höchste Sicherheit, niedrigste Performance). In dieser Implementierung hauptsächlich als konzeptionelle Sub-Achse vorhanden; operative Umsetzung erfolgt über die Caching-Strategie.

Bausteine

Variante	Beschreibung	Quelle
InMemoryOnly	Rein RAM-basierter Index ohne Persistierung. Direkter, sequentieller Record-Zugriff ohne Dispatch-Overhead. Schnellster Pfad und Vergleichs-Nullpunkt der Achse. Verwendet für Mess-Baseline und Performance-Referenzierung unter Bedingungen ohne I/O-Latenz.	Code/Baseline
BufferedIo	Standard <code>read()/write()</code> via OS-Page-Cache mit automatischem read-ahead und write-back. Default für allzweck-DBMS. Trade-off: bequem, aber Double-Buffering (App-Cache + OS-Cache führt zu erhöhter Speichernutzung). Papier-Anker: Stonebraker CACM 1981 (Konzept-/Position-Paper Operating System Support for Database Management, DOI 10.1145/358699.358703). Confidence: medium (Engineering-Pattern, kein Algo-Code).	P-Stonebraker-1981
DirectIo	Bypass des OS-Page-Cache mittels <code>O_DIRECT</code> -Flag (<code>open(2)</code>), NVMe-SSD-optimal. 512B/4KB-Alignment erforderlich. Ermöglicht DBMS-eigene Cache-Strategien (RocksDB, MySQL). Höhere CPU-Kosten, aber keine Cache-Pollution durch Background-Activity. Keine dedizierte Papier-Quelle (Linux-API seit 2.4.10); wird als bewusste Engineering-Baseline behandelt.	Code/Baseline
MmapIo	Memory-Mapped File I/O (<code>mmap</code>) mit Persistent Memory oder Read-Heavy Workloads. OS pagiert automatisch, kein Copy zwischen User- und Kernel-Space. Trade-off: page-fault-Latenz-Spitzen vs. Reuse-Freundlichkeit. Papier-Anker: Crotty et al. CIDR 2022 (Anti-Pattern-Baseline Are You Sure You Want to Use MMAP in Your DBMS?, db.cs.cmu.edu/.../cidr2022-p13-crotty.pdf). Bewusst als Anti-Pattern-Referenz zitiert, um Grenzen und Overhead zu dokumentieren. OSS-Referenz MIT-lizenziert.	P-Crotty-2022

Beitragende Quellen: Stonebraker-1981, Crotty-2022 [Sto81, CLP22]

D.16 T15 Migrations-Strategie-Achse

Diese Achse ist das *Migrations-Organ* (T15) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2). Die Migrations-Strategie-Achse (T15, `axis_migration`) kapselt Entscheidungslogiken zur Daten-Platzierung across mehrschichtige Speicher-Hierarchien (RAM/SSD/HDD, Hot/Cold-Tiers). Sie modelliert, wann und wie häufig zugegriffene Daten zu schnellerem Storage promoviert bzw. selten zugegriffene demoviert werden, ohne echte Tier-Migration durchzuführen (Entscheidungslogik-Messung nur). Unterstützt vier konkrete Strategien: static (Baseline), Heat/Cold-Separation, Multi-Tier-Hierarchie, ML-driven Adaptive.

Statische Sub-Achsen

ID	Name	Beschreibung
MG1	Trigger-Mechanismus (MG1)	Bestimmt, wann eine Migrations-Entscheidung auslöst wird: keine (static), Schwellwert-basiert (Threshold), beobachtungsgesteuert (Observation). Compile-Time Variante pro Strategy.
MG2	Migrations-Richtung (MG2)	Bestimmt die Richtung von Daten-Bewegung: keine (static), aufwärts (RAM→SSD), abwärts (SSD→RAM), bidirektional. Compile-Time Variante.
MG3	Migrations-Granularität (MG3)	Spezifiziert die Skalierungseinheit für Migrations-Entscheidungen: keine (static), Page (4KB), Block (64KB), Object (einzelnes Item). Compile-Time Kategorie.

Dynamische Sub-Achsen

ID	Parameter	Beschreibung
D1	Entscheidungs-Schwellwert (Decision Threshold)	Laufzeit-Parameter: Hot/Cold-Klassifikations-Grenzwert (z.B. Recency-Counter-Limit für LRU-basierte Promotions/Demotions). Benutzer-konfigurierbar.
D2	Gewichtungen für Feature-Scoring	Laufzeit-Parameter für AdaptiveMigration: Gewichte (<code>w_freq</code> , <code>w_rec</code>) für Häufigkeits- und Recency-Features in der ML-Linearkombination. Self-tuning möglich.
D3	Tier-Konfiguration	Laufzeit: Anzahl Speicher-Tiers (z.B. <code>kTiers=3</code> für RAM/SSD/HDD in Tier BasedMigration), Kapazitäten pro Tier, Promotions/Demotion-Schwellen.

Bausteine

Variante	Beschreibung	Quelle
<code>NoMigration</code>	Statische Platzierung ohne Migration (Baseline). Daten werden einmalig positioniert, keine Bewegung zwischen Tiers. Benchmark-Nullpunkt für Vergleich mit aktiven Strategien.	Code/Baseline
<code>HotColdMigration</code>	Heat/Cold-Separation via LRU + probabilistisches Recency-Counting. Häufig zugegriffene Daten (hot) werden zu schnellerem Cache promoviert, selten zugegriffene (cold) zu langsamem Storage demoviert. Standard für LRU-K und LFU-Cache-Systeme.	Anti-Caching (PVLDB 2013) [DPT ⁺ 13]

Variante	Beschreibung	Quelle
TierBased Migration	Multi-Ebenen-Migration RAM→NVM→SSD→HDD mit Modulo-basiertem Tier-Mapping. Jedes Datenitem wird anhand eines Feldwertes modulo-auf Ziel-Tier gemappt. Standard für RocksDB, LSM-Trees und Tiered-Cache-Systeme mit konfigurierbaren Schwellwerten.	Managing NVM in DBMSs (SIGMOD 2018) [RLK ⁺ 18]
AdaptiveMigration	ML-driven adaptive Migration mit Online-Learning (Cachelib/ LeCaR-Stil). Gewichtete Linearkombination von Häufigkeits- und Recency-Features berechnet adaptiven Score; gleitender Lern-Akkumulator modelliert Online-Learning. Höherer Overhead vs. optimale Hit-Rate.	ML-based LeCaR (USENIX HotStorage 2018) [VRM ⁺ 18]

Beitragende Quellen: Anti-Caching (PVLDB 2013), LeCaR (HotStorage 2018), Managing NVM (SIGMOD 2018) [DPT⁺13, VRM⁺18, RLK⁺18].

D.17 T16 Filter-Achse

Diese Achse ist das *Filter-Organ* (T16) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2). Die Filter-Achse T16 bietet wählbare probabilistische und deterministische Membership-Query-Filter (Punkt- und Bereichsabfragen). Sie deckt klassische Bloom-Filter mit k Hash-Funktionen, den deletable-Cuckoo-Filter mit Bucket-Hashing und Fingerprints, den succinct Range-Filter SuRF mit LOUDS-Trie-Navigation sowie den platzsparenden Xor-Filter mit konstanter 3er-XOR-Latenz ab. Zweck: Trade-offs zwischen Raum, Abfragelatenz, Mutability und Falsch-Positiv-Rate direkt im Cache-Engine-Experiment messbar machen.

Statische Sub-Achsen

ID	Name	Beschreibung
FT1	Abfrage-Typ	Unterscheidung zwischen Punkt-Abfragen (membership: key enthalten?) und Bereichs-Abfragen (range: keys in [a,b]?). Punkt-Filter sind Bloom/Cuckoo/Xor; nur RangeSurfFilter unterstützt Range-Semantik.
FT2	Mutierbarkeit	Unterteilung in unveränderbare Filter (Bloom, RangeSurf, Xor nach Build immutable) vs. Mutable (Cuckoo mit delete-Operation). Bestimmt Design des Wrapper und Cachingmöglichkeiten während Laufzeit.
FT3	Fehler-Profil	Charakterisierung der fehlertoleranten Semantik: False-Positive-Only (Bloom, Cuckoo, Xor — Filter sagt 'ja' falsch), vs. lossless (theoretisch deterministische Antwort). Bloom: approx. $k \cdot n$ Hash-Tests; Xor: exakt 3 XOR-Operationen (branch-frei).

Bausteine

Variante	Beschreibung	Quelle
BloomFilter	Klassischer Bloom-Filter nach Bloom 1970: m-Bit-Bitmap, k unabhängige Hash-Funktionen (double-hashing nach Kirsch/Mitzenmacher 2006). Punkt-Abfragen nur; False-Positives möglich, keine Deletes. Implementierung mit echter persistenter m-Bit-Bitmap (8 KiB, m=65536, k=4 double-hashing), die im Mess-Pfad B über die tatsächlich eingefügten Schlüssel geprobt wird; die ehrlich deklarierte Pseudo-Bitmap-Probe (k=4 Hash-Tests pro Query, charakteristische early-exit-Latenz bei Mismatch) verbleibt nur im Pfad-A-Segment-Timing-Scan.	Bloom 1970 (CACM 13:422-426)
CuckooFilter	Cuckoo Filter nach Fan et al. CoNEXT 2014: Bucketed Cuckoo-Hashing mit 1-Byte-Fingerprints. Unterstützt Deletes und Lookup-Counting (Vorteil vs Bloom). Probe-Charakteristik: 2 Bucket-Lookups + Fingerprint-Vergleich (partielle-Schlüssel-Cuckoo nach CoNEXT 2014 §3: $i2 = i1 \text{ XOR } \text{hash}(fp)$). Höhere Insert-Kosten, aber flexible Mutierbarkeit.	Fan et al. CoNEXT 2014 (DOI:10.1145/2674005.2674994)
RangeSurfFilter	SuRF (Succinct Range Filter) nach Zhang et al. SIGMOD 2018: LOUDS-Bitmap-Trie für Range-Abfragen. Einziger Filter mit Range-Semantik (lower_bound/upper_bound auf Byte-Prefix-Ebene). Probe-Charakteristik: mehrstufiger Trie-Abstieg mit succinct-LOUDS-Navigation (bis kMaxDepth=6), nicht exakt 1 Hash-Test, daher latenz-variabel je Prefix-Länge. Praktisch in RocksDB SST-Optimierungen.	P10 (Zhang et al. SIGMOD 2018)
XorFilter	Xor Filter nach Graf+Lemire 2020 (JEA 25): XOR-basierter Filter mit ca. 9 bits/key (vs. 10 bits/key Bloom bei gleicher FP-Rate). Immutable nach Build. Probe-Charakteristik: exakt 3 unabhängige Tabellen-Zugriffe (3 disjunkte Tabellen-Drittel h0, h1, h2) + 3er-XOR, branch-frei, konstante Latenz ohne early-exit. Sicherheit gegen Wegoptimierung, kleinster Speicherfootprint aller Filter.	Graf+Lemire 2020 (JEA 25:Article 1.5, DOI:10.1145/3376122)

Beitragende Quellen: Bloom 1970, Fan et al. CoNEXT 2014, Zhang et al. SIGMOD 2018, Graf+Lemire JEA 2020 [Blo70, FAKM14, ZLL⁺18, GL20].

D.18 T17 Pufferung Q1 (Operations-Puffer)

Diese Achse ist das *Pufferungs-Organ (Q1)* (T17) des Lebewesens *SearchAlgorithm* (vgl. Kapitel 3.2).

Die Achse *axis_q1_queueing* realisiert verschiedene Pufferungsstrategien für Operationen im Cache-Engine-Kontext. Sie umfasst Lehrbuch-ADTs (FIFO, LIFO, Append-Only, Priority-Heap), spezialisierte Versioning-Techniken (Delta-Chain, Copy-on-Write, Epoch, Tombstone) sowie moderne Lock-Free-Queues (SPSC/MPMC nach Lamport und Vyukov). Die Strategien werden übersetzungszeitlich über die Achsen-Registry (*mp_list* der W2-Recherche, Enable-Flag-gefiltert) pro Permutations-Binary selektiert (kein Runtime-Switch) und sind klassifizierbar nach Zugriffsmuster (sequenziell, sortiert, zyklisch, versioniert, batched, lock-free).

Statische Sub-Achsen

ID	Name	Beschreibung
QS1	Sequenzieller Zugriff	FIFO/LIFO/Append-Only: Zugriff am Anfang oder Ende, typisch Write-Coalescing + MemTable-Pufferung

ID	Name	Beschreibung
QS2	Sortierter Zugriff	Skip-List/Priority-Heap: Sortierter Drain, typisch LSM-Compact und Hot-Key-Eviction
QS3	Zyklischer Zugriff	Bounded Ring-Buffer: Feste Kapazität, Disruptor-Pattern, wrap-around mit Overflow-Handling
QS4	Versionierter Zugriff	Delta-Chain/Tombstone/CoW/Epoch: Multi-Version Concurrency Control, Snapshot-Isolation, Reclamation-Windows
QS5	Batch-Operationen	Batched-Insert-Buffer: Sammlung von Operationen zu Sub-Batches, SIMD-Vectorization, Cache-Locality-Optimierung
QS6	Lock-Free Concurrent	SPSC/MPMC: CAS-Loop oder wait-free Lamport-Modell, echte Lock-Freedom ohne Mutex

Bausteine

Variante	Beschreibung	Quelle
NoBuffer	Passthrough/Identität (Baseline): <code>put()</code> ist no-op, <code>get()</code> liefert immer <code>std::nullopt</code> . Sinnvoll als Vergleichspunkt und in klassischen B+/ART-Szenarien mit direktem Leaf-Zugriff. Family Q01, QS1 <code>sequential_access</code> .	Code/Baseline
AppendOnlyBuffer	Append-Only Linear Buffer (LSM-MemTable, Bw-Tree-Delta): monoton wachsendes <code>std::vector</code> ohne mittiges Löschen. Optimiert für reine Append + Bulk-Drain. Family Q02, QS1 <code>sequential_access</code> . Literatur: Levandoski/Lomet/Sengupta Bw-Tree ICDE 2013, O'Neil LSM 1996.	Code/Baseline
FIFOQueueBuffer	FIFO-Queue (<code>std::deque</code>): First-In-First-Out via <code>deque::push_back / deque::pop_front</code> . Unbounded, kann mittig effizient wachsen. Lehrbuch-ADT, typisch für Write-Coalescing. Family Q03, QS1 <code>sequential_access</code> .	Code/Baseline
LIFOStackBuffer	LIFO-Stack (<code>std::vector</code>): Last-In-First-Out via <code>vector::push_back / vector::pop_back</code> . Unbounded, optimiert für Hot-Path-Wiederverwendung und Versions-Tombstones. Family Q04, QS1 <code>sequential_access</code> .	Code/Baseline
BoundedRingBuffer	Bounded Ring-Buffer (Disruptor-Pattern): Feste Kapazität, zyklischer Zugriff mit wrap-around. Overflow: <code>drop_oldest</code> (default). <code>iterable_aspect_t</code> : Capacity als Laufzeit-Loop. Family Q05, QS3 <code>cyclic_access</code> . Literatur: LMAX Disruptor 2011.	LMAX 2011
PriorityHeapBuffer	Priority-Heap (<code>std::priority_queue</code>): Max-Heap, <code>get()</code> liefert höchste Priorität zuerst. Erste Strategie mit <code>supports_priority_ordering()=true</code> . Unbounded, Lehrbuch-ADT (Williams 1964). Anwendung: LRU-Approximation, Hot-Key-Promotion, Eviction-Queues. Family Q06, QS2 <code>ordered_access</code> .	Code/Baseline
DeltaChainBuffer	Delta-Chain Buffer (Bw-Tree, Levandoski 2013): Erste versioned-Strategie (<code>is_versioned()=true</code>). Jeder <code>put()</code> hängt Delta-Record mit aufsteigender Version-ID an Spitze der Chain an. <code>get()</code> LIFO-Drain mit Versions-ID. Append-Versioning-Paradigma. Family Q07, QS4 <code>versioned_access</code> .	Bw-Tree (Levandowski et al. ICDE 2013) [LLS13]
SkiplistBuffer	Skip-List Buffer (sortierter Buffer, RocksDB-Stil): <code>put()</code> fügt in sortierter Reihenfolge ein via <code>std::set</code> . <code>get()</code> ASCENDING-drain (minimum first), typisch für LSM-Compact-Output. Unterschied zu <code>PriorityHeapBuffer</code> : Skiplist=ascending, Heap=descending. Literatur: Pugh CACM 1990. Family Q08, QS2 <code>ordered_access</code> .	Pugh 1990, RocksDB
TombstoneBuffer	Tombstone-Marker Buffer (LSM + MVCC): <code>put()</code> erzeugt Live-Eintrag, deleted-Einträge bleiben als Tombstones (<code>std::nullopt</code>) bis Compact-Lauf. <code>get()</code> überspringt Tombstones FIFO-artig, <code>is_versioned()=true</code> (Erase-Versioning, anders als Delta-Chain Append-Versioning). <code>size()</code> zählt nur Live-Slots. Family Q09, QS4 <code>versioned_access</code> . Literatur: O'Neil LSM-tree 1996.	O'Neil 1996

Variante	Beschreibung	Quelle
CopyOnWriteBuffer	Copy-on-Write Snapshot-Buffer (Persistent Data Structures): <code>put()</code> / <code>get()</code> inkrementiert Snapshot-Version, frühere Snapshots bleiben logisch erhalten via <code>shared_ptr</code> . Snapshot-Versioning-Paradigma (anders als Delta-Chain Append, Tombstone Erase). <code>is_versioned()=true</code> . Family Q10, QS4 <code>versioned_access</code> . Literatur: Driscoll/Sarnak/Sleator/Tarjan JCSS 1989.	Driscoll et al. 1989
EpochBuffer	Epoch-based Buffer / QSBR (Quiescent-State-Based-Reclamation): Einträge in aktueller Epoche akkumuliert. Nach <code>epoch_threshold</code> Übergang => frühere Epoche kann reclaimed werden. <code>iterable_aspect_t: epoch_threshold</code> als Laufzeit-Parameter. <code>is_versioned()=true</code> (Reclamation-Window-Versioning). Family Q11, QS4 <code>versioned_access</code> . Literatur: McKenney RCU OLS 2001, Masstree EuroSys 2012, SMART OSDI 2023.	P29 (RCU OLS 2001), Masstree 2012
BatchedInsert Buffer	Batched-Insert Buffer (OLAP-Indexing, ART-Bulk-Insert): <code>put()</code> puffert in Sub-Batches der Größe <code>batch_size_</code> , vollständige Batches werden übergeben (bulk-Drain). Ziel: amortisierter Insert-Cost durch Bulk-Operations, SIMD-Vectorization, Cache-Locality. Erste QS5-Strategie. Family Q12, QS5 <code>batched_access</code> . Literatur: Leis ART ICDE 2013 §6.	P01 (Leis ART ICDE 2013)
LockFree SPSCBuffer	Lock-Free SPSC Ring-Queue (Lamport 1977): Single-Producer/Single-Consumer. <code>put()</code> wait-free (1 <code>atomic.store</code> + 1 <code>atomic.load</code>), <code>get()</code> wait-free. Kein CAS nötig dank disjoint Modified-Sets. <code>ProgressGuarantee::LockFree</code> (sogar wait-free). <code>iterable_aspect_t: Capacity</code> . Overflow: dropping. Pflicht-Vertrag: genau 1 Producer + 1 Consumer. Family Q13a, QS6 <code>lock_free_access</code> .	Lamport 1977
LockFree MPMCBuffer	Lock-Free Bounded MPMC Queue (Vyukov 2010-11): Multi-Producer/Multi-Consumer via per-cell atomic Sequence-Number und CAS-Loop. <code>Cell.sequence == pos</code> (Producer schreibt), dann <code>pos+1</code> (Consumer liest). Retries bei Contention (wait-free pro Op NICHT garantiert). Capacity MUST Power-of-2. <code>iterable_aspect_t</code> . Family Q13b, QS6 <code>lock_free_access.is_original_eligible=true</code> (Vyukov BSD-2).	Vyukov 1024cores 2010-11, Michael/Scott PODC 1996
OriginalLockFree MpmcConcurrent Queue	<code>moodycamel::ConcurrentQueue</code> (Cameron Desrochers, Block-Based MPMC): Habich-Compliance-Wrapper mit Original-Code-Linking (2/6 Methoden original: <code>put</code> , <code>get</code> ; 4/6 Lücken: <code>emplace</code> , <code>peek_front</code> , <code>peek_back</code> , <code>clear</code> sind Re-Impl). <code>is_original_module()=false</code> wegen Lückenfüllungen. Header-only, BSD-2/BSL-1.0 dual. Analog <code>LockFreeMPMCBuffer</code> , aber mit echter Submodule-Bindung. Family Q15, QS6 <code>lock_free_access.is_original_eligible=true</code> (moodycamel BSD-2).	moodycamel 2014, BSD-2/BSL-1.0

Beitragende Quellen: LMAX 2011, P01 (Leis ART ICDE 2013), Bw-Tree (Levandowski et al. ICDE 2013), P29 (RCU OLS 2001), Lamport 1977, Vyukov 1024cores 2010-11, Michael/Scott PODC 1996, Pugh CACM 1990, O'Neil LSM-tree 1996, Driscoll et al. 1989, Masstree EuroSys 2012, SMART OSDI 2023, Williams 1964, moodycamel 2014 [TFB⁺11, LKN13, LLS13, McK01, Lam77, Vyu11, MS96, Pug90, OCGO96, DSST89, MKM12, LZS⁺23, Wil64, Des14].

D.19 T18 Pufferung Q2 (Flush-Policy)

Diese Achse ist das *Pufferungs-Organ (Q2)* (T18) des Lebewesens `SearchAlgorithm` (vgl. Kapitel 3.2).

Die Achse Q2 definiert automatische Spülstrategien für Schreibpuffer. Sie entscheidet, WANN ein gefüllter

Buffer sein Ausgabeergebnis an die nächste Stufe propagiert — nach je einer Operation (Eager), gar nicht (Lazy), bei Schwellwertüberschuss (Watermark), nach Zeitfenster (Timed) oder adaptiv nach Workload-Häufigkeit (EWMA-gesteuert LSM-Stil). Dies ist zentral für den Speicher-/Durchsatz-Tradeoff und die Batching-Effizienz in Cache-Engines.

Statische Sub-Achsen

ID	Name	Beschreibung
FS1	Ereignisgetriggert (Event-Triggered)	Flush wird durch explizite Ereignisse (put/get-Operation oder Eviction) ausgelöst. Umfasst EagerFlush (nach jeder Operation) und LazyFlush (nur bei Eviction).
FS2	Schwellwertgetriggert (Threshold-Triggered)	Flush wird ausgelöst, wenn die Pufferfüllung einen Schwellwert (Default 75%) überschreitet. Reduziert unnötige Spülungen bei sparsen Traffic.
FS3	Zeitgetriggert (Time-Triggered)	Flush wird periodisch nach einem Zeitfenster (Micro-Batching) ausgelöst, unabhängig von Pufferfüllung. Klassisches Hadoop/Spark/Kafka-Pattern für bounded-latency Guarantees.
FS4	Adaptiv (Adaptive-Triggered)	Flush-Schwellwert wird adaptiv aus Workload-Häufigkeit via EWMA gelernt. Hohe Write-Rate → früher spülen (60% Schwelle); niedrige Rate → später spülen (95% Schwelle). Erste Q2-Strategie mit self-tuning.

Bausteine

Variante	Beschreibung	Quelle
EagerFlush	Pessimistische Write-Through-Policy: spült nach JEDER <code>put()</code> -Operation vollständig (FullFlush). Niedrigste Latenz, aber maximale Spülfrequenz und höchster Durchsatzaufwand — klassisches Synchronous-Write-Pattern.	Lehrbuch Write-Through Policy / Code Baseline
LazyFlush	Optimistische Write-Back-Policy: spült NIE automatisch, nur bei expliziter Eviction aus dem Buffer. Maximales Batching, minimale Spülfrequenz — klassisches Deferred-Write-Pattern. <code>should_flush()</code> gibt IMMER NoFlush zurück.	Lehrbuch Write-Back Policy / Code Baseline
WatermarkFlush	Schwellwert-gesteuerte Flush-Policy (Default 75% Kapazität). Iterable Aspekt mit 5 Varianten (50%, 65%, 75%, 85%, 95%) für Laufzeit-Permutation. Verwandt mit LSM-Tree Memtable-Trigger (O’Neil 1996).	LSM-Tree O’Neil 1996 (verwandt) / Code Implementation
TimedFlush	Zeit-Fenster-basierte Micro-Batching-Policy (iterable Aspekt: 10/100/1000/10000 ms). Inspiriert von Kafka (NetDB 2011) und Spark D-Streams (SOSP 2013) für Bounded-Latency-Guarantees. Unabhängig von Pufferfüllung.	Kafka NetDB 2011 / Spark SOSP 2013 / Code Implementation
AdaptiveLsm Flush	Selbst-tunende adaptive Watermark-Policy mit EWMA über Burst-Rate. Threshold interpoliert zwischen 60% (hohe Write-Rate) und 95% (niedrige Rate). Konzeptuell RocksDB Dynamic Leveled Compaction + O’Neil LSM 1996; EWMA-Logik ist CE-eigen. Erste Q2-Strategie mit <code>is_adaptive=true</code> .	RocksDB Dynamic Leveled Compaction / O’Neil LSM-Tree 1996 / Code Implementation

Beitragende Quellen: O’Neil et al. 1996 (LSM-Tree), Kafka (NetDB 2011), Spark D-Streams (SOSP 2013) [OCGO96, KNR11, ZDL⁺13].

D.20 Build-PG Seitentyp-Achse (PAGE-TYPE)

Die Seitentyp-Achse definiert die Struktur- und Indexierungsstrategien von Branch-Seiten im Trie-Index: von dicht-indiziertem Direktadressierungsindex (DenseByte), über mehrstufige Dichte-Varianten (ExtendedDense, SparsePatricia) bis zu Pfadkollaps-Optimierungen (Redirect, CustomCache) und B+-artige sortierte Seiten (BPlus). Alle Seitentypen sind Cache-Line-bewusst und implementieren das Concept-Interface `PageTypeStrategy` mit garantierter Austauschbarkeit (F15-Permutation).

Statische Sub-Achsen

ID	Name	Beschreibung
PG1	Struktur-Rolle (Branching)	Klassifizierung der Verzweigungsseiten-Architektur: Branch-Seite mit voller Verzweigung, kollabierter Single-Path-Knoten (Redirect) oder spezielle Cache-optimierte Variante.
PG2	Dichte-Klasse (Density)	Grad der Slot-Auslastung und daraus resultierende Indexierungsstrategie: dicht besiedelt (0-256 Slots), erweitert (>256 Slots) oder sparse mit Patricia-Kompression.
PG3	Pfad-Kollaps-Strategie	Handhabung eindeutiger Restpfade: keine Kompression (Standard), oder Kollaps durch Redirect/CoCo-Technik zur Speicherminimierung.

Bausteine

Variante	Beschreibung	Quelle
<code>DenseBytePageType</code>	Dichte byte-indizierte Verzweigungsseite mit direkter 256-Slot-Adressierung (direct-addressed). Seitentyp 1: direkte Abbildung von Schlüssel-Byte auf Slot-Index ohne Umleitung. Quelle: Adaptive Radix Tree (ART), Leis et al. 2013.	P01
<code>ExtendedDensePageType</code>	Erweiterte dichte Seite für >256 Einträge mit mehrstufigem Indexierungsschema (z.B. 2-Level-Lookup oder variable k-Byte-Span). Seitentyp 2: Dichte-Klasse für komplexere Slot-Anordnungen. Quelle: Adaptive Radix Tree Erweiterungen, PRT-ART.	P01 (extended)
<code>SparsePatriciaPageType</code>	Spärlich besetzte Seite mit Patricia-Pfadkompression (Single-Bit-Split). Seitentyp 3: Dichte-Klasse mit Bit-Vektor-Traversal für niedrige Branchingness. Quelle: HOT (Height Optimized Trie, Binna et al. 2018, basierend auf Patricia-Trie, Morrison 1968).	P02
<code>RedirectPageType</code>	Kollabierter eindeutiger Restpfad (Single-Path-Knoten). Seitentyp 4: Pfad-Kollaps-Strategie zur Speicheroptimierung bei monopol eindeutigen Nachfolgersequenzen. Weder klassischer Branch noch Leaf, speichert nur das Suffix bis zum nächsten echten Verzweigungs- oder Wert-Knoten. Quelle: CoCo-Trie (Boffa et al. 2024).	P04
<code>CustomCachePageType</code>	Cache-Line-orientierte Custom-Seite mit explizitem Layout-Design für 64-Byte-Alignment. Seitentyp 5: Struktur-Rolle Branch mit Hardware-bewusster Geometrie. Quelle: PRT-ART Eigenentwicklung mit TLB-Offset + Cache-Line-Aligned Memory Layout (Achse 5).	PRT-ART
<code>BPlusPageType</code>	B+-artige sortierte Verzweigungsseite mit Range-Walk-Traversal. Seitentyp 6: Struktur-Rolle Branch, aber mit sortierter Key-Array-Organisation statt Byte-Indexierung. Quelle: Masstree (Mao et al. 2012), B+-Tree-Familie.	P03

Beitragende Quellen: P01, P02, P03, P04, Morrison 1968, PRT-ART [LKN13, BZP⁺18, MKM12, BFTV24, Mor68].

Verwandte Literatur: P05, P06, P10, P11, P12, P13, P20 [FJKN20, SSL⁺21, ZLL⁺18, RR99, RR00, HP03, MBL25].

D.21 Build-SIMD: Vektor-/Accelerator-Erweiterungs-Achse (Hardware-Beschleuniger)

Die Achse `axis_09b_simd_extension` modelliert die verfügbaren SIMD-/Accelerator-Erweiterungen der CPU-Instruktionssatz-Architektur als Sub-Achse zu `axis_09` (Haupt-ISA). Sie ermöglicht Compile-Time-Auswahl zwischen verschiedenen Vektor-Erweiterungen (SSE2/AVX2/AVX-512 für x86; NEON/SVE2 für ARM; RVV für RISC-V) sowie Baseline (Skalar) und GPU-Beschleunigung (CUDA), mit rückwärts-kumulativer Schichthierarchie und hardwarespezifischer Topologie (SIMD-Units pro Sockel, P/E-Core-Zugriff).

Statische Sub-Achsen

ID	Name	Beschreibung
SE1	Vector-Width (Vektorbreite)	Compile-Time-bekannte maximale Vektorbreite der SIMD-Extension in Bits: 0 (keine), 128, 256, 512, oder -1 (skalierbar). Bestimmt Lane-Breite und Parallelisierungsgrad.
SE2	Accelerator-Type (Beschleuniger-Typ)	Kategorisiert die Beschleuniger-Klasse: none (Baseline), simd (Vektor-SIMD), matrix (Matrix-Engine), gpu (GPU), npu (Neural Processing Unit). Influenziert Komposition mit hardware-Topologie (<code>axis_12</code>).
SE3	Compat-Family (Kompatibilitäts-Familie)	Constraint zur Haupt-ISA (<code>axis_09</code>): none (universell), x86, arm, riscv, cuda. Sichert Compile-Time-Validierung: z.B. <code>Sse2SimdExtension</code> kompatibel nur mit <code>Amd64Isa</code> .

Bausteine

Variante	Beschreibung	Quelle
<code>NoSimdExtension</code>	Baseline-Variante ohne SIMD-Beschleunigung. Skalare Fallback-Strategie für alle Permutationen ohne Accelerator. Universal kompatibel zu allen Haupt-ISAs (x86/ARM/RISC-V/PowerPC). Pflicht-Vergleichspunkt.	Code/Baseline
<code>Sse2SimdExtension</code>	Intel SSE2 128-bit Vektor-Extension (x86_64 ABI-Pflicht seit 2003). Nur kompatibel mit <code>Amd64Isa</code> . Baseline auf x86_64; bietet 4x uint32-Lanes/Iteration. Standard auf Cluster-Knoten Barnard/Capella.	Intel ISA Ext Ref 319433-060
<code>Avx2SimdExtension</code>	Intel AVX2 256-bit Vektor-Extension (Haswell+ 2013). Nur x86_64. Rückwärts-kumulativ: umfasst alle SSE+AVX. Default-Optimierungs-Stufe für Server-DBMS. 2 AVX2-Units/Sockel. ZIH-Standard auf Barnard (AMD EPYC Zen 3) und Capella (Intel Xeon).	Intel ISA Ext Ref 319433
<code>Avx512SimdExtension</code>	Intel AVX-512 512-bit Vektor-Extension (Skylake-X+, Ice Lake Server, Zen 4+). Rückwärts-kumulativ: alle SSE/AVX/AVX2 + 15 separate Flags (VNNI für Vector-Indexes, VPOPCNTDQ für Bitmap-Indexes). Nicht auf allen x86_64-CPU's (z.B. Haswell/Raptor Lake disabled). 1 AVX-512-Unit/Sockel; E-Cores kein Zugriff (Intel Hybrid).	Intel ISA Ext Ref 319433-060

Variante	Beschreibung	Quelle
NeonSimdExtension	ARM NEON / Advanced SIMD 128-bit (AArch64 ABI-Pflicht ARMv8+). Kompatibel nur mit AArch64Isa. Baseline auf allen AArch64-CPU's (Apple M-Series, AWS Graviton 2/3/4, NVIDIA Grace). 1 NEON-Unit/Socket; alle Cores zugänglich (big.LITTLE-Architektur).	ARM ARM DDI 0487 / Cortex-A8 TRM
Sve2SimdExtension	ARM SVE2 (Scalable Vector Extension 2) ARMv9-Pflicht (skalierbar 128–2048 bit). Kompatibel nur mit AArch64Isa. ZIH-Cluster Grace Hopper (NVIDIA Grace = ARM Neoverse V2 mit 128-bit SVE2). Skalierbare Lane-Breite; 1 Unit/Socket; alle Cores.	SVE [SBB ⁺ 17] (IEEE Micro 2017); SVE2/ Armv9 per ARM ARM [Arm24]
RvvSimdExtension	RISC-V Vector Extension (RVV v1.0, skalierbar VLEN 128–65536 bit). Kompatibel nur mit RiscvIsa. Optional (nicht Baseline wie SSE2/NEON). Wachsende Relevanz: SiFive Performance P870 (128-bit), T-Head Xuantie C908 (128-bit). Forschungs-Boards an TU Dresden/ZIH.	RVV-v1.0-Spec [RIS21]; Vitruvius+ (TACO 2023) [MPU+23]
CudaGh200SimdExtension	NVIDIA CUDA auf Hopper GH200 (Grace Hopper Superchip GPU-Beschleuniger). ZIH-Cluster Alpha Centauri mit NVIDIA HGX. Kompatibel mit AArch64Isa (Grace CPU) UND Amd64 Isa (Hopper-Host). NVLink-C2C 900 GB/s zwischen Grace-CPU und Hopper-GPU. Massive parallel (96 GB HBM3 GPU; ~16.896 CUDA Cores). <code>units_per_socket()</code> = -1 (GPU ist separate Device).	NVIDIA Tesla [LNOM08] (IEEE Micro 2008) + NVIDIA GH200 Whitepaper

Quellen: ISA-Spezifikationen und Whitepaper (Intel, ARM, IEEE Micro, ACM TACO, NVIDIA); Einzelnachweise in der Quellenspalte der Bausteine.

D.22 Build-HW — Plattform-Hardware-Strategie

Definiert die Plattform-Hardware-Familie (CPU-Architektur, SIMD-Capabilities, Cache-Topologie, Memory-Paging) als Compile-Time-Konstanten für alle Cache-Engine-Algorithmen. Die Achse erfasst, welche Plattformen zur Build-Zeit konfigurierbar sind, und liefert statische Properties (Cache-Line-Größe, Page-Größe, SIMD-Bit-Breite, NUMA-Fähigkeit, HugePage-Fähigkeit) für algorithmengerechte Datenstrukturen.

Statische Sub-Achsen

ID	Name	Beschreibung
hw1_simd_family	12.1 SIMD-Familie	Welche Vector-Instruction-Set wird aktiv genutzt? (AVX2, AVX-512, NEON, SVE2, scalar/none). Bestimmt die maximale Vektorbreite für parallele Operationen innerhalb eines Algorithmus.
hw2_cache_level_targeting	12.2 Cache-Level-Targeting	Welcher Cache-Level wird optimiert (L1-aware, L2-aware, L3-aware, HBM-aware)? Entscheidend für Prefetch-Strategien und Speicher-Layout-Entscheidungen.
hw3_numa_strategy	12.3 NUMA-Strategie	Wie wird NUMA-Topologie berücksichtigt (single-node, multi-node, NUMA-aware-pinning, NUMA-balanced)? Beeinflusst die Allocator-Lokalität und Thread-Pinning-Strategie.
hw4_prefetch_hardware	12.4 Prefetch-Hardware-Instruktionen	Welche expliziten Hardware-Prefetch-Instruktionen sind verfügbar (PREFETCH, PREFETCHNTA, PREFETCHW, none)? Kernbestandteil von Distance-Estimator und path-oriented Prefetch.

ID	Name	Beschreibung
hw5_ atomic_ instruc tion_ family	12.5 Atomic-Befehl-Familie	Welche atomaren Read-Modify-Write-Befehle sind verfügbar (CAS, LL-SC, RMW-extended via HTM/XBEGIN/XEND, none)? Fundamentale Basis für Lock-Free-Concurrency und Optimistic-Lock-Coupling.

Bausteine

Variante	Beschreibung	Quelle
X86_64Hardware Profile	Moderne x86-64 Desktop/Server-Plattform (Intel/AMD). Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=256</code> (AVX2 konservativ), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Basis für desktop- und server-seitige Experimente.	Code/Baseline
Aarch64Hardware Profile	ARM64/AArch64 Server und Mobile Plattformen (Ampere Altra, Apple Silicon M-Serie, Linux ARM). Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=128</code> (NEON Standard), <code>numa_capable=true</code> , <code>huge_page_capable=true</code> . Ermöglicht Cross-Plattform-Vergleiche.	Code/Baseline
GenericHardware Profile	Konservatives Fallback-Profil für plattform-agnostische Konfigurationen. Build-Time-Konstanten: <code>cache_line_size=64</code> , <code>memory_page_size=4096</code> , <code>simd_width_bits=0</code> (scalar-only), <code>numa_capable=false</code> , <code>huge_page_capable=false</code> . Lowest-Common-Denominator wenn kein spezialisierter Adapter passt.	Code/Baseline

Quellen: Plattform-Profil als Code/Baseline (keine externen Paper).

D.23 A-Korpus Allokator-Korpus (A01-A23)

Die Allokator-Achse dokumentiert 25 speicherverwaltungs-Strategien (Kennungen A01–A21, die drei Standardbibliotheks-Baselines A22a–A22c sowie A23), die das dynamische Speichermanagement in Such-Workloads beeinflussen. Sie umfasst klassische Lehrbuch-Allokatoren (Buddy, Slab), industrielle Standards (dlmalloc, jemalloc, TCMalloc, mimalloc, snmalloc), spezialisierte Varianten (NUMA-aware, PIM-Malloc, Cache-Aware) und moderne gehärtete Implementierungen (StarMalloc). Alle 25 Wrapper erfüllen das `AllocatorStrategy`-Konzept und ermöglichen systematische Messung des Speichermanagement-Einflusses auf Suchbaum-Performance (F15-Operativität).

Statische Sub-Achsen

ID	Name	Beschreibung
AA1	Free-List-Topologie	Struktur der freien Speicher-Blöcke: Per-Page-Sharding (mimalloc), Per-Thread-Caching (snmalloc, rpmalloc, TCMalloc), Buddy-Baum (Buddy-Allocator), Single-Global-Free-List (dlmalloc), Per-Heap-Locks (Hoard).
AA2	Size-Class-Schema	Klassifizierung von Block-Größen: Slab-pro-Objektgröße (Slab), dlmalloc Bins, jemalloc Size-Classes, scallocc Spans, Buddy-Power-of-2.
AA3	Thread-Lokalität	Grad der Thread-Isolation: tcmalloc thread-local-caches, rpmalloc per-thread-spanning, snmalloc message-passing, Hoard per-heap-locks.

ID	Name	Beschreibung
AA4	Synchronisation	Concurrency-Mechanismus: Lock-Free CAS (Michael, LRMalloc), Per-Heap-Locks (Hoard), Wait-Free (Crystalline), Mutex-basiert.
AA5	Allokations-Richtlinie	Routing von Allokationsanfragen: NUMA-origin-aware (NUMAAlloc), Cache-Set-aware (CAMA), NUCA-aware (TCMalloc-Warehouse), PIM-aware (PIM-Malloc).
AA6	Rückgewinnung	Freigabe gelöschter Blöcke: Magazine-Cache (Slab), Page-Heap-Release (TCMalloc), Lock-Free-Reclamation (LRMalloc), Deferred-Free (mimalloc).
AA7	Fragmentierungsstrategie	Vermeidung von Fragmentierung: dlmalloc Coalescing, Object-Coloring (Slab), Buddy-Splitting, Page-Local-Sharding (mimalloc), Span-Pooling (scalloc).

Bausteine

Variante	Beschreibung	Quelle
HoardAllocator (A01)	Per-Heap Free-Lists mit SMP-Skalierung. Drei-Ebenen-Struktur: Thread-lokale Heaps, globale Heap für Cross-Thread-Migration, False-Sharing-Reduktion durch segregierte Allocations-Anfragen.	A01 [BMBW00] (Berger/McKinley/ Blumofe/Wilson, ASPLOS 2000, 10.1145/378993.379232) + Code/ Baseline
SlabAllocator (A02)	Object-Caching Kernel-Allocator. Slab-pro-Objektgröße mit Magazine-Cache pro CPU (Bonwick 1994/2001). Klassische Solaris-Kernel-Implementierung für vorallokierte Objekt-Pools.	A02 [Bon94] (Bonwick, USENIX 1994) + Code/Baseline
MichaelLockFree Allocator (A03)	Lock-Free CAS-Allocator nach Michael 2004. Hazard-Pointer-geschützte Lock-Free-Datenstrukturen, MIT-Re-Implementierung mit IBM-Patent-Hintergrund.	A03 [Mic04b] (Michael, PLDI 2004, 10.1145/996841.996848) + Code/ Baseline
MimallocAllocator (A04)	Free-List-Sharding (3 Free-Lists pro Page). Microsoft Research 2019: deferred_free Hooks, temporal Cadence, Page-lokale Sharding. is_original: MIT-Direktbindung.	A04 [LZM19] (Leijen/Zorn/de Moura, APLAS 2019, 10.1007/ 978-3-030-34175-6_13) + is_original (github.com/microsoft/mimalloc, MIT)
JemallocAllocator (A05)	Arena-based Size-Class-Allocator. Evans 2006 / Free BSD. Pro-CPU Arenas mit Tcache-Layer für Thread-lokale Caches. Facebook-Standard. is_original: BSD-2-Clause-Direktbindung.	A05 [Eva06] (Evans, BSDCan 2006, papers.freebsd.org/.../jemalloc) + is_ original (github.com/jemalloc/jemalloc, BSD-2-Clause)
TCMallocAllocator (A06)	Thread-Cache + Central-Cache + Page-Heap. Google gperftools 2005. Später TEMERAIRE (OSDI 2021) mit Warehouse-Scale-Optimierungen.	A06 [HKT ⁺ 21] (TCMalloc 2005; TEMERAIRE, OSDI 2021) + Code/ Baseline
SnmallocAllocator (A07)	Message-Passing-Allocator. Paul Liétar et al. ISMM 2019: Free-Operationen via Lock-Free-Queue an Allocating-Thread, Cross-Thread-Lock-Contention-Vermeidung. is_ original: MIT-Direktbindung.	A07 [LDPB19] (Liétar, ISMM 2019, 10.1145/3315573.3329980) + is_ original (github.com/microsoft/snmalloc, MIT)
ScallocAllocator (A08)	Span-basiert, Virtual-Spans-Technik. Aigner/Kirsch/Lippautz/Sokolova OOPSLA 2015. Multicore-skalierbar mit niedriger Fragmentierung.	A08 [AKLS15] (Aigner et al., OOPSLA 2015) + Code/Baseline
NUMAAllocAllocator (A09)	NUMA-Origin-Aware Allocator. UTSASRG ISMM 2023. Knoten-Hinweis für lokale NUMA-Allokation, Multi-Knoten-Systeme optimiert.	A09 [YZZ ⁺ 23] (Yang et al., ISMM 2023) + Code/Baseline
RPMallocAllocator (A10)	Per-Thread Span-Caching. Mattias Jansson 2017, OSS ohne Paper. Sonderfall: Pflicht-Init (rpmalloc_initialize, rpmalloc_thread_initialize). is_original: Public-Domain/MIT-Direktbindung.	Code/Baseline (github.com/mjansson/ rpmalloc, Public-Domain/MIT) + is_ original

Variante	Beschreibung	Quelle
LRMallocAllocator (A11)	Lock-Free + Hazard-Pointers. Leite/Rocha VECPAR 2018 (LNCS 11333). Moderner Lock-Free-Allocator mit ABA-Protection. is_original: MIT-Direktbindung.	A11 [LR18] (Leite/Rocha, VECPAR 2018, 10.1007/978-3-030-15996-2_17) + is_original (github.com/ricleite/lrmalloc, MIT)
CAMAAAllocator (A12)	Cache-Aware Memory Allocator. Herter/Backes/Hauptenthal/Reineke ECRTS 2011 (Saarland). Vorhersagbare Cache-Behavior, keine öffentliche Code-Referenz.	A12 [HBHR11] (Herter et al., ECRTS 2011) + Code/Baseline
StarMalloc Allocator (A13)	Formal Verified Hardened Allocator. Inria-Prosecco (Reitz/Fromherz/Protzenko et al.) OOPSLA 2024. F*/Steel-formale Verifikation, Apache-2.0-Lizenz.	A13 [RFP24a] (Reitz et al., OOPSLA 2024) + Code/Baseline
TCMallocWarehouse Allocator (A14)	TCMalloc-Warehouse / Hyperscale. Google Cloud OSDI 2021 / ASPLOS 2024. Hupage-aware Variante, Marketing-Variante ohne eigenständiges Artefakt.	A14 [HKT ⁺ 21, ZGV ⁺ 24] (Hunter OSDI 2021 / Zhou ASPLOS 2024) + Code/Baseline
HMallocAllocator (A15)	Hybrid, Scalable, Lock-Free Memory Allocator. Li/Yao/Tang/Lin ICPADS 2019. Hybrid-Ansatz für Skalierbarkeit, keine öffentliche Code-Referenz.	A15 [LYTL19] (Li/Yao/Tang/Lin, ICPADS 2019, 10.1109/ICPADS47876.2019.00064) + Code/Baseline
PIMMalloc Allocator (A16)	Processing-In-Memory Allocator. VIA-Research HPCA 2026 (arXiv:2505.13002). Speicherverteilung über Rechenknoten, PIM-Hardware-optimiert.	A16 [VIA26] (VIA-Research, HPCA 2026, arxiv.org/abs/2505.13002) + Code/Baseline
Crystalline Reclamation (A17)	Wait-free bzw. lock-/wait-free Safe-Memory-Reclamation-Familie für nicht-blockierende Datenstrukturen. Kein eigenständiger malloc/free-Allokator, sondern Reclamation-Policy.	A17 [NR21, NR24] (Nikolaev/Ravindran, DISC 2021 / PLDI 2024) + Code/Baseline
ExgenAllocator (A18)	Single-Threaded Specialized Allocator. UT Austin IEEE CAL 2025 (arXiv:2510.10219). 'Old is Gold'-Optimierungen für Einthread-Workloads, keine öffentliche Code-Referenz.	A18 [LJY25] (UT Austin, IEEE CAL 2025, arxiv.org/abs/2510.10219) + Code/Baseline
BuddyAllocator (A19)	Buddy System (Power-of-2-Splitting). Knuth TAOCP Vol.1 (1968), CACM 1965. Klassischer Lehrbuch-Allocator, Binärbaum-Splitting ohne OSS-Code-Referenz.	A19 [Kno65] (Knuth/Carlson, CACM 1965, 10.1145/365628.365655) + Code/Baseline
DlmallocAllocator (A20)	Doug Lea Malloc (Bins-based). Lea 1987-2012, Public-Domain (CC0). Klassiker seit 1987, Bins-Size-Class-Schema, älteste produktive Implementierung. is_original: Public-Domain-Direktbindung.	A20 [Lea96] (Lea, gee.cs.oswego.edu/.../malloc.html, 1996) + is_original (github.com/ennorehling/dlmalloc, Public-Domain/CC0)
PtMalloc2 Allocator (A21)	ptmalloc2 (glibc malloc). Wolfram Gloger 1990er-2000er. LGPL-2.1+ Copyleft, Linking blockiert (no is_original). Standard-Basis für glibc-basierte Systeme.	Code/Baseline (malloc.de/en/) + License restriction
StdMalloc (A22a)	Standard libc malloc (Concept-Beweis), portabler Fallback-Allokator. Baseline für alle Standard-Bibliotheks-Implementierungen ohne Vendor-Linking.	Code/Baseline (C++ Standard Library) + Baseline
PmrResource Allocator (A22b)	std::pmr::memory_resource. WG21 N3916 (2014 → C++17). Polymorphic Memory Resources Standard-Bibliothek, Fallback auf new/delete-Allokator.	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + Baseline
PoolResource Allocator (A22c)	std::pmr::unsynchronized_pool_resource. WG21 N3916 (C++17). Eigener Size-Class-Pool, F15-operativ (erste Wrapper mit eigenem Speicherhaltungsverhalten ohne Vendor-Linking).	Code/Baseline (C++17 Standard Library, open-std.org/.../n3916.pdf) + is_operative

Variante	Beschreibung	Quelle
VmemMagazines Allocator (A23)	Vmem + Magazines (Slab-Erweiterung). Bonwick 2001 USENIX ATC. Per-CPU-Magazine für Skalierung, Kernel-Klassiker ohne öffentliche Code-Referenz.	A23 [BA01] (Bonwick, USENIX ATC 2001, usenix.org/.../bonwick.html) + Code/Baseline

Beitragende Quellen: je Allokator A01–A23 einzeln in der Quellenspalte der Bausteine-Tabelle nachgewiesen.

E Architekturentscheidungen

F Vergleichsinterfaces: `std::map` und `std::vector`

Dieser Anhang führt — ergänzend zu Abschnitt 2.3 — die vollständige Auflistung aller Standard-Interface-Funktionen der beiden Vergleichshüllen `std::map` (geordnet-assoziativ) und `std::vector` (Sequenz) nach dem Stand von C++23 mit ihrer offiziell erwarteten Semantik und ihrer Komplexitäts-Garantie gemäß dem ISO-C++-Standard [ISO24]. Er dient zugleich als zentrale Referenz für die Weiterentwicklung der Cache-Engine: Da die Suchalgorithmus-Hülle ihr Innenleben durch dynamisch austauschbare Achsen-Algorithmen ersetzt (Abschnitt 2.3), wird je Standard-Funktion erklärungsbedürftig, welches Verhalten der äußere Vertrag garantiert.

Variadische Abbildung. Ein Template-Parameter stellt die `std::vector`-API bereit, zwei Parameter die `std::map`-API und $N > 2$ eine `std::map<K, std::tuple<...>` (Abschnitt 2.3); die geteilten Funktionen (Iteration, `size/empty/clear`, `swap`, `get_allocator`) tragen in beiden Hüllen dieselbe Semantik.

Zusammengesetzte Funktionen. Einige Funktionen ergeben sich „unter der Haube“ als Komposition primitiverer Operationen — etwa `operator[]` als `lower_bound` mit anschließendem Default-Einfügen bei fehlendem Schlüssel oder `at` als `find` mit Wurf einer `std::out_of_range`-Ausnahme bei Fehlen —; solche Fälle sind in der Spalte „Anmerkung“ gekennzeichnet. Die Spalte „seit“ nennt die einführende Standard-Fassung. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Konformitäts-Teilmenge. Das Konformitäts-Gatter (Abschnitt 2.3) prüft vor jeder Messung eine Kern-Teilmenge der `std::map`-Operationen, damit nur Vergleichbares verglichen wird; sie ist in der `std::map`-Tabelle als solche gekennzeichnet.

F.1 `std::map` (geordnet-assoziativ)

Tabelle F.1 listet die vollständige Schnittstelle von `std::map<Key, T, Compare, Allocator>` nach C++23. Ein † markiert die Kern-Operationen, die das Konformitäts-Gatter (Abschnitt 2.3) vor jeder Messung prüft, damit nur Vergleichbares verglichen wird. Komplexitäten beziehen sich auf $n = \text{size}()$; bei Bereichs-Operationen ist N die Zahl der Eingabe-Elemente.

Tabelle F.1: Vollständige `std::map`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			

Funktion	Offizielle Semantik	Komplexität	seit
(Konstruktor)	leer; aus Compare/Allocator; Bereich; Kopie/Move (auch allokator-erweitert); initializer_list; ab C++23 from_range	$O(1)$ bis $O(N \log N)$	C++98
(Destruktor)	zerstört alle Elemente und Knoten	$O(n)$	C++98
operator=	Kopier-/Move- /initializer_list-Zuweisung mit Allokator-Propagation	$O(n)$	C++98
get_allocator	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
at †	Referenz auf den Mapped-Wert; wirft std::out_of_range bei Fehlen	$O(\log n)$	C++11
operator[] †	Referenz; fügt bei Fehlen wertinitialisiert ein (wirft nie)	$O(\log n)$	C++98
Iteratoren			
begin/end †	Iterator auf erstes Element bzw. Ende, in Schlüsselordnung (cbegin/cend ab C++11)	$O(1)$	C++98
rbegin/rend	Rückwärts-Iteratoren (crbegin/crend ab C++11)	$O(1)$	C++98
Kapazität			
empty †	leer-Test (begin() == end()); [[nodiscard]] ab C++20	$O(1)$	C++98
size	Anzahl der Elemente	$O(1)$	C++98
max_size	theoretische Maximalgröße	$O(1)$	C++98
Modifikatoren			
clear †	alle Elemente entfernen	$O(n)$	C++98
insert †	Element(e) ohne Überschreiben (Wert, P&&, Hint, Bereich, initializer_list, Node-Handle ab C++17)	$O(\log n)$; Hint amort. $O(1)$	C++98
insert_range	Eingabe-Bereich einfügen, keine Überschreibung	$O(N \log(n+N))$	C++23
insert_or_ assign †	einfügen oder vorhandenen Wert zuweisen; bool meldet Einfügung	$O(\log n)$	C++17
emplace †	in-place konstruieren, nur falls Schlüssel fehlt (sonst verworfen)	$O(\log n)$	C++11

Funktion	Offizielle Semantik	Komplexität	seit
<code>emplace_hint</code>	wie <code>emplace</code> , positionsnah zum Hint	$O(\log n)$; amort. $O(1)$	C++11
<code>try_emplace†</code>	wie <code>emplace</code> , rührt die Argumente bei vorhandenem Schlüssel nicht an	$O(\log n)$	C++17
<code>erase†</code>	nach Iterator/Bereich/Schlüssel (Schlüssel: 0 oder 1 entfernt)	Iter. amort. $O(1)$; Key $O(\log n)$	C++98
<code>swap</code>	Inhalt und Comparator tauschen; Iteratoren bleiben gültig	$O(1)$	C++98
<code>extract</code>	Element als Node-Handle entnehmen (ohne Kopie/Move)	Iter. amort. $O(1)$; Key $O(\log n)$	C++17
<code>merge</code>	Knoten aus Quell-Map splicen; vorhandene Schlüssel bleiben dort	$O(N \log(n+N))$	C++17
Suche			
<code>find†</code>	Iterator auf den Schlüssel oder <code>end()</code>	$O(\log n)$	C++98
<code>count†</code>	Anzahl (0 oder 1)	$O(\log n)$	C++98
<code>contains†</code>	Mitgliedschaft (bool)	$O(\log n)$	C++20
<code>lower_bound†</code>	erster Schlüssel $\geq$ key	$O(\log n)$	C++98
<code>upper_bound†</code>	erster Schlüssel $>$ key	$O(\log n)$	C++98
<code>equal_range†</code>	Paar <code>[lower_bound, upper_bound)</code>	$O(\log n)$	C++98
Beobachter			
<code>key_comp†</code>	Kopie des Schlüssel-Comparators	$O(1)$	C++98
<code>value_comp</code>	<code>value_compare</code> (vergleicht Paare über den Schlüssel)	$O(1)$	C++98
Nicht-Member-Funktionen			
<code>operator==</code>	gleiche Größe und paarweise gleiche Elemente	$O(n)$	C++98
<code>operator<=></code>	lexikografischer Drei-Wege-Vergleich; synthetisiert <code><=>=></code>	$O(n)$	C++20
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions-Leitfäden	CTAD aus Bereich/ <code>initializer_list</code> / <code>from_range</code>	— (Compile-Zeit)	C++17

Die vor C++20 eigenständigen Vergleichsoperatoren $< <= > >=$ werden seit C++20 aus `operator<=>` synthetisiert, `!=` aus `operator==` umgeschrieben. Die heterogenen Überladungen von `operator[]`, `at`, `try_emplace` und `insert_or_assign` (transparenter Schlüssel) gehören erst zu C++26 und bleiben hier ausgeklammert; die C++23-Neuzugänge sind genau der `from_range`-Konstruktor, `insert_range` und die transparenten Schlüssel-Überladungen von `erase/extract`.

F.2 `std::vector` (Sequenz)

Tabelle F.2 listet die vollständige Schnittstelle von `std::vector<T, Allocator>` nach C++23 ($n = \text{size}()$). Sämtliche Member sind seit C++20 `constexpr`.

Tabelle F.2: Vollständige `std::vector`-Schnittstelle (C++23)

Funktion	Offizielle Semantik	Komplexität	seit
Konstruktion, Zuweisung, Allokator			
(Konstruktor)	leer/Allocator; $n \times$ Default/Wert; Bereich; Kopie/Move (auch allokator-erweitert); <code>initializer_list</code> ; ab C++23 <code>from_range</code>	$O(1)$ oder $O(n)$	C++98
(Destruktor)	zerstört alle Elemente, gibt Speicher frei	$O(n)$	C++98
<code>operator=</code>	Kopie/Move/ <code>initializer_list</code>	$O(n)$	C++98
<code>assign</code>	Inhalt durch $n \times$ Wert/Bereich/ <code>initializer_list</code> ersetzen	$O(n)$	C++98
<code>assign_range</code>	Inhalt durch Eingabe-Bereich ersetzen	$O(n)$	C++23
<code>get_allocator</code>	Kopie des zugeordneten Allokators	$O(1)$	C++98
Elementzugriff			
<code>at</code>	geprüfter Zugriff; wirft <code>std::out_of_range</code>	$O(1)$	C++98
<code>operator[]</code>	ungeprüfter Zugriff (UB außerhalb des Bereichs)	$O(1)$	C++98
<code>front/back</code>	Referenz auf erstes/letztes Element (UB bei leer)	$O(1)$	C++98
<code>data</code>	Zeiger auf den zusammenhängenden Speicher (nicht bei <code>vector<bool></code>)	$O(1)$	C++11
Iteratoren			

Funktion	Offizielle Semantik	Komplexität	seit
begin/end, rbegin/rend	Vorwärts-/Rückwärts-Iteratoren in Indexordnung (c...-Varianten ab C++11)	$O(1)$	C++98
Kapazität			
empty/size/ max_size	leer-Test, Elementanzahl, Maximalgröße	$O(1)$	C++98
reserve	Kapazität $\geq n$; realloziert und invalidiert; wirft <code>length_error</code> bei $n > \text{max_size}()$	$O(n)$	C++98
capacity	aktuell allozierte Kapazität	$O(1)$	C++98
shrink_to_fit	nicht-bindende Reduktion der Kapazität auf <code>size()</code>	$O(n)$	C++11
Modifikatoren			
clear	alle Elemente entfernen (Kapazität bleibt)	$O(n)$	C++98
insert	vor Position: Wert/ $n \times$ /Bereich/ <code>initializer_list</code> ; verschiebt den Schwanz	$O(n)$	C++98
insert_range	Eingabe-Bereich vor Position einfügen	$O(n)$	C++23
emplace	Element in-place vor Position konstruieren	$O(n)$	C++11
erase	Position/Bereich entfernen, Schwanz nachrücken	$O(n)$	C++98
push_back	Element anhängen (Kopie/Move)	amort. $O(1)$	C++98
emplace_back	in-place anhängen; Referenz-Rückgabe ab C++17	amort. $O(1)$	C++11
append_range	Eingabe-Bereich anhängen	amort. $O(n)$	C++23
pop_back	letztes Element entfernen (UB bei leer)	$O(1)$	C++98
resize	auf n kürzen oder mit Default/Wert auffüllen	$O(n)$	C++98
swap	Inhalt und Kapazität mit anderem Vektor tauschen	$O(1)$	C++98
Nicht-Member-Funktionen			
operator==	gleiche Größe und elementweise Gleichheit	$O(n)$	C++98
operator<=>	lexikografischer Drei-Wege-Vergleich	$O(n)$	C++20

Funktion	Offizielle Semantik	Komplexität	seit
<code>std::swap</code>	Spezialisierung; ruft <code>lhs.swap(rhs)</code>	$O(1)$	C++98
<code>std::erase</code>	entfernt alle Elemente gleich dem Wert; gibt die Anzahl zurück	$O(n)$	C++20
<code>std::erase_if</code>	entfernt alle Elemente mit <code>pred==true</code> ; gibt die Anzahl zurück	$O(n)$	C++20
Deduktions- Leitfäden	CTAD aus Bereich/ <code>from_range</code>	— (Compile-Zeit)	C++17

Die Spezialisierung `std::vector<bool>` packt Wahrheitswerte bitweise: Sie besitzt kein `data()` und keine Zusammenhangs-Garantie, liefert über `operator[]/at/front/back` einen Proxy-Typ `reference` statt `bool&` und bietet zusätzlich `flip()` (auf Vektor- und auf `reference`-Ebene), ein statisches `swap(reference, reference)` sowie eine `std::hash`-Spezialisierung. Die Vergleichsoperatoren verhalten sich wie bei `std::map`.

F.3 Zusammengesetzte Funktionen und Konformitäts-Teilmenge

Mehrere Schnittstellen-Funktionen sind „unter der Haube“ Kompositionen primitiverer Operationen (Tabelle F.3); die Tabelle beschreibt den ISO-Standardvertrag. Die eigene Mess-Hülle weicht an zwei Stellen bewusst davon ab, weil der ABI-Konformitätsvertrag `noexcept` verlangt und eine werfende Hülle als nicht-konform gilt: `operator[]` ist dort ein nicht-mutierendes `std::optional`-Lesen (der Schreibpfad läuft über `set()/insert_or_assign`), und `at` wirft nicht (`optional`-/Status-Rückgabe). Für diese Arbeit ist das die zentrale Beobachtung: Sämtliche Map-Zugriffe und -Modifikatoren ruhen auf einem einzigen Ordnungs-Such-Primitiv (`lower_bound/find`); eine Achsen-Implementierung muss daher nur eine korrekte Ordnungs-Suche samt Knoten-Verkettung bereitstellen, der Rest ergibt sich als feste Komposition — genau die Austauschbarkeit, deren Vergleich der zentrale Mess-Akt ist (Abschnitt 2.3). Das Konformitäts-Gatter prüft die in Tabelle F.1 mit † markierte Kern-Teilmenge.

Tabelle F.3: Zusammengesetzte Funktionen („unter der Haube“)

Funktion	Zusammensetzung	Anmerkung
<code>map::</code>	<code>lower_bound</code> + Default-Einfügen	mutiert bei Fehlen;
<code>operator[]</code>	bei Fehlen → Referenz	wirft nie
<code>map::at</code>	<code>find</code> + Wurf	einziger werfender
	<code>std::out_of_range</code> bei Fehlen	Zugriff; mutiert nie

Funktion	Zusammensetzung	Anmerkung
<code>map::insert / emplace</code>	Ordnungs-Suche + Knoten verlinken nur bei fehlendem Schlüssel	kein Überschreiben; <code>emplace</code> kann Argumente bei Treffer dennoch konsumieren
<code>map::try_ emplace</code>	wie <code>emplace</code> , aber Argumente bei Treffer unangetastet	bewahrt move-only- Argumente
<code>map::insert_ or_assign</code>	Ordnungs-Suche + Einfügen oder Zuweisen	überschreibt (anders als <code>insert</code>)
<code>map::count / contains / equal_range</code>	reduzieren auf <code>find</code> bzw. <code>lower_bound+upper_bound</code>	bei Unikat-Map stets 0 oder 1
<code>vector:: push_back / emplace_back</code>	ggf. Wachstum (Realloc + Umzug) + Konstruktion am Ende	versteckte Kosten + Invalidierung beim Realloc
<code>vector::insert / emplace</code>	ggf. Wachstum + Schwanz verschieben + Einsetzen	zusätzlicher $O(n)$ -Shift
<code>vector::resize</code>	anhängen (Default/Wert) oder Schwanz kürzen	bidirektional; verkleinert die Kapazität nicht
<code>vector::assign</code>	<code>clear</code> + Bereich/ $n \times$ einfügen	Kapazität wird ggf. wiederverwendet

Funktionen, die beide Hüllen dem Namen nach *teilen* — `size`, `empty`, `clear`, die Iterator-Familien, `swap`, `get_allocator` sowie `insert/emplace/erase` —, tragen in der Map Schlüssel- und im Vektor Positions-/Index-Semantik; `operator[]` etwa fügt in der Map ein, ist im Vektor hingegen ein ungeprüfter Indexzugriff. Die exakten cache-engine-internen Funktions-Sprünge (*function-handle-hops*) hinter diesen Verträgen dokumentiert ein eigenes Entwickler-Dokument der Cache-Engine.

Danksagung

Platzhalter – Danksagung folgt.

